

Kafka + Spark Structured Streaming

Your Name

2026-07-02

Table of contents

Data Management	1
Foreword	1
I. Data Manegment	3
1. From Data to Datasets	5
1.0.1. Data Pipeline Stages	6
1.0.2. Monte Carlo Simulations as a Data Source	6
1.1. Metadata	7
1.1.1. CMS Metadata Example: Path-as-Metadata	7
1.2. Data Structures	8
1.2.1. Structured Data	8
1.2.2. Semi-Structured Data	9
1.2.3. Unstructured Data	10
1.3. Summary: Data Structure Types	11
1.4. What is Data Management?	11
1.5. Exercises	12
2. Storage	15
2.1. Memory Devices	16
2.1.1. DRAM (Dynamic Random Access Memory)	16
2.1.2. NVM (Non-Volatile Memory) / Persistent Memory	17
2.1.3. SSD (Solid State Drives)	17
2.1.4. HDD (Hard Disk Drives)	18
2.1.5. Magnetic Tapes	19
2.2. Kryder's Law	20
2.3. Scalability	21
2.4. Storage Media at a Glance	22
2.5. Measuring Storage Performance	22
2.5.1. <code>fio</code> — Production-Grade I/O Benchmarking	23
2.6. Exercises	23
3. Reliability and Security	27
3.0.1. Failure Rate Lifecycle: The Bathtub Curve	27
3.1. Reliability Strategies	28
3.1.1. RAID 0	29
3.1.2. RAID 1	29
3.1.3. RAID 1+0	30
3.1.4. RAID 5	30
3.1.5. RAID 6	31
3.2. Cryptography	33
3.3. Authentication and Authorization	34
3.4. Encoding and Decoding of Information	35
3.5. Hash Functions	37

Table of contents

3.6. Encryption vs Hashing: Summary	38
3.7. Exercises	38
4. File System	47
4.1. Characteristics	47
4.2. File Access Modes	48
4.3. Space Allocation	48
4.3.1. Contiguous Allocation	48
4.3.2. Linked Allocation	49
4.3.3. Indexed Allocation	49
4.4. Limits	50
4.5. Common File Systems Compared	51
4.6. Exercises	52
5. Distributed File System	55
5.1. Operation	55
5.2. Example: HDFS	56
5.3. CAP Theorem	58
5.4. Exercises	60
6. Databases	69
6.1. Database Models	69
6.2. Relational Databases	69
6.2.1. Transaction Control in SQL	70
6.2.2. Transaction Isolation Levels	71
6.3. SQL	71
6.3.1. Creating Databases and Tables	72
6.3.2. Keys	72
6.4. Relationships in Relational Databases	74
6.4.1. One-to-One (1:1)	74
6.4.2. One-to-Many (1:N)	75
6.4.3. Many-to-Many (N:M)	75
6.4.4. Constraints	76
6.4.5. Altering Tables	76
6.4.6. Data Insertion	77
6.4.7. Modifying Records	78
6.4.8. Deleting Records	78
6.4.9. Conditional Expressions: CASE	78
6.4.10. Data Selection	79
6.4.11. Other JOIN Types	82
6.5. Subqueries	83
6.5.1. Data Control (DCL)	83
6.6. Exercises	84
6.6.1. Conceptual Questions	84
6.6.2. ACID Questions	87
6.6.3. Schema Design Exercises	88
7. RDBMS - Hands on MySQL	93
7.1. 00 - Introduction	93
7.2. 01 - Create a Connection to MySQL	93
7.3. 02 Database and Table creation	97
7.4. 03 Declaring constraints and altering tables	101

7.5.	04 Inserting Updating and Deleting records	107
7.6.	04 Basic queries	113
7.7.	05 Queries on well-formed DBs	118
7.8.	06 More on queries	122
7.8.1.	Check which tables are available	123
7.8.2.	Describe the content of table <code>Names_</code>	123
7.8.3.	Study the distribution of the year of birth of all people included in the database	123
7.8.4.	Movie stats	123
7.8.5.	Is IMDB only about movies?	124
7.8.6.	Identify interesting outliers	124
7.8.7.	It's all about the ratings	124
7.9.	SQL String Functions Reference	125
7.10.	SQL Date and Time Functions Reference	125
7.11.	SQL Query Exercises	126
8.	NoSQL Databases	133
8.1.	Database Partitioning	133
8.1.1.	Consistent Hashing	134
8.2.	Characteristics of NoSQL Databases	137
8.2.1.	BASE vs ACID	137
8.3.	NoSQL Database Categories	138
8.4.	Examples of NoSQL Databases	138
8.4.1.	Key-Value Databases	138
8.4.2.	Document Databases	140
8.4.3.	Time Series Databases	140
8.4.4.	FullText Search Engine	140
8.5.	Wide-Column Databases	140
8.6.	Graph Databases	141
8.7.	Vector Databases	141
8.8.	Exercises	142
II.	Processing	145
9.	Processing and Concurrency	147
9.1.	Introduction to Data Processing	147
9.2.	CPU Processing	147
9.2.1.	Processing Time	147
9.2.2.	Historical Context: Moore's Law	148
9.2.3.	The Power Wall	148
9.2.4.	Pipelining and Alternative Strategies	149
9.3.	Multi-Processing Architectures	149
9.4.	Parallelism	150
9.5.	Flynn's Taxonomy	150
9.6.	Speedup: Amdahl's Law	151
9.7.	Gustafson's Law	151
9.8.	Processes and Threads	152
9.8.1.	Definitions	152
9.8.2.	Concurrency vs Parallelism	152
9.8.3.	Multi-Tasking	152
9.8.4.	Multi-Threading	153
9.9.	Race Conditions	153

9.10. Thread Synchronisation	153
9.11. The Python GIL	154
9.12. Threading vs Multiprocessing in Python	154
9.13. Hands-on: Concurrency in Python	155
9.13.1. 00 — Single-Threaded Baseline	156
9.13.2. 01 — Two Concurrent Threads	156
9.13.3. 02 — Thread-Local Data	156
9.13.4. 03extra — Minimal Race Condition	156
9.13.5. 03 — Multi-Threaded Without Lock (Race Condition)	156
9.13.6. 04extra — Minimal Race Condition Fixed with Lock	157
9.13.7. 04 — Multi-Threaded With Lock	157
9.13.8. 05 — Thread Pool with Lock (<code>concurrent.futures</code>)	157
9.13.9. 06 — Thread Pool with Queue	157
9.13.10 07 — Multi-Processing	157
9.13.11 Synchronous vs Asynchronous Execution	158
9.13.12 08 — Multiprocessing Pool (<code>starmap</code> , blocking)	158
9.13.13 09 — Multiprocessing Pool (<code>starmap_async</code> , non-blocking)	158
9.14. Benchmarking: I/O-bound vs CPU-bound Tasks	158
9.14.1. I/O-Bound Tasks	158
9.14.2. CPU-Bound Tasks	159
10. Distributed Processing and MapReduce	161
10.1. Scaling Processing	161
10.2. Parallel Computing Architectures	161
10.2.1. Shared Memory Multiprocessor (SMP)	161
10.2.2. Distributed Memory Multiprocessor (Cluster)	162
10.3. Distributed Computing: Clusters	162
10.3.1. Supercomputers as Clusters	163
10.4. Batch Systems	163
10.5. Programming Frameworks for Parallel-Distributed Systems	164
10.6. Big Data and the 3 Vs	164
10.7. Hadoop	165
10.8. HDFS: A Reminder	165
10.9. The MapReduce Paradigm	166
10.9.1. 1. Map	166
10.9.2. 2. Reduce	166
10.9.3. Word Count Example	166
10.10 MapReduce in Hadoop	167
10.10.1 Map Phase	167
10.10.2 Reduce Phase	167
10.11 YARN: Yet Another Resource Negotiator	167
10.12 Hadoop Higher-Level Abstractions	167
10.13 The Hadoop Ecosystem	168
11. Apache Spark	169
11.1. Motivation	169
11.2. Spark Ecosystem	169
11.3. Spark Cluster Architecture	170
11.3.1. Driver and Executors	170
11.3.2. Cluster Resource Managers	170
11.4. RDD: Resilient Distributed Dataset	171

11.5. Transformations and Actions	171
11.5.1. Transformations	171
11.5.2. Actions	171
11.6. Laziness and the DAG Scheduler	172
11.7. Spark DataFrame	172
11.8. Real-Time Analytics and Structured Streaming	172
11.8.1. Spark Structured Streaming	173
11.8.2. Sources and Sinks	173
11.9. Hands-on: Apache Spark	174
11.9.1. Lecture 1: Introduction to PySpark — RDD	174
11.10 Docker Cluster	174
12. Spark Session	177
12.1. Parallelize!	178
12.2. Map and Reduce	180
12.2.1. Exercise 1	182
12.2.2. Exercise 2	182
12.2.3. — <i>Intermezzo</i> — Broadcasting small shared data	182
12.2.4. Exercise 3 - compute π	183
12.2.5. Exercise 4 - Distributed linear regression	187
12.3. Caching	194
13. Stop worker and master	197
13.0.1. Lecture 2: Spark DataFrames	197
13.1. Create and Start a Spark Session	197
13.2. Creating a Spark DataFrame	198
13.2.1. Create a Spark DataFrame from a List of Rows	198
13.2.2. Create a Spark DataFrame from a List of Tuples	199
13.2.3. Create a Spark DataFrame from a Pandas DataFrame	199
13.3. Loading Structured Data	200
13.4. The <code>pyspark.pandas</code> API	202
13.5. DataFrame Analysis Example	203
13.5.1. The Dimuon Mass Spectrum	203
13.6. Stop worker and master	219
13.6.1. Lecture 3: Stream Processing with Spark	219
13.7. Structured Streaming	219
13.8. Create and Start a Spark Session	221
13.8.1. TCP Socket Source	222
13.8.2. Creating the streaming DataFrame from a TCP socket source	222
13.8.3. Start the python producer.py script	223
13.8.4. Running the first streaming application	223
13.9. Example of streaming query: data parsing	223
13.9.1. From static to streaming queries	226
13.9.2. Process each batch to identify possibly fraudulent transactions	227
13.10 Stop spark worker	229
14. Dask	231
14.1. Introduction	231
14.2. Dask Fundamentals	231
14.2.1. Lazy Evaluation and Task Graphs	231
14.2.2. Partitioning	232
14.3. Cluster Setup	233

14.4. Hands-on: Dask	233
14.4.1. Lecture 1: Task Parallelization with Dask	233
14.5. Recap on Task Parallelization (this time using Dask)	233
14.6. Starting a cluster	234
14.7. Simple parallelization of python functions	234
14.8. Futures and Eager computation with <code>submit</code>	237
14.9. Exercise 1: Parallelization of a for loop	240
14.10 Exercise 2: Map Reduce with Dask	242
14.11 Exercise 3: Count how many words are present in a series of documents	244
14.12 Exercise 4: Calculate the first n Fibonacci numbers	246
14.13 Exercise 5: Monte Carlo Integration	247
14.14 Exercise 6: π via MonteCarlo	250
14.14.1. Take-away Message	252
14.15 Stop client	252
14.15.1 Lecture 2: Dask Data Structures	252
14.16 Start the Dask cluster	253
14.17 Dask Bag	253
14.17.1 Create and Take from a Bag	253
14.17.2 MapReduce in Dask: using <code>fold</code>	256
14.17.3 Considerations for Partitioning	256
14.17.4 Broadcasting data to all workers	257
14.18 Exercise 1 - Open and Preprocess JSON Data	258
14.18.1 Additional operations on Dask Bags: Groupby and Foldby	261
14.18.2 Understanding <code>foldby</code> in Dask	262
14.19 Exercise 2 - Account Data	264
14.20 Exercise 3 - Computing Total Transfers Amount per Name	265
14.21 From Bag to pre-processed output datasets	266
14.22 Flattening Deeply Nested Account Data	266
14.23 Stop client	268
14.23.1 Lecture 3: Dask DataFrames and Arrays	268
14.24 Start the Dask cluster	268
14.24.1 (On Data Partitioning)	269
14.25 Basic operations on DataFrames	269
14.26 Computations with Dask DataFrames	273
14.27 Shared, repeated and intermediate computations	276
14.28 Re-run the same exercise from the Spark notebook using Dask	278
14.29 Stop client	280
15. Apache Kafka	281
15.1. Introduction to Message Streaming	281
15.1.1. Core Concepts	281
15.2. Kafka Architecture	282
15.2.1. Brokers and Clusters	282
15.2.2. Consumer Offset Management	282
15.3. Delivery Semantics	282
15.3.1. Pub/Sub Decoupling	283
15.3.2. Partitions and Keys	283
15.4. Hands-on: Apache Kafka	284
15.4.1. Producer	284
15.5. Interacting with Kafka Producer from the Shell	284
15.6. Programmatically produce messages to Kafka with Python	285

15.7. Messages with Key	288
15.8. Create a topic from kafka-python	288
15.9. Publish messages for the Spark Structured Streaming example	290
15.9.1. Consumer	291
15.10 Creating a Consumer Accessing Only One Partition	294
15.11 Creating a Consumer Group	295
15.11.1. Consuming messages as a group	296
15.12 Reading from the Kafka+Spark results topic	296
15.12.1. Consumer Groups	297
15.12.2. Kafka and Spark Structured Streaming	298
15.13 Read data from Kafka	299
15.14 Simple Fraud Detection + Kafka Sink	300
15.15 Stop worker and master	302

Data Management

Foreword

Disclaimer

I am **NOT** the original author of the content in this document. My contribution is limited solely to the translation and re-organization of the original material. All credit, recognition, and compliments for the ideas, structure, and substance of the work should be directed to the original creator **Original Author**.

Original Author is **NOT** responsible for the management of this version or any errors that it might contain.

No copyright

All the source code of this document is private and therefore is not public for any uses. Any feedback and/or questions related to the document should be directed to the original author of this document **Original Author**.

Contributing

If you want to contribute to this version of the notes open an issue at **my repository of the master**.

Below we will discuss the various steps that lead from data acquisition through sensors to the production of real *information*. The process can be divided into two parts:

1. **Data acquisition system**, divided into the following parts:

- **Sensors**, used to convert physical quantities into electrical signals.
- **Frontend electronics**, with the role of interface between the sensors and the signal processing system; the latter is in fact amplified and filtered in order to remove unwanted noise.
- **Readout electronics**, responsible for the digital conversion of the analog signal; it can also perform the function of temporary storage of the signal.
- **Trigger & DAQ**: the first determines when the system can acquire data, for example after exceeding a certain input threshold or after a certain time interval; the second further filters the signal (now digitized) and stores it.

2. **Calculation model**, divided into the following aspects:

- **Storage**, that is the way and the place where the data obtained from the acquisition system are physically stored.

- **Reliability and security**, that is the techniques adopted to preserve the data and the permissions to access them.
- **Querying and filtering**, that is the process of searching and selecting data; *querying* is that process by which it is possible to extract data from memory according to certain criteria.
- **Processing**, which involves the application of algorithms and calculation tools to derive useful information from the stored data.

It should be borne in mind that the one presented is a system for acquiring *physical* data; more generally, the data source can be of a very different type: it is possible to work with *simulated* data (think of the various Monte Carlo techniques), with text documents, with images and so on.

Part I.

Data Manegment

1. From Data to Datasets

By **dataset** we mean a set of *information* (data or simulations) collected from a *series of sources* (detectors and devices in general) in a certain *time frame* associated with a single *domain* or a single *purpose*.

The data, that is the basic unit of information contained in a dataset, is often indicated as ***datum*** or also as ***record***.

Below is illustrated the formation of a dataset:

1. A data acquisition system (**DAQ**) constantly writes to memory every piece of information that is collected by the sensors at the highest possible frequency.
2. The stored data are then subjected to a first processing, on average after a certain number of hours or days, which includes cleaning, correction of any errors, normalization and — if necessary — also dimensionality reduction.
3. Over a longer period of time (on average every year) the data are subjected to further processing involving more sophisticated techniques, such as Monte Carlo simulations.
4. The data that are processed in any period of time are long-term archived in a dedicated memory space.

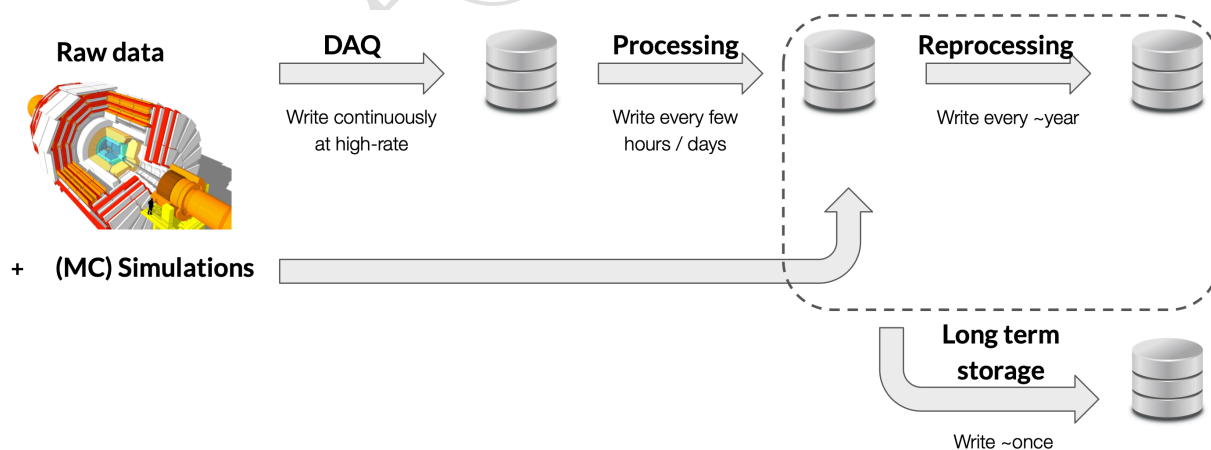


Figure 1.1.: Formation of a dataset

The acquired data sequences are usually divided according to the purpose of the collection: the division can be trivially divided by time frames (for example by year) or by data acquisition campaigns (think of the various *Runs* that are performed at the LHC); another possibility is the division by version of the software used, a choice adopted mainly for simulated data models.

1.0.1. Data Pipeline Stages

The journey from raw detector output to physics results passes through several well-defined pipeline stages. Each stage has a distinctive read/write profile that shapes the storage and computing infrastructure required.

Stage	Write frequency	Read frequency	Description
DAQ	Continuous / high-rate	Rarely read back	Raw detector data written once as sensors fire; write-heavy by design
Processing	Every few hours or days	Moderate	Calibration and reconstruction applied to DAQ output; derived datasets written
Reprocessing	Every ~1 year	High (re-reads raw)	When algorithms improve the original raw data are re-processed; read-heavy on the DAQ tier
Analysis	Infrequent	Very high	Physicists repeatedly read small derived samples; essentially read-only

A key pattern emerges: **write frequency decreases** and **read frequency increases** as data move from DAQ through to analysis. This asymmetry drives many design decisions in physics data management — raw DAQ storage must sustain high ingest rates, while analysis storage must deliver low-latency random reads.

i Write-once, read-many

Raw DAQ data are typically written *once* and subsequently only read. Long-term archives therefore optimise for storage density and read throughput rather than write speed. Tape-based systems, which excel at exactly this profile, remain dominant for DAQ-tier storage at large experiments.

1.0.2. Monte Carlo Simulations as a Data Source

Alongside real detector data, physics experiments produce large volumes of **Monte Carlo (MC) simulated data**. MC simulations generate synthetic datasets by modelling physics processes using random sampling: particles are generated according to theoretical cross-sections, propagated

through a virtual detector model, and digitised to produce output in the same format as real detector readout.

MC datasets are not a shortcut — they pass through *exactly the same processing pipeline* as real data (reconstruction, calibration, tier conversion) so that simulated and measured distributions can be compared on equal footing. This comparison is the basis of most physics analyses: deviations between observed and expected (MC) distributions reveal new phenomena or constrain theoretical parameters.

In practice, MC datasets are often **larger than the corresponding real-data samples**. Because each simulated event is statistically independent, many millions of events are needed to reduce Monte Carlo statistical uncertainty below the level of the real-data uncertainty. A single physics analysis may consume more CPU-hours of simulation than all the data-taking combined.

1.1. Metadata

Datasets are often accompanied by **metadata**, so called because they are data whose function is to give information about the data itself contained in the dataset.

They perform three main functions: identification of the dataset and its contents, description of the acquisition conditions, and indication of access methods and permissions.

To understand the importance of metadata, let's look at two very different files. The first is a generic photo of a cat (*a_nice_cat_photo.jpeg*): in this case the metadata may be superfluous, since any image viewing program will show the image; the metadata could however provide further information, such as who took the photo, in which folder it is located, what is the name of the cat, and so on.

Let's now move on to the other file: *3CBC07C2-4B64-E611-B423-02163E0124E9.root*. Without any metadata it is impossible to trace the meaning of what is contained in the file: it is, in fact, proton-proton collision data at 13 TeV that occurred in the CMS, part of a dataset collected in 2018 belonging to the *Run-D* campaign, 3 GB in size, to which only users of the CMS experiment have access. None of this would have been conceivable without data that explicitly told us what has just been written.

1.1.1. CMS Metadata Example: Path-as-Metadata

A concrete illustration of metadata in practice is the CMS (Compact Muon Solenoid) dataset naming convention used on the WLCG grid. A typical dataset path looks like:

`/CMS/Run2018D/SingleMuon/MINIAOD/UL2018_MiniAODv2-v1/`

Every component of that path encodes structured metadata:

Path component	Meaning
CMS	Experiment
Run2018D	Data-taking period (year + run era)

1. From Data to Datasets

Path component	Meaning
SingleMuon	Physics stream (events passing the single-muon trigger)
MINIAOD	Processing tier (reduced Analysis Object Data format)
UL2018_MiniAODv2-v1	Reprocessing campaign and software version

In addition, each dataset carries per-file and per-dataset metadata records stored separately, for example:

```
Run number:      325170
Luminosity section: 47
Centre-of-mass energy: 13 TeV
Dataset:         SingleMuon
Data tier:        MINIAOD
Number of files:  ~3 000
Total size:       ~12 TB
```

This design is an example of **path-as-metadata**: the file system path *itself* carries structured, human- and machine-readable metadata. It allows data management tools to locate, filter, and catalogue datasets without opening a single file, and it ensures that the provenance of every sample is unambiguous throughout its lifetime.

i Why path-as-metadata matters

When hundreds of petabytes are spread across thousands of storage nodes, scanning file contents to identify datasets is prohibitively expensive. Encoding key metadata in the path allows lightweight string-matching queries to replace full catalogue scans, dramatically reducing the cost of data discovery.

1.2. Data Structures

Datasets can be divided into three different types depending on the way they are sorted:

1.2.1. Structured Data

They are organized in a *rigid* tabular format, so that each element is “boxed” in a unique position determined by a row and a column.

In the following figure we can observe an example of structured data: on the left is shown the *raw* format (the actual bits stored in memory in hexadecimal form), while on the right the same data are in a human-readable text format.

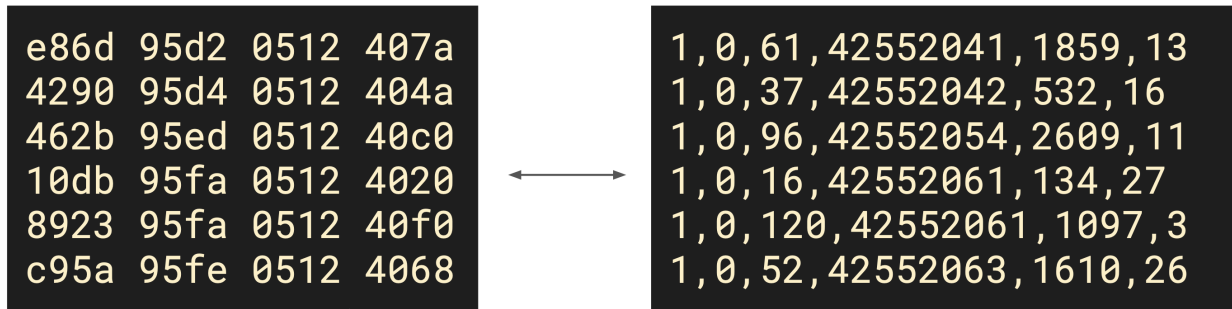


Figure 1.2.: Example of structured data

The tabular form can be displayed using various options, including the standard one offered by the Python *pandas* library.

HEAD	FPGA	TDC_CHANNEL	ORBIT_CNT	BX_COUNTER	TDC_MEAS
1	0	61	42552041	1859	13
1	0	37	42552042	532	16
1	0	96	42552054	2609	11
1	0	16	42552061	134	27
1	0	120	42552061	1097	3
1	0	52	42552063	1610	26

Figure 1.3.: Display via *pandas* of the data shown in the previous figure

Examples of structured data are files in *.csv* (*comma-separated values*) format and *RDBMS* (*Relational DataBase Management System*), also called *relational databases* suitable for the SQL language.

The main advantage of structured data is offered precisely by their structure, being already “packaged” according to a clear and established order. The disadvantage is given instead by the rigidity of this scheme, which limits the possibility of adding new columns and of acting on any errors in positioning.

1.2.2. Semi-Structured Data

They are organized according to structured logics but do not have a fixed scheme a priori, being deducible from the data themselves through *tags* and *markup*.

Examples of semi-structured data are files in *.xml* (*extensible markup language*) format, files in *.json* (*JavaScript Object Notation*) format and files in *.yaml* (*YAML ain't markup language*) format.

Example of semi-structured data in two different formats

To verify the integrity of the data it is always possible to convert semi-structured data into structured data.

1. From Data to Datasets

```
<person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd Street</streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumbers>
    <phoneNumber>
      <type>home</type>
      <number>212 555-1234</number>
    </phoneNumber>
  </phoneNumbers>
  <sex>
    <type>male</type>
  </sex>
</person>
```

(a) Format .xml

```
<person>
  <firstName>John</firstName>
  <lastName>Smith</lastName>
  <age>25</age>
  <address>
    <streetAddress>21 2nd Street</streetAddress>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <phoneNumbers>
    <phoneNumber>
      <type>home</type>
      <number>212 555-1234</number>
    </phoneNumber>
  </phoneNumbers>
  <sex>
    <type>male</type>
  </sex>
</person>
```

(a) Format .json

For example, a JSON document such as:

```
{"name": "Alice", "age": 30, "city": "Bologna"}
```

can be mapped to a table row:

name	age	city
Alice	30	Bologna

Schema validation tools (e.g. JSON Schema) can enforce a structure on semi-structured data before ingestion into a relational system.

First name	Last name	Age	Address	Phone numbers	Sex
John	Smith	25			M

Street address	City	State	Postal code
21 2nd Street	New York	NY	10021

Type	Number
Home	212 555-1234

Figure 1.6.: Semi-structured data converted into structured data

1.2.3. Unstructured Data

They are not schematized through any model nor is there any structure suggested by the data themselves.

These are usually data that define a *continuous flow* of information, such as files of a textual, audio, video and image nature.

The main advantage of this type of format is its maximum flexibility, adapting well to any type of data, while the other side of the coin is the difficult and laborious work to extract single pieces of information from the dataset.

1.3. Summary: Data Structure Types

Data Type	Schema Fixed?	Suitable for RDBMS?	Typical Formats	Main Challenge
Structured	Yes — defined before storing	Yes	CSV, TSV, SQL tables	Rigidity; schema changes are costly
Semi-structured	Optional / flexible	With conversion	JSON, XML, YAML, HDF5	Heterogeneity; variable fields
Unstructured	No	No	Images, audio, text, video	Extracting information requires specialised tools

1.4. What is Data Management?

This chapter introduced the types of data we will work with throughout the course. The subsequent chapters address the full data management lifecycle:

Topic	Chapter
Data storage — media, performance metrics, scalability	Storage and Scalability
Reliability and security — RAID, encryption, authentication	Reliability and Security
File systems — local and distributed	File Systems, Distributed File Systems
Database management systems — relational and NoSQL	Databases, NoSQL Databases

Effective data management means choosing the right storage medium, ensuring data is durable and secure, and providing efficient access through appropriate file systems and database systems.

1.5. Exercises

Exercise 1.1 (*Structured vs Semi-structured Data*)

Explain the difference between structured and semi-structured data. For each, describe the main advantages and disadvantages and provide a concrete example of such data.

Solution

The difference between structured data and semi-structured data is that structured data has well defined structure (like columns in an excel sheet) while semi-structured data does not have a structure but it has a set of rules (like json or yaml files), not all files will have the same disposition of the data but all files can be read in the same way

The main advantage of structured data are that:

- It is easy to read many files as they all have the same structure
- It has clear structure and order

The main disadvantages are:

- It is difficult to add new data (because of the rigid structure)
- It is less human readable

In the case of semi-structured data, the main advantages are:

- They can be used in more complex type of data
- They can be mapped in to a structured format in some cases

The disadvantages are:

- It is harder to keep them consistent as the schema is not fixed
- Each record can carry its own keys, so parsing, querying and validating cost more than for a fixed schema

A typical structured example is a CSV or SQL table; a typical semi-structured example is JSON or YAML.

Exercise 1.2 (*Metadata*)

What data management concept is key to interpreting and contextualising individual records in a dataset? Define the concept in detail and explain why it is essential.

Solution

The key concept is metadata, this can be present in different ways, but the most common is at the beginning of the file, and is there to describe the data. It helps to understand what the data is about, it can give us information about what the data is, how it has been treated, if there is missing information. It helps us understand the origin and the nature of the dataset, otherwise they are just numbers in a selected format.

Exercise 1.3 (*Types of Metadata*)

Name at least two different types of metadata. Describe metadata you have already encountered in your physics experience and explain how you used it to interpret data.

Solution

Metadata can be described in a separate file, in the same file at the beginning, the name of the file, etc. Basically anywhere where we can get information of the data. Two types that I have seen:

- Metadata in the path: When we run an experiment under different conditions I have seen how all the csv files are saved under different folders with different names describing the conditions of the experiments. Imagine an experiment where we vary the Temperature, Pressure and Volume of a gas, then I take measurements of the temperature for different pressure and volume conditions so each file lives in a folder named `P_{ }_bar_V_{ }.ml` where in the brackets is placed the values of pressure and volume of that run.
- Metadata in a separate file: I have seen many times that when someone sends me data they also share an auxiliary file (in markdown or yaml format) describing what each column is, in what units the measurement was taken and how it has been treated. This is important as maybe the name of a column is not self-describing, it can be named `T` and without the file you can not know if it is temperature or period if it's measured in kelvins or celsius.

2. Storage

Each storage unit plays a different role, although the purpose was always to keep the data received as input in memory:

- The DAQ writes to a memory at very high frequencies, performing several write cycles in a short time; however, this data will not remain in that memory unit for long — it will be processed and sent elsewhere, freeing up space for new information from the data acquisition system.
- The processed data are distributed in two different units: in both cases the writing takes place in much wider time frames, and at the same time the amount of memory used grows considerably.
- The same processed data are copied to a further memory, which must therefore be of huge dimensions but from which not many read cycles are expected.

We therefore notice *different needs* regarding the memory units: a decreasing number of read and write operations corresponds to an increasing amount of available space. These different needs translate into *different technologies* suitable to satisfy them.

These technologies are distinguished by several aspects, all related to **I/O** (input/output) requests — read, write, update and delete:

- **Latency:** time taken to perform a single I/O
- **Frequency:** number of I/Os performed per second (**IOPS**, *I/O per second*)
- **Block size:** size in bytes of a single I/O
- **Throughput:** product of *block size* and *frequency*, i.e. the amount of bits transferred in a certain time interval, conventionally measured in MB/s

i Bandwidth vs Throughput

Bandwidth is the *theoretical maximum* capacity of a storage channel — the ideal rate under perfect conditions. **Throughput** is the *actually measured* transfer rate in practice, which is always lower due to overhead, contention, and access patterns. Benchmarking tools measure throughput; vendor datasheets typically quote bandwidth.

Example 2.1 (*and Throughput*)

Given a storage device with the following characteristics: - Block size: 4 kB - Average access latency: 11.1 ms

$$\text{IOPS} = \frac{1}{0.0111 \text{ s}} \approx 90 \text{ ops/s}$$

$$\text{Throughput} = 90 \times 4 \text{ kB} \approx 360 \text{ kB/s}$$

This is typical of a spinning HDD under random-access workloads. For large sequential reads the same HDD may achieve 100–200 MB/s by amortising seek and rotation latency across large blocks.

i Throughput = IOPS × Block Size

The two metrics are linked by a simple identity:

$$\text{Throughput} = \text{IOPS} \times \text{Block size}$$

For example, a device sustaining **1 000 IOPS** with **128 kB blocks** delivers:

$$1,000 \times 128 \text{ kB} = 128 \text{ MB/s throughput}$$

This formula highlights a fundamental trade-off in storage tuning:

- **Maximising throughput** favours large sequential I/Os (big block size, fewer individual operations) — typical of streaming workloads such as video ingest or bulk data analysis.
- **Maximising IOPS** favours many small random I/Os (small block size, high operation count) — typical of database transaction workloads or metadata-heavy file systems.

Optimising for one goal generally comes at the expense of the other: a device configured for peak IOPS (small blocks, high parallelism) will show lower throughput than the same device streaming large sequential blocks, and vice versa.

2.1. Memory Devices

The various devices are listed from the one with the lowest latency to the one with the highest latency.

2.1.1. DRAM (Dynamic Random Access Memory)

It stores data bits in **NAND cells**, composed of a transistor connected to the bus line coupled with a capacitor.

The term *RAM* indicates that it is possible to access data in any position at the same speed; the term *dynamic* specifies that the data do not remain in memory persistently — when the power to the device is interrupted all the data are lost.

Its main flaw lies precisely in this characteristic, which makes it a *temporary storage*; at the same time, its main advantage is the speed in executing I/O operations, with a latency of $\sim 0.1 \mu\text{s}$. DRAMs are used for processes that require a very high writing frequency.

2.1.2. NVM (Non-Volatile Memory) / Persistent Memory

An alternative to DRAM that has a similar latency ($\sim 1 \mu s$) but keeps the data saved even when the device's power is cut off. These qualities require a very high cost: for this reason, devices of this type are not used in standard applications, but in specific projects that require devices with these characteristics.

Unlike DRAM, NVM is **byte-addressable** and **non-volatile** — data survives a power loss just as it does on an SSD, while still being accessible at near-DRAM speeds. A well-known commercial example is **Intel Optane** (based on 3D XPoint technology), which sits between DRAM and NVMe SSD in the storage hierarchy:

Property	DRAM	NVM / Persistent Memory	NVMe SSD
Latency	$\sim 100 \text{ ns}$	$\sim 1 \mu s$	$\sim 100 \mu s$
Persistence	Volatile	Non-volatile	Non-volatile
Byte-addressable	Yes	Yes	No
Cost/GB	Very high	High	Medium–high

Typical use cases include: - **High-speed persistent cache**: absorbing bursty writes before flushing to slower storage, without the risk of data loss on power failure. - **In-memory databases**: replacing DRAM as main memory in latency-sensitive applications (e.g., financial trading, real-time analytics) where durability is also required.

i NVM in the Storage Hierarchy

NVM fills the gap between volatile DRAM and durable SSDs. It is faster than SSD, slower than DRAM, and — unlike DRAM — survives a power loss. This makes it attractive wherever both low latency *and* persistence are required simultaneously.

2.1.3. SSD (Solid State Drives)

Due to their depreciation over time, SSDs have become widespread even for the general public, being included in common personal computers.

The novelty lies in the way the information is stored: no longer in NAND cells but in **multi-level charge traps** — capacitors that distinguish between multiple charge levels. In most devices these levels are eight, thus encoding a total of three bits of information.

2. Storage

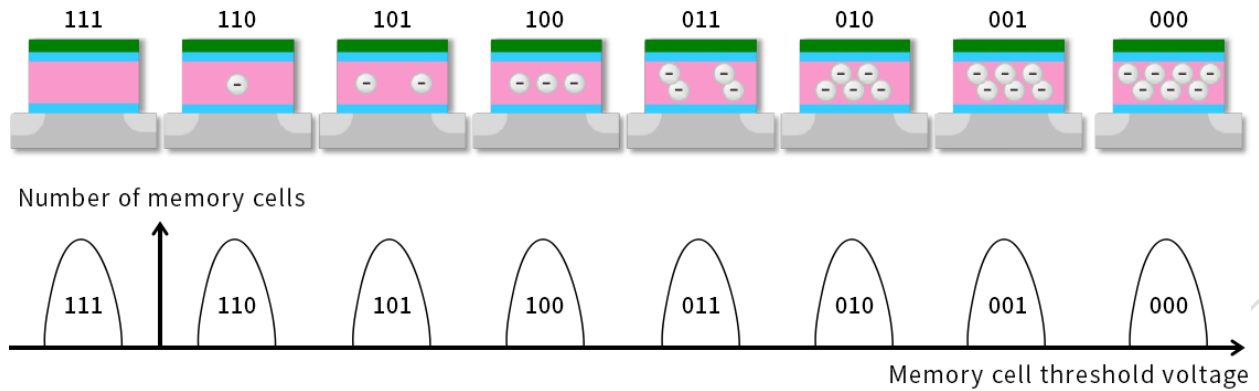


Figure 2.1.: TLC technology: at the top the encoding of the various charge states in three bits; at the bottom the voltage *thresholds* that define the eight states

This technology, called *TLC* (*Triple-level cell*), allows to expand the *storage density*, resulting in much more compact devices with the same memory. On the other hand, it limits the writing capacity: every time it is necessary to write a new piece of information, the entire cell must be erased first. These are therefore devices with a high reading frequency ($\sim 10 \mu s$) but with a considerably lower writing frequency ($\sim 100 \mu s$).

Another limitation lies in the inevitable process of hysteresis that the various cells undergo in the writing cycles: each cycle corresponds to a small residual charge that no longer allows correct operation beyond a certain accumulation. Every SSD has an “expiration date” foreseen from the beginning, the closer the more frequent the use.

2.1.4. HDD (Hard Disk Drives)

They are the most widespread memory devices today, being an excellent compromise in terms of cost relative to their performance.

An HDD is formed by disks magnetized on both sides whose domains are distributed along concentric circles: the individual magnetic domains are called *tracks*, which belong to *sectors* in turn distributed on the side of a disk, called a *platter*. The bits are not associated each with a single track but with two consecutive tracks — bit 0 corresponds to two tracks with the same polarization and bit 1 to two tracks with opposite polarization.

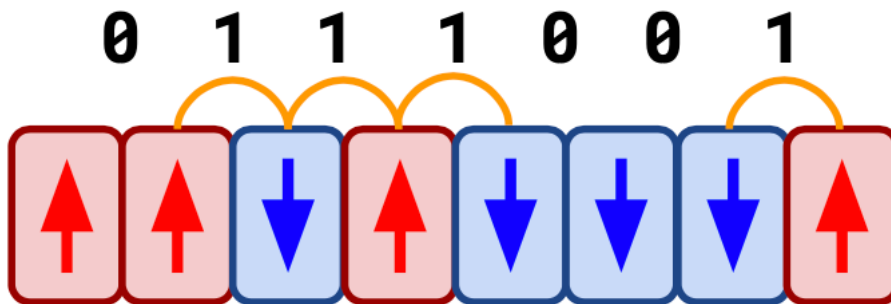


Figure 2.2.: Encoding of bits as a function of the variation of polarization between two contiguous tracks

Data reading does not take place by track but by sector, which usually contain from 0.5 to 4 kB, so even writing to disk is broken down according to this block size.

Reading and writing takes place through mechanical arms, which read the various sectors as a stylus does with a turntable. The limits in reading and writing speed depend on the mechanical nature of the device. The latency ($\sim$ ms) can be divided into two contributions:

- **Seek time:** time taken to align the arm to the track
- **Rotation delay:** time taken by the disk to rotate by aligning itself with the sector

2.1.5. Magnetic Tapes

Magnetic tapes use the same technology as VHS cassettes: magnetic domains arranged one after the other on a tape that can be wound into a compact form. They are designed for data that will not be retrieved except rarely and after a long time. All large scientific projects have a copy of all their data in the form of tapes — CERN currently stores almost 1 EB (10^3 PB) in tape form.

Reading takes place through robotic arms that pick up the requested tape and carry it to the reader. The latency order of magnitude is ~ 100 s, with a wide range due to high randomness.

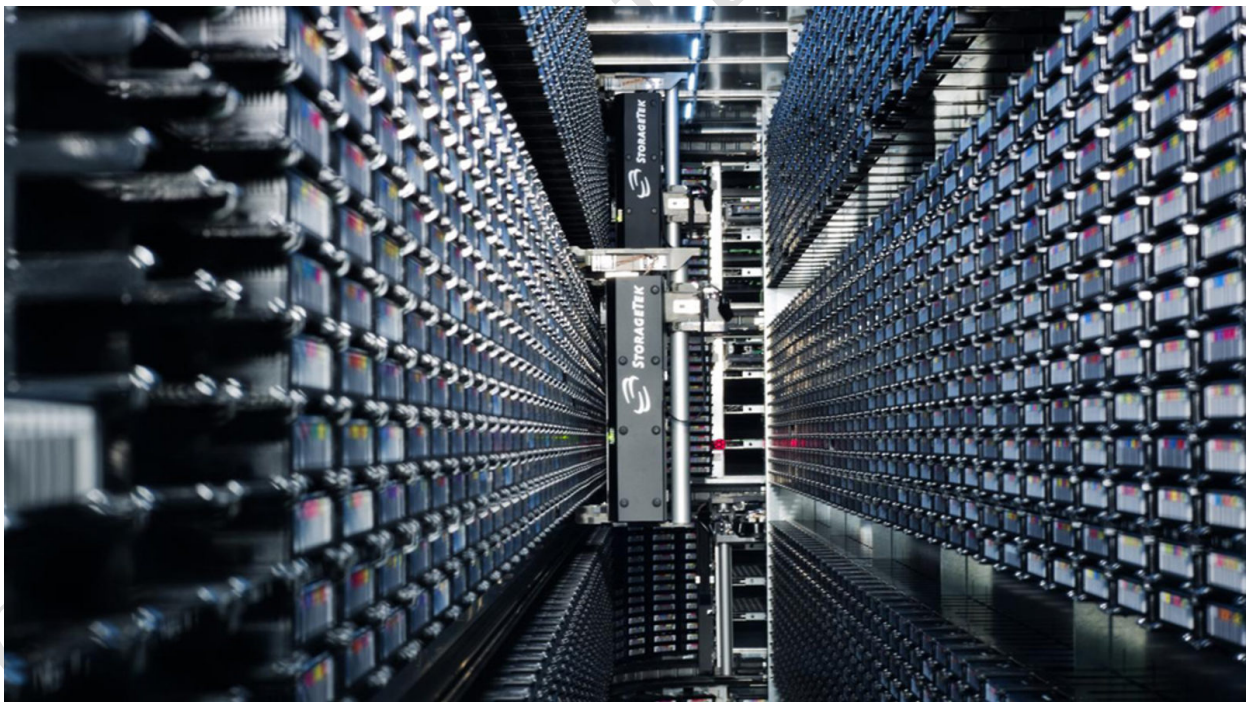


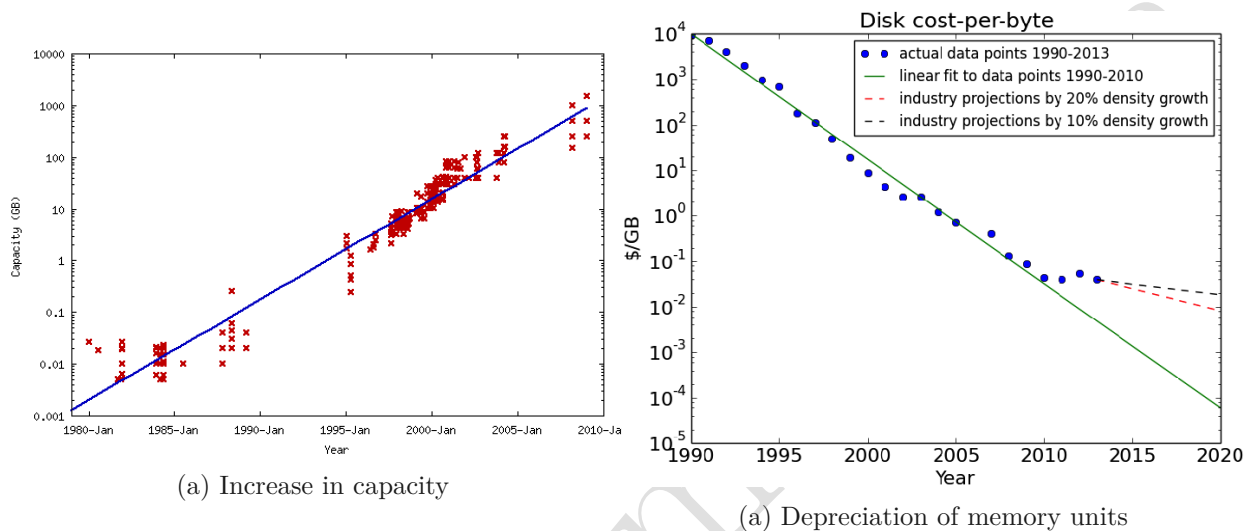
Figure 2.3.: Magnetic tape library at CERN

The reading itself is very fast — the high latency is exclusively attributable to the mechanical retrieval process described above. The main advantages of this format are its low production cost and its robustness: even in extreme cases such as flooding, the data would remain intact.

2. Storage

2.2. Kryder's Law

A factor to be taken into account for the long-term storage of data is the constant development of technologies in this sector. Analogous to Moore's Law for transistors, **Kryder's Law** (Mark Kryder, 2005) states that storage density doubles roughly every 13 months — implying that the cost per GB halves on a similar timescale. A side effect is the constant *depreciation* over time of the same amount of memory.



Kryder's Law: capacity growth and memory depreciation over time

⚠ Kryder's Law Has Significantly Slowed

The original 2005 prediction has **not held for well over a decade**. Areal density improvements in HDDs — the primary driver of the law — stalled around 2010 as magnetic bit cells approached fundamental physical limits. Progress since then has relied on incremental engineering advances (heat-assisted magnetic recording, shingled magnetic recording) rather than the exponential density scaling Kryder envisioned.

Practical consequences:

- The **cost-per-GB gap** between HDD and flash/SSD has narrowed far more slowly than Kryder's Law predicted.
- **Magnetic tape** remains cost-competitive for cold storage at scale precisely because HDD prices have not fallen as quickly as expected — tape continues to offer the lowest cost per GB for write-once archival workloads.
- Long-term capacity planning for large experiments cannot safely assume exponential cost reduction; actual procurement costs should be benchmarked against current market prices.

This law, first proposed in 2005 on an empirical basis, has not been strictly valid for about two decades. Nevertheless, its intrinsic meaning remains: technologies in this field evolve over time, and it is important to take this into account in the context of datasets that grow over the years.

2.3. Scalability

How do we manage a constantly growing dataset? How will performance change as a result of the increase in memory? This problem is called *scalability*, and there are two different approaches:

- **Vertical scalability:** the increase in memory is performed by replacing the current resources with more modern and advanced resources.
- **Horizontal scalability:** the increase in memory is performed by connecting new resources in a chain, forming a *distributed system*.

Vertical	Horizontal
No real change in the way the user accesses the resources	Possible to increase space consistently with a relatively low price
If performance is satisfactory, it can only improve	Scalability is facilitated at the hardware level
Expensive	Relies on the network connection — bandwidth can be limiting
Limit to the amount of memory available for a single device	Laborious to access resources distributed on different local systems
Operations must be interrupted during the update and data transfer	

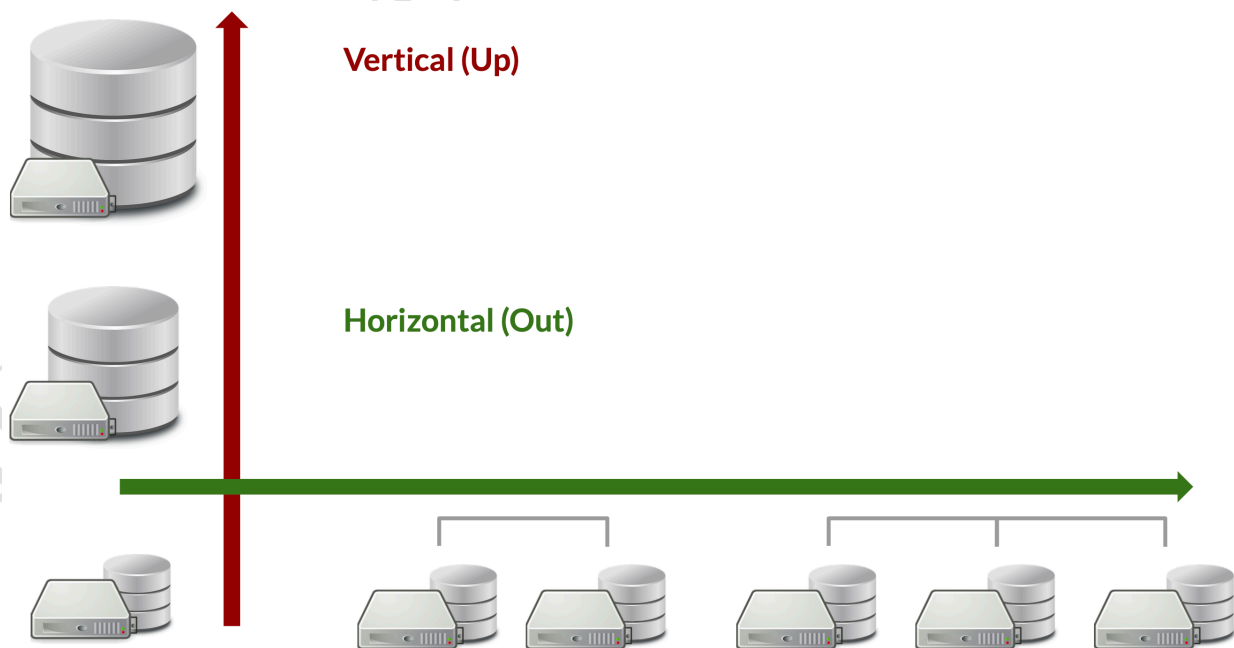


Figure 2.6.: The two possible scalability approaches starting from a given storage

2.4. Storage Media at a Glance

Medium	Typical Latency	Typical Throughput	Capacity	Cost/GB	Persistence
DRAM	~100 ns	~50 GB/s	GBs	Very high	Volatile
NVM / Persistent Memory	~1 μ s	~10–40 GB/s	GBs–TBs	High	Non-volatile
SSD (NVMe)	~100 μ s	~3–7 GB/s	TBs	Medium–high	Non-volatile
SSD (SATA)	~500 μ s	~500 MB/s	TBs	Medium	Non-volatile
HDD	~10 ms	~100–200 MB/s	TBs–PBs	Low	Non-volatile
Magnetic Tape	~100 s	~250–400 MB/s (sequential)	PBs	Very low	Non-volatile

The right storage tier depends on the access pattern: latency-sensitive workloads require DRAM or NVMe; archival workloads with infrequent access are well served by tape.

2.5. Measuring Storage Performance

The `dd` command-line utility is a standard tool for benchmarking raw storage throughput on Linux systems.

Write test (write 1 GB of zeros to a file, bypassing the OS page cache with `oflag=direct`):

```
dd if=/dev/zero of=/tmp/test_write bs=1M count=1024 oflag=direct
```

Read test (read the file back, discarding output):

```
dd if=/tmp/test_write of=/dev/null bs=1M iflag=direct
```

The `dd` output reports the elapsed time and measured throughput.

i Why `/dev/zero` and `/dev/null`?

In the write test, the input file is `/dev/zero` — a special Linux *device file* that produces an unlimited stream of null bytes (all zeros) without reading any real storage. This ensures the benchmark measures **write performance only**, not how fast the source medium can be read.

In the read test, the output file is `/dev/null` — a special device file that silently discards

everything written to it. This ensures the benchmark measures **read performance only**, not how fast the destination medium can be written.

Both devices are effectively instantaneous from the kernel's perspective, so the elapsed time is dominated by the storage under test.

2.5.1. fio — Production-Grade I/O Benchmarking

`dd` is useful for a quick sequential-throughput estimate, but it cannot measure **IOPS**, **random access patterns**, **queue depth effects**, or **latency distributions**. The industry-standard tool for comprehensive storage characterisation is **fio** (Flexible I/O Tester).

Key capabilities of **fio**:

- Configurable block sizes (e.g., 4 kB for IOPS-focused tests, 1 MB for throughput tests)
- Sequential and random read/write patterns (`--rw=read`, `--rw=randread`, `--rw=randrw`, ...)
- Adjustable I/O queue depth (`--iodepth`) to stress the device's internal parallelism
- Multi-job concurrency (`--numjobs`) to simulate real-world workloads
- Detailed output: IOPS, bandwidth, and latency percentiles (p50, p99, p99.9)

Minimal fio example — random 4 kB reads (typical IOPS benchmark):

```
fio --name=randread --ioengine=libaio --rw=randread --bs=4k --numjobs=4 \
    --size=1G --runtime=30 --group_reporting
```

This runs four parallel jobs, each performing random 4 kB reads over a 1 GB working set for 30 seconds, and reports aggregated IOPS, throughput, and latency.

2.6. Exercises

Exercise 2.1 (*Evolving Physics Systems*)

In physics, a large portion of data is structured. What issues might arise in an evolving system (e.g., an experiment adding more sensing devices over time), and what alternative data management solutions could be implemented to address these challenges?

Solution

The main problem with structured data is that the schema is completely fixed, so if you add more sensor that are measuring something that is not already in the schema, you will have to change the schema and update all previous records. Another problem might be the scalability of the system, the new sensors might produce a bigger amount of data than expected. The first problem has not an easy solution, but apart from updating the schema you can change to a semi-structured data format which is more flexible. The second problem can be solved if it was foreseen before making measurements, if the system was designed to scale up (updating the hardware) then the fix is just to update the hardware, if not, it means that it was designed

2. Storage

to scale horizontally, so the solution is to add more nodes to the system and distribute the data among them.

Exercise 2.2 (*Scaling Up Storage*)

When scaling up a storage system, what types of issues are you likely to encounter? Discuss both technical and organisational challenges.

Solution

Depending on the type of scalability, the issues might be different. In vertical scalability, the main problem is that there is a limit to the amount of memory that can be added to a single device, so if the dataset grows too much, it might not be possible to scale up anymore. In horizontal scalability, the main problem is that the network connection can become a bottleneck, especially if the data needs to be accessed frequently and from multiple nodes. Organisational challenges include coordinating updates across multiple nodes, ensuring data consistency, and managing the increased complexity of a distributed system.

Exercise 2.3 (*Throughput vs Bandwidth*)

Explain the concept of storage throughput and highlight how it differs from bandwidth. Why is the measured throughput always lower than the theoretical bandwidth?

Solution

Storage throughput is the amount of data that can be transported (or used in I/O operations) per second. The bandwidth is the theoretical maximum of the throughput, which assumes optimal conditions for the device, and that's why it is always bigger than the throughput.

Exercise 2.4 (*Throughput Calculation*)

A storage device transfers 100 blocks of 4 kB in 9.5 ms. Compute the throughput (in MB/s) and the IOPS. Based on these values, identify the most likely type of storage device being used.

Solution

The throughput is:

$$\frac{100 \times 4 \times 10^{-3} [MB]}{9.5 \times 10^{-3} [s]} = 42.105 [MB/s]$$

and the IOPS are:

$$\frac{100}{9.5 \times 10^{-3} [s]} = 10526.31579 [IO/s^{-1}]$$

In order to determine the device we should compute the latency, which is:

$$IOPS^{-1} = \frac{1}{10526.31579} = 95 \times 10^{-6} [s] = 95 [\mu s]$$

Given that the latency is closer to the 100 μs we could say that the device is an SSD.

Exercise 2.5 (*dd Command Exercise*)

The following Unix `dd` command is used to benchmark a storage element:

```
dd if=/dev/zero of=./test bs=16M count=64 oflag=direct
```

This command performs 64 consecutive writes of 16 MB blocks and measures the total elapsed time. If the overall time is measured at 0.436 seconds, what is the throughput of the storage element? From this throughput value, what kind of storage element is most likely being used?

Solution

The throughput is:

$$\frac{64 \times 16 \text{ [MB]}}{0.436 \text{ [s]}} = 2348.624 \text{ [MB/s]} \approx 2.35 \text{ [GB/s]}$$

and the IOPS are:

$$\frac{64}{0.436 \text{ [s]}} = 146.789 \text{ [IO/s]}.$$

With 16 MB blocks $1/\text{IOPS} = 6.8 \text{ ms}$ is time-per-block, not a device latency suitable for classification. The relevant clue is the **throughput**: a sustained $\sim 2.3 \text{ GB/s}$ on streaming writes is well above SATA SSD ($\sim 500 \text{ MB/s}$) and HDD ($\sim 150 \text{ MB/s}$), so the device is an **NVMe SSD**.

Exercise 2.6 (*HDD Latency from IOPS*)

Under ideal conditions, a given HDD is capable of 85 IOPS. What is the latency of the device? What are the main physical sources of this latency in a mechanical hard drive?

Solution

The latency is:

$$\text{IOPS}^{-1} = \frac{1}{85} = 11.764 \times 10^{-3} \text{ [s]} = 11.764 \text{ [ms]}$$

Which is very high compared with other devices, this high latency it's due to 2 main factors:

- * **Seek time**: Is the time the disk takes to align with the sector where the data lies
- * **Rotation time**: Is the time it takes the disk to rotate into the sector where the data lies

Exercise 2.7 (*Latency and Storage Media Identification*)

Given a storage device with the following characteristics: block size = 512 B, throughput = 3.2 GB/s. Calculate the IOPS and the latency of the device. Based on the obtained latency value, suggest what type of storage media (DRAM, NVMe SSD, SATA SSD, HDD, tape) is suitable.

2. Storage

Solution

With $3.2 \text{ [GB/s]} = 3.2 \times 10^9 \text{ [B/s]}$, the IOPS are throughput divided by block size:

$$\frac{3.2 \times 10^9 \text{ [B/s]}}{512 \text{ [B]}} = 6.25 \times 10^6 \text{ [IOPS]}.$$

Latency:

$$\text{IOPS}^{-1} = \frac{1}{6.25 \times 10^6} = 160 \times 10^{-9} \text{ [s]} = 160 \text{ [ns]}.$$

Sub-microsecond random latency combined with multi-GB/s throughput is in the **DRAM** range — neither SATA SSD ($\sim 100 \mu\text{s}$) nor NVMe SSD ($\sim 10 \mu\text{s}$) can hit it.

Exercise 2.8 (MTTF and Data Centre Reliability)

How does the Mean Time To Failure (MTTF) of storage devices influence the overall reliability of large-scale data centres? What strategies can be adopted to mitigate the risk of data loss due to device failures?

Solution

The Mean Time To Failure (MTTF) measures what is the average time that a device will work before it can loss part or all of the data. It influences the reliability of large-scale data center because they need to ensure that the data loss is reduced to the minimum and the more devices you manage, the more devices will brake and will have to be raplaced. One solution can be implemented by schedueling replacement of the devices before the MTTF, other solution can be to implement a RAID system to ensure that if there is a malfunctioning device the data is not lost, then replace that device, in this approach the device will only be replaced after failure.

Exercise 2.9 (CERN Failure Rate Calculation)

In 2013, the CERN Data Centre hosted approximately 52,000 tape cartridges and 17,000 HDD disks. Assuming a realistic MTTF of 150,000 hours for HDDs and 300,000 hours for tape cartridges, how many combined disks and tapes should be expected to fail every week?

Solution

For the tape cartridges (MTTF = 300,000 [h]):

$$\frac{52,000 \times 24 \text{ [h/day]} \times 7 \text{ [day/week]}}{300,000 \text{ [h]}} \approx 29.1 \text{ tapes per week.}$$

For the HDDs (MTTF = 150,000 [h]):

$$\frac{17,000 \times 24 \text{ [h/day]} \times 7 \text{ [day/week]}}{150,000 \text{ [h]}} \approx 19.0 \text{ disks per week.}$$

So on average ≈ 48 devices fail every week at the CERN Data Centre.

3. Reliability and Security

Physical media contain our data, but what matters is the integrity of the data, not of the memory units themselves. The threats from which we must protect data are of three types:

1. **Data loss:** if a memory device is subject to a failure (electrical or mechanical), usually all the data contained within it are lost.
2. **Data corruption:** certain environmental factors (lowering of the input voltage, increase in ambient temperature, cosmic radiation) can cause a *bit-flip*, that is the inversion of a bit. For HDDs, on average there is a bit-flip every 10^{14} bits involved in read or write cycles. A 10 TB read therefore spans roughly 10^{14} bits — meaning a single full scan of a 10 TB disk could produce one silent, undetected data corruption.
3. **Transfer errors:** occur when there are no adequate checks in the copy or transfer of data.

Definition 3.1 (Reliability). **Reliability** corresponds to the probability that a system will continue to operate correctly beyond a certain period of time or a certain number of operations.

In quantitative terms:

- **MTTF** (*Mean Time To Failure*): the average time that elapses between the first power-on of the device and the moment a failure occurs.
- **MTBF** (*Mean Time Between Failure*): defined for *repairable* systems, the average time that elapses between one failure and the next.

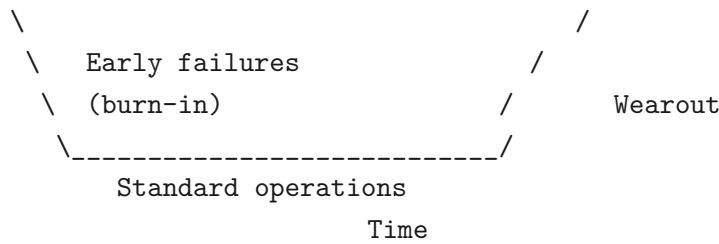


Figure 3.1.: MTTF and MTBF as a function of time

3.0.1. Failure Rate Lifecycle: The Bathtub Curve

The failure rate of hardware components is not constant over time. It follows a characteristic pattern known as the **bathtub curve**:

Failure
Rate



- **Early failure period (burn-in):** newly deployed components have an elevated failure rate due to manufacturing defects; these typically manifest within the first hours or days of operation.
- **Useful life period:** failure rate is low and roughly constant; this is the expected operating lifetime quoted by manufacturers.
- **Wearout period:** as components age, mechanical wear, electromigration, and cell degradation cause the failure rate to rise again.

MTTF and MTBF statistics are measured during the useful life period and do not capture burn-in or wearout behaviour.

Realistically, the MTTF of an HDD is about ten years. Assuming an equally distributed probability of failure:

$$p = \frac{1}{10 \cdot 365} = 0.00027$$

Considering 5000 independent HDDs:

$$5000 \cdot 0.00027 = 1.35$$

So, on average, it is expected that every day an HDD will fail and have to be replaced. At CERN, with $\sim 100\,000$ HDDs in daily use, that translates to about 27 damaged memory units per day.

3.1. Reliability Strategies

To store data securely, three main strategies are used:

1. **Mirroring:** the data are replicated on different devices, sometimes located in different places.
2. **Striping:** the data are divided into “pieces” and distributed on different memory elements.
3. **Checking:** errors relating to the consistency of the data within the devices are identified (and possibly corrected), such as a *checksum* or a *parity check*.

The basis on which to apply these strategies is the so-called *disk redundancy*, indicated by the acronym **RAID** (*Redundant Array of Independent Disks*), in which multiple devices are combined to form a single *logical* device in order to improve reliability and performance.

i Software RAID

RAID can be implemented entirely in software without dedicated hardware. On Linux, the `mdadm` utility manages software RAID arrays:

```
# Create a RAID 5 array across three devices
mdadm --create /dev/md0 --level=5 --raid-devices=3 /dev/sdb /dev/sdc
      ↪ /dev/sdd

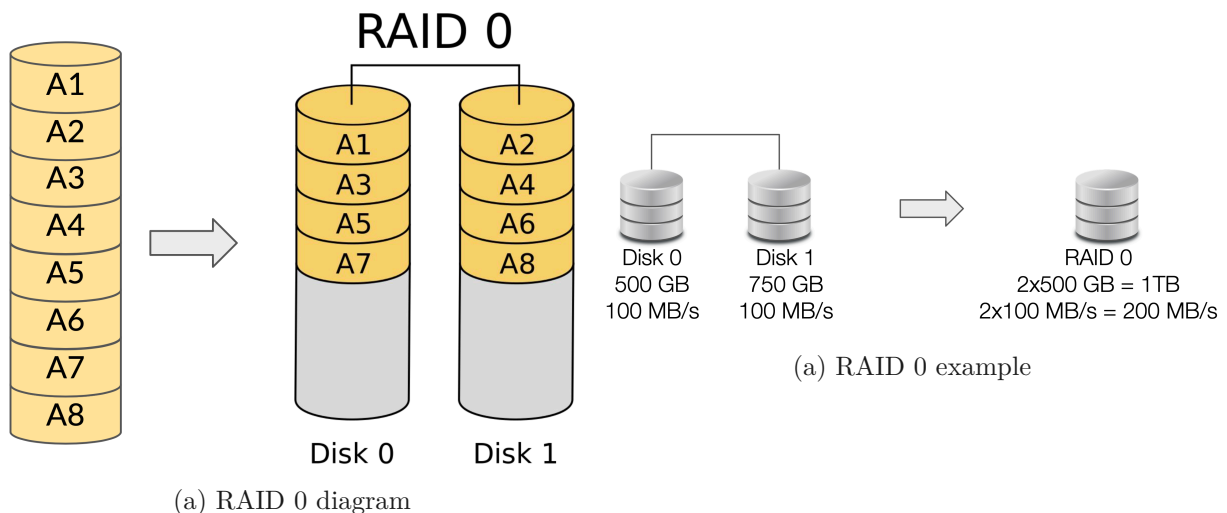
# Check array status
mdadm --detail /dev/md0
```

Software RAID is less expensive than hardware RAID controllers and is widely used in commodity server deployments.

3.1.1. RAID 0

Property	Value
Strategies	Striping (no mirroring, no checking)
Min. disks	2
Total capacity	Lowest single disk capacity × number of disks
Throughput	Conditioned by simultaneous read/write from all disks

Prioritizes performance over reliability — losing a disk implies losing all data.

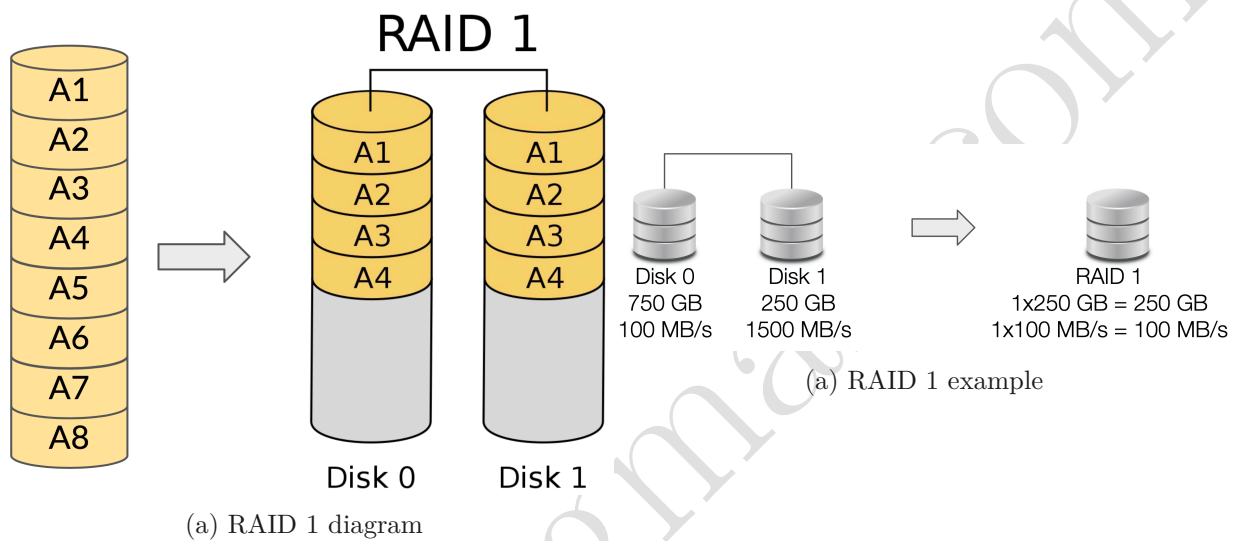


3.1.2. RAID 1

3. Reliability and Security

Property	Value
Strategies	Mirroring (no striping, no checking)
Min. disks	2
Total capacity	Lowest single disk capacity
Throughput	Throughput of the single lowest disk

Prioritizes reliability over performance — data is safe as long as at least one disk is operational.



3.1.3. RAID 1+0

Property	Value
Strategies	Mirroring and striping (no checking)
Min. disks	4
Total capacity	Lowest RAID 1 unit capacity × number of RAID 1 units
Throughput	Read: RAID 0 speed (fast); Write: RAID 1 speed (slow)

Maintains the high performance of RAID 0 while improving reliability with mirroring.

3.1.4. RAID 5

Property	Value
Strategies	Striping and parity checking (no mirroring)
Min. disks	3

Property	Value
Total capacity	Lowest single disk capacity $\times$ (number of disks $- 1$)
Throughput	Non-trivial due to parity read/write overhead

A portion of each disk is “sacrificed” in favor of parity information, so the data can be recovered if a single disk fails. The parity is calculated through a chain of XORs:

$$p = D_1 \oplus D_2 \oplus \dots \oplus D_n$$

Parity recovery by XOR inversion:

XOR is self-inverse: $A \oplus B \oplus B = A$. If disk D2 fails, the missing data is recovered as:

$$D2 = p \oplus D1 \oplus D3$$

Concrete 4-bit example:

Block	Bits
D1	1010
D2	1100 (failed)
D3	0111
$p = D1 \oplus D2 \oplus D3$	0001

Recovery: $p \oplus D1 \oplus D3 = 0001 \oplus 1010 \oplus 0111 = 1100$

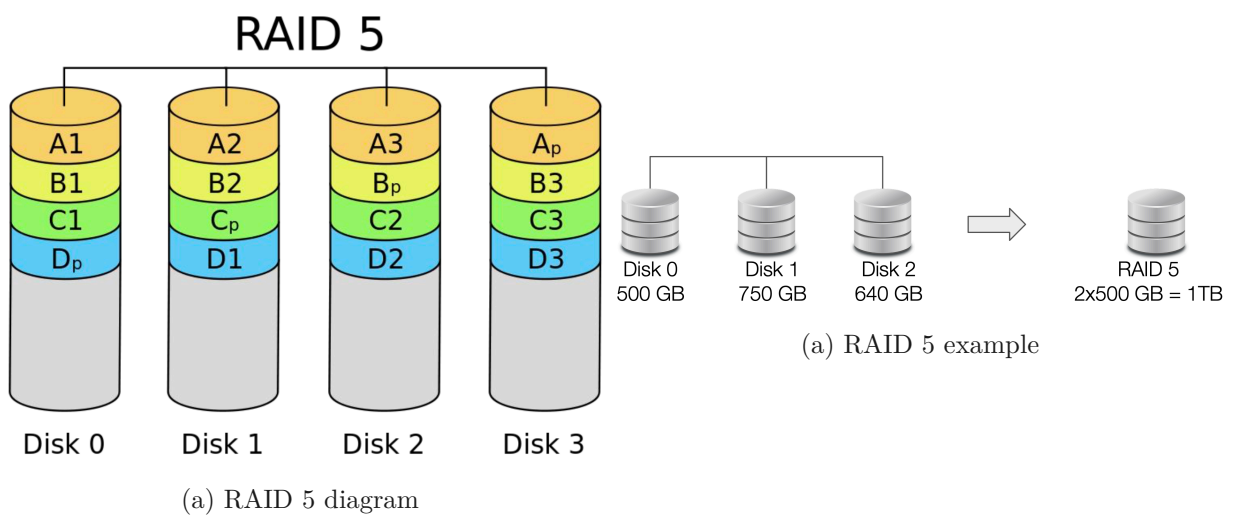
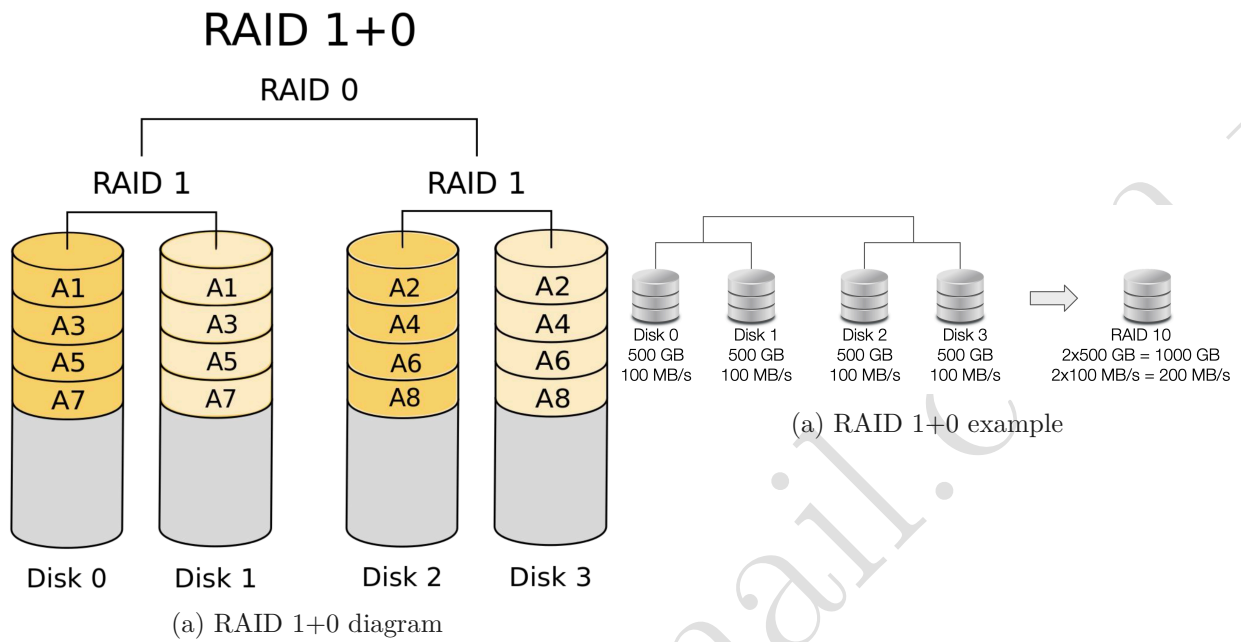
3.1.5. RAID 6

Property	Value
Strategies	Striping and parity checking (no mirroring)
Min. disks	5
Total capacity	Lowest single disk capacity $\times$ (number of disks $- 2$)
Throughput	Non-trivial due to parity read/write overhead

A portion of two disks is “sacrificed” in favor of the parity information, encoded in two bits per section. In order for the data to be recoverable after the failure of two disks, these parity bits must be *linearly independent* — the Reed-Solomon code is the most famous solution:

$$p = D_1 \oplus D_2 \oplus D_3$$

$$q = D_1 \oplus (D_2 \ll 1) \oplus (D_3 \ll 2)$$



where p is the standard XOR parity (identical to RAID 5) and q is a second independent parity computed using Galois-field bit-shifts (). Because p and q are linearly independent, the system has enough information to solve for any **two simultaneously failed disks** — RAID 6 tolerates any two disk losses without data loss.

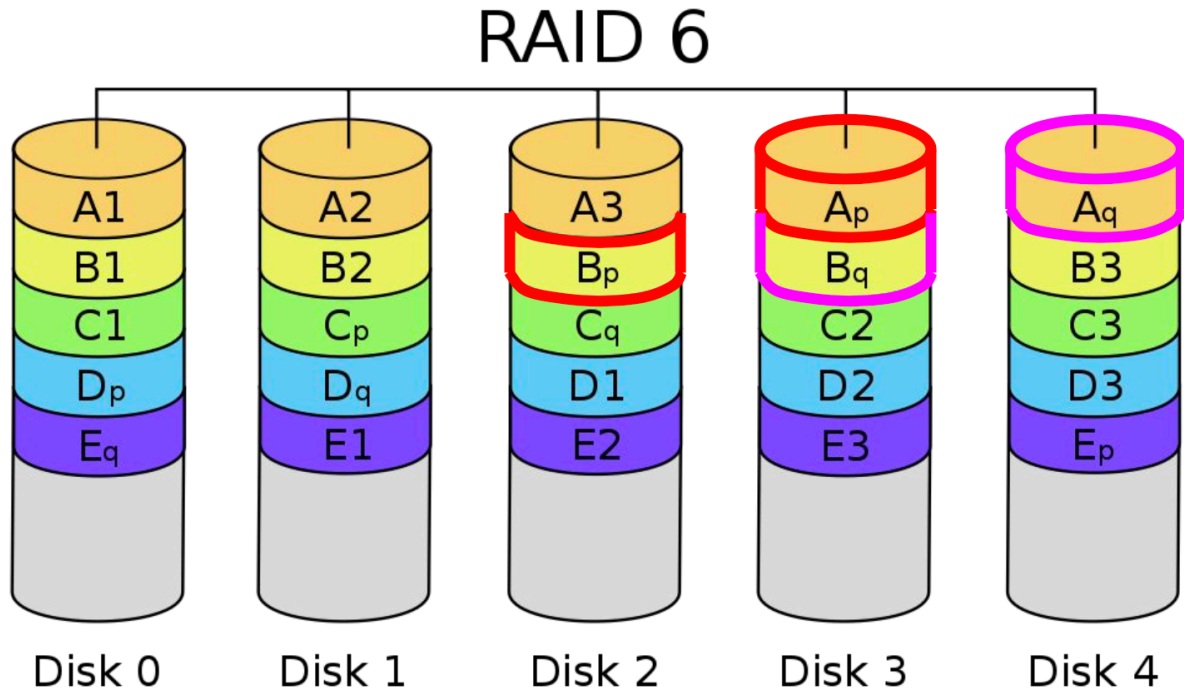


Figure 3.10.: RAID 6 diagram

3.2. Cryptography

Cryptography is a field of mathematics that deals with encoding information in order to make it secure and intelligible only to people authorized to access it.

The main actions involving security are:

- Protect the data saved on a device
- Establish secure connections with remote servers
- Exchange information between two *peers*
- Prevent malicious users from tampering with the data

For a message exchange to be considered *secure*, we must answer the following questions:

- Am I sure I know who I'm talking to?
- Can someone else intercept the data? (*Yes — it is **always** possible.* This is the basis of the security systems to be implemented.)
- Is the message I received identical to the one sent?
- Is the user I am talking to responsible for the message sent?

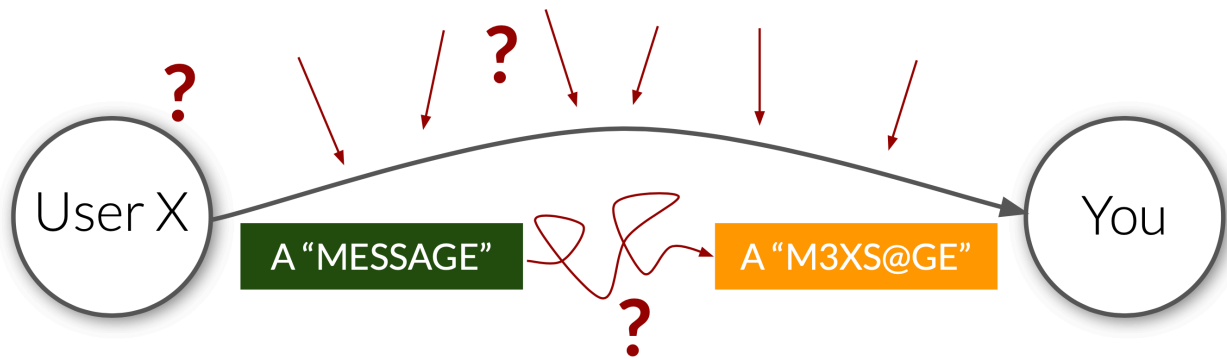


Figure 3.11.: Schematic of the message exchange with all the dangers that threaten its integrity and reliability

The main concepts of cryptography in data management:

1. **Confidentiality:** ensuring that no one can extract information even by intercepting the entire conversation.
2. **Integrity:** ensuring that the message has not been modified during the transfer.
3. **Authenticity:** verifying that you are communicating with the entity you think is at the other end.
4. **Identity:** verifying the identity of the specific individual.
5. **Non-repudiation:** ensuring that the individual responsible for a certain activity cannot deny being associated with it.

The five properties can be summarised as follows:

Property	Description
Confidentiality	Information is accessible only to authorised parties
Integrity	Data has not been altered without authorisation
Authenticity	The identity of the communicating party is verified
Identity	The ability to uniquely identify an entity in the system
Non-repudiation	A party cannot deny having performed an action

3.3. Authentication and Authorization

- **Authentication:** identifying users and validating their identities. Methods include username/password, certificates, keys, SSO, and biometric data (fingerprints, facial recognition, etc.).
- **Authorization:** granting a user permissions to access or use certain resources.

Authentication typically takes place through an *authorization server*, which certifies the user's identity; only after authentication does the system verify the user's permissions through an *Access Control List*, contained in the *resource server*.

3.4. Encoding and Decoding of Information

All messages exchanged through computer tools are constantly exposed. The only way to prevent information from being extracted is to modify the message so that an interceptor cannot understand it.

The most basic idea is to apply a *mask* — called a **key** — to the message, which shuffles and reassembles it. In cryptography these actions are called **encrypting** and **decrypting**.

Symmetric key encryption (*private-key cryptography*) uses a single shared key (applied for example via XOR or AES-128/192/256). It is simple to implement but both parties must share the same key, which is a security risk.

Asymmetric key encryption (*public-key cryptography*) uses two keys: a *public* key for encrypting and a *private* key for decrypting. They are produced together by the same algorithm (RSA, ECC, etc.) so that from one key it is impossible to derive the other. The public key is made openly available; only the intended recipient can decrypt with their private key. This is slower computationally but far more secure, and is today considered the standard.

Standard Cryptographic Algorithm Families

Category	Algorithms
Symmetric	AES-128, AES-192, AES-256
Asymmetric	RSA, ECC (Elliptic Curve Cryptography)
Hashing	MD5 (<i>deprecated</i>), SHA-1 (<i>deprecated</i>), SHA-2 family (SHA-256, SHA-512)

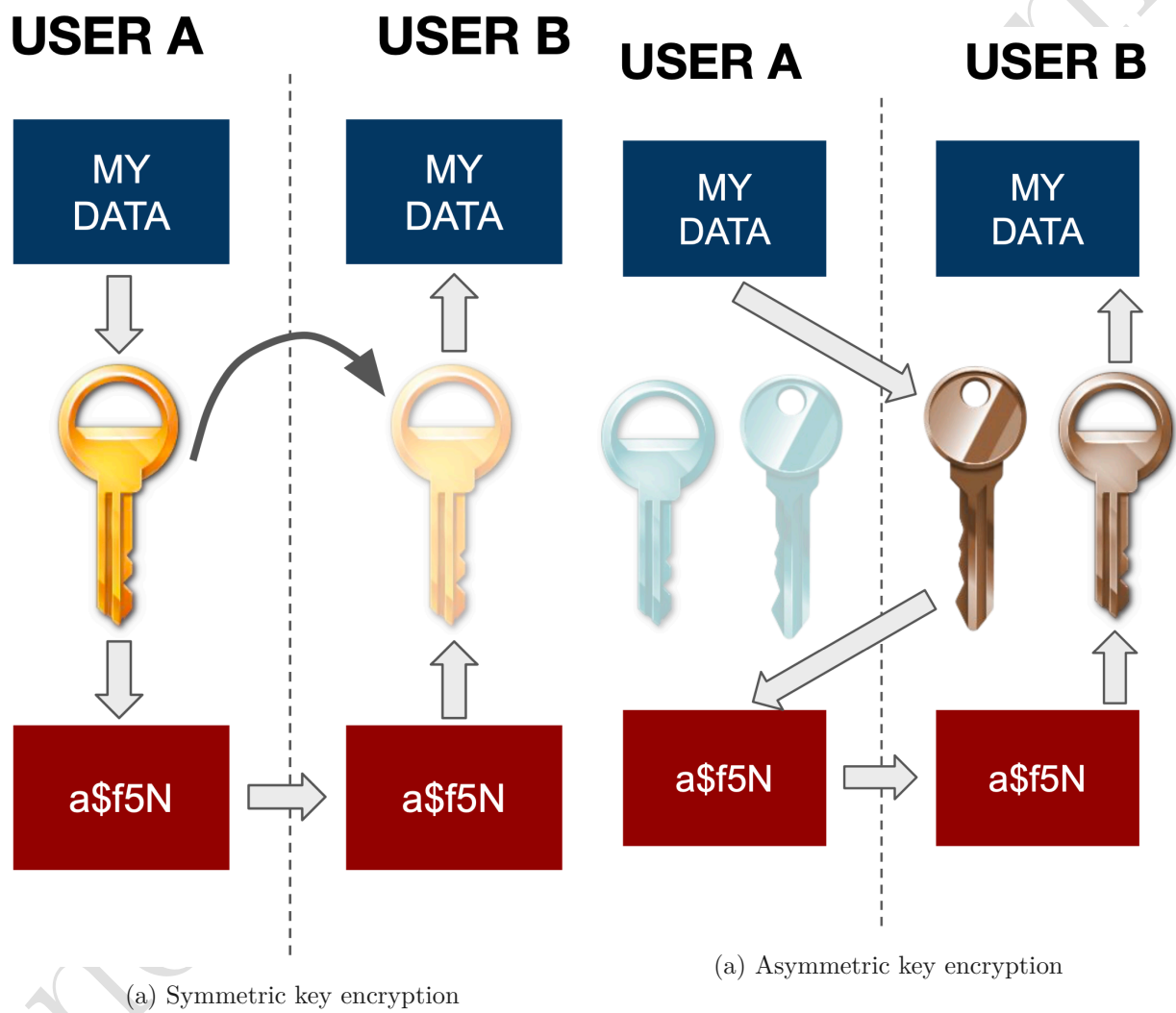
Symmetric algorithms are fast and suitable for bulk data encryption once a secure channel is established. Asymmetric algorithms are slower but allow secure key exchange without a pre-shared secret. Hashing algorithms are one-way and produce a fixed-length digest regardless of input size.

SSH in Practice

Asymmetric key authentication is the standard mechanism for connecting to remote Linux servers. A typical connection to a cloud VM using a private key:

```
ssh -i ~/private/my_private_key.pem ubuntu@10.67.22.231
```

When the server is behind a gateway (jump host), use the `-J` flag:



```
ssh -J gate.cloudveneto.it -i ~/private/my_private_key.pem
↪ ubuntu@10.67.22.231
```

The private key file must have restricted permissions (`chmod 400`). The server stores only the corresponding public key in `~/.ssh/authorized_keys`.

3.5. Hash Functions

Hash functions apply *hashing* to the data — scrambling them to the point that the original form *can no longer be reproduced*. Hashing algorithms (MD5, SHA-1, SHA-2) produce a **digest** of fixed length for all inputs. Modifying the input even slightly will completely change the final digest.

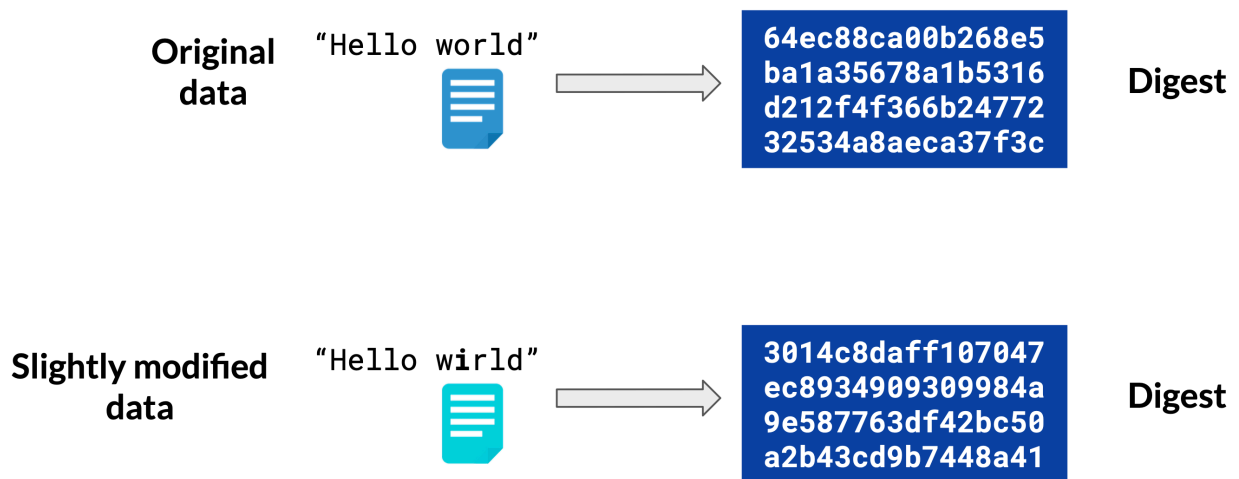


Figure 3.14.: Digest of two very similar data sets

A hash function maps an **infinite input space** to a **finite output space** (for example, 2^{256} possible digests for SHA-256). By the **pigeonhole principle**, collisions must therefore exist in theory: given enough distinct inputs, at least two will always map to the same digest. A cryptographically secure hash function does not eliminate collisions — it makes finding them computationally infeasible, meaning no practical algorithm can produce a collision in a tractable amount of time.

What the algorithms must prevent are **hash collisions** — cases in which two different data sets give rise to the same digest — since the uniqueness of each digest is their primary purpose.

The three main uses of hashing:

- **Data protection:** sensitive data such as passwords are stored as hashes, not in plaintext. Authentication systems compare input hashes with stored hashes.
- **Data integrity check:** comparing the hash of a received file with the expected hash confirms that the file was not tampered with during transfer.
- **Digital signatures:** due to their uniqueness and irreproducibility, hashes are used to certify documents stored in digital systems.

3.6. Encryption vs Hashing: Summary

	Encryption	Hashing
Reversible?	Yes (with key)	No (one-way function)
Purpose	Confidentiality — protect data in transit/at rest	Integrity — verify data has not changed
Output size	Same order as input	Fixed size (e.g. 256 bits for SHA-256)
Key required?	Yes (symmetric) or key pair (asymmetric)	No
Use when	Storing sensitive data, securing communications	Storing passwords, verifying file integrity, digital signatures
Examples	AES-256, RSA, ECC	SHA-256, SHA-3, bcrypt

3.7. Exercises

Exercise 3.1 (*Mirroring, Striping, Parity*)

Define mirroring, striping, and parity check. Discuss the pros and cons of each technique, and compare them in terms of reliability in RAID configurations.

Solution

Mirroring: Duplicates the data in to different devices. Pros: In case of disk failure the data is still available. Cons: You duplicate the data using mor storage
Striping: Divides the data into smaller portions and the portions are distributed in different devices. Pros: It increases throughput (it can be parallelized). Cons: If there is disk failure the data is lost
Parity checks: It assigns a parity to the data and checks the data with XOR operations. Pros: It makes sure the data is not corrupted and it can reverse data loss in some cases. Cons: It reduces throughput due to performing checkings.

In terms of reliability the best methods are mirroring and parity checking since are the only ones where disk failure does not necessarilly mean data loss. Stripping is only prefered when throughput needs to be high.

Exercise 3.2 (*RAID 0 vs RAID 1*)

What is the performance impact of using RAID 0 vs RAID 1? What factors should be considered when choosing between them?

Solution

The two main factors are the reliability and throughput. RAID 0 is prefered when multiple operations needs to be done at the same time and fast, as it uses striping. RAID 1 is prefered when reliability is main factor to guarantee, as it uses mirroring. A third factor would be

storage capacity, as it is a limiting factor, specially in a RAID 1 configuration since you need twice or more storage capacity than data.

Exercise 3.3 (*RAID 0 Throughput and Risks*)

How does RAID 0 increase data throughput, and what are the risks associated with this configuration?

Solution

The RAID 0 throughput is increased via striping (dividing the data into multiple disks), since this make reading and writting faster via parallelization of the task. The main risk affecting the RAID 0 configuration is data loss, as there is no back up copies of the data when a disk fails all data is lost.

Exercise 3.4 (*RAID 5 Capacity*)

You have five disks with sizes x_1, x_2, x_3, x_4, x_5 . If you arrange them in a RAID 5 system, what will be the usable storage capacity of the system? Express the answer in terms of the individual disk sizes.

Solution

The usable storage capacity will be $4 \times$ the lowest storage capacity as one disk is kept for allocating the parity of the data.

Exercise 3.5 (*Parity Placement*)

Can you store the parity of a data block on the same disk as the data block itself? Explain your answer in the context of RAID reliability.

Solution

The answer is NO. If we stored the data in the same disk as the data it would be impossible to recover the data as we would have lost the data and the parity of the data. If the parity is stored in a different disk is possible to compute the parity of the data and compared with the stored parity in order to recover it, but if we lose both parts we can not computed or know which parity had. That is why is important in terms of reliability to store them in different disks

Exercise 3.6 (*RAID 6 Configuration*)

You have a RAID 6 configuration with 5 disks. What is the usable storage capacity of the filesystem (expressed as a fraction of total raw capacity)? How many disks can fail simultaneously without data loss?

Solution

In a RAID 6 configuration we can lose up to 2 disks. The total capacity (assuming all disks have equal capacity) would be $3/5$ of the raw capacity as we use 2 disks for parity checking. If the disks have different capacity the usable storage would be smaller than $3/5$.

Exercise 3.7 (RAID 10 vs RAID 6)

Compare RAID 10 and RAID 6 in terms of pros and cons regarding storage efficiency and reliability (fault tolerance).

Solution

- **Storage efficiency.** RAID 6 wins: $(n-2)/n$ usable vs $1/2$ for RAID 10. With $n=6$: $4/6$ vs $3/6$.
- **Write performance.** RAID 10 wins: mirror + stripe with no parity computation. RAID 6 pays double-parity overhead on every write.
- **Fault tolerance.** RAID 6 tolerates *any* 2 simultaneous failures. RAID 10 also tolerates 2 in the worst case (and up to one per mirror in the best), but loses data if both halves of any single mirror fail together.

Rule of thumb: RAID 6 for capacity-bound archives, RAID 10 for write-heavy databases.

Exercise 3.8 (RAID Configuration Exercise (1))

You are tasked with choosing a RAID configuration to store a 3.5 TB dataset. You have 6 disks available with the following capacities:

Disk	Capacity (TB)
Disk-0	1.0
Disk-1	1.0
Disk-2	1.2
Disk-3	1.5
Disk-4	1.5
Disk-5	1.8

The dataset needs to be stored with high reliability and fault tolerance. Which RAID configuration would you choose, and why? Consider both the storage requirements and fault tolerance.

Solution

The RAID 6 configuration is the best choice as it has a storage capacity of $(6-2) \times 1.0 \text{ [TB]} = 4.0 \text{ [TB]}$ (RAID 5 would give us more storage, but is less reliable) where we could lose 2 disks before data loss. RAID 0 would give us more storage (6.0 [TB]) but no reliability and RAID 1 or RAID 10 do not meet the storage requirements.

Exercise 3.9 (RAID Configuration Exercise (2))

Arrange the following 6 disks in a RAID 10 configuration:

Disk	Capacity (GB)	Throughput (MB/s)
0	750	120
1	640	200
2	500	250
3	640	150
4	1000	120
5	720	160

- How can the 6 disks be arranged into mirrored pairs?
- What is the estimated overall usable storage capacity?
- How many disks can fail without incurring data loss in this configuration?

Solution

Pair similar capacities so the mirror loss is small: $(1000 + 750)$, $(720 + 640)$, $(640 + 500)$, giving mirror sizes 750, 640, 500 GB.

Striping across the three pairs is limited by the smallest pair-min, so the usable capacity is $3 \times 500 \text{ [GB]} = 1.5 \text{ [TB]}$.

Up to three disks can fail (one per pair); if both halves of any single mirror fail, data is lost.

Exercise 3.10 (Encryption vs Hashing)

What is the difference between encryption and hashing? Why are both techniques important and useful for ensuring data security? Give an example use case for each.

Solution

Encryption is used in secure communication as data can be coded and decoded if the users have the keys or methods to do it. Hashing instead can not be decoded and is used to check and store data.

We use encryption we want to have a controlled access to data, like when we send a message in an app to a person, we want only that person to be able to read it and replying. Instead we would use hashing when we want to check the data (the same data will produce the same digest, so if we know the digest and want to check the data we can compare it) or to store data (like when we enter a password into a web service the platform will check the digest, so they don't know or have your password but they can authenticate you by the digest)

Exercise 3.11 (Symmetric vs Asymmetric Encryption)

What are the differences between symmetric and asymmetric encryption? In what scenarios is each type preferable?

Solution

In symmetric encryption only one key is generated and is meant to be shared only with the entity we want to communicate, in asymmetric encryption 2 keys are generated and one is meant to be public for anyone and the other one is meant to be completely private, in that way entities can communicate with you but only you can read what they send to you. Asymmetric encryption is always preferable, since you don't need to share anything that is meant to be private. Symmetric key encryption can be used in private communications and asymmetric encryption can be used when you want to share data with other people but have control about who can read it.

Exercise 3.12 (*Asymmetric Encryption Vulnerabilities*)

How can data be compromised in an asymmetric key encryption scheme? Describe at least two attack vectors or points of failure.

Solution

- **Man-in-the-middle.** Without a trusted PKI / certificate authority, an attacker can substitute their own public key for the recipient's and intercept all subsequent traffic.
- **Private-key compromise.** If the private key is stolen, copied, or generated with a weak RNG, every past and future message encrypted to it becomes decryptable. RSA is also vulnerable to factorisation attacks if the key length is too small (below ~2048 bits today).

Exercise 3.13 (*Data Confidentiality in Network Communications*)

What common methods are used for protecting data confidentiality in network communications? Name and briefly describe at least three techniques.

Solution

- **TLS / HTTPS.** An asymmetric handshake establishes a session key, then the payload is symmetrically encrypted (AES). Used in essentially all modern web traffic.
- **VPN / IPsec.** Encrypts the whole IP packet between two endpoints, hiding source/destination as well as the payload.
- **End-to-end encryption** (PGP, Signal, age). Application-level encryption so the transport provider cannot read content; only the recipient's private key decrypts.

Exercise 3.14 (*Authentication vs Authorisation*)

What is the difference between authentication and authorisation? Provide a concrete example illustrating the distinction.

Solution

In authentication you want to verify the person who is logging in and in authorisation is where you manage what permission (what the authenticated user can do) they have. An example could be a university website, where students and professors are authenticated in order to access, but they have different authorisations as the students can not alter grades.

Exercise 3.15 (Data Integrity)

What is integrity in data security, and what strategies can be used to ensure it? Describe at least two mechanisms used in practice.

Solution

Integrity is the guarantee that data has not been altered — accidentally or maliciously — since it was written. Two practical mechanisms:

- **Cryptographic hashes / checksums** (SHA-256, CRC32). The producer publishes the digest; the consumer recomputes it on receipt and compares.
- **Digital signatures**. The producer signs the hash with their private key; anyone with the matching public key verifies *integrity* and *authenticity* in one step.

Exercise 3.16 (Confidentiality in Cryptography)

Describe how confidentiality can be ensured through cryptographic techniques. What properties must a secure encryption algorithm have?

Solution

Confidentiality is achieved by encrypting data so only holders of the right key can recover the plaintext. A secure encryption algorithm must be:

- **Hard to invert without the key** — no efficient decryption beyond brute force.
- **Resistant to chosen-plaintext attacks** — many (*plaintext*, *ciphertext*) pairs must not leak the key.
- **Semantically secure** — ciphertexts of two chosen plaintexts must be indistinguishable to an attacker.
- **Randomised** via fresh IVs / nonces so identical plaintexts yield different ciphertexts.

Examples: AES-256-GCM (symmetric), RSA-OAEP or ECIES (asymmetric).

Exercise 3.17 (Cloud Confidentiality for Patient Data)

How could confidentiality be ensured in a Cloud system that contains sensitive data about patients? What technical and organisational measures would you implement?

Solution

Technical.

- **Encryption at rest** (AES-256) and **in transit** (TLS).
- **Client-side encryption** so the provider stores only ciphertext; keys held in a separate KMS or on-premise HSM.
- **Pseudonymisation** of identifying fields (name, ID, date of birth) before upload.

Organisational.

- **Role-based access control** with audit logs of every read.
- **Vetted provider** under a compliant data-processing agreement (HIPAA, GDPR).
- **Geographic restrictions** so the data never leaves the legal jurisdiction.

Exercise 3.18 (*Managing a Sensitive Physics Dataset*)

You are tasked with defining the strategy to manage and share with others (e.g., physicists) a large dataset of potentially sensitive data, such as records of hospital patients. What techniques and strategies would you consider effective? Consider data access control, anonymisation, encryption, and audit trails.

Solution

- **Access control.** Authenticated users only, with RBAC: clinicians see identifiable records, researchers see pseudonymised ones.
- **Anonymisation / pseudonymisation.** Strip direct identifiers before release and replace with stable tokens so re-linking requires a separate key held by the data custodian.
- **Encryption.** AES-256 at rest, TLS in transit, plus client-side encryption for highly sensitive fields.
- **Audit trails.** Immutable logs of every access (who, when, what record) for compliance review.
- **Data-sharing agreements** with collaborators specifying allowed analyses and prohibiting re-identification.

Exercise 3.19 (*Hash Collisions*)

What is a hash collision? Why do we want to avoid it, and is it totally avoidable? How do modern hash functions (e.g. SHA-256) minimise the probability of collisions?

Solution

A **collision** is two distinct inputs $x \neq y$ with the same digest $h(x) = h(y)$. We want to avoid them because an attacker who finds a collision can substitute one input for another — e.g. a malicious file for a benign one — without changing the published hash.

Not totally avoidable: any fixed-length hash maps an infinite input space into 2^n outputs, so collisions exist by pigeonhole. Modern hashes like SHA-256 make them *computationally*

infeasible: the birthday-bound cost of finding one is $2^{n/2} = 2^{128}$, well beyond any feasible compute.

Exercise 3.20 (Parity Checks and Checksums)

What is the role of parity checks and checksums in data management? In what contexts are they used, and what are their limitations?

Solution

Both detect accidental data corruption.

- **Parity bit.** A single bit added so the count of 1s in a word is even (or odd). Detects any odd number of bit-flips but cannot locate or correct them, and is useless against bursts. Used in RAM ECC, RAID 5/6, serial links.
- **Checksum** (CRC, Adler-32, MD5). A short fixed-size digest of a longer block. Detects most random errors and many bursts. Used in TCP/IP packets, `md5sum` file checks, ZFS / ext4 block validation.

Limitations. Neither resists *intentional* tampering — an attacker who modifies the data can recompute the checksum. For that you need a cryptographic hash or a MAC.

Exercise 3.21 (Brute-Force Cracking Exercises)

Answer the following two sub-questions:

(a) Imagine your password is hashed via the SHA-256 algorithm and it contains 10 characters drawn from a standard alphanumeric character set (letters a–z, A–Z, digits 0–9: 62 characters total). A hacker can try 1,000 different hash combinations per hour. How much time does it take for the hacker to crack your password in the worst case?

(b) If you use SHA-256 hashing on a text and an algorithm can guess at a rate of 2×10^{18} guesses per second, how long will the algorithm take in the worst case to find the original input? (Recall that SHA-256 produces a 256-bit digest, giving 2^{256} possible hash values.)

Solution

(a) Search space = $62^{10} \approx 8.4 \times 10^{17}$. At 10^3 hashes/hour,

$$\frac{62^{10}}{10^3 \text{ h}^{-1}} \approx 8.4 \times 10^{14} \text{ h} \approx 9.6 \times 10^{10} \text{ years.}$$

About a hundred billion years — astronomically safe at that hash rate.

(b) With $2^{256} \approx 1.16 \times 10^{77}$ possible digests and 2×10^{18} guesses/s,

$$\frac{2^{256}}{2 \times 10^{18} \text{ s}^{-1}} \approx 5.8 \times 10^{58} \text{ s} \approx 1.8 \times 10^{51} \text{ years.}$$

Brute force on the digest space is infeasible regardless of how short the input was; this is why pre-image resistance of cryptographic hashes is the load-bearing property in password

3. Reliability and Security

hashing.

4. File System

There are three main solutions for storing and working with datasets:

1. **File:** the best known and most used. *Files* are sets of data blocks managed by a system called a *File System*, which distributed over multiple memory units (horizontal scalability) will act as a *Distributed File System*.
2. **DataBase:** each single element is accessible at a *granular* level and modifiable through systems called *DataBase Management System*; if distributed over multiple memory units, Databases are called *Distributed Databases*.
3. **ObjectStore:** the data are saved in any arbitrary form (files, blocks of specific length, entire operating systems) inside containers called *buckets*; they are mainly used in *cloud storage*.

This chapter deals with file systems; the next chapter covers databases.

4.1. Characteristics

The role of a file system is to manage and keep track of the position of data saved in memory in the form of files through a **hierarchical** structure, making use of folders (also called *directories*) and subfolders. Its main characteristics can be summarized in three aspects:

1. **Data saving:** the data are physically stored in the form of *blocks* of fixed length, distributed within the memory devices. The file system both divides and allocates the files in memory, and keeps track of all the positions of the blocks.
2. **Abstraction:** the file system identifies the data through the *name* of the file, the *path* organized according to a hierarchical structure, and the *metadata* attributed to the file.
3. **Low-level operations:** the file system interfaces with the operating system to allow *low-level* operations on the storage blocks, such as opening, closing, reading, writing, and searching. This interface follows precise standards, the most widespread of which is **POSIX** (*Portable Operating System Interface for Unix*).

The **POSIX (Portable Operating System Interface)** standard defines the minimum set of file operations that a conforming OS must provide:

Operation	Description
OPEN	Open or create a file; returns a file descriptor
CLOSE	Release the file descriptor
READ	Read bytes from the current position
WRITE	Write bytes at the current position

Operation	Description
SEEK	Move the current position within the file

POSIX compliance ensures that programs written for one Unix-like OS (Linux, macOS, BSD) will work on another without modification.

4.2. File Access Modes

File systems support three fundamental access patterns:

Access Mode	Description	Use Case
Sequential	Records are read/written in order from beginning to end	Log files, streaming data, tape archives
Direct (Random)	Any record can be accessed directly by its position/address	Databases, indexed data
Indexed	An index structure maps keys to physical locations; lookups use the index	B-tree indexes in databases, directory listings

Most operating systems implement direct access at the OS level; sequential and indexed access are typically layered on top by applications or database engines.

4.3. Space Allocation

The main distinction between the various file systems concerns the way in which space is allocated to save files in memory. Below are three types of allocation, proceeding by scale of efficiency.

4.3.1. Contiguous Allocation

The various blocks that make up the individual files are physically arranged next to each other. Access to the files is of a **sequential** type.

The limit of this type of allocation is evident: when you want to expand the content of a file, beyond a certain point you will encounter an already occupied block from another file that cannot be overwritten. For this reason, a file system based on this allocation is suitable for magnetic tapes, where files are saved to be kept long-term without ever being modified.

File	Start	Length
count	0	2
f	6	2

File	Start	Length
tr	14	3
mail	19	6
list	28	4

Example directory table for contiguous allocation with five files

4.3.2. Linked Allocation

Through linked allocation it is possible to “jump” from one block to another to store a single file. This is done by inserting in each block the information on where the next data block is located, so a small part of the memory is “sacrificed” to allow the file system to move and retrieve information.

Compared to contiguous allocation, a better optimization of the available space is possible, not being bound to a series of contiguous blocks to compose the file.

File	Start	End
jeep	9	25

Example directory table for linked allocation

- **Advantage:** no external fragmentation; files can grow dynamically without needing contiguous free space.
- **Disadvantage:** random access is $O(n)$ — to reach block k you must follow k pointers sequentially. Additionally, a single corrupt pointer loses the rest of the file.

i FAT: Linked Allocation with an In-Memory Table

FAT (File Allocation Table) is a variant of linked allocation where the linked list of block pointers is stored in a separate table kept in memory rather than within the data blocks themselves. This avoids the per-block pointer overhead and allows faster traversal, but the FAT table must fit in memory and becomes a single point of failure.

4.3.3. Indexed Allocation

Linked allocation optimizes space on the disk but not the time in operations involving the files: every time an operation is requested, the system must start from the initial block and “bounce” between all subsequent blocks until it reaches the desired destination.

Indexed allocation optimizes time through the use of direct and indirect blocks, with the latter having the function of indicating the positions of the data.

The first block is constituted as follows:

- The initial part contains the *metadata* relating to the file.

4. File System

- Then follow the *direct blocks* — addresses pointing directly to the data.
- Optionally, *single indirect blocks* point to blocks that in turn point to the data; *double indirect blocks* extend this one level further, and so on.

An inode's data block pointers are organized in tiers:

1. **Direct blocks** (e.g., 12 pointers) — each pointer points directly to a data block.
2. **Single indirect block** — one pointer to a block that contains N pointers, each pointing to a data block.
3. **Double indirect block** — one pointer to a block of pointers, each of which points to a block of pointers to data blocks.
4. **Triple indirect block** — one further level of indirection beyond double indirect.

This tiered structure keeps the inode small while supporting very large files via indirection: small files use only direct blocks (fast access), while large files extend through indirect blocks as needed.

Example (Berkeley Fast File System): 12 direct blocks + 1 indirect block, each 4 kB; memory addresses occupy 4 bytes.

The 12 direct blocks contain:

$$12 \times 4 \text{ kB} = 48 \text{ kB}$$

The indirect block contains $\frac{4 \text{ kB}}{4 \text{ B}} = 1000$ addresses, pointing to $1000 \times 4 \text{ kB} = 4 \text{ MB}$ of data.
Total:

$$48 \text{ kB} + 4 \text{ MB} \approx 4 \text{ MB}$$

Adding a *double* indirect block would yield $1000 \times 1000 \times 4 \text{ kB} = 4 \text{ GB}$ more. Increasing the layer of indirect blocks exponentially increases the memory dedicated to a file.

4.4. Limits

A file system is mainly limited by the space it can dedicate to a single file, and by the total space it can manage for a single memory unit (due to counters with a limited number of bits).

Another limitation is that the file system cannot give information on the content within a single file — it is up to the user to exploit the hierarchical tree organization to order the data conveniently. This does not allow operations such as filtering based on the content of the files.

Pros	Cons
Can handle a large amount of data	Scaling a growing amount of data according to the hierarchical view is complex
Low-level operations via POSIX are very fast	No structure underlying that of the files is accessible

Pros	Cons
Offers a hierarchical view of all data	It is not possible to group, select and filter data based on file content
Single entities (files) can be easily transferred, copied, shared and processed	

4.5. Common File Systems Compared

File System	OS	Max File Size	Max Volume Size	Max Filename	Permissions	Journal
FAT32	Windows/cross-platform	4 GB	8 TB	255 UCS-2 characters	No	No
exFAT	Windows/cross-platform	128 PB	128 PB	255 UTF-16 characters	No	No
NTFS	Windows	16 TB	256 TB	255 characters	Yes	Yes
HFS+	macOS	8 EB	8 EB	255 UTF-16 characters	Yes	Yes
ext3	Linux	2 TB	32 TB	255 bytes	Yes	Yes
ext4	Linux	16 TB	1 EB	255 bytes	Yes	Yes
XFS	Linux	8 EB	8 EB	255 bytes	Yes	Yes

ext4 is the default file system on most modern Linux distributions and is suitable for the majority of workloads. **FAT32/exFAT** are used for cross-platform compatibility (USB drives, SD cards).

File System Paths in Practice

A file path represents the full location of a file within the hierarchy. In the context of physics data, paths can encode experiment, run, and file type information:

```
/eos/cms/store/data/Run2012D/SingleMu/RAW-RECO/22Jan2013-v1/20000/
00259D38-8A6F-E211-8E23-003048F118C4.root
```

Each component — experiment (**cms**), data type (**data**), run period (**Run2012D**), stream (**SingleMu**), format (**RAW-RECO**) — is a directory level, making the hierarchy self-documenting.

4.6. Exercises

Exercise 4.1 (*Indirect Blocks and Maximum File Size*)

How does increasing the number of indirect blocks impact the maximum file size in an indexed file system? Explain the relationship between indirect block levels and file size limits.

Solution

Indirect blocks optimize the space used to store data as we store the pointer and not the data itself (which has a lower size in memory), so the more pointers we have the bigger the file we can store. This comes with a trade-off as the number of pointers increase the time to read the data also increases. If we use double indirected blocks, which is an indirect block that points to another indirect block that then points to the data, it increases exponentially as it would n^2

Exercise 4.2 (*File System Size Calculation*)

You have a simple indexed file system with the following parameters:

- Block size: 512 bytes
- Pointer size: 4 bytes
- 3 direct blocks
- 2 singly-indirect blocks
- 1 doubly-indirect block

What is the maximum size of a file that can be stored in this file system? Show all calculation steps.

Solution

First let's compute the number of pointers per block, that is:

$$\frac{512 [B]}{4 [B]} = 128 \text{ pointers}$$

The maximum size for a file would be:

$$3 \times 512 [B] + 2 \times 128 \times 512 [B] + 128 \times 128 \times 512 [B] = 8,521,216 [B] \approx 8.5 [MB]$$

As we account for the direct blocks, the 2 singly-indirect blocks and the doubly-indirect block. The space used to store that file would be:

$$3 \times 512 [B] + 2 \times 128 \times 4 [B] + 128 \times 128 \times 4 [B] = 68,096 [B] = 68.096 [kB]$$

Exercise 4.3 (*Types of File Access*)

What are the types of memory access in a local file system? Describe each type and give an example use case where it is most appropriate.

Solution

The first type is the sequential access, where the data is stored in order so the data is stored in order and can be accessed by simply moving forward in the memory. Like a movie tape where we can see each fotogram of the movie displayed in order. You just need to unroll the tape to see everything but you can not go directly to the end as you would have to see everything before. The second type is direct access, in this case you only store the information of where is the data and can be accessed from anywhere in the system. The last one is indexed access, here the information is stored in an index table (or key-value table) which contains pointers to the data. You can see it as a menu, where the file system (you) select what file you want (the dish) and then that is retrieved for you to use (eat).

5. Distributed File System

To apply a file system to a horizontally scaled system we must move from *local file systems* to **distributed file systems**, indicated with **DFS** (*Distributed File Systems*). In technical terms, a *cluster* is a set of computing resources — computers called *nodes* of the cluster. The user typically connects from outside the cluster through a *client* program. The individual nodes are connected to each other through a very fast *communication network*, which the external user also accesses directly, meaning the user does not have an *atomic* view of the individual nodes in the cluster.

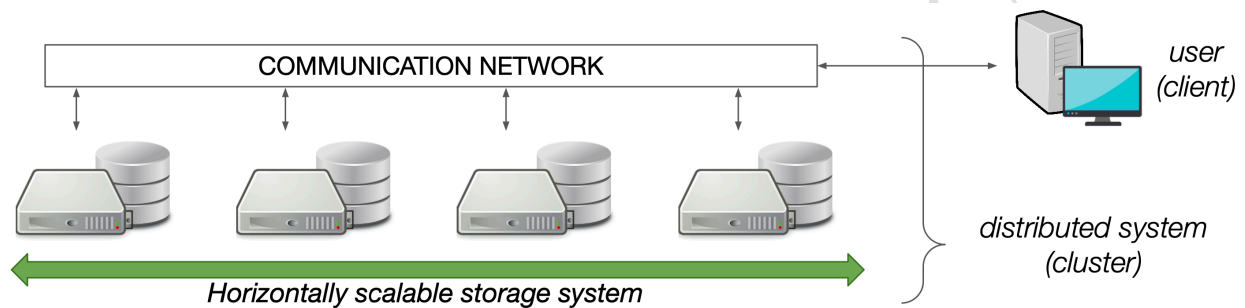


Figure 5.1.: Diagram of a horizontally scaled storage system

5.1. Operation

Some well-known DFS implementations:

- **NFS** (*Network File System*): built for UNIX operating systems, the most widespread at the user level.
- **AFS** (*Andrew File System*): suitable for academic purposes in research environments.
- **Lustre**: specifically designed for the HPC (*High Performance Computing*) field to maximize throughput and manage huge amounts of data.
- **EOS** (*Elastic Object Storage*): developed at CERN to mainly manage large amounts of unstructured data.

The table below summarises the major DFS implementations and their release years:

DFS	Year	Notes
NFS (Network File System)	1984	Sun Microsystems; still widely used
AFS (Andrew File System)	1986	Carnegie Mellon; predecessor to OpenAFS

DFS	Year	Notes
Lustre	2003	High-performance computing clusters
GFS (Google File System)	2003	Inspired HDFS design
HDFS	2006	Apache Hadoop; based on GFS paper
EOS	2010	CERN; used for LHC data

All DFS share the same basic operation: data (together with metadata) are divided into files, in turn broken into memory blocks of fixed size distributed in various positions but always traceable through indexes and pointers.

The main differences with respect to local file systems are three:

1. The block sizes are divided on different machines, so for the position not only the memory address must be specified but also the machine itself.
2. Since operations must act on different machines to work on the same file, the latency inevitably increases, adding the communication network latency to the inherent mechanical latency.
3. Due to the previous point, block sizes are larger than those typical of local file systems — on the order of megabytes — in order to maximize the amount of data extracted from each position.

Three *ideal* requirements for a DFS:

1. **Transparency:** the user should not feel any difference working with a DFS compared to a local file system; the dispersion of data on multiple computers must be as little evident as possible.
2. **Fault tolerance:** the ability to keep all operations active despite the failure of some infrastructures.
3. **Scalability:** extensibility to new nodes in the cluster in a simple and practical way.

5.2. Example: HDFS

The **HDFS** (*Hadoop Distributed File System*) is an open-source DFS created in 2006 to emulate through reverse engineering the *GFS* (*Google File System*) introduced by Google three years earlier. It was created to manage and process large amounts of data on *commodity hardware* — relatively cheap and easily available devices — using standard TCP/IP protocols.

In HDFS there is a special node called the **namenode**, which contains the metadata of all the files and has the role of accepting or rejecting requests from users. All other nodes, called **datanodes**, host file blocks of 128 MB each.

i Block size: local vs distributed file systems

Local file systems (e.g. ext4, NTFS) typically use a block size of **4 KB**. HDFS uses a dramatically larger default block size of **128 MB** (increased from 64 MB in Hadoop 2.x; the value is configurable). The large block size serves two purposes:

- **NameNode memory:** the NameNode must keep an entry in memory for every block across the entire cluster. Fewer, larger blocks means far fewer metadata entries to track, reducing NameNode RAM pressure.
- **Seek overhead:** for the large sequential reads that are typical in data-analysis workloads, spending time seeking to the start of a 128 MB block is amortised over much more useful data transfer than it would be for a 4 KB block.

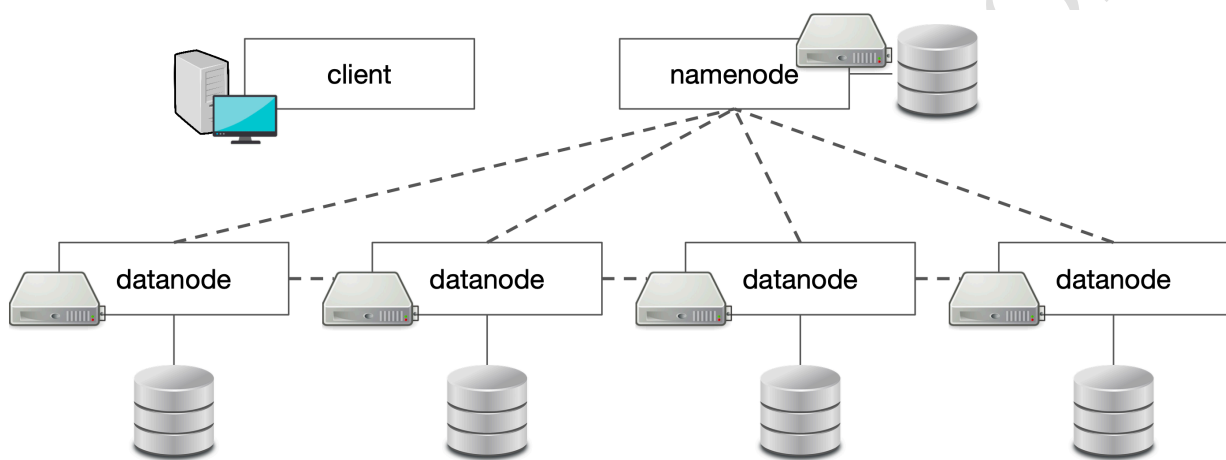


Figure 5.2.: HDFS system diagram

Each operation is divided into four steps:

1. The client sends its request (read, write, delete, etc.) directly to the namenode.
2. The namenode verifies authorizations and permissions, then provides the client with the positions of the relevant data blocks.
3. The client acts directly on the datanodes containing those blocks, without going through the namenode.
4. The datanodes signal to the client that the operation is complete.

Fault tolerance in HDFS involves three types of failures:

- **Network instability:** each datanode repeatedly sends a *heartbeat* signal to the namenode. When this heartbeat is missing, the namenode excludes that node until the heartbeat is restored.
- **Datanode failure:** data blocks are copied to different nodes (by default, three copies) so that if a datanode fails, the data is not lost.
- **Namenode failure:** this is the worst possible failure — a *Single Point Of Failure* (SPOF). The data itself is not lost, but the information on how to compose files from blocks is lost. To mitigate this, HDFS introduces a **Secondary NameNode**.

i Secondary NameNode — what it is and what it is not

The Secondary NameNode is **not** a hot standby. If the Primary NameNode crashes, HDFS does **not** automatically recover — manual intervention is required.

Its actual job is *checkpointing*: it periodically fetches the Primary NameNode's **fsimage** (a snapshot of the entire namespace) together with the **edit log** (a journal of every change made since the last snapshot), merges them into a new, up-to-date fsimage, and sends the result back to the Primary NameNode.

Without this merging the edit log would grow without bound, and restarting the Primary NameNode after a crash would require replaying an arbitrarily long sequence of edits — making startup extremely slow. Periodic checkpointing keeps the edit log short and the startup time bounded.

5.3. CAP Theorem

For distributed systems, a theorem mathematically demonstrated by Eric Brewer — known as the **CAP theorem** — states the following:

Theorem 5.1. *It is **impossible** for a distributed system to provide simultaneously more than **two** of the three CAP guarantees (consistency, availability and partition tolerance).*

The three guarantees:

1. **Consistency (C)**: *every* request from *every* client to *any* datanode must result in the *same* copy of the saved information. To apply this, a “waiting line” is introduced: after a client modifies a file, all other copies are updated before any further operation is allowed.

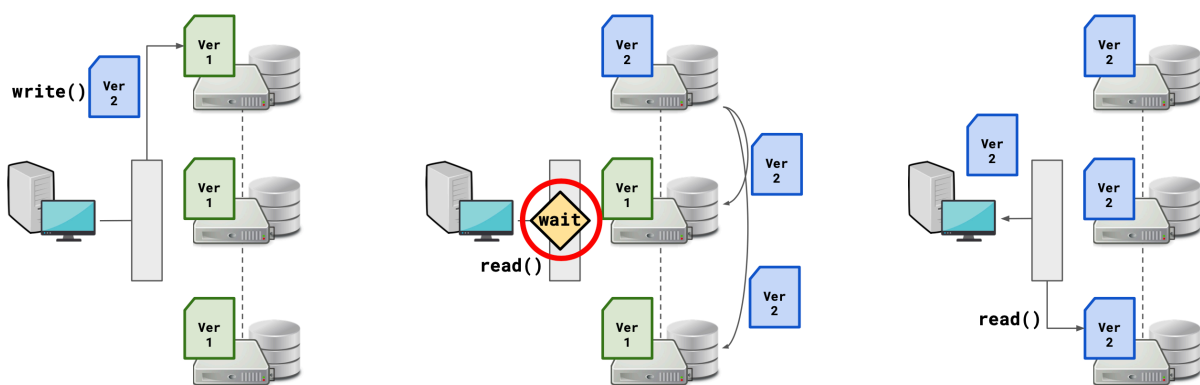


Figure 5.3.: Example of consistency in case of writing to a file

2. **Availability (A)**: *every* request from *every* client must correspond to an *immediate* response, *regardless* of the state of the data and the non-functioning of some network nodes. This means minimizing latency and supporting multiple simultaneous clients. However, this creates a *race competition* — a client may read an outdated version right after writing.

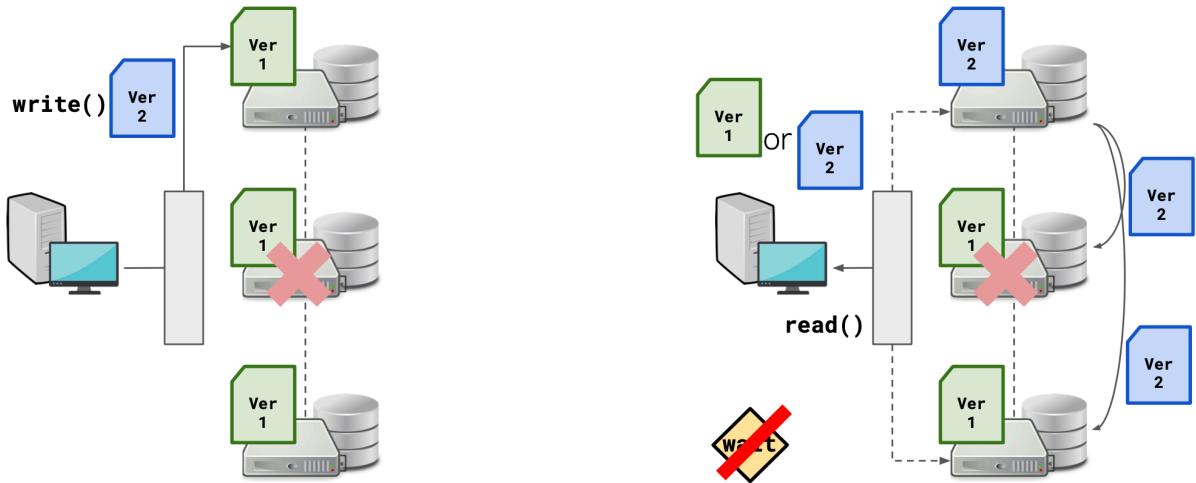


Figure 5.4.: Example of availability in case of writing and subsequent reading

3. **Partition tolerance (P):** the system *must* continue to operate *regardless* of cuts to the communication lines between nodes (*network partitions*).

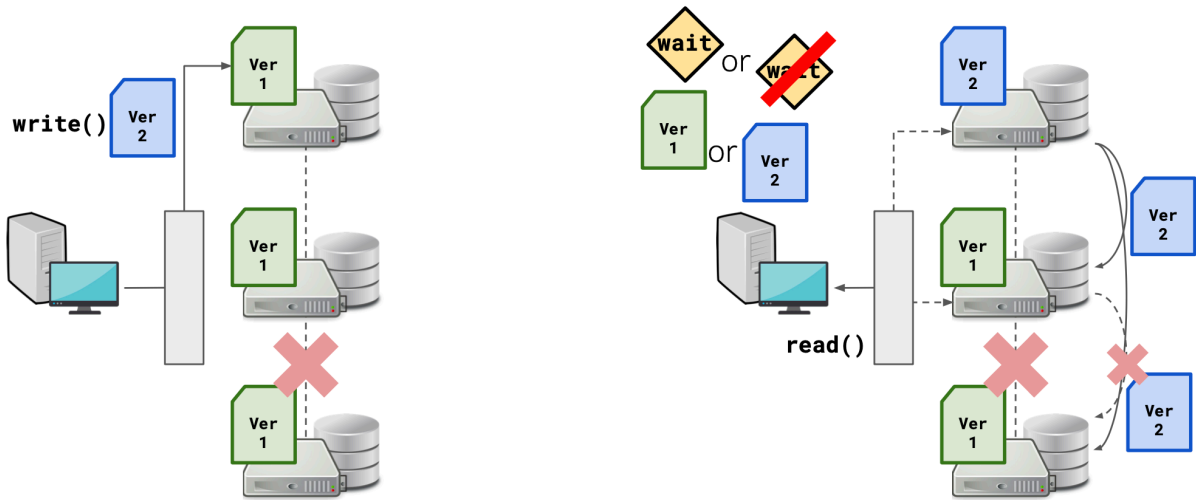


Figure 5.5.: Example of partition tolerance in case of writing to a file

In a distributed system it is not possible to avoid partition tolerance, so the practical choice reduces to:

- **CP system:** tendentially slower in response but guarantees consistency in reading by multiple clients.
- **AP system:** fast in reading but no guarantee that the available data is updated to the latest version.

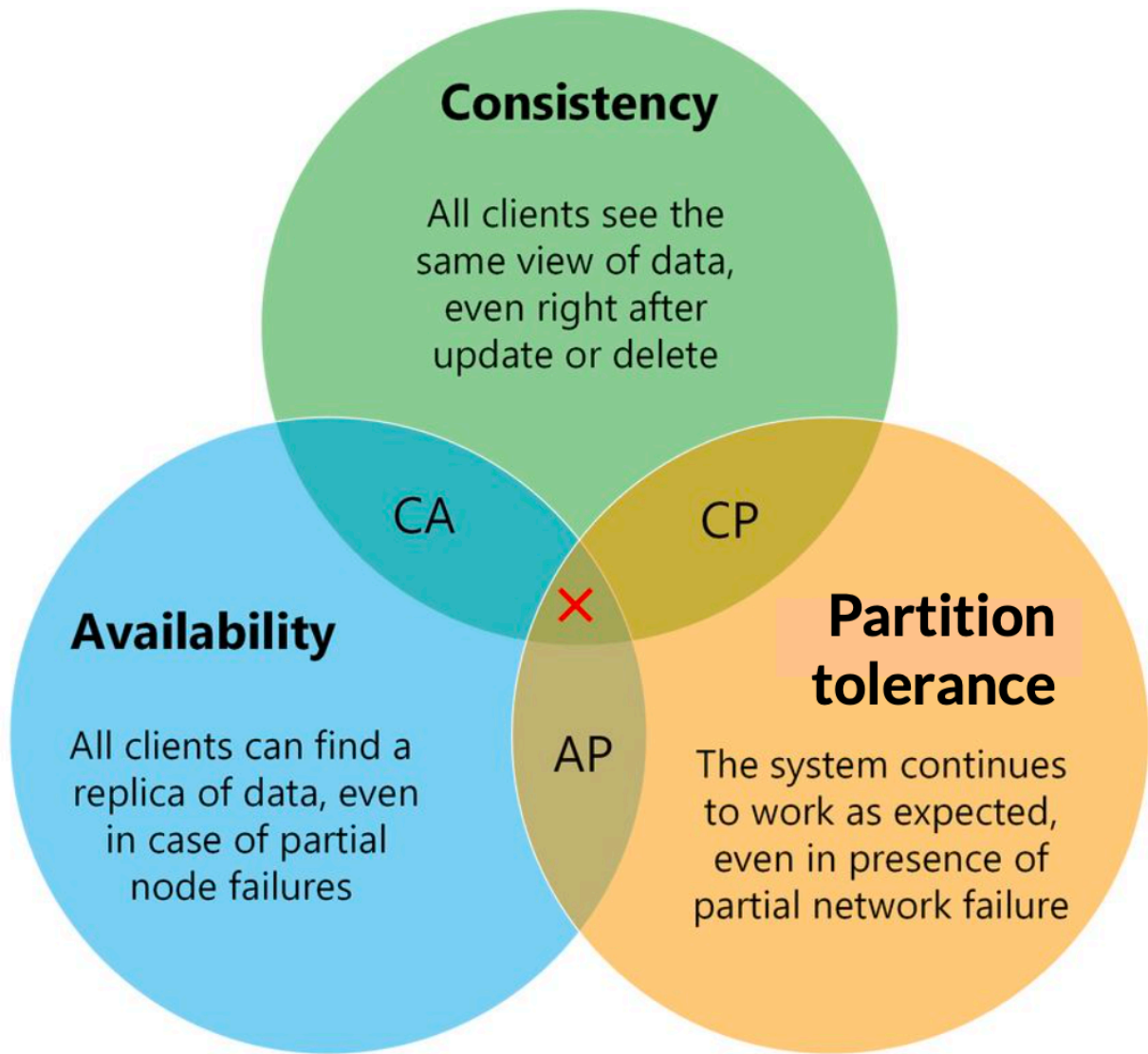


Figure 5.6.: CAP theorem diagram

The choice depends on the use case: high-level computing systems like *Lustre* prefer AP for rapid access to constantly updated data; banking systems prefer CP for guaranteed consistency. The AC combination is not viable for distributed systems but is used for vertically scalable systems where partition tolerance is not needed.

5.4. Exercises

Exercise 5.1 (*Writing a File to HDFS*)

Explain how a client interacts with HDFS when it wants to write a file. Describe the roles of the NameNode and DataNodes in this process.

Solution

The client first contacts the **NameNode** with the write request. The NameNode checks authorisation and returns the list of DataNodes that should host each block. The client then connects **directly** to those DataNodes — data is **not** routed through the NameNode. Each block is written through a **replication pipeline**: client → DN1 → DN2 → DN3 (default replication factor 3). Acknowledgements propagate back to the client and the DataNodes report the new block to the NameNode.

Exercise 5.2 (*Heartbeat in HDFS*)

What is meant by a heartbeat in a distributed system such as HDFS? What is its main role, and what happens if heartbeats stop?

Solution

It is a periodic signal the DataNodes send to the NameNode to make sure the DataNodes are working and available. Its main functionality is to prevent Network Failures, so in case that “heartbeat” stops the NameNode excludes that DataNode until it is available and redirects to another DataNode containing the same data.

Exercise 5.3 (*DataNode Failure*)

What happens in HDFS when a DataNode fails? How does HDFS detect the failure and what corrective actions does it take?

Solution

The NameNode detects the failure when heartbeats from the DataNode stop. The data on the dead node is “lost” from that copy, but other replicas survive on different DataNodes, so the NameNode redirects clients to those. Crucially, the NameNode also triggers **re-replication**: it identifies every block whose replication factor dropped below the configured value and instructs surviving DataNodes to copy those blocks to other live DataNodes, restoring the target factor. Without this step durability would degrade with every subsequent failure.

Exercise 5.4 (*Fault Tolerance in HDFS*)

How is fault tolerance implemented in HDFS? Describe the mechanisms used to ensure data durability in the presence of hardware failures.

Solution

Fault tolerance is divided in three categories:

- Network instability: The NameNode can not interact with a DataNode, this is avoided by having multiple DataNodes with copies of the data, as DataNodes are continuously sending signals to the NameNode when the signal fails that Node is excluded and redirected to another working node while the signal is restored.
- DataNode failure: The data of a DataNode is lost, to avoid that multiple copies of the

data is stored in different DataNodes, so the NameNode uses one of the copies while the failed one is being restored

- NameNode failure: The worst case, this can be avoided by having a secondary NameNode that is activated, but needs to be done manually.

Exercise 5.5 (*AC in a Distributed File System*)

Can we create a system that behaves as AC (Available + Consistent) in a distributed file system? Relate your answer to the CAP theorem.

Solution

No. The CAP theorem says a distributed system can guarantee at most two of {C, A, P}. **Partition tolerance cannot be designed away** — network links between nodes will eventually fail (cable cuts, switch failures, congestion), and when they do the system is partitioned whether we like it or not. Since P is forced upon us, the practical choice is **CP** or **AP**. AC is only viable for non-distributed (single-node, vertically scaled) systems where there is no inter-node network to fail.

Exercise 5.6 (*CAP Theorem Explained*)

Explain the CAP Theorem in the context of a distributed computing system. Define each of the three properties and explain why only two can be fully guaranteed simultaneously.

Solution

- **Consistency (C)**. Every read returns the most recent write — all nodes see the same value at the same time.
- **Availability (A)**. Every request receives a (non-error) response, regardless of the state of any individual node.
- **Partition tolerance (P)**. The system continues to operate correctly even when the inter-node network is split into disconnected groups (a *partition*).

Only two can hold simultaneously because during a partition the system must choose: either refuse some requests on the minority side to keep the data consistent (sacrifices A) or accept divergent writes on each side and reconcile later (sacrifices C). Since real networks always eventually partition, the practical choice in any distributed system is **CP** or **AP**.

Exercise 5.7 (*CAP Theorem and Implementation*)

Describe the CAP theorem and discuss how it guides the design and implementation of distributed databases.

Solution

CAP says a distributed database must pick **CP** or **AP**. That choice dictates the architecture:

- **CP designs** use leader-based replication with quorum writes; during a partition the minority side refuses writes to preserve consistency (e.g. HBase, MongoDB default, Spanner).
- **AP designs** use multi-master replication with eventual consistency and conflict resolution (last-write-wins, vector clocks, CRDTs) — every node keeps serving requests during a partition and reconciles state afterwards (e.g. Cassandra, DynamoDB, Riak).

So CAP doesn't just classify databases — it shapes the replication protocol, the consistency model, and the failure-mode behaviour the application must handle.

Exercise 5.8 (CAP and Databases)

Does the CAP theorem apply to databases? Discuss with examples of real database systems.

Solution

Yes — any database that is replicated across nodes is subject to CAP.

- **CP**: HBase, MongoDB (default), Google Spanner, etcd, ZooKeeper.
- **AP**: Cassandra, DynamoDB, Riak, CouchDB.
- **Single-node RDBMS** (PostgreSQL, MySQL on one host) are effectively **CA** because there is no inter-node partition to tolerate; clustering them turns them into CP systems.

Some systems are **tunable** per operation (Cassandra: $R + W > N$ gives stronger consistency; MongoDB: read-concern levels), letting the application slide along the CP–AP spectrum.

Exercise 5.9 (CAP and ATM Booths)

Describe how the CAP theorem applies to ATM booths of a banking system. Which two properties does this system typically prioritise, and why?

Solution

Banking systems prioritise **CP**. During a network partition between an ATM and the central ledger, the ATM will **refuse** the withdrawal rather than risk inconsistent balances or overdrafts — i.e. availability is sacrificed to preserve consistency. Money has integrity constraints that cannot tolerate divergent replicas: a brief “sorry, please try again later” is acceptable; a double-spend is not.

Exercise 5.10 (CAP and Live Chat Systems)

Describe how the CAP theorem applies to a live chat system that wants to deliver messages as quickly as possible. Which CAP combination does this system choose?

Solution

Live chat picks **AP**. Users care that messages are delivered fast and that the service stays up under network turbulence; they tolerate brief inconsistencies (messages temporarily showing in a different order on different devices, late delivery on a slow link). Strong consistency would force every message through a single coordinator and add latency the users would notice. Eventual consistency is fine here.

Exercise 5.11 (*Social Media DBMS*)

How would you classify the database management system of a social media platform that wants to upload messages as quickly as possible? Describe it using the CAP theorem framework (as an AP system).

Solution

AP. Posts must be accepted quickly and the timeline must stay reachable at all times. It is acceptable that a new post appears on followers' feeds with a few seconds' lag, or that the like-count is briefly stale on different servers — eventual consistency suffices. The cost of refusing a write (lost engagement) is much higher than the cost of a slightly stale read. Hence the typical storage layer is a multi-master, eventually consistent system like Cassandra or DynamoDB.

Exercise 5.12 (*AP Systems*)

Discuss Availability–Partition tolerance (AP) distributed systems according to the CAP theorem. What consistency guarantees do AP systems provide, and what do they sacrifice?

Solution

AP systems guarantee that every node keeps responding to requests during a partition, sacrificing strict consistency. They typically provide **eventual consistency**: replicas may diverge temporarily but converge to the same state once the partition heals and writes propagate. Conflicts are resolved by mechanisms such as **last-write-wins** timestamps, **vector clocks** or **CRDTs**. The application has to be prepared to read stale data and, sometimes, see anomalies like read-your-own-writes failures unless extra session guarantees are layered on top.

Exercise 5.13 (*Consistency vs Availability Trade-off*)

Discuss the consistency and availability trade-off in distributed NoSQL databases. How do different systems position themselves along this spectrum?

Solution

NoSQL stores span the whole spectrum:

- **Strongly consistent (CP)**. HBase, Spanner, ZooKeeper, MongoDB (default). Slow writes during partitions, may refuse requests, but never serve stale data.

- **Tunable.** Cassandra (per-query consistency level: $R + W > N$ for quorum), DynamoDB (eventual or strong reads), MongoDB (write-concern levels). The operator picks the trade-off per workload.
- **Eventually consistent (AP).** Riak, Voldemort, CouchDB. Lowest latency, highest availability, reads may be stale, conflicts resolved post-hoc.

The choice is essentially how much **staleness** / **divergence** the application tolerates in exchange for **latency** / **availability**.

Exercise 5.14 (*CAP in Relational and NoSQL DBs*)

Explain the CAP theorem in the context of both relational and NoSQL databases. How do traditional RDBMS systems typically position themselves compared to NoSQL systems?

Solution

Traditional single-node RDBMS (PostgreSQL, MySQL) are effectively **CA** — there is no partition because there is only one node. As soon as they are clustered with synchronous replication they become **CP**: writes are blocked on the minority side of a partition to preserve consistency.

NoSQL systems were created precisely to give up some consistency for **horizontal scale and availability**, so most lean **AP** (Cassandra, DynamoDB, Riak). A few NoSQL systems remain CP (HBase, MongoDB-default) when consistency is non-negotiable. The shift from RDBMS to NoSQL was, in CAP terms, the explicit acceptance of eventual consistency as the price of always-on, geo-distributed scale.

Exercise 5.15 (*"C" in CAP vs "C" in ACID*)

What is the difference between the “C” (Consistency) in the CAP theorem and the “C” (Consistency) in the ACID model? Are they the same concept?

Solution

No, they are different concepts.

- **CAP-C** is *linearizability*: every read sees the most recent write, and all replicas agree on the value at every instant. It is a property of how replicas are kept in sync across the network.
- **ACID-C** is *integrity*: a transaction moves the database from one valid state to another, preserving constraints (foreign keys, uniqueness, business rules). It is a property of a single transaction on a single logical database, independent of replication.

A single-node RDBMS satisfies ACID-C without CAP ever applying; a distributed AP store may violate CAP-C while still respecting ACID-C within each replica.

Exercise 5.16 (*CAP Consistency*)

Explain the concept of Consistency from the perspective of the CAP theorem. How does it differ from eventual consistency?

Solution

CAP-Consistency (linearizability) requires that **every read returns the most recent committed write** — as if there were a single global copy of the data. To uphold it on N replicas, every write must be propagated and acknowledged before any later read can succeed. **Eventual consistency** is weaker: it only promises that **if no new writes occur, all replicas will converge to the same value**. In the meantime, reads may return stale data, and different clients may briefly see different values. Eventual consistency is the consistency model of AP systems; CAP-C is the one CP systems uphold.

Exercise 5.17 (*Partition Tolerance*)

What is the meaning of “Partition” in the context of the CAP theorem? Why is partition tolerance generally considered non-negotiable in distributed systems?

Solution

A **partition** is a state of the network in which some subset of nodes can no longer communicate with the rest — caused by cable cuts, switch failures, congestion, routing problems, or whole-datacentre outages. **Partition tolerance** is the property of continuing to operate correctly when this happens.

It is non-negotiable because partitions are not a design choice — they will happen in any sufficiently long-lived networked system. A “non-P” design would simply fail (or worse, silently corrupt data) on the first network blip, which is unacceptable.

Exercise 5.18 (*Why Partition Tolerance is Crucial*)

Why is partition tolerance crucial in distributed systems? What would happen to a distributed system that does not tolerate network partitions?

Solution

Because network failures (link cuts, switch faults, congestion, dropped packets) are **routine**, not exotic, in any system that spans more than one host. A system that did not tolerate partitions would either become completely unavailable on the first link failure, or — worse — keep accepting writes on both sides without reconciliation, silently producing inconsistent state. Neither is acceptable in production, which is why P is mandatory and the real CAP choice is between **CP** and **AP**.

Exercise 5.19 (*Ways to Partition Databases*)

What are the different ways to partition (shard) databases? Describe at least three partitioning strategies and their trade-offs.

Solution

- **Range partitioning.** Each shard owns a contiguous key range (rows 0–1M on node A, 1M–2M on node B). Simple, range scans are cheap, but **hotspots** appear on popular ranges.
- **Hash partitioning.** $\text{Shard} = \text{hash}(\text{key}) \bmod N$. Distributes load evenly, but adding a node reshuffles every record unless you use **consistent hashing**, and range scans become expensive (a scan visits every shard).
- **Directory / lookup-based partitioning.** A separate metadata service maps each key to a shard, allowing arbitrary placement. Maximally flexible, but the lookup service is a SPOF and an extra hop.
- **Vertical partitioning.** Different *columns* or *tables* live on different nodes (split hot vs cold attributes). Improves cache locality but joins across partitions become expensive.

Choosing a strategy trades off **load balance**, **range-query locality**, **rebalance cost** and **operational complexity**.

Exercise 5.20 (MongoDB and CAP)

MongoDB is a NoSQL database focused on consistency. Does it mean it is CA or CP (or perhaps somewhere in the middle)? Motivate your answer with reference to MongoDB's replication and failover mechanisms.

Solution

MongoDB is **CP** (not CA — CA is impossible for any real distributed system). A replica set has a single **primary** that accepts writes and replicates them to secondaries via the oplog. On a partition, only the side holding a majority of nodes can elect a primary and continue serving writes; the minority side **rejects writes** to preserve consistency — that is the CP trade-off, sacrificing availability on one side of the partition.

It is “in the middle” only in the sense of being **tunable on reads**: reading from the primary gives linearizable behaviour, while reading from secondaries (`readPreference: secondary`) gives faster but potentially stale data — an AP-like operation point on top of the otherwise CP architecture.

6. Databases

Definition 6.1 (Database). A **database** is a set of *records* (generally related to each other) organized and processed by a dedicated *management system*.

The term *database* encompasses three concepts: the *records*, the *collections* that group the individual records, and the *relations* that exist both between records of the same collection and between records of different collections.

The dedicated management system is called a **database management system** (DBMS), whose main functionalities are:

1. *Create* and *define* data
2. *Select* and *query* data
3. *Manipulate* and *process* data

The main differences between management through a file system and through a DBMS:

1. With DBMS, operations can take place at the level of the *single records*, not just whole files.
2. The data are no longer “scattered” between different files and directories, but all located within a single entity that also includes the relationships between the data.
3. In data analysis, different algorithms applied to the same data set do not require a new copy of the file — it is possible to select a subset of data without replicating it.

6.1. Database Models

In databases we distinguish two aspects:

1. **Model** (or **schema**): the *description* of the data and the *relations* between the various parts of the dataset.
2. **Instance** (or **state**): the actual content in the database, that is the *data*.

In general, we distinguish two types of models: *relational* and *non-relational*; the former are the standard for structured data, and are also called *SQL databases*; the latter are typically made for semi-structured and unstructured data, and are referred to as *NoSQL* (“Not Only SQL”).

6.2. Relational Databases

The foundations of relational databases were laid in **1970** by **Edgar F. “Ted” Codd** at IBM, in his landmark paper “*A Relational Model of Data for Large Shared Data Banks*”. Codd proposed

organising data into **relations** (tables) and defined **relational algebra** — a set of formal operations on those relations, including *select* (filter rows), *project* (choose columns), *join* (combine tables), *union*, *intersection*, and *difference*. Every SQL query can be understood as a composition of these algebraic operations. The most widely used RDBMS today is *MySQL*, an open source management system.

Relational databases require a predefined schema and are perfectly suited to structured data. The collections of RDBs (*Relational DataBases*) are **tables** organized in columns (categories) and rows (individual records).

Three elements are clearly distinguished:

- The *records*, corresponding to the single rows of the table, also called *tuples*.
- The **attributes**, corresponding to the single columns.
- The **primary key**: a specific attribute (or combination of attributes) that must be *unique* (different tuples cannot have the same key) and *minimal* (the minimum number of attributes required to define the key).

To link the various tables, **foreign keys** are used: attributes or combinations of attributes in one table that act as a primary key in another table.

Each set of operations performed on a relational database is called a **transaction**, and each must respect four criteria summarized in the acronym **ACID**:

- **Atomicity** (*all-or-nothing*): all operations in a transaction are either performed completely or the entire transaction is aborted.
- **Consistency**: each transaction must bring a database from a *valid state* to *another valid state*, preserving all imposed constraints.
- **Isolation**: each transaction must generate the same results that would have been obtained in a single-user environment, avoiding conflicts when multiple users act simultaneously.
- **Durability**: following a transaction, the state of the database must remain the same until it is modified by another transaction.

6.2.1. Transaction Control in SQL

ACID properties are enforced through **transactions**. SQL provides three transaction control statements:

```
START TRANSACTION;                                -- begin a transaction

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

COMMIT;                                             -- make changes permanent
```

If an error occurs, roll back all changes made since **START TRANSACTION**:

```
START TRANSACTION;
DELETE FROM Orders WHERE order_date < '2020-01-01';
ROLLBACK;                                -- undo the DELETE
```

SAVEPOINT allows partial rollbacks within a transaction:

```
START TRANSACTION;
INSERT INTO log VALUES ('step 1');
SAVEPOINT sp1;
INSERT INTO log VALUES ('step 2');
ROLLBACK TO SAVEPOINT sp1;              -- undo only step 2
COMMIT;                                -- commits step 1
```

6.2.2. Transaction Isolation Levels

SQL defines four isolation levels that control the trade-off between consistency and concurrency:

Level	Dirty Read	Non-repeatable Read	Phantom Read
READ UNCOMMITTED	Possible	Possible	Possible
READ COMMITTED	Prevented	Possible	Possible
REPEATABLE READ	Prevented	Prevented	Possible
SERIALIZABLE	Prevented	Prevented	Prevented

MySQL's default is REPEATABLE READ. Higher isolation levels reduce anomalies but increase lock contention. Set the level with:

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

6.3. SQL

The standard language for any DBMS related to relational databases is **SQL** (pronounced *se-quel*, from the original name **SEQUEL** — *Structured English Query Language* — given to it by IBM researchers in 1974). Development at IBM began that year, and SQL was adopted as an **ISO/ANSI standard in 1987**. The standard has been revised many times since: SQL:1992 added important query features, SQL:1999 introduced recursive queries and triggers, SQL:2003 added window functions and XML support, and SQL:2016 added JSON support, among other revisions.

SQL is a **declarative** language: what is written describes *what* must be done, rather than *how* it should be done. It is structured in three parts:

- **DDL** (*Data Definition Language*): defines and modifies the structure of the database.

6. Databases

- **DML** (*Data Manipulation Language*): inserts, updates, deletes and queries records.
- **DCL** (*Data Control Language*): manages access permissions and security.

6.3.1. Creating Databases and Tables

Each SQL instruction must end with a semicolon.

```
-- View all databases
SHOW DATABASES;

-- Create a database
CREATE DATABASE IF NOT EXISTS MyDataBase;

-- Select a database to work with
USE MyDataBase;

-- View tables in the current database
SHOW TABLES;

-- Create a table
CREATE TABLE Users (
    UserID    varchar(30),
    BadgeNum  int,
    FirstName varchar(255),
    LastName  varchar(255),
    Age       int,
    OtherAttr float
);
```

SQL variable types include: int, float, varchar(n), char(n), date, datetime, boolean, text, and more.

6.3.2. Keys

```
-- Single-attribute primary key
CREATE TABLE Users (
    UserID    varchar(30) PRIMARY KEY,
    BadgeNum  int,
    FirstName varchar(255),
    LastName  varchar(255),
    Age       int,
    OtherAttr float
);
```

```

-- Multi-attribute primary key
CREATE TABLE Users (
    UserID    varchar(30),
    BadgeNum  int,
    FirstName varchar(255),
    LastName  varchar(255),
    Age       int,
    OtherAttr float,
    PRIMARY KEY (UserID, BadgeNum)
);

-- Foreign key
CREATE TABLE Authorizations (
    BadgeID int PRIMARY KEY,
    Since   date,
    Level   int,
    Sector  int
);

CREATE TABLE Users (
    UserID    varchar(30) PRIMARY KEY,
    BadgeNum  int,
    FirstName varchar(255),
    LastName  varchar(255),
    Age       int,
    OtherAttr float,
    FOREIGN KEY (BadgeNum) REFERENCES Authorizations(BadgeID)
);

```

6.3.2.1. AUTO_INCREMENT / SERIAL

When using a simple integer surrogate key, you rarely want to assign IDs manually. The `AUTO_INCREMENT` attribute (MySQL syntax) tells the database to generate a unique, incrementing integer value automatically whenever a new row is inserted:

```

CREATE TABLE users (
    id    INT AUTO_INCREMENT PRIMARY KEY,
    name  VARCHAR(100)
);

```

Every `INSERT` that omits `id` (or passes `NULL`) receives the next available integer. PostgreSQL uses the `SERIAL` shorthand (or the standard `GENERATED ALWAYS AS IDENTITY`) for the same effect.

⚠ Common mistake

Do **not** write `PRIMARY KEY` next to each individual attribute for a multi-attribute key — SQL will interpret each as a separate primary key and return `ERROR 1068 (42000): Multiple primary key defined`. Use the `PRIMARY KEY (attr1, attr2)` syntax at the end of the table definition instead.

6.4. Relationships in Relational Databases

A relational database models **relationships** between entities. There are three fundamental relationship types, each requiring a different schema design.

The table below summarises the notation commonly used in **Entity-Relationship (ER) diagrams** alongside the corresponding SQL implementation:

Relationship	Crow's foot notation	SQL implementation
One-to-one (1:1)	<code>\ --\ </code>	FOREIGN KEY with UNIQUE constraint
One-to-many (1:N)	<code>\ ---<</code>	FOREIGN KEY on the “many” side
Many-to-many (N:M)	<code>>---<</code>	Junction/bridge table with two FOREIGN KEYs

6.4.1. One-to-One (1:1)

Each record in table A corresponds to **at most one** record in table B. The Foreign Key is placed on either side, but must have a `UNIQUE` constraint to enforce the 1:1 cardinality.

Example — Student Thesis: each student has at most one thesis.

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100)
);

CREATE TABLE Theses (
    thesis_id INT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    student_id INT UNIQUE,           -- UNIQUE enforces 1:1
    FOREIGN KEY (student_id) REFERENCES Students(student_id)
);
```


6.4.2. One-to-Many (1:N)

Each record in table A (the “one” side) corresponds to **many** records in table B (the “many” side). The Foreign Key must be placed on the “**many**” side — never on the “one” side.

Example — Course Lectures: one course has many lectures.

```
CREATE TABLE Courses (
    course_id    INT          PRIMARY KEY,
    name         VARCHAR(100) NOT NULL
);

CREATE TABLE Lectures (
    lecture_id   INT          PRIMARY KEY,
    title        VARCHAR(200) NOT NULL,
    course_id    INT,          -- FK on the "many" side
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

i Note

Placing the FK on the “one” side (e.g., storing a list of lecture IDs inside the Course row) violates First Normal Form and is not possible in standard SQL. The FK always belongs on the table that can have multiple records per parent.

6.4.3. Many-to-Many (N:M)

Records in table A can relate to **many** records in table B and vice versa. SQL cannot express this directly with a single FK — a **junction table** (also called association or bridge table) is required.

Example — Student Course: students enrol in many courses; each course has many students.

```
CREATE TABLE Enrollments (
    student_id   INT NOT NULL,
    course_id    INT NOT NULL,
    enrollment_date DATE,
    PRIMARY KEY (student_id, course_id),          -- composite PK ensures
                                                ↪ uniqueness
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

The composite Primary Key (student_id, course_id) guarantees that a student cannot be enrolled in the same course twice, and the combination is the minimal unique identifier for each

enrolment record.

6.4.4. Constraints

Constraints are keywords written after the variable type:

- **UNIQUE**: each value in the column must be unique.
- **NOT NULL**: no NULL values allowed.
- **DEFAULT <value>**: sets a default value when none is provided.
- **CHECK (<condition>)**: returns an error if the condition is not respected.

```
CREATE TABLE Users (
  UserID    varchar(30) PRIMARY KEY,
  BadgeNum  int          NOT NULL UNIQUE,
  FirstName varchar(255) NOT NULL DEFAULT '',
  LastName  varchar(255) NOT NULL DEFAULT '',
  Age       int          DEFAULT 18,
  OtherAttr float        CHECK (OtherAttr > 0)
);
```

To inspect a table's structure:

```
DESCRIBE Users;
```

Field	Type	Null	Key	Default	Extra
UserID	varchar(30)	NO	PRI	NULL	
BadgeNum	int	NO	UNI	NULL	
FirstName	varchar(255)	NO			
LastName	varchar(255)	NO			
Age	int	YES		18	
OtherAttr	float	YES		NULL	

6.4.5. Altering Tables

Altering a table's structure involves all the records present in it. This is one of the main challenges in working with relational databases — maintaining consistency is critical (a process called **database normalization**).

```
-- Add a column
ALTER TABLE Users
ADD COLUMN Role char(1) DEFAULT 'A' CHECK (Role IN ('A','B','C'));
```

```

-- Change default value
ALTER TABLE Users
ALTER Role SET DEFAULT 'B';

-- Add a constraint (redefines the attribute)
ALTER TABLE Users
MODIFY Role char(1) NOT NULL;

-- Add a primary key after creation
ALTER TABLE AddressBook
ADD PRIMARY KEY (TelephoneNumber, User);

-- Add a foreign key
ALTER TABLE AddressBook
ADD CONSTRAINT FK_user
FOREIGN KEY (User) REFERENCES Users(UserID);

-- Remove a constraint
ALTER TABLE AddressBook
DROP CONSTRAINT FK_user;

-- View table creation info
SHOW CREATE TABLE AddressBook;

-- Drop a column
ALTER TABLE Users
DROP COLUMN Role;

-- Drop a table
DROP TABLE IF EXISTS Users;

```

Warning

Dropping a table deletes all records within it **and** all foreign key connections to other tables, potentially making the entire database unusable. Use with caution.

6.4.6. Data Insertion

```

INSERT INTO Users (UserID, BadgeNum, FirstName, LastName, OtherAttr)
VALUES ('usr:00001', 100, 'Rei', 'Ayanami', 1.8);

-- Multiple records at once
INSERT INTO Users (UserID, BadgeNum, FirstName, LastName, OtherAttr)

```

VALUES

```

('usr:00002', 101, 'Asuka', 'Soryu Langley', 3.5),
('usr:00003', 102, 'Shinji', 'Ikari', 2.9),
('usr:00004', 103, 'Toji', 'Suzuhara', 4.7),
('usr:00005', 104, 'Kaworu', 'Nagisa', 3.0);

```

6.4.7. Modifying Records

```

UPDATE Users
SET     Role = 'C'
WHERE   (LastName = 'Ayanami');

```

Wildcards extend WHERE conditions for strings: `_` is a char wildcard (any single character) and `%` is a string wildcard (any sequence of characters). They are used with **LIKE**:

```

-- All records where FirstName contains 'o'
UPDATE Users
SET     Role = 'B'
WHERE   (FirstName LIKE '%o%');

-- All records where the third character of FirstName is 'i'
UPDATE Users
SET     OtherAttr = 5.0
WHERE   (FirstName LIKE '__i%');

```

6.4.8. Deleting Records

```

DELETE FROM Users
WHERE   (LastName LIKE '%p_');

```

6.4.9. Conditional Expressions: CASE

The **CASE** expression evaluates conditions row-by-row and returns a value, similar to an if-else chain:

```

SELECT name,
       salary,
       CASE
         WHEN salary < 30000 THEN 'Low'
         WHEN salary < 70000 THEN 'Medium'
         ELSE 'High'

```

```
END AS salary_band
FROM Employees;
```

CASE can also be used in ORDER BY, GROUP BY, and aggregate functions.

6.4.10. Data Selection

i How to Read (and Write) a SELECT Statement: Execution Order

Although you write SELECT first, the database executes the clauses in this order:

Step	Clause	What it does
1	FROM / JOIN	Identify the source tables and combine them
2	WHERE	Filter rows before grouping
3	GROUP BY	Group the filtered rows
4	HAVING	Filter groups (like WHERE but on aggregate results)
5	SELECT	Evaluate expressions and select columns
6	ORDER BY	Sort the result
7	LIMIT / OFFSET	Restrict the number of returned rows

This order explains why you **cannot** use a column alias defined in SELECT inside a WHERE clause — WHERE is evaluated before SELECT.

The SELECT command is the most powerful and widely used in SQL, enabling the extraction of records according to conditions specified by the user — the so-called **queries**.

```
-- View entire table
SELECT * FROM Users;

-- Limit displayed records
SELECT * FROM Users
LIMIT 3;

-- Select specific attributes
SELECT FirstName, BadgeNum FROM Users;

-- Distinct values (no repetitions)
SELECT DISTINCT FirstName FROM Users;

-- Sort (ascending by default; DESC for descending)
SELECT FirstName FROM Users
ORDER BY FirstName;
```

```

-- Filter with WHERE
SELECT FirstName, BadgeNum FROM Users
WHERE (OtherAttr > 0);

-- Create a VIEW
CREATE VIEW OneThree AS
SELECT UserID, BadgeNum, LastName
FROM Users
WHERE (OtherAttr BETWEEN 1 AND 3);

SELECT * FROM OneThree;
SHOW CREATE VIEW OneThree;

-- Mathematical operations on attributes
SELECT OtherAttr * 2, POWER(BadgeNum, 2) FROM Users
WHERE (OtherAttr > 1.8);

```

Built-in aggregate functions: COUNT(), MAX(), MIN(), SUM(), AVG().

Aliases for attributes and tables:

```

SELECT DISTINCT FirstName AS UniqueNames
FROM Users;

-- AS can be omitted
SELECT FirstName NamesEndingI
FROM Users U
WHERE (FirstName LIKE '%i');

```

GROUP BY and HAVING:

```

SELECT FromCountry, COUNT(DISTINCT FromCity) TotalCities
FROM Ships
GROUP BY FromCountry
ORDER BY TotalCities DESC;

-- HAVING filters on aggregated results
SELECT FromCountry, COUNT(DISTINCT FromCity) TotalCities
FROM Ships
GROUP BY FromCountry
HAVING TotalCities > 1
ORDER BY TotalCities DESC;

```

⚠ Implicit JOIN = Cartesian Product + Filter

An **implicit join** uses a comma-separated FROM clause and a WHERE condition to filter the result:

```
SELECT c.name, o.total
FROM Customers c, Orders o
WHERE c.customer_id = o.customer_id;
```

This is equivalent to an explicit INNER JOIN, but the database engine first computes the full **Cartesian product** (every row of Customers × every row of Orders) and then filters. For large tables this is highly inefficient and easy to make incorrect by forgetting the WHERE condition. Always prefer explicit JOIN ... ON syntax.

JOINS combine tables through matching attribute values:

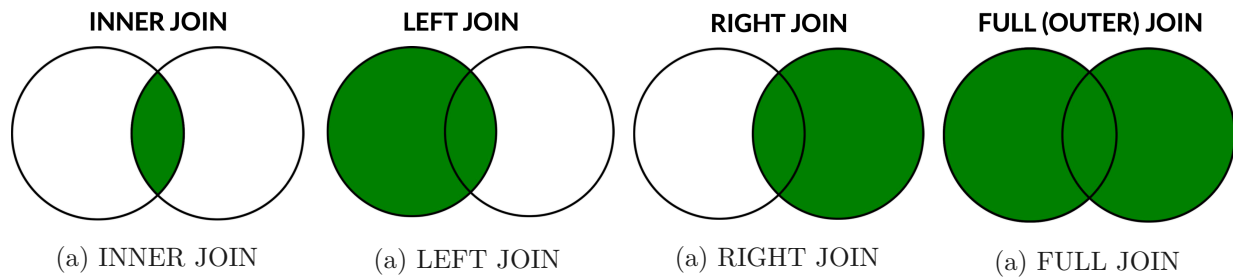
```
-- INNER JOIN: only records with matching keys in both tables
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
INNER JOIN Orders o ON s.Code = o.Code;

-- LEFT JOIN: all records from the left table; NULLs for non-matching right
-- records
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
LEFT JOIN Orders o ON s.Code = o.Code;

-- RIGHT JOIN: all records from the right table; NULLs for non-matching left
-- records
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
RIGHT JOIN Orders o ON s.Code = o.Code;

-- FULL (OUTER) JOIN: all records from both tables; NULLs where no match
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
LEFT JOIN Orders o ON s.Code = o.Code
UNION
SELECT s.FromCountry, o.Name, s.Rate
FROM Ships s
RIGHT JOIN Orders o ON s.Code = o.Code;
```

Graphic illustration of the join processes



6.4.11. Other JOIN Types

JOIN Type	Description
CROSS JOIN	Returns the Cartesian product of two tables — every combination of rows. No <code>ON</code> condition. Rarely used directly; equivalent to implicit join without a <code>WHERE</code> clause.
SELF JOIN	Joins a table to itself. Useful for hierarchical data (e.g. an employee table with a <code>manager_id</code> column referencing the same table).
NATURAL JOIN	Automatically joins on all columns that share the same name and data type in both tables. No <code>ON</code> clause is written — the database infers the join condition.

```
-- CROSS JOIN example
SELECT a.colour, b.size FROM Colours a CROSS JOIN Sizes b;

-- SELF JOIN example (employees and their managers)
SELECT e.name AS employee, m.name AS manager
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.employee_id;

-- NATURAL JOIN example
SELECT * FROM orders NATURAL JOIN customers;
```

⚠ NATURAL JOIN is fragile

`NATURAL JOIN` infers its join condition from column names at query time. If either table later gains a new column that happens to share a name with a column in the other table, the join condition **silently changes** — the query continues to run but returns different (wrong) results with no error or warning. For this reason, explicit `JOIN ... ON` clauses are strongly preferred in any production or long-lived code.

6.5. Subqueries

A **subquery** (or nested query) is a **SELECT** statement embedded inside another SQL statement. Subqueries can appear in the **WHERE**, **FROM**, or **SELECT** clauses.

```
-- Find customers who have placed at least one order over €1000
SELECT customer_name
FROM Customers
WHERE customer_id IN (
    SELECT customer_id
    FROM Orders
    WHERE total > 1000
);
```

Subqueries can also use **ANY** or **ALL**:

```
-- Products more expensive than ALL products in category 5
SELECT name FROM Products
WHERE price > ALL (SELECT price FROM Products WHERE category_id = 5);
```

Warning

When possible, rewrite correlated subqueries as explicit **JOINs** — a well-optimised **JOIN** is typically faster because the planner can choose the join algorithm. Subqueries that are executed once per row of the outer query (“correlated subqueries”) can be very slow on large tables.

6.5.1. Data Control (DCL)

The DCL (*Data Control Language*) manages access to databases, user permissions and data security.

```
-- Create a user
CREATE USER 'new_user'@'localhost'
IDENTIFIED BY 'new_password';

-- Grant privileges
GRANT SELECT, UPDATE ON Orders.Name TO 'new_user'@'localhost';

-- Revoke privileges
REVOKE UPDATE ON Orders.Name FROM 'new_user'@'localhost';
```

6.6. Exercises

6.6.1. Conceptual Questions

Exercise 6.1 (*Schema vs Instance*)

Explain the difference between the database schema and a database instance. Give an example to illustrate the distinction.

Solution

The database schema is how the data is organized, it affects the structure and the dataset and can constrain the values of the data is stored. An instance is the data itself that follows the schema. We could see it as a party with a dress code, the organizers set the dress code and ensures everyone follows (the schema), each person in the party follows those rules (each person is an instance) and if they don't follow it they can not attend.

Exercise 6.2 (*File System vs Database*)

Compare file system-based data management with database management systems. What are the main advantages of using a DBMS over raw file storage?

Solution

In file systems you can not impose rules to the data, they can store more data but there is no control about what is being stored, in DBMS you can store less data, but you can control what is stored and you can access the data directly by applying filters. The main advantage is that you have the fixed schema that ensures the data has the proper format and you can search and make operations directly in the data.

Exercise 6.3 (*Primary Key*)

What is a primary key and why is it important? Can a table have more than one primary key? What properties must a primary key satisfy?

Solution

A primary key is an attribute (or set of attributes) that uniquely identifies each row of a table.

A table has **exactly one** primary key. It can be **composite** (made of several columns), but there is only one PK per table. Other column-sets that could also serve as a unique identifier are called **candidate keys**; the DBA picks one and promotes it to PK.

Properties a PK must satisfy: **uniqueness**, **non-null**, **minimality** (no proper subset is itself unique) and **stability** (the value should not change over the row's lifetime).

Exercise 6.4 (Foreign Key)

What is a foreign key? Provide a concrete example showing a foreign key relationship between two tables.

Solution

A foreign key is a key that is related to another table and can be used to link tables, the key that relates to needs to be a primary key of the other table. For example in a table named Cities we have information of each city in a country and there is a Country_Code key and in another table named Country we have the information at the Country level, and there is a Code key. In this case Country_code is a foreign key in the Cities table so now both are linked and we can get the information of each city in a country.

Exercise 6.5 (DDL and DML)

What is the meaning of DDL (Data Definition Language) and DML (Data Manipulation Language) in SQL? Provide at least one example statement for each category.

Solution

- **DDL** defines or modifies the *schema* — the structure of the database. Examples: CREATE TABLE, ALTER TABLE, DROP TABLE, CREATE INDEX.
- **DML** manipulates the *data* inside existing tables. Examples: INSERT INTO, UPDATE, DELETE, and (in some texts) SELECT — though SELECT is sometimes split off into a separate **DQL** (Data Query Language) category.

Exercise 6.6 (ALTER vs UPDATE)

Briefly explain the difference between ALTER and UPDATE in SQL. When would you use each?

Solution

The ALTER statements is used when you want to alter the tables schema, the UPDATE statement is used when you want to update the records in the table.

Exercise 6.7 (DELETE vs DROP)

Briefly explain the difference between the DELETE and DROP statements in SQL. What are the consequences of each?

Solution

The DELETE statements is used to delete specific records and the DROP statements is used to delete the table from the schema, so the entire atble is erased from the DBMS and all that dta is lost. A DELETE statement with a weak WHERE condition can have the same effect but it wont lose the schema of the table

Exercise 6.8 (CRUD Operations)

What are the CRUD operations and their corresponding SQL commands? Provide a simple example for each command.

Solution

CRUD = **Create, Read, Update, Delete** — the four basic operations on persistent data.

```
-- Create
INSERT INTO Students (FirstName, LastName, Email)
VALUES ('Ada', 'Lovelace', 'ada@example.com');

-- Read
SELECT * FROM Students WHERE LastName = 'Lovelace';

-- Update
UPDATE Students SET Email = 'ada.l@example.com' WHERE BadgeID = 1;

-- Delete
DELETE FROM Students WHERE BadgeID = 1;
```

Exercise 6.9 (Database Normalisation)

Why is it important to avoid storing all records in a single table in relational databases, and why should databases be normalised across multiple tables? What problems does normalisation solve?

Solution

A single fat table duplicates information and creates three classes of *anomalies*:

- **Update anomaly.** Changing one fact (e.g. a course's name) requires updating every row that mentions it; missing one row leaves the data inconsistent.
- **Insertion anomaly.** Cannot insert a course until at least one student is enrolled (or vice versa) without nullable filler columns.
- **Deletion anomaly.** Removing the last student of a course also erases the course itself.

Normalisation (1NF → 2NF → 3NF → BCNF) splits the data into linked tables so each fact lives in exactly one place, joined back at query time via foreign keys.

6.6.2. ACID Questions

Exercise 6.10 (*Atomicity*)

What is Atomicity in the ACID model? Provide an example of a database transaction where atomicity is critical.

Solution

Atomicity means a transaction is all-or-nothing: either every statement succeeds and the result is committed, or none of them take effect (rollback).

Bank transfer example. Moving \$100 from account A to account B requires two updates: $A -= 100$ and $B += 100$. If the system crashes between them and atomicity is not enforced, money disappears (debit applied, credit lost) or is created (credit applied, debit lost). Atomicity guarantees the pair either both happen or both do not.

Exercise 6.11 (*Isolation*)

What does “I” (Isolation) refer to in the ACID database transaction model? Describe a scenario where insufficient isolation causes a data integrity problem.

Solution

Isolation means concurrent transactions do not see each other’s intermediate state — each runs as if it were alone on the database. Without it, anomalies appear: dirty reads, lost updates, non-repeatable reads, phantoms.

Lost-update example. Two cashiers process refunds against an account with balance 100.

1. Cashier 1 reads balance = 100.
2. Cashier 2 reads balance = 100.
3. Cashier 1 writes $100 - 30 = 70$.
4. Cashier 2 writes $100 - 20 = 80$.

Both refunds happened but only one was deducted. Proper isolation (serializable or row-level locking) forces the second read to wait, giving the correct final balance 50.

Exercise 6.12 (*Durability vs BASE*)

What is the meaning of Durability in the ACID model? How does ACID Durability compare to the approach taken by BASE models?

Solution

Durability means that once a transaction has been acknowledged as committed, the result survives any subsequent crash — power loss, OS panic, hardware failure. It is enforced via write-ahead logs (WAL) and `fsync` of the log to non-volatile storage before the commit returns.

BASE (**B**asically **A**vailable, **S**oft state, **E**ventual consistency) is the NoSQL counterpart that explicitly relaxes ACID for horizontal scale: writes may be acknowledged after being

stored on only a subset of replicas, soft state may evolve in the background, and the system promises only that replicas will eventually converge. Durability is weakened in exchange for higher availability and throughput.

6.6.3. Schema Design Exercises

Exercise 6.13 (*University Schema: Students and Courses*)

The University is preparing a database to store all Students and Courses.

- Each Course can be attended by multiple Students.
- Each Student can be enrolled in multiple Courses.

Students table:

```
BadgeID (INT, AUTO_INCREMENT, PK),
FirstName (VARCHAR(30), NOT NULL),
LastName (VARCHAR(30), NOT NULL),
Email (VARCHAR(30), NOT NULL).
```

Courses table:

```
CourseID (CHAR(10), UNIQUE NOT NULL, PK),
CourseName (VARCHAR(255), NOT NULL),
Degree (VARCHAR(30), NOT NULL),
B_or_M (CHAR(1),
CHECK (B_or_M IN ('B', 'M'))).
```

Given this schema, what modifications are necessary (new tables, constraints, etc.) to relate the Students and Courses tables? If creating a new table, identify its Primary and Foreign Keys.

Solution

A *many-to-many* relationship requires a **junction (associative) table** linking the two existing primary keys. (*The schema below is illustrative — not a real University database.*)

```

CREATE TABLE Enrollments (
  BadgeID INT NOT NULL,
  CourseID CHAR(10) NOT NULL,
  Year INT NOT NULL,
  Grade DECIMAL(3,1),
  -- Composite primary key: a student can take the same course in
  -- different academic years, so the year is part of the PK.
  PRIMARY KEY (BadgeID, CourseID, Year),
  FOREIGN KEY (BadgeID) REFERENCES Students(BadgeID)
    ON DELETE CASCADE,
  FOREIGN KEY (CourseID) REFERENCES Courses(CourseID)
    ON DELETE CASCADE
);

```

- **Primary key:** composite (BadgeID, CourseID, Year).
- **Foreign keys:** BadgeID → Students(BadgeID) and CourseID → Courses(CourseID).
- Optional payload columns on the link (e.g. Grade, Year) capture facts that belong to the *pair*, not to a student or a course alone.

Exercise 6.14 (Library Schema: Books and Authors)

A library is preparing a relational database to manage all Books and Authors.

- Each Author can write multiple Books.
- Each Book can have multiple Authors.

What modifications are necessary to relate the Books and Authors tables? If creating a new table, what are the Primary and Foreign Keys of that table?

Solution

Same many-to-many pattern as the University case — add a junction table. (*Hypothetical schema, not a real library database.*)

```

CREATE TABLE Authors (
    AuthorID INT AUTO_INCREMENT PRIMARY KEY,
    Name     VARCHAR(100) NOT NULL
);

CREATE TABLE Books (
    ISBN     CHAR(13)     PRIMARY KEY,
    Title    VARCHAR(255) NOT NULL,
    PubYear  INT
);

CREATE TABLE BookAuthors (
    ISBN     CHAR(13) NOT NULL,
    AuthorID INT      NOT NULL,
    PRIMARY KEY (ISBN, AuthorID),
    FOREIGN KEY (ISBN) REFERENCES Books(ISBN)
        ON DELETE CASCADE,
    FOREIGN KEY (AuthorID) REFERENCES Authors(AuthorID)
        ON DELETE CASCADE
);

```

- **PK** of BookAuthors: composite (ISBN, AuthorID).
- **FKs**: ISBN → Books(ISBN), AuthorID → Authors(AuthorID).

Exercise 6.15 (Twitter: User and Follows)

A Twitter user can follow multiple people. How must the **User** table relate to the **Follows** table? Write the SQL statement to alter the **Follows** table to define the attributes that constitute the primary and foreign keys.

Solution

Follows is a **self-referencing** many-to-many link on **Users**: both the follower and the followee are users. (*Sample schema, not the real Twitter one.*)


```

CREATE TABLE Users (
    UserID  INT AUTO_INCREMENT PRIMARY KEY,
    Handle  VARCHAR(30) UNIQUE NOT NULL
);

-- Assume Follows(FollowerID, FolloweeID) already exists.
ALTER TABLE Follows
    ADD CONSTRAINT pk_follows
        PRIMARY KEY (FollowerID, FolloweeID),
    ADD CONSTRAINT fk_follower
        FOREIGN KEY (FollowerID) REFERENCES Users(UserID)
        ON DELETE CASCADE,
    ADD CONSTRAINT fk_followee
        FOREIGN KEY (FolloweeID) REFERENCES Users(UserID)
        ON DELETE CASCADE,
    ADD CONSTRAINT chk_no_self_follow
        CHECK (FollowerID <> FolloweeID);

```

- **PK** of Follows: composite (FollowerID, FolloweeID) — each directed follow relationship is unique.
- **FKs**: both columns reference Users(UserID).
- The CHECK prevents users from following themselves.

Exercise 6.16 (*Many-to-Many: Actors and Plays*)

How do you implement a many-to-many relationship between two existing SQL tables, Actors and Plays? Write the SQL statements required to create the necessary structure.

Solution

Create a junction table whose primary key is the composite of the two foreign keys. (*Illustrative schema, not a real one.*)

```

-- Existing tables (sketched here for context):
CREATE TABLE Actors (
    ActorID INT AUTO_INCREMENT PRIMARY KEY,
    Name    VARCHAR(100) NOT NULL
);
CREATE TABLE Plays (
    PlayID  INT AUTO_INCREMENT PRIMARY KEY,
    Title   VARCHAR(255) NOT NULL,
    Year    INT
);

-- Junction table linking them:
CREATE TABLE ActorPlays (
    ActorID INT NOT NULL,
    PlayID  INT NOT NULL,
    Role    VARCHAR(100),
    PRIMARY KEY (ActorID, PlayID),
    FOREIGN KEY (ActorID) REFERENCES Actors(ActorID)
        ON DELETE CASCADE,
    FOREIGN KEY (PlayID)  REFERENCES Plays(PlayID)
        ON DELETE CASCADE
);

```

The optional `Role` column on the link captures information that belongs to the *pair* (the actor played a specific role in that play), not to the actor or the play individually.

7. RDBMS - Hands on MySQL

This notebook is not executed since it needs to be executed inside a docker container

7.1. 00 - Introduction

Among the Relational DBMSs, the most widely known are Oracle, SQLite, Postgres, MariaDB, and MySQL.

Each one of them is characterized by a different set of “variations” on top of the base SQL language, although usually minor.

We will use **MySQL** to run exercises about Relational DB transactions using the SQL language, as it is *currently* among the top of the list of most widely used RDBMSs.

DB Engines Ranking Trend

[Stack Overflow 2025 Developer Survey - Most Popular Technologies \(Language\)](#)

[Stack Overflow 2025 Developer Survey - Most Popular Technologies \(Database\)](#)

MySQL programming interfaces exist for many languages, including C, C++, Go, Java, Perl, PHP, Python, etc...

Even simply referring to Python, we can count on several options to connect and interact with the DBs.

Let's explore a couple of them...

7.2. 01 - Create a Connection to MySQL

A MySQL server is running within one of your Docker containers.

You can access it using the MySQL server credentials defined in the `docker-compose.yml` file.

By default, a `root` user is created with the following credentials: - **Username:** `root` - **Password:** `root_pwd`

Additionally, a non-root user is created by default with: - **Username:** `my_user` - **Password:** `my_pwd`

7.2.0.1. Interactive

To interact with the MySQL server, you can log in interactively to its Docker container.

First, check the `CONTAINER ID` of the container named `mysql_db` by running:

```
$ docker ps
```

Then, initiate an interactive bash session within the container by executing:

```
$ docker exec -it <YOUR CONTAINER ID> bash
```

Once inside the container, you can access the MySQL server as if you were connected to a remote server hosting your databases:

```
$ mysql -u <EITHER ROOT OR USER> -p
```

Replace <YOUR CONTAINER ID> with the actual container ID obtained from the `docker ps` command output, and <EITHER ROOT OR USER> with either `root` or `my_user`, depending on the user you want to log in as.

We can then inspect the MySQL server to display all databases by executing the following SQL command:

```
SHOW DATABASES;
```

NB: Remember to end a SQL command with a semicolon `;`.

7.2.0.2. mysql.connector

A more pythonic way to interact with a MySQL server is by using a dedicated driver such as `mysql.connector`, which is already installed in the Jupyter Notebook container.

This connector offers an interface to establish a connection object to the MySQL server and provides various functionalities for executing transactions and fetching records from databases.

We can start by declaring a connection to our MySQL server.

NB: The server is hosted in a different container from the notebook, simulating a connection between two different machines: a client and a server.

```
# import the connector
import mysql.connector

# declare a connection to the MySQL server
# the hostname (or IP) of the server machine should be passed
db = mysql.connector.connect(
    user="root",
    password="root_pwd",
    host="db", # in this case 'db' is the equivalent of the IP address of the
               server
               # or 'localhost' if running mysql locally
```

```
)

print(db)
```

At this point, a database connection is established.

To interact with the database using Python and `mysql.connector`, we need a “handle” known as a cursor, which must be instantiated.

When using this connector to interact with the database, all SQL statements are executed using the cursor.

```
# create our db cursor object
dbc = db.cursor()
```

The cursor allows us to execute all SQL transactions on the database as if we were logged into the server.

If a transaction fetches results from the database (returns a set of records), the connector object will retain the results. This feature can be useful for later use in a more “pythonic” way, integrating our database queries with other Python statements.

The syntax for issuing a generic SQL statement is as follows:

```
connector_object.execute(my_sql_query)
```

For example, let’s execute the same check as before by asking MySQL to show us all available databases...

```
# show all available databases
dbc.execute(
    "SHOW DATABASES"
)
```

No output is printed because the result of this SQL transaction is collected into the cursor as a list of records (tuples).

To retrieve the results, we can simply loop over them.

```
# loop over the container to fetch the results
for _ in dbc:
    print(_)
```

If we are `root` users in this example, we can access the inner structure of the MySQL server, which already contains a few databases by default.

After fetching the output of the transaction, the cursor reaches “the end” of the records and becomes available to perform a new transaction.

7. RDBMS - Hands on MySQL

```
# use fetch to retrieve one, many, or all entries in the cursor
# dbc.fetchone()

dbc.fetchmany(2)

# dbc.fetchall()
```

```
dbc.reset()
```

If one wants to perform a new transaction without collecting the output of the previous one from the cursor, a reset must be issued first:

```
connector_object.reset()
```

If a generic user (i.e., not the `root` user) is logged into the database server, they will not have access to certain functionalities and won't be able to see the previous list of all databases.

Some of these databases and tables are protected and used to store additional information about the state of the MySQL server and the connected users.

```
# connect to the database as a non-root user
udb = mysql.connector.connect(
    user="my_user",
    password="my_pwd",
    host="db",
)

# create a cursor
udbc = udb.cursor()

# execute the transaction
udbc.execute(
    "SHOW DATABASES"
)

# fetch all results
udbc.fetchall()
```

7.2.0.3. iPython Magic

Another way to execute queries from Python within a Jupyter Notebook is by utilizing notebook “magic” functions, which are based on iPython and SQLAlchemy.

This approach is similar to loading inline figures from Matplotlib using `%matplotlib inline`.

To instruct iPython to interpret SQL commands and connect to the appropriate database, we need to:

1. Load the extension for the SQL language.
2. Set the environment variable to point to the connection to the database.

```
%load_ext sql

%env DATABASE_URL=mysql+mysqlconnector://root:root_pwd@db
%config SqlMagic.style = '_DEPRECATED_DEFAULT'
```

Now, notebook cells can execute SQL transactions directly in plain SQL by starting the cell with `%%sql`.

By default, the result is also printed using the `pretty` format, similar to the pandas-like format.

```
%%sql

SHOW DATABASES;
```

An additional benefit is that a single `%sql` (not a double `%%sql`) can be used to combine a SQL query with any Python command.

```
my_python_var = %sql SHOW DATABASES

print(my_python_var)
print(f'First record={my_python_var[0]}')
```

The combination of either of the two Pythonic methods can be used to easily redirect the output of database transactions to leverage other well-known Python modules, such as plotting with Matplotlib.

```
import matplotlib.pyplot as plt
%matplotlib inline

plt.hist(my_python_var);
plt.xticks(rotation='vertical');
```

7.3. 02 Database and Table creation

We have already seen how to inspect the server to see which databases are present using the command:

```
SHOW DATABASES;
```

To start interacting with a specific database, we can select it using **USE**:

```
USE <DB_NAME>;
```

If the user has been granted the right to access this database, it's possible to start investigating its content.

If we want to create a new database to work with instead, we can use the command:

```
CREATE DATABASE <DB_NAME>;
```

A more complex syntax can be used to create a new database only if no other database with the same name already exists:

```
CREATE DATABASE IF NOT EXISTS <DB_NAME>;
```

- Check if a DB named `my_db` exists.
- If not, create one.
- Then, switch to the new DB to start using it.

```
%%sql
```

```
CREATE DATABASE IF NOT EXISTS my_db;  
USE my_db;
```

All tables contained in the DB are shown in a similar way to what used to show all DBs on the server:

```
SHOW TABLES;
```

NB: we are inspecting the content of a specific DB (the one selected with `USE <DB_NAME>`)

```
%%sql
```

```
SHOW TABLES;
```

The analogous Pythonic way to execute the same list of commands (using the `mysql.connector` driver) is the following:

NB: This time we are not retrieving results from the server, but are instead asking the DBMS to *create* a new database. We are issuing a request that requires us to **commit** a change in the MySQL server


```

# defining the sql command line
sql_ = "CREATE DATABASE IF NOT EXISTS my_db;"

# instructing the cursor to execute it
dbc.execute(sql_)

# in this case the result of the sql transaction is not to return a list of
  ↪ items
# it is instead a change in the DB structure
# therefore, for the change to have an effect, the change is to be committed
  ↪ to the DB
db.commit()

```

All other commands discussed so far (`USE` and `SHOW`), do not require any change to be committed, therefore we can simply issue them via the cursor.

Creating new databases, or changing the structure (*schema*) or the content (*instance*) of a database, will instead require a `commit`.

```

# use the my_db database
dbc.execute("USE my_db")

# show all tables
dbc.execute("SHOW TABLES")
dbc.fetchall()

```

So far the `my_db` database is empty, so let's use the same lines presented in the slides to create a dummy `Users` table.

Any table can then be inspected to check its schema by listing all attributes.

1. Begin by creating a `Users` table. Define the schema by specifying each attribute.
2. Verify if it has been successfully created in the database. Use the `SHOW` command for this purpose.
3. Inspect the schema of the `Users` table, execute the `DESCRIBE <TABLE_NAME>` command.

```

%%sql

# create a table following the schema on the slides
CREATE TABLE Users (
    UserID    varchar(30),
    BadgeNum  int(16),
    FirstName varchar(255),
    LastName  varchar(255),
    Age       int,

```

```
    OtherAttr float
);
```

```
%%sql

# show the tables in the current database
SHOW TABLES;
```

```
%%sql

# describe the table `Users`
DESCRIBE Users;
```

Following the previous examples, let's perform the same transactions using the `mysql.connector` driver. Since the `Users` table already exists, we will create a new table called `Users_py`.

Remember the following points:

- Utilize the DB Cursor to execute the transactions.
- Transactions that change the content of the DB must be *committed* to the DB.
- Transactions returning the content of the DB host their results *in the cursor*.

```
# create a table following the schema on the slides
sql = 'CREATE TABLE Users_py (\
    UserID    varchar(30),\
    BadgeNum  int(16),\
    FirstName varchar(255),\
    LastName  varchar(255),\
    Age       int,\
    OtherAttr float);'

dbc.execute(sql)
db.commit()
```

```
# show the tables in the current database
dbc.execute("SHOW TABLES")
dbc.fetchall()
```

```
# describe the table `Users_py`
dbc.execute("DESCRIBE Users_py")
dbc.fetchall()
```

7.4. 03 Declaring constraints and altering tables

Before moving on, let's clean up all existing tables in our database.

To remove an existing table from the database, you can use the following instruction:

```
DROP TABLE IF EXISTS <NAME OF YOUR TABLE>;
```

CAUTION: Dropping a table deletes all records from that table and severs all connections with other tables based on foreign keys. This action can render the entire database unusable. Be extremely careful when executing this command.

```
%%sql

# drop the table Users
DROP TABLE IF EXISTS Users;
```

Let's also drop the `Users_py` table using the Python connector. This is once again an action that alters the database, thus we must commit it.

```
# drop the table Users_py
dbc.execute("DROP TABLE IF EXISTS Users_py")
db.commit()
```

```
# inspect the content of the database
dbc.execute("SHOW TABLES")
dbc.fetchall()
```

Create a new `products` table with the following schema:

Attribute	Type	Extra info	Constraint
productID	Integer		NOT NULL AUTO_INCREMENT
productName	String	Max length 30	DEFAULT ''
unitPrice	Float		NOT NULL DEFAULT 99999.99
unitsInStock	Integer	Unsigned	NOT NULL DEFAULT 0

The `productID` attribute will be our **PRIMARY KEY**.

This can be declared either inline, or at the end of the table definition:

```
productID INT NOT NULL UNIQUE AUTO_INCREMENT PRIMARY KEY
```

or

```
productID INT NOT NULL UNIQUE AUTO_INCREMENT
[...]
PRIMARY KEY (productID)
```

```
%%sql

# create the products table
CREATE TABLE IF NOT EXISTS products (
    productID      INT          NOT NULL UNIQUE AUTO_INCREMENT PRIMARY
    ↪ KEY,
    productName    VARCHAR(30)  DEFAULT ' ',
    unitPrice      FLOAT        NOT NULL DEFAULT 99999.99,
    unitsInStock   INT UNSIGNED NOT NULL DEFAULT 0
);
```

```
%%sql

# inspect the table schema
DESCRIBE products;
```

The table does not contain any constraints, default values, or foreign keys.

The **schema** of a given table can be altered by adding/removing Attributes, as well as by adding constraints.

Let's first add a new attribute to the table:

Attribute	Type	Extra info	Constraint
productID	Integer		NOT NULL AUTO_INCREMENT
productName	String	Max length 30	DEFAULT ''
unitPrice	Float		NOT NULL DEFAULT 99999.99
unitsInStock	Integer	Unsigned	NOT NULL DEFAULT 0
isDiscontinued	Bool		NOT NULL

To alter the **schema** of a table and create a new column we can act on it with:

```
ALTER TABLE <THE TABLE>
ADD COLUMN <NEW COL> <THE NEW ATTRIBUTE DECLARATION>;
```

7.4.0.1. Alter the products table and add the isDiscontinued attribute

```
%%sql

# alter the table
ALTER TABLE products
ADD COLUMN isDiscontinued BOOL NOT NULL;
```

```
%%sql

# inspect the table schema
DESCRIBE products;
```

The `isDiscontinued` attribute does not contain any meaningful constraint or default value. Let's consider 0 as a good default value for this attribute:

Attribute	Type	Extra info	Constraint
productID	Integer		NOT NULL AUTO_INCREMENT
productName	String	Max length 30	DEFAULT ''
unitPrice	Float		NOT NULL DEFAULT 99999.99
unitsInStock	Integer	Unsigned	NOT NULL DEFAULT 0
isDiscontinued	Bool		NOT NULL DEFAULT 0

We can use the same `ALTER TABLE` instruction to do this, and set a new default:

```
ALTER TABLE <THE TABLE>
ALTER <ATTRIBUTE> SET DEFAULT <NEW DEFAULT>;
```

```
%%sql

# alter the table
ALTER TABLE products
ALTER isDiscontinued SET DEFAULT 0;
```

```
%%sql

# inspect the table schema
DESCRIBE products;
```

We can also impose `productName` to be not-null by altering the table one final time.

7. RDBMS - Hands on MySQL

Attribute	Type	Extra info	Constraint
productID	Integer		NOT NULL AUTO_INCREMENT
productName	String	Max length 30	NOT NULL DEFAULT ''
unitPrice	Float		NOT NULL DEFAULT 99999.99
unitsInStock	Integer	Unsigned	NOT NULL DEFAULT 0
isDiscontinued	Bool		NOT NULL DEFAULT 0

In MySQL, this implies modifying the entire attribute, thus the Type will have to be re-stated:

```
ALTER TABLE <THE TABLE>
MODIFY <ATTRIBUTE> <ATTRIBUTE TYPE> <CONSTRAINT>;
```

```
%%sql

# alter the table
ALTER TABLE products
MODIFY productName VARCHAR(30) NOT NULL;
```

```
%%sql

# inspect the table schema
DESCRIBE products;
```

Let's now create a `suppliers` table with the following description:

Attribute	Type	Extra info	Constraint
supplierID	Integer		NOT NULL AUTO_INCREMENT
supplierName	String	Max length 30	NOT NULL DEFAULT ""
countryCode	Char	Length 3	NOT NULL
email	String	Max length 30	NOT NULL

`supplierID` is going to act as the table's primary key in this case.

```
%%sql

# create the suppliers table
CREATE TABLE suppliers (
    supplierID      INT NOT NULL UNIQUE AUTO_INCREMENT PRIMARY KEY,
    supplierName    VARCHAR(30) NOT NULL DEFAULT "",
    countryCode     CHAR(3) NOT NULL,
```

```
email          VARCHAR(30) NOT NULL
);
```

```
%%sql
```

```
# inspect the table schema
DESCRIBE suppliers;
```

Alter the table to include a default “ITA” for `countryCode`

```
%%sql
```

```
# alter the table
ALTER TABLE suppliers
ALTER countryCode SET DEFAULT "ITA";
```

```
%%sql
```

```
# inspect the table schema
DESCRIBE suppliers;
```

To establish a relation between the `products` and `suppliers` tables, we have to consider that:

- a supplier can supply multiple products
- a product can be supplied by multiple suppliers

This is a **many-to-many** relationship, which requires a supporting external table.

To establish a many-to-many relationship between the `products` and `suppliers` tables, we create a junction table named `supplier_to_product` with foreign keys referencing the `productID` from the `products` table and the `supplierID` from the `suppliers` table.

Create a new `supplier_to_product` table with the following attributes:

Attribute	Type	Extra info	Constraint
prodID	Integer		NOT NULL
supID	Integer		NOT NULL

```
%%sql
```

```
# create the support table
CREATE TABLE supplier_to_product (
    prodID    INT NOT NULL,
```

```
    supID      INT NOT NULL  
);
```

Neither `prodID` nor `supID` is a good primary key in this case, as we can have both appearing multiple times in the table.

However, a good primary key for this table is the combination (`prodID, supID`) which is unique, and thus, a good key.

We can alter the table to add the primary key statement:

```
ALTER TABLE <TABLE>  
ADD PRIMARY KEY (<ATTRIBUTE 1>, <ATTRIBUTE 2>);
```

```
%%sql  
  
# alter the table  
ALTER TABLE supplier_to_product  
ADD PRIMARY KEY (prodID, supID);
```

```
%%sql  
  
# inspect the table schema  
DESCRIBE supplier_to_product;
```

Finally, we can link the 3 tables together by stating which attributes act as the foreign keys in the `supplier_to_product` table, and to which attributes in the parent tables they refer to.

- `prodID` points to the `productID` attribute of `products` table
- `supID` points to the `supplierID` attribute of `supplier` table

This translates into: - `prodID` is a foreign key for `supplier_to_product` - `subID` is a foreign key for `supplier_to_product` - the combination (`prodID, supID`) is the primary key of `supplier_to_product`

We can declare an attribute as a foreign key by altering the table:

```
ALTER TABLE <TABLE>  
ADD FOREIGN KEY (<ATTRIBUTE IN THIS TABLE>) REFERENCES <OTHER  
    ↪ TABLE>(<ATTRIBUTE IN OTHER TABLE>);
```

```
%%sql  
  
# alter the table
```



```
ALTER TABLE supplier_to_product
ADD CONSTRAINT FK_prod
FOREIGN KEY (prodID) REFERENCES products(productID);
```

```
%%sql
```

```
# alter the table
ALTER TABLE supplier_to_product
ADD CONSTRAINT FK_sup
FOREIGN KEY (supID) REFERENCES suppliers(supplierID);
```

```
%%sql
```

```
# inspect the table schema
DESCRIBE supplier_to_product;
```

7.5. 04 Inserting Updating and Deleting records

The tables are currently empty. We can inspect their content by performing a simple “read” query using `SELECT`. We will discuss later the full syntax of the `SELECT` statement... for the time being, it’s important to realize that this command will show the entire content of all columns of the table we are inspecting.

```
SELECT * FROM <TABLE>;
```

To insert a single record into a table we can proceed in either of the two following ways:

1. By explicitly specifying the names of the columns when passing the tuple containing the values

```
INSERT INTO <TABLE> (<COLUMN1>, <COLUMN2>, <COLUMN3>, ...)
VALUES (<VALUE1>, <VALUE2>, <VALUE3>, ...);
```

This will work also in the case attributes with a default value will not be explicitly passed

2. By implicitly using all the columns when passing the tuple containing the values

```
INSERT INTO <TABLE>
VALUES (<VALUE1>, <VALUE2>, ..., <VALUEn>);
```

NB: 1. is the usually recommended way to perform a data insertion in SQL, as (pretty much always in coding) **explicit is better than implicit**

Using the `mysql.connector` we can formulate the same data insertion in a more Pythonic way, e.g. to include this statement programmatically into a loop or a function call:

```
sql_ = "INSERT INTO <TABLE> (<COLUMN1>, <COLUMN2>, <COLUMN3>, ...) VALUES
      ↪ (%s, %s, %s, ...)"
val_ = (<VALUE1>, <VALUE2>, <VALUE3>, ...)
db_cursor.execute(sql_, val_)
db.commit()
```

Being an active change to the database instance, remember to commit the insert.

7.5.0.1. Perform a simple data insertion

- Verify that the product table is actually empty.
- Insert the following list of products into the `products` table.
- Check the content of the table after the transaction to verify that both the `AUTO_INCREMENT` key and the default values have been taken into account.

productName	unitPrice	unitsInStock	isDiscontinued
"Founders Porter"	4.9	12	0

```
%%sql

# retrieve the content of the table
SELECT * FROM products;
```

```
%%sql

# insert records into the table
INSERT INTO products (productName, unitPrice, unitsInStock)
VALUES ('Founders Porter',4.9 ,12);
```

```
%%sql

# retrieve the content of the table
SELECT * FROM products;
```

Multi-values insert can be performed in a similar way by separating multiple tuples with a comma:

```

INSERT INTO <TABLE> (<COLUMN1>, <COLUMN2>, <COLUMN3>, ...)
VALUES
(<VALUE11>, <VALUE12>, <VALUE13>, ...),
(<VALUE21>, <VALUE22>, <VALUE23>, ...),
(<VALUE31>, <VALUE32>, <VALUE33>, ...);

```

The Python-based command is similar to the previous one, but it drops the `execute` in favor of `executemany`, using a list of tuples as values:

```

sql_ = "INSERT INTO <TABLE> (<COLUMN1>, <COLUMN2>, <COLUMN3>, ...) VALUES
      ↪ (%s, %s, %s, ...)"
val_ = [
    (<VALUE11>, <VALUE12>, <VALUE13>, ...),
    (<VALUE21>, <VALUE22>, <VALUE23>, ...),
    (<VALUE31>, <VALUE32>, <VALUE33>, ...) ]
db_cursor.executemany(sql_, val_)
db.commit()

```

Add a few more products to the `product` table:

productName	unitPrice	unitsInStock	isDiscontinued
"Punk IPA"	5.2	45	
"HB Dunkel"	4.5	250	
"Forst Xmas"	8.0	2	
"Damm Complot"	3.7	120	

```

# insert records into the table
sql_ = "INSERT INTO products (productName, unitPrice, unitsInStock) VALUES
      ↪ (%s, %s, %s)"
val_ = [
    ('Punk IPA',      5.2,  45),
    ('HB Dunkel',    4.5, 250),
    ('Forst Xmas',   8.0,   2),
    ('Damm Complot', 3.7, 120)]
dbc.executemany(sql_, val_)
db.commit()

dbc.execute("SELECT * FROM products");
dbc.fetchall()

```

Updating entries is a delicate issue in databases, as we can potentially modify all records associated with a number of conditions. This makes the update operation quite powerful and possibly error-prone if not performed with extreme caution.

```
UPDATE <TABLE>
SET <ATTRIBUTE> = <NEW VALUE>
WHERE <CONDITION>;
```

or

```
sql_ = "UPDATE <TABLE> SET <ATTRIBUTE> = <NEW VALUE> WHERE <CONDITION>"
db_cursor.execute(sql)
db.commit()
```

7.5.0.2. Let's update a single record with a “very explicit” condition statement:

- Assign isDiscontinued equal to 1 to the “Forst Xmas” product.
- Check if the update has taken place.

```
%%sql

# update the content of the table
UPDATE products
SET isDiscontinued = 1
WHERE productName = "Forst Xmas";
```

```
%%sql

SELECT * FROM products;
```

Similarly, deleting entries is quite risky if not done with plenty of caution, as a wrong WHERE condition might inadvertently lead to the deletion of a large part of your database records.

Remember, with DELETE, we remove entire records (i.e., tuples) from the database, not just the content of the WHERE conditional statement.

```
DELETE FROM <TABLE>
WHERE <CONDITION>;
```

or

```
sql_ = "DELETE FROM <TABLE> WHERE <CONDITION>"
db_cursor.execute(sql)
db.commit()
```

7.5.0.3. Delete all records under a set of conditions:

- Delete all products whose name begins with ‘F’ and ends with ‘s’.
- Check if the deletion has taken place.

```
%%sql

# delete records from the table
DELETE FROM products
WHERE productName LIKE "F%s";
```

```
%%sql

# retrieve the content of the table
SELECT * FROM products;
```

7.5.0.4. Populate the suppliers table according to the following list

supplierName	countryCode	email
"A hop in the park"	"USA"	"contactme@ahop.com"
"Allez Hops! Cave à bière"	"FRA"	"itsme@allezhops.com"
"Wild Hops"	"ITA"	"myemail@wildhops.it"
"Beer Paradise"	"ITA"	"beer@beerparadise.it"

```
%%sql

# insert records into the table
INSERT INTO suppliers (supplierName, countryCode, email) VALUES
("A hop in the park","USA","contactme@ahop.com"),
("Allez Hops! Cave à bière","FRA","itsme@allezhops.com"),
("Wild Hops","ITA","myemail@wildhops.it"),
("Beer Paradise","ITA","beer@beerparadise.it");
```

```
%%sql

# retrieve the content of the table
SELECT * FROM suppliers;
```

Complete the database with the two additional steps:

- Update the “A hop in the park” email address to “anewemail@ahop.com”

7. RDBMS - Hands on MySQL

- Populate the table of connections between products and suppliers
 - Remember that the `supplier_to_product` table is linked to the other two via the respective Foreign Keys

supplier	products provided
"A hop in the park"	"Punk IPA", "HB Dunkel"

```
sql_ = "UPDATE suppliers SET email = %s WHERE supplierName LIKE %s;"
val_ = ('anewemail@ahop.com', 'A hop%')
dbc.execute(sql_, val_)
db.commit()

dbc.execute("SELECT * FROM suppliers");
for _ in dbc:
    print(_)
```

```
%%sql
```

```
SELECT * FROM products;
```

```
%%sql
```

```
SELECT * FROM suppliers;
```

```
%%sql
```

```
SELECT * FROM supplier_to_product;
```

```
%%sql
```

```
INSERT INTO supplier_to_product
VALUES
(2,1),
(3,1);
```

```
%%sql
```

```
SELECT * FROM supplier_to_product;
```

Export the DB you have created into a `.sql` file, and store it for later use.

1. Connect to the “server” Docker container using bash
2. Dump the database using the `mysqldump` command

```
$ mysqldump -u <USER> -p <DB NAME> > <FULL/PATH/TO/NEW/DB_DUMP.sql>
```

3. use the folder `/mapd-workspace/dbs` to store the file, as it’s the one currently visible from both in- and outside the container

7.6. 04 Basic queries

The `SELECT` statement allows us to perform queries and retrieve data from a DB.

We can perform queries on a single table in the DB, as well as perform complex queries that span multiple interconnected tables, with one-to-one, one-to-many, and many-to-many relationships across records.

Any `SELECT` statement is composed of at least:

```
SELECT <TABLE1.ATTRIBUTE1>, <TABLE2.ATTRIBUTE2>  --# the list of attributes
    ↪ to show                                     --# the * wildcard symbol
                                                    ↪ means ALL attributes

FROM <TABLE1>, <TABLE2>                          --# the table, or list of
    ↪ tables                                       --# from which to extract
                                                    ↪ the attributes

WHERE <CONDITIONS>                               --# (optional) condition
    ↪ statement
```

The same structure applies to Python-based connectors:

```
db_cursor.execute("""
    SELECT <TABLE1.ATTRIBUTE1>, <TABLE2.ATTRIBUTE2>
    FROM <TABLE1>, <TABLE2>
    WHERE <CONDITIONS>
""")
res_ = db_cursor.fetchall()
for _ in res_:
    print(_)
```

As an alternative, `db_cursor.fetchone()` can be used to print out only one of possibly many records.

7. RDBMS - Hands on MySQL

7.6.0.1. Get the email of all suppliers from Italy

```
%%sql

SELECT email
FROM suppliers
WHERE countryCode="ITA";
```

7.6.0.2. Get all information for products with unit price greater than 4

```
%%sql

SELECT *
FROM products
WHERE unitPrice>4;
```

7.6.0.3. Select all products with unit cost less than 5, and list them in descending order of units in stock

```
%%sql

SELECT *
FROM products
WHERE unitPrice<5
ORDER BY unitsInStock DESC;
```

Group by is a common statement that allows us to perform data aggregation.

If you're familiar with **pandas**, this should not be new to you.

Just remember: 1. No group by statement is possible without an aggregation function. 2. Group by operations can be quite computationally expensive. In the case of large databases, group by operations might require a long time to execute.

7.6.0.4. Plot the distribution of suppliers by country

```
%%sql

DESCRIBE suppliers;
```

```
res_ = %sql SELECT COUNT(countryCode) AS count, countryCode FROM suppliers
↪ GROUP BY countryCode;
```



```
print (res_);

res_.bar();
```

The main strength of Relational DBs is the possibility of matching information from different tables through **Join** operations.

A join, either performed explicitly or implicitly is:

1. Computationally expensive. In the case of large databases, join operations might require a long time to execute.
2. Very easy to be done wrong, thus resulting in non-meaningful results.

Be sure of which attributes (most often keys) you are performing a join operation on.

7.6.0.5. Join the `supplier_to_product` table with the `suppliers` table

```
%%sql

SELECT *
FROM suppliers, supplier_to_product
WHERE supplier_to_product.supID = suppliers.supplierID;
```

```
%%sql

SELECT *
FROM suppliers
INNER JOIN supplier_to_product ON supplier_to_product.supID =
    ↪ suppliers.supplierID;
```

```
%%sql

SELECT *
FROM suppliers
LEFT JOIN supplier_to_product ON supplier_to_product.supID =
    ↪ suppliers.supplierID;
```

```
%%sql

SELECT *
FROM suppliers
```

```
RIGHT JOIN supplier_to_product ON supplier_to_product.supID =
  ↪ suppliers.supplierID;
```

We can concatenate multiple join operations to extract higher-level information out of the connections stored in the database.

7.6.0.6. Add the corresponding product names to the previous join

```
%%sql

SELECT *
FROM suppliers
INNER JOIN supplier_to_product ON supplier_to_product.supID =
  ↪ suppliers.supplierID
INNER JOIN products ON supplier_to_product.prodID = products.productID;
```

The conjunction of SELECT, GROUP BY, and JOIN statements will allow us to extract non-trivial information out of any Relational database.

7.6.0.7. Add the following records to the supplier_to_product table, and perform the following queries:

supplier	products provided
"Wild Hops"	"Founders Porter", "Punk IPA", "HB Dunkel", "Damm Complot"
"Beer Paradise"	"Punk IPA", "Damm Complot"
"Allez Hops! Cave à bière"	"Founders Porter", "HB Dunkel"

1. show all suppliers of "Damm Complot"
2. retrieve the email address of all possible suppliers of "Punk IPA"
3. list the unit price of all products supplied by Italian firms, listed in descending order (of unit price)

```
sql_ = "INSERT INTO supplier_to_product (prodID, supID) VALUES (%s, %s)"
val_ = [
    # supplier Allez Hops
    (1, 2),
    (3, 2),

    # supplier Wild Hops
    (1, 3),
    (2, 3),
```

```

    (3, 3),
    (5, 3),

    # supplier Beer Paradise
    (2, 4),
    (5, 4),
]

dbc.executemany(sql_, val_)
db.commit()

dbc.execute("SELECT * FROM supplier_to_product");
for _ in dbc:
    print(_)

```

```

%%sql

SELECT *
FROM suppliers
JOIN supplier_to_product ON supplier_to_product.supID = suppliers.supplierID
JOIN products ON supplier_to_product.prodID = products.productID
WHERE productName = "Punk IPA";

```

```

%%sql

SELECT suppliers.supplierName
FROM supplier_to_product
INNER JOIN suppliers ON supplier_to_product.supID = suppliers.supplierID
INNER JOIN products ON supplier_to_product.prodID = products.productID
WHERE products.productName = "Punk IPA";

```

```

%%sql

SELECT suppliers.email
FROM supplier_to_product
INNER JOIN suppliers ON supplier_to_product.supID = suppliers.supplierID
INNER JOIN products ON supplier_to_product.prodID = products.productID
WHERE products.productName = "Punk IPA";

```

```
%%sql
```

```
SELECT DISTINCT products.unitPrice, products.productName
FROM supplier_to_product
INNER JOIN suppliers ON supplier_to_product.supID = suppliers.supplierID
INNER JOIN products ON supplier_to_product.prodID = products.productID
WHERE suppliers.countryCode = "ITA"
ORDER BY unitPrice DESC;
```

7.7. 05 Queries on well-formed DBs

In MySQL, we can load entire databases from external files.

The files will contain the full database structure (tables and constraints) and will populate all tables with all records.

To load a database into the MySQL server from the interactive Docker shell, we can simply do the following:

1. Create a new database named `world`.
2. Import the content of the file `/mapd-workspace/dbs/world.sql` into the newly created database via:

```
$ mysql -u root -p world < /mapd-workspace/dbs/world.sql
```

NB: The `/mapd-workspace` directory has been defined as a Docker Volume and is a shared location with your computer.

7.7.0.1. Using the `world` DB, perform queries to answer the following questions.

Additional references can be found here <https://dev.mysql.com/doc/world-setup/en/>

Use a **single query** to answer the questions:

1. Which country has the largest population?
2. Get the total number of cities in Canada.
3. Select the country with the most cities.
4. List all the languages spoken in the Southeast Asia region.
5. Identify the 5 most spoken languages by number of countries.
6. What is the total population of the cities in the Caribbean region?
7. What Countries have a City named "Kingston"?
8. Plot the population density by country, limiting to the 25 with the least density.
9. Plot the GNP (Gross Nation Product) of all countries that become independent between 1850 and 1910, limiting to the 10 with the highest GNP.
10. Create a scatter plot of the life expectancy vs the GNP of all countries that have Spanish as their Official language.

7.7.0.1.1. Suggestions:

- Start by inspecting the database tables' structure
- When in doubt, check the table content by selecting only a few lines (use `LIMIT` to avoid returning all records)
- Before finalizing the query, you can always perform “intermediate” queries to check if complex operations (e.g. `GROUP BY` and `JOIN`) are stated correctly

```
%%sql
```

```
USE world;
```

```
SHOW TABLES;
```

```
%%sql
```

```
DESCRIBE city
```

```
%%sql
```

```
SELECT *  
FROM city  
LIMIT 5
```

```
%%sql
```

```
DESCRIBE country
```

7.7.0.2. 1 - Which country has the largest population?

```
%%sql
```

```
# SELECT ,  
# FROM  
# ORDER BY DESC  
# LIMIT 1;
```

7. RDBMS - Hands on MySQL

7.7.0.3. 2 - Get the total number of cities in Canada.

```
%%sql

# SELECT AS CitiesInCanada
# FROM
# WHERE ;
```

7.7.0.4. 3 - Select the country with the most cities.

```
%%sql

# SELECT
# FROM
# GROUP BY
# ORDER BY
# LIMIT ;
```

7.7.0.5. 4 - List all the languages spoken in the Southeast Asia region.

```
%%sql

# SELECT DISTINCT
# FROM
# INNER JOIN ON
# WHERE
# ORDER BY ;
```

7.7.0.6. 5 - Identify the 5 most spoken languages by number of countries.

```
%%sql

# SELECT
# FROM
# GROUP BY
# ORDER BY
# LIMIT ;
```

7.7.0.7. 6 - What is the total population of the cities in the Caribbean region?

```
%%sql

# SELECT
# FROM
# (LEFT) JOIN ON
# WHERE ;
```

7.7.0.8. 7 - What Countries have a City named “Kingston”?

```
%%sql

# SELECT
# FROM
# (RIGHT) JOIN ON
# WHERE ;
```

7.7.0.9. 8 - Plot the population density by country, limiting to the 25 with the least density.

The population density is the total Population of a country divided by the SurfaceArea. We can compute this quantity on the fly during a SELECT statement and rename it with an ALIAS for convenience.

```
res_ = %sql SELECT      FROM      ORDER BY      LIMIT ;

print(res_)

plt.figure(figsize=(16,3))
res_.bar();
plt.xticks(rotation='vertical');
```

7.7.0.10. 9 - Plot the GNP (Gross Nation Product) of all countries that become independent between 1850 and 1910, limiting to the 10 with the highest GNP

We can use BETWEEN a AND b to select only those entries between two extremes;

```
res_ = %sql SELECT      FROM      WHERE      ORDER BY      LIMIT 10;

print(res_)
```

```
plt.figure(figsize=(16,3))
res_.bar();
plt.xticks(rotation='vertical');
```

7.7.0.11. 10 - Create a scatter plot of the life expectancy vs the GNP of all countries that have Spanish as their Official language

```
res_ = %sql SELECT \
            FROM \
            RIGHT JOIN ON \
            WHERE \
            AND ;

print(res_)

plt.figure(figsize=(8,6))
for _ in res_:
    plt.scatter(_[0],_[1],s=50);
    plt.annotate(_[2], (float(_[0])*1.002,float(_[1])*1.002));
plt.ylabel('GNP')
plt.xlabel('LifeExpectancy')
```

7.8. 06 More on queries

Let's import a well-structured database, with several tables and relationships, to be used as a sandbox for further exercises on queries.

The database is freely available and is taken from the IMDb (Internet Movie Database) website (<https://www.imdb.com/interfaces/>), although severely slimmed down.

This database contains several tables with information about movies, TV shows, people involved in the entertainment industry, and more, spanning from the 1890s to recent years.

To simplify the execution of queries, we will work with a slimmed-down version of the entire database, with still plenty of records for us to extract interesting information.

To start working with the IMDb database, we need to import it into our environment. The database .sql file should be downloaded from the link on Moodle and copied into the `../mysql/dbs` folder of the GitHub repository so that it will be found in the mounted volume at the location: `/mapd-workspace/dbs/IMDb_pruned.sql`.

1. Create a new DB named IMDb
2. Import the content of the file `/mapd-workspace/dbs/IMDb_pruned.sql` into the newly created DB (this might take a minute or two...)

7.8.1. Check which tables are available**7.8.2. Describe the content of table Names_****7.8.3. Study the distribution of the year of birth of all people included in the database**

1. Start by limiting the query to 1000 entries
 1. Draw a histogram of the year of birth
 2. Draw a second histogram with the year of birth of the 1000 youngest people in the list

1.A

```
res_ = %sql SELECT D.name_id, COUNT() AS Directorsperyear\
          FROM Directors D\
```

1.B

2. Extend the query to all records in the table
 1. Check for outliers (inspect min and max)
 2. If any outlier is found, limit the query to a reasonable range (say... at least people born after 1850)
 3. Draw a new histogram with the year of birth
 4. Compute the mean and standard deviation of the year of birth

2.A

2.B

2.C

2.D

7.8.4. Movie stats

1. Plot the number of titles available in the database for each year over the past 25 years.
2. List the names of all actors who have portrayed the character Spider-Man in movies, along with the corresponding movie titles.
3. List and count the top 10 people who have appeared the most in movies directed by Wes Anderson.

7.8.4.1. 1 - Plot the number of titles available in the database for each year over the past 25 years.

7.8.4.2. 2 - List the names of all actors who have portrayed the character Spider-Man in movies, along with the corresponding movie titles.

7.8.4.3. 3 - List and count the top 10 people who have appeared the most in movies directed by Wes Anderson.

7.8.5. Is IMDB only about movies?

1. Check the number of different media types contained in the database
2. Plot a histogram summarizing the number of titles per media type
3. Find the name and rating of the oldest videogame stored in the database

7.8.5.1. 1 - Check the number of different media types contained in the database

7.8.5.2. 2 - Plot a histogram summarizing the number of titles per media type

7.8.5.3. 3 - Find the name and rating of the oldest videogame stored in the database

7.8.6. Identify interesting outliers

1. List the names of all movies whose titles begin with “The” and have a runtime between 3 and 4 hours.
2. What actor had the most roles as ‘Self’?
3. Identify what TV series had more than 1,000 episodes, and identify in what year it was aired the first- and last-episode

7.8.6.1. 1 - List the names of all movies whose titles begin with “The” and have a runtime between 3 and 4 hours.

7.8.6.2. 2 - What actor had the most roles as ‘Self’?

7.8.6.3. 3 - Identify what TV series had more than 1,000 episodes, and identify in what year it was aired the first episode

7.8.7. It’s all about the ratings

1. Group all movies by decade and plot their overall rating distribution trend. Limit the query to the movies with at least 1000 ratings.
2. Plot the average rating of each episode of “Games of Thrones”. Group by season and compute the average and standard deviation for each season
3. Plot the total number of movies produced per year, divided by genre (Action, Comedy, Drama, Thriller, Fantasy), and the yearly rating average per genre

- 7.8.7.1. 1 - Group all movies by decade and plot their overall rating distribution trend. Limit the query to the movies with at least 1000 ratings.
- 7.8.7.2. 2 - Plot the average rating of each episode of “Games of Thrones”. Group by season and compute the average and standard deviation for each season.
- 7.8.7.3. 3 - Plot the total number of movies produced per year, divided by genre (Action, Comedy, Drama, Thriller, Fantasy), and the yearly rating average per genre

7.9. SQL String Functions Reference

Function	Description	Example
CONCAT(a, b, ...)	Concatenate strings	CONCAT(first_name, ' ', last_name)
SUBSTRING(str, pos, len)	Extract a substring	SUBSTRING(name, 1, 3)
UPPER(str) / LOWER(str)	Change case	UPPER('hello') → 'HELLO'
TRIM(str)	Remove leading/trailing spaces	TRIM(' hello ') → 'hello'
LEFT(str, n) / RIGHT(str, n)	First/last n characters	LEFT('hello', 3) → 'hel'
LENGTH(str)	String length in bytes	LENGTH('hello') → 5
REPLACE(str, from, to)	Replace all occurrences	REPLACE('aababc', 'ab', 'X') → 'aXXc'
LOCATE(substr, str)	Find position of substring	LOCATE('ll', 'hello') → 3
FORMAT(num, d)	Format number with d decimal places	FORMAT(1234567.891, 2) → '1,234,567.89'

```
-- Example: build a full name and show first 10 characters
SELECT UPPER(CONCAT(first_name, ' ', last_name)) AS full_name,
       SUBSTRING(bio, 1, 100) AS bio_preview
FROM Authors;
```

7.10. SQL Date and Time Functions Reference

Function	Description	Example
NOW() / CURRENT_TIMESTAMP	Current date and time	NOW() → 2024-05-28 14:32:00
CURRENT_DATE	Today's date	CURRENT_DATE → 2024-05-28

Function	Description	Example
DATE_ADD(date, INTERVAL n unit)	Add an interval	DATE_ADD('2024-01-01', INTERVAL 30 DAY)
DATE_SUB(date, INTERVAL n unit)	Subtract an interval	DATE_SUB(NOW(), INTERVAL 1 YEAR)
DATEDIFF(d1, d2)	Days between two dates	DATEDIFF('2024-12-31', '2024-01-01') → 365
TIMESTAMPDIFF(unit, t1, t2)	Difference in specified unit	TIMESTAMPDIFF(MONTH, '2023-01-01', '2024-05-01') → 16
DATE_FORMAT(date, fmt)	Format a date	DATE_FORMAT(NOW(), '%Y-%m-%d')
EXTRACT(unit FROM date)	Extract a component	EXTRACT(YEAR FROM '2024-05-28') → 2024
YEAR(d) / MONTH(d) / DAY(d)	Extract components	YEAR(NOW()) → 2024

```
-- Example: find orders placed in the last 30 days
SELECT order_id, order_date, total
FROM Orders
WHERE order_date >= DATE_SUB(CURRENT_DATE, INTERVAL 30 DAY)
ORDER BY order_date DESC;
```

7.11. SQL Query Exercises

Exercise 7.1

Write a single SQL query to retrieve all wines made with 'Merlot' or 'Pinot Noir' grapes since 2010, sorted by alcohol percentage in descending order.

Solution

(Assumed schema — not a real one: *Wines(name, grape, year, alcohol).*)

```
SELECT *
FROM Wines
WHERE grape IN ('Merlot', 'Pinot Noir')
AND year >= 2010
ORDER BY alcohol DESC;
```

Exercise 7.2

Write a single SQL query to retrieve the names of professors whose average student grade across all their courses is greater than 24. Consider only sufficient grades (at least 18).

Solution

(Assumed schema — not a real one: *Professors*(*prof_id*, *name*), *Courses*(*course_id*, *prof_id*), *Exams*(*student_id*, *course_id*, *grade*).)

```
SELECT p.name
FROM Professors p
JOIN Courses c ON c.prof_id = p.prof_id
JOIN Exams e ON e.course_id = c.course_id
WHERE e.grade >= 18
GROUP BY p.prof_id, p.name
HAVING AVG(e.grade) > 24;
```

Exercise 7.3

Write a single SQL query to retrieve the maximum and average employee salary per department, limited to departments located in 'New York' and 'Chicago'. Sort the result by department name.

Solution

(Assumed schema — not a real one: *Departments*(*dept_id*, *name*, *location*), *Employees*(*emp_id*, *dept_id*, *salary*).)

```
SELECT d.name           AS department,
       MAX(e.salary)    AS max_salary,
       AVG(e.salary)    AS avg_salary
FROM Departments d
JOIN Employees e ON e.dept_id = d.dept_id
WHERE d.location IN ('New York', 'Chicago')
GROUP BY d.dept_id, d.name
ORDER BY d.name;
```

Exercise 7.4

Write a single SQL query to retrieve earthquakes with magnitudes between 6.5 and 7.7 that occurred after 1940 from the *quaketable*, sorted by epicentral depth (greater than 0) from highest to lowest.

Solution

(Assumed schema — not a real one: *quaketable(id, magnitude, year, depth).*)

```
SELECT *
FROM quaketable
WHERE magnitude BETWEEN 6.5 AND 7.7
      AND year > 1940
      AND depth > 0
ORDER BY depth DESC;
```

Exercise 7.5

Provide SQL queries demonstrating the following:

- One query using WHERE, AND, and OR.
- Two queries each using JOIN, ORDER BY (A–Z by name), GROUP BY, and the LIKE operator.

Solution

(All schemas below are hypothetical, not real.)

```
-- WHERE / AND / OR
SELECT *
FROM Employees
WHERE department = 'Sales'
      AND (salary > 50000 OR years_experience > 5);

-- JOIN + ORDER BY + GROUP BY + LIKE (1)
SELECT d.name AS department, COUNT(e.emp_id) AS headcount
FROM Departments d
JOIN Employees e ON e.dept_id = d.dept_id
WHERE e.title LIKE 'Senior%'
GROUP BY d.name
ORDER BY d.name ASC;
```

```
-- JOIN + ORDER BY + GROUP BY + LIKE (2)
SELECT a.name AS author, COUNT(b.book_id) AS books_written
FROM Authors      a
JOIN BookAuthors  ba ON ba.author_id = a.author_id
JOIN Books        b  ON b.book_id    = ba.book_id
WHERE b.title LIKE '%Physics%'
GROUP BY a.name
ORDER BY a.name ASC;
```

Exercise 7.6

Write a single SQL query to retrieve the average years of experience of employees who are older than 30 years and have a salary greater than \$50,000, grouped by department.

Solution

(Assumed schema — not a real one: *Employees(emp_id, department, age, salary, years_experience).*)

```
SELECT department,
       AVG(years_experience) AS avg_years_experience
FROM Employees
WHERE age > 30
       AND salary > 50000
GROUP BY department;
```

Exercise 7.7

Using the world database, write a single SQL query to retrieve the average population of cities in each country that has a population greater than 10 million, grouped by continent.

Solution

(Standard world schema — *country(Code, Continent, Population), city(CountryCode, Population).*)

```
SELECT c.Continent,
       AVG(ci.Population) AS avg_city_population
FROM country c
JOIN city    ci ON ci.CountryCode = c.Code
WHERE c.Population > 10000000
GROUP BY c.Continent;
```

Exercise 7.8

Using the `world` database, write a single SQL query to retrieve the total number of languages spoken in countries that are located in Asia and have a population greater than 50 million, grouped by region.

Solution

(Standard `world` schema — `country(Code, Continent, Region, Population)`, `countrylanguage(CountryCode, Language)`.)

```
SELECT c.Region,
       COUNT(DISTINCT cl.Language) AS total_languages
FROM country      c
JOIN countrylanguage cl ON cl.CountryCode = c.Code
WHERE c.Continent = 'Asia'
      AND c.Population > 50000000
GROUP BY c.Region;
```

Exercise 7.9

Using the IMDb database, write a single SQL query to retrieve the average movie rating of movies in the “Action” genre that have a runtime of more than 90 minutes, grouped by year.

Solution

(Assumed IMDb-style schema — not the official one: `Movies(movie_id, year, runtime)`, `Genres(movie_id, genre)`, `Ratings(movie_id, rating)`.)

```
SELECT m.year,
       AVG(r.rating) AS avg_rating
FROM Movies m
JOIN Genres g ON g.movie_id = m.movie_id
JOIN Ratings r ON r.movie_id = m.movie_id
WHERE g.genre = 'Action'
      AND m.runtime > 90
GROUP BY m.year;
```

Exercise 7.10

Using the IMDb database, write a single SQL query to retrieve the top 5 persons who have worked in the most movies, ordered by the number of movies they have worked in (descending).

Solution

(Assumed schema — not real: *Persons*(*person_id*, *name*), *Crew*(*person_id*, *movie_id*).)

```
SELECT p.name,
       COUNT(DISTINCT c.movie_id) AS movie_count
FROM Persons p
JOIN Crew c ON c.person_id = p.person_id
GROUP BY p.person_id, p.name
ORDER BY movie_count DESC
LIMIT 5;
```

Exercise 7.11

Using the IMDb database, retrieve the average rating of movies in the “Comedy” genre that were released after 2000 and have more than 100,000 votes, grouped by director. Include only directors who have directed more than 5 such movies.

Solution

(Assumed schema — not real: *Movies*(*movie_id*, *year*, *votes*), *Genres*(*movie_id*, *genre*), *Ratings*(*movie_id*, *rating*), *Crew*(*person_id*, *movie_id*, *role*), *Persons*(*person_id*, *name*).)

```
SELECT p.name                AS director,
       AVG(r.rating)         AS avg_rating
FROM Persons p
JOIN Crew c ON c.person_id = p.person_id
              AND c.role     = 'director'
JOIN Movies m ON m.movie_id = c.movie_id
JOIN Genres g ON g.movie_id = m.movie_id
JOIN Ratings r ON r.movie_id = m.movie_id
WHERE g.genre = 'Comedy'
      AND m.year > 2000
      AND m.votes > 100000
GROUP BY p.person_id, p.name
HAVING COUNT(DISTINCT m.movie_id) > 5;
```

Exercise 7.12

Using the IMDb database, retrieve the total number of movies and the average runtime of movies in each genre, considering only movies released after 2010. Group by genre. Exclude genres with 10 or fewer movies and genres where the average runtime is less than 90 minutes.

Solution

(Assumed schema — not real: *Movies(movie_id, year, runtime)*, *Genres(movie_id, genre)*.)

```
SELECT g.genre,
       COUNT(DISTINCT m.movie_id) AS total_movies,
       AVG(m.runtime)              AS avg_runtime
FROM Movies m
JOIN Genres g ON g.movie_id = m.movie_id
WHERE m.year > 2010
GROUP BY g.genre
HAVING COUNT(DISTINCT m.movie_id) > 10
      AND AVG(m.runtime) >= 90;
```

8. NoSQL Databases

The SQL language is perfectly suited to relational databases, which were predominant until the 1990s, when the amount of data generated by the explosion of the internet and web applications became increasingly large. In 1998, computer scientist Carlo Strozzi coined the term *NoSQL* by creating a database management system that did not require the use of SQL, promoting the idea of open source databases that departed from the traditional relational model.

8.1. Database Partitioning

A first important obstacle in managing a relational database arises with the growth of the quantity of records, which forces the **partitioning** of the databases — their subdivision into smaller parts.

Vertical partitioning: all records are saved on each disk so that the attributes are divided between disks. This is not a problem for the relational model, since the various partitions can be seen as different tables with the same primary key.

Horizontal partitioning (also called **sharding**): the records are divided among the different disks so that all attributes associated with the same record are maintained together. This case is not supported by traditional RDBMSs, which require each table to be entirely included in a single memory element.

The idea is to apply a hash function to the primary key, obtaining a number, then perform integer division with the number of available partitions P : the remainder — a number between 0 and $P - 1$ — indicates the disk on which to store the record. Formally, given a key k :

$$\text{partition} = \text{hash}(k) \bmod P$$

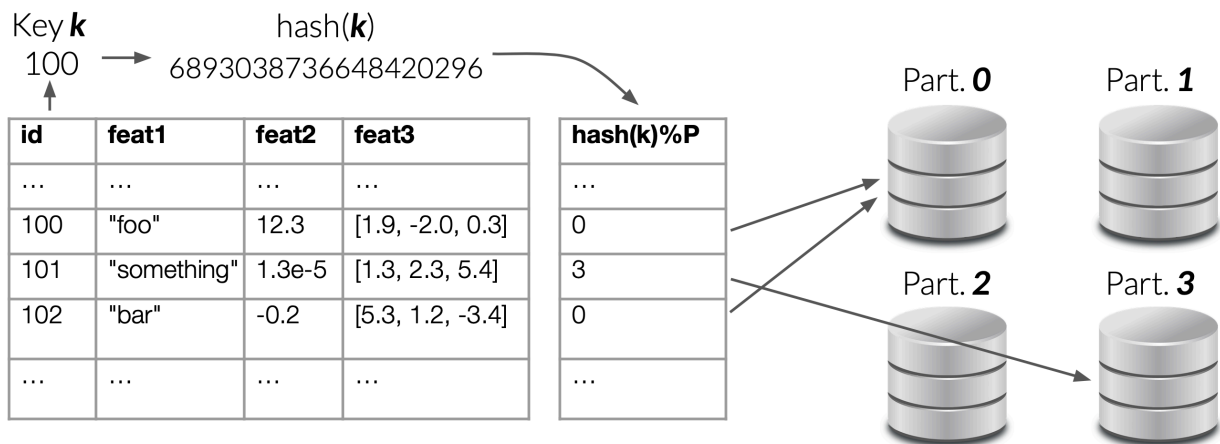


Figure 8.1.: Example of sharding via hashing with integer division and $P = 4$

This method does not respond well in terms of scalability: adding a new disk means going from P to $P + 1$, requiring all integer divisions to be recalculated and all records moved according to the new indexing. Because the modulus changes, nearly **all** K keys must be remapped to different partitions — an $O(K)$ data movement that is prohibitively expensive in large clusters.

8.1.1. Consistent Hashing

The practical solution is **consistent hashing**: instead of using modulo arithmetic, both keys and nodes are mapped to positions on a **circular hash ring** — a continuous space of values from 0 to $2^N - 1$ (for SHA-256, $N = 256$, giving 2^{256} possible positions).

Each node (partition/disk) is assigned one or more positions on the ring. To look up where a key k belongs, compute $\text{hash}(k)$ and walk **clockwise** around the ring; the first node encountered is responsible for that key.

A “sector” of the ring is associated with each disk, and each digest in the ring is “rounded” to the largest value within the sector where the association with the partition takes place.

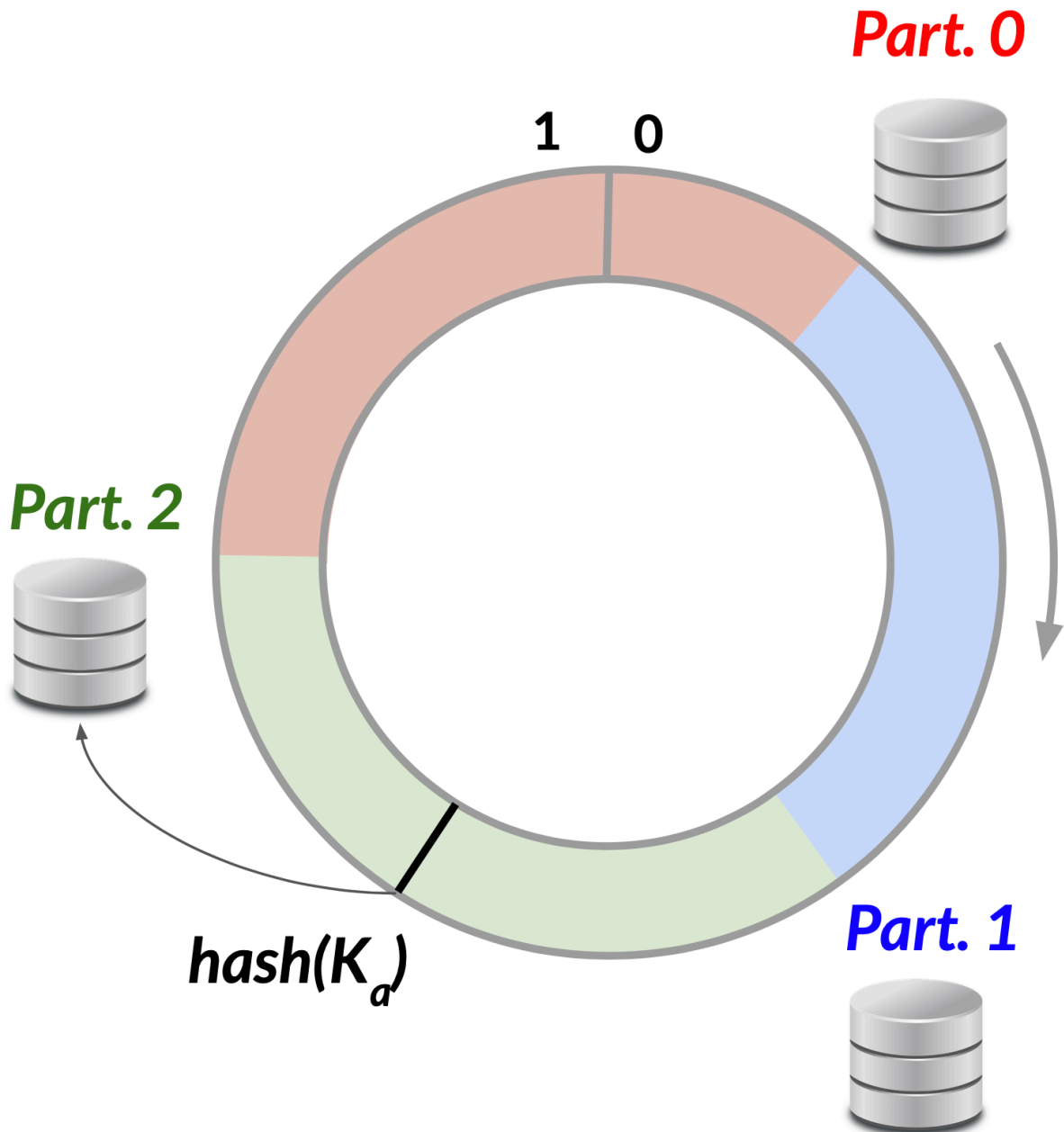


Figure 8.2.: Example of consistent hashing with three disks

When introducing a new disk, it occupies part of a sector previously occupied by another disk. Only the records in that sector need to be transferred — not the entire database. Concretely, only the keys that fall between the new node and its predecessor on the ring need to move — roughly $O(K/N)$ remappings instead of the $O(K)$ required by naive modular hashing.

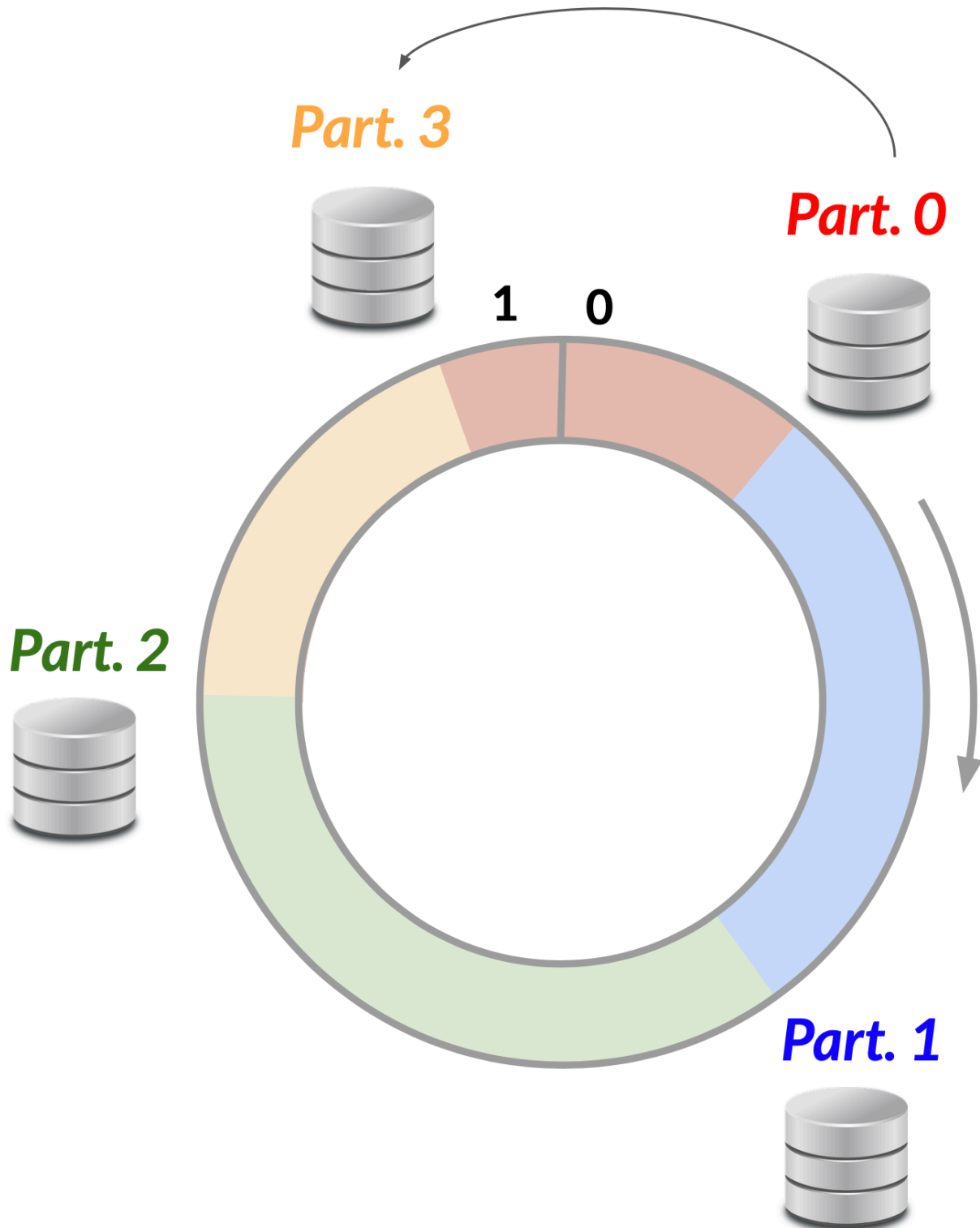


Figure 8.3.: Introduction of a new partition with consequent transfer of records

Virtual nodes: to distribute load more uniformly, a single physical node is mapped to *multiple* positions on the ring (virtual nodes). This smooths out hash-space imbalances and makes rebalancing even more granular when nodes join or leave.

Consistent hashing also lends itself well to **data replication**: the user chooses a replication factor

R ; a record stored at its primary node is additionally copied to the next $R - 1$ nodes clockwise on the ring, ensuring fault tolerance without a separate replication mechanism.

8.2. Characteristics of NoSQL Databases

The four main points that distinguish NoSQL databases from traditional relational ones:

1. **Flexible models:** support for the storage and management of semi-structured and unstructured data, departing from the rigid structured nature of relational databases.
2. **Horizontal scalability:** distribution of databases on multiple nodes of a cluster without significant degradation of performance.
3. **Simplified queries:** languages that do not use complex operations such as JOIN, favoring faster response times especially in distributed environments.
4. **Models with less consistency:** adherence to ACID properties is reduced in favor of the scalability and *availability* necessary in distributed systems.

8.2.1. BASE vs ACID

The transaction model followed by most NoSQL databases is **BASE** — **B**asically **A**vailable, **S**oft state, **E**ventually consistent (a deliberate contrast with ACID as in chemistry):

- **Basically Available:** the individual nodes in a network are generally accessible *almost* always, unlike the ACID model where access could be interrupted due to *isolation* during a read or write operation.
- **Soft state:** the state of a database is not rigidly *persistent* — it may change without a specific intervention by the user, as in the case of data replication across cluster nodes.
- **Eventually consistent:** *consistency* is guaranteed only after a certain time, not necessarily at the exact moment a transaction is concluded.

Where ACID prioritises correctness and strict consistency (at the cost of availability under network partitions), BASE trades strict consistency for higher availability and partition tolerance — occupying the *AP* corner of the CAP theorem. This makes BASE-compliant systems well suited to large-scale distributed deployments where brief inconsistency windows are acceptable.

Note

The last point does not imply that all NoSQL databases are *AP*. Both availability *A* and consistency *C* are guaranteed along a “gradient” — from more stringent conditions (*high availability*) to more labile conditions (*eventual availability*). This flexibility allows the same DBMS to accommodate different services and use cases.

C in CAP $\neq$ C in ACID

The letter **C** appears in both acronyms but means something completely different:

- **C in ACID** (*Consistency*): every transaction leaves the database in a **valid state**

— all integrity constraints (foreign keys, uniqueness, check constraints) are preserved before and after the transaction.

- **C in CAP** (*Consistency*): every **read** receives the **most recent write** or an error. All nodes in the distributed system see the same data at the same time — this is the property of *linearisability* (or *serializability* in a broader sense).

A system can satisfy ACID consistency while violating CAP consistency (e.g. a single-node RDBMS that cannot guarantee up-to-date reads in a network-partitioned cluster), and vice versa.

8.3. NoSQL Database Categories

NoSQL databases are not a monolithic family — they cover several distinct data models:

Category	Core abstraction	Typical strengths
Key-Value Store	Opaque value per key	Ultra-fast reads/writes, easy sharding, in-memory caching
Document Store	Self-describing JSON/BSON document	Flexible schema, rich queries on nested fields
Wide-Column Store	Sparse row with variable columns per row	High-throughput writes, efficient column-family scans
Graph Database	Nodes and labelled edges with properties	Relationship traversal, multi-hop queries
Time Series Database	Value indexed by timestamp	Append-heavy workloads, time-window aggregations; optimised for metrics, sensor readings, and financial ticks. Examples: InfluxDB, Prometheus
Full-Text Search Engine	Inverted index over text documents	Ranked keyword/phrase search over unstructured text. Examples: Elasticsearch, Solr
Vector Database	High-dimensional embedding vectors	Approximate nearest-neighbour (similarity) search

8.4. Examples of NoSQL Databases

8.4.1. Key-Value Databases

The simplest way to store data: a key is associated with each record, similarly to Python dictionaries. Each key can be associated with a different type of data (a number, a string, a list, etc.). This

type of storage is *schema-less*, not requiring metadata to illustrate the structure of the data.

The absence of a schema allows for easy and rapid distribution among the nodes of a network. The key-value model is particularly suitable for datasets requiring a large number of reads and writes in rapid succession, so data are often saved directly in RAM for significantly higher performance. Databases that manage data located in main memory are called **in-memory databases**.

Redis — common commands:

```
# Store a value (string)
SET user:1001 "Alice"

# Retrieve a value
GET user:1001

# Delete a key
DEL user:1001

# Set a key with an expiry time (TTL in seconds)
SET session:abc123 "token_data" EX 3600

# Work with a list
LPUSH myqueue "job1"
RPOP myqueue

# Work with a hash (like a mini-document)
HSET user:1001 name "Alice" age 30
HGET user:1001 name
```

Redis supports five core data types:

Type	Description / typical use case
String	Binary-safe byte sequence; counters, cached HTML fragments, session tokens
Hash	Map of field→value pairs inside one key; user profiles, configuration objects
List	Ordered sequence with O(1) push/pop at both ends; task queues, activity feeds
Set	Unordered collection of unique strings; tag systems, unique visitor tracking
Sorted Set	Set where every member has a numeric score; leaderboards, rate-limiting windows

These types make Redis suitable for caching, session management, rate limiting, leaderboards, and pub/sub messaging.

Notable implementations: Redis, Amazon DynamoDB, Memcached, Apache Ignite.

8.4.2. Document Databases

Instead of saving key-value pairs, document databases save *documents* composed of field-value pairs. These are essentially information encoded in JSON format that are perfectly suited to semi-structured data. The various documents can also be organized into “collections” through a field — usually called *id* — that uniquely identifies the document, just like primary keys in relational databases. Queries are performed using the fields of the documents.

Notable implementations: MongoDB, CouchDB, Amazon DocumentDB, Couchbase.

8.4.3. Time Series Databases

They are designed to store data indexed by timestamps and to group them according to time windows. They are suitable for empirical data produced by sensors and provide functionality to calculate metrics and statistics that help identify trends and patterns.

Notable implementations: InfluxDB, Prometheus, TimescaleDB, OpenTSDB.

8.4.4. FullText Search Engine

Suitable for unstructured data collected as textual documents. The queries — similarly to what search engines like Google do — return a “ranking” of documents on the basis of the matches with the keywords specified by the user.

Notable implementations: Elasticsearch, Apache Solr, OpenSearch, Meilisearch.

8.5. Wide-Column Databases

Wide-column databases (also called column-family stores) organise data as rows with a row key, but unlike relational tables, **each row can have a different set of columns**. Columns are grouped into “column families” and sparse rows are stored efficiently without null padding. This means that different rows genuinely store different columns — absent columns consume no space at all — which is the key advantage over a fixed-schema RDBMS, where every row must carry every column (or an explicit NULL) regardless of whether the value exists.

Row Key	Name	Age	Email	Temperature	Pressure
sensor_001				23.5	1013.2
sensor_002				24.1	
user_001	Alice	30	alice@example.com		

The `sensor_*` rows store only measurement columns; the `user_*` rows store only profile columns. Both coexist in the same table without wasted storage.

Strengths:

- Excellent for **time-series and event data** where columns vary per record
- High write throughput across distributed nodes
- Efficient for queries that scan a column family rather than full rows

Typical use cases: IoT sensor data, financial ticks, clickstream logs, audit trails.

Notable implementations: Apache Cassandra, Apache HBase, Google Bigtable.

8.6. Graph Databases

Graph databases model data as a network of **nodes** (entities) and **edges** (relationships), each of which can carry arbitrary properties. This structure is natural for highly interconnected data where the relationships themselves are first-class objects.

```
(Alice:Person {age:30}) -[:KNOWS]-> (Bob:Person {age:25})
(Bob:Person)           -[:WORKS_AT]-> (Acme:Company {since:2020})
(Alice:Person)         -[:ACTED_IN]-> (Matrix:Movie {year:1999})
```

When to prefer a graph DB over a relational DB:

Relational databases can model graph-like data via self-joins and junction tables, but performance degrades with query depth (e.g., “friends of friends of friends”). Graph databases store adjacency natively and traverse relationships in $O(1)$ per hop.

Applications: social networks, knowledge graphs, recommendation engines, fraud detection, molecular biology (protein interaction networks), epidemiology (contact tracing).

Query language: Cypher (neo4j), SPARQL (RDF graphs), Gremlin (Apache TinkerPop).

Notable implementations: neo4j, Dgraph, Amazon Neptune, ArangoDB (multi-model).

8.7. Vector Databases

Vector databases store **high-dimensional embedding vectors** and are optimised for **similarity search** — finding the k nearest vectors to a query vector, typically using cosine similarity or Euclidean distance.

Motivation: machine learning models (language models, image encoders) represent objects as dense vectors in a high-dimensional space (e.g. 768 or 1536 dimensions). Traditional RDBMS and even standard NoSQL databases do not support efficient approximate nearest-neighbour (ANN) search.

Typical workflow:

1. Encode documents, images, or records as embedding vectors using a neural model.
2. Store vectors in the vector database with their metadata.
3. At query time, encode the query and retrieve the top- k most similar vectors.

Applications: semantic search, Retrieval-Augmented Generation (RAG) for LLMs, recommendation systems, image similarity, anomaly detection.

Notable implementations: Pinecone, Weaviate, ChromaDB, Milvus, pgvector (PostgreSQL extension).

8.8. Exercises

Exercise 8.1 (*Non-relational DBMS Examples*)

List at least four examples of non-relational (NoSQL) database management systems. For each, identify the NoSQL category it belongs to (Key-Value, Document, Wide-Column, Graph, etc.).

Solution

- **Key-Value** — Redis, DynamoDB, Riak. Fast lookups by opaque key; values are blobs.
- **Document** — MongoDB, CouchDB, Elasticsearch. Self-describing JSON / BSON records, queryable by field.
- **Wide-Column** — Cassandra, HBase, ScyllaDB. Sparse rows with very many columns, column families grouped on disk.
- **Graph** — Neo4j, Amazon Neptune, ArangoDB. First-class nodes + edges with traversal queries (Cypher, Gremlin).

Some systems (Elasticsearch, Solr) are sometimes split off as **full-text search engines**, and time-series stores (InfluxDB, TimescaleDB) as a category of their own.

Exercise 8.2 (*ACID vs BASE*)

Describe the differences between the ACID and BASE consistency models. In what scenarios is each model preferable?

Solution

- **ACID** = **A**tomicity, **C**onsistency, **I**solation, **D**urability. Strict, transactional guarantees. Every committed write is immediately visible to all subsequent reads and survives crashes. Cost: latency and limited write throughput under contention.
- **BASE** = **B**asically **A**vailable, **S**oft state, **E**ventually consistent. Relaxed guarantees in exchange for availability and horizontal scale. Reads may return stale data; replicas converge in the background.

ACID is preferable when correctness is non-negotiable: banking and accounting, inventory, order management, anything with money or audit constraints. **BASE** wins in high-throughput, eventually consistent workloads: social media feeds, CDNs, IoT telemetry, large-scale caches and analytics ingestion.

Exercise 8.3 (*Horizontal vs Vertical Scaling*)

What are horizontal and vertical scaling (partitioning)? Discuss their pros and cons, and argue which one is generally preferable for large-scale data systems and why.

Solution

- **Vertical scaling (scale-up).** Bigger single machine: more CPU, RAM, faster disks. Pros: simple — software is unchanged, no distributed-systems complexity, no consistency trade-offs. Cons: hard ceiling on hardware, exponential price per unit performance at the top end, single point of failure.
- **Horizontal scaling (scale-out).** More machines, data sharded and replicated across them. Pros: near-unlimited growth, commodity hardware, built-in fault tolerance via replication. Cons: needs partitioning, replication, distributed transactions or eventual consistency; operational complexity is much higher.

For **large-scale data systems**, horizontal scaling is generally preferred. Vertical scaling cannot keep up with terabyte- or petabyte-scale workloads at any reasonable cost, and a single super-server is a single point of failure. NoSQL and modern distributed RDBMS were designed precisely to make horizontal scaling tractable.

Exercise 8.4 (*Consistent Hashing for Sharding*)

What are the advantages of using consistent hashing for sharding distributed databases compared to a “standard” partitioning technique (e.g. via modulo-operations)? What problem does consistent hashing solve when nodes are added or removed?

Solution

With **plain hash-mod sharding** ($\text{shard} = \text{hash}(\text{key}) \bmod N$), changing the number of nodes from N to $N \pm 1$ changes the modulus and therefore **reshuffles almost every key** — i.e. roughly $\mathcal{O}(K)$ key movements for K keys. That stalls the cluster during any resize.

Consistent hashing places nodes and keys on a virtual ring; each key is owned by the next node clockwise. Adding or removing a node only changes ownership of the keys in its **immediate neighbourhood** on the ring, so on average only $\mathcal{O}(K/N)$ keys move. Combined with **virtual nodes** (each physical node owns many ring positions), the load stays balanced. The problem it solves: **graceful, low-impact rebalancing** when nodes join or leave a dynamic cluster — exactly the day-to-day reality of large NoSQL and CDN deployments (DynamoDB, Cassandra, Riak, memcached).

Part II.

Processing

9. Processing and Concurrency

9.1. Introduction to Data Processing

Processing refers broadly to the set of operations performed on a dataset to extract higher-level information. In a very simplified view:

$$\text{DATA(SET)} \xrightarrow{\text{PROCESSING}} \text{RESULTS}$$

This encompasses a wide variety of tasks such as analytics, data reduction, data cleansing, and feature enrichment. In practice, processing often involves multiple sequential stages, each consuming the output of the previous one:

$$\text{DATA} \xrightarrow{\text{PROCESSING}} \text{DATA} \xrightarrow{\text{PROCESSING}} \text{DATA} \xrightarrow{\text{PROCESSING}} \text{RESULTS}$$

The fundamental question that runs through all of these stages is: **how does the processing scale with the data size, task complexity, and available resources?**

9.2. CPU Processing

At its core, processing is the sequential execution of instructions by the CPU, operating on data held in memory. The CPU follows the classic **fetch–decode–execute** cycle for each instruction:

- **Fetch** — retrieve the instruction from memory
- **Decode** — interpret the instruction
- **Execute** — carry out the operation (Von Neumann architecture)

9.2.1. Processing Time

The **CPU processing time** is the time needed to execute the entire instruction set. Two complementary metrics matter:

- **Wall-time** (response time / elapsed time): the total latency to complete a task, including all I/O operations (memory and storage access), network communication, and OS overhead.
- **CPU time**: the time attributed solely to instruction processing by the CPU, excluding time spent waiting for other activities.

CPU time can be modelled as:

$$t_{\text{CPU}} = \sum_i \frac{IC_i \cdot CPI_i}{CF}$$

where:

Symbol	Meaning
IC_i	Instruction count — number of instructions in the program
CPI_i	Clock cycles per instruction
CF	Clock frequency (Hz)

9.2.2. Historical Context: Moore's Law

In 1965, Gordon Moore observed that the number of transistors on a chip doubled approximately every 18–24 months — a trend that held for nearly five decades and became known as **Moore's Law**.

Era	Key trend
1970s–1990s	Transistor count and clock frequency both doubled regularly
Early 2000s	Clock frequency plateau: thermal limits prevented further frequency scaling
2004 onwards	Industry shifted to multi-core CPUs rather than faster single cores
Today	Transistor scaling continues (though slowing) but single-thread performance improvements are incremental

The plateau in single-thread performance — roughly 3–4 GHz since the mid-2000s — is why parallel and distributed computing became the dominant paradigm for improving application throughput.

9.2.3. The Power Wall

A straightforward way to increase processing performance is to raise the clock frequency. This strategy drove chip performance for many years, alongside transistor scaling.

However, from the early 2000s onwards, processors started hitting a fundamental limit: **power dissipation**. The power consumed by a chip scales as:

$$P \propto n \cdot C \cdot V^2 \cdot f$$

where n is the number of transistors, C is transistor capacitance, V is the supply voltage, and f is the clock frequency. Early 32-bit processors such as the Intel 80386 consumed around 2 W; a

2021 4.0 GHz Intel Xeon Gold 5318Y consumes more than 150 W. Continuing to scale frequency without also reducing voltage became thermally and energetically unsustainable.

9.2.4. Pipelining and Alternative Strategies

As performance gains from raw clock-frequency scaling became harder to sustain, chip designers turned to **instruction pipelining**, which allows multiple instructions to overlap in their fetch–decode–execute stages on the same ALU. This achieves a speedup without raising the clock frequency.

A broader change of paradigm was therefore required to continue improving performance.

9.3. Multi-Processing Architectures

The industry response to the power wall was **multi-core** CPUs: placing multiple independent processing units (cores) on the same die, and further combining multiple CPUs (sockets) on the same motherboard.

This can be realised in several ways:

- **Symmetric Multiprocessing (SMP)** — homogeneous CPUs, all with equal access to shared memory.
- **Asymmetric Multiprocessing** — heterogeneous CPUs with different roles (e.g. big.LITTLE mobile architectures).
- **Master/Slave** — one master CPU assigns tasks to subordinate CPUs.

A modern laptop (e.g. a Lenovo X1 Gen 8 with an Intel Core i7-10510U) might show the following `lscpu` output:

```
Architecture:      x86_64
CPU(s):            8          ← 1 socket × 4 cores × 2 threads/core = 8 logical PUs
Thread(s) per core: 2        ← Intel Hyper-Threading: 2 independent "feeding lanes" per core
Core(s) per socket: 4        ← 4 physical processing units (cores) per CPU
Socket(s):         1          ← 1 CPU socket on the motherboard
Model name:        Intel(R) Core(TM) i7-10510U CPU @ 1.80GHz
CPU max MHz:       4900.0000
L1d cache:         128 KiB
L2 cache:          1 MiB
L3 cache:          8 MiB
```

i Hyper-Threading

Intel's Hyper-Threading technology provides two independent **fetch/decode** lanes per physical core, but only one execution unit (ALU). The OS sees 8 logical CPUs on a 4-core processor, but at most 4 instructions can execute truly in parallel at any moment.

The total number of logical CPUs visible to the OS is:

$$\text{CPUs} = \text{Sockets} \times \text{Cores per Socket} \times \text{Threads per Core}$$

A production-grade server (e.g. a Dell PowerEdge R750 with two Intel Xeon Gold 5318Y) might expose 96 logical CPUs to the OS (2 sockets $\times$ 24 cores $\times$ 2 threads). Multi-processor machines are now the standard — the question is how to take advantage of them.

9.4. Parallelism

Parallelism means exploiting multi-processing architectures to solve problems faster. Two orthogonal forms of parallelism exist:

1. **Data-Level Parallelism (DLP)** — many data items can be operated on simultaneously. The same operation is applied to all elements in parallel.
2. **Task-Level Parallelism (TLP)** — multiple sub-tasks of a process operate independently and in parallel.

In practice, the most efficient solutions combine both.

Parallel processing is the simultaneous execution of a single job split into n sub-tasks across m CPUs/cores. The primary goal is to reduce the total execution time (**speedup**).

9.5. Flynn's Taxonomy

Flynn's taxonomy (1966) classifies computer architectures by the number of simultaneous instruction streams and data streams:

	Single Data	Multiple Data
Single Instruction	SISD	SIMD
Multiple Instruction	MISD	MIMD

- **SISD** — one instruction on one data stream at a time. No parallelism; e.g. a single-threaded process on a single-core CPU.
- **SIMD** — one instruction performed on multiple data streams simultaneously. Enables data-level parallelism. Found in modern CPU AVX/SSE/MMX instructions and most GPUs.
- **MISD** — multiple instructions on a single data stream. Rare in practice; used in fault-tolerant applications.
- **MIMD** — multiple instructions on multiple data streams simultaneously. Supports both DLP and TLP; each processing unit runs independently. Most modern parallel computers fall into this category.

i SPMD

A widely used sub-category of MIMD is **SPMD (Single Program Multiple Data)**: the same entire program (not just single instructions) runs on multiple “chunks” of data across

several processing units. This is the natural model for **embarrassingly parallel** problems.

9.6. Speedup: Amdahl's Law

Is it possible to speed up a computation indefinitely by adding more processing units?

Given a process with a fraction p that can be parallelised and a fraction $(1 - p)$ that **must** remain serial, and with n processing units, the speedup predicted by **Amdahl's law** is:

$$S(n) = \frac{1}{(1 - p) + \frac{p}{n}}$$

As $n \rightarrow \infty$:

$$S_{\max} = \frac{1}{1 - p}$$

No matter how many processing units are available, the **serial fraction** of the computation remains the bottleneck. To maximise performance, the process itself must be redesigned to reduce its serial portion.

Amdahl's law is often considered somewhat pessimistic, as it assumes the serial fraction grows proportionally with problem size.

9.7. Gustafson's Law

Gustafson's **scaled speedup** addresses parallelism from a different angle. It observes that in practice the “problem size” often scales with the number of available processors: as more processors are added, a larger problem is tackled, so the parallelisable component grows more than the serial component.

$$S_{\text{scaled}}(n) = 1 + (n - 1) \cdot p$$

Either way, **heterogeneous computing** — integrating CPUs, GPUs, and specialised accelerators — aims to maximise both the serial and parallel components of a computation.

9.8. Processes and Threads

9.8.1. Definitions

i Process

A **process** is an instance of a program in execution. Every process is an independent entity with its own executable code and memory space. Multiple instances of the same program can run simultaneously — they share the same code but are distinct processes.

i Thread

A **thread** is the basic unit of CPU utilisation. Every thread consists of a program counter, a set of registers, and its own execution stack. A thread is part of a process, and each process can contain one or more threads.

Within a multi-threaded process:

- All threads **share** the process memory and common resources (“global” for all threads).
- Each thread has an **independent** set of registers, memory, and execution state.

A word processor, for example, might use one thread to check spelling, another to process keystrokes, a third to load images, and a fourth to perform periodic automatic saves.

9.8.2. Concurrency vs Parallelism

These two concepts are often confused:

- **Concurrency** — multiple tasks are *in progress* at the same time. Tasks can start, run, and stop independently and in any order. On a single-core CPU, concurrency is achieved through rapid context switching.
- **Parallelism** — multiple tasks are *executing* at the same time. True simultaneous execution, requiring multiple processing units.

9.8.3. Multi-Tasking

Multi-tasking is the OS-enabled execution of multiple concurrent tasks. The OS instructs the CPU to share its time between tasks via a **scheduler**:

- **Pre-emptive scheduling** — the scheduler alternates between N tasks, sharing CPU time approximately equally (with priority adjustments).
- **Collaborative scheduling** — the system switches processes only when the active one voluntarily releases control.

A **context switch** saves the state of the currently running process or thread so it can be resumed later.

9.8.4. Multi-Threading

Multi-threading is the execution of multiple threads within a process. The execution mode depends on the architecture:

- **Concurrent**, on single-core CPUs with a multi-tasking OS (the CPU alternates between threads via context switching).
- **Parallel**, on multi-core CPUs (different threads run simultaneously on different cores).

Context switching between threads is lighter than between full processes, since threads within the same process share memory. Multiple sub-components of a single application can thus proceed independently while sharing global resources.

9.9. Race Conditions

Multi-threaded execution can lead to subtle bugs when multiple threads access the same shared memory simultaneously.

Race Condition

A **race condition** occurs when a shared resource (e.g. a variable in memory) is accessed concurrently by multiple threads, and the outcome depends on the particular order in which the accesses take place.

Consider `var_x = 5` shared between two threads:

- Thread 1 reads `var_x` (value 5) and computes $5 * 2 = 10$.
- Thread 2 **also** reads `var_x` (still 5, before Thread 1 has written back) and computes $5 * 3 = 15$.
- Thread 1 writes 10 back to `var_x`.
- Thread 2 writes 15 back to `var_x`.
- Final value: 15 — but the correct sequential result (multiply by 2 then by 3) should be 30.

9.10. Thread Synchronisation

Thread synchronisation mechanisms prevent race conditions by controlling access to shared resources:

- **Lock / Mutex (Mutual Exclusion)** — only one thread at a time can hold the lock and access the protected resource. Other threads block until the lock is released.
- **Binary semaphore** — similar to a mutex, with a 0/1 counter.
- **Counting semaphore** — controls access to a pool of resources.

⚠ Deadlock

A **deadlock** occurs when a thread waits indefinitely for an event that can only be triggered by another thread that is itself blocked. Locks must always be released after use, and circular wait conditions must be avoided.

9.11. The Python GIL

The **Global Interpreter Lock (GIL)** is a mutex on the CPython interpreter itself, ensuring that only one thread at a time can hold the GIL and thus execute Python bytecode. This design:

- Prevents many deadlocks and race conditions at the interpreter level.
- **Disables true parallelism** for CPU-bound work in CPython: even on a multi-core machine, two Python threads from the same process cannot execute Python bytecode simultaneously.

The GIL is an implementation detail of CPython (the standard Python interpreter), not a requirement of the Python language. Alternative interpreters such as Jython and IronPython do not rely on it.

! Python 3.13 and the GIL

Starting with **Python 3.13** (October 2024), the GIL became optional. CPython now provides a “free-threaded” build mode in which the GIL is disabled, enabling full multi-threading for CPU-bound Python code. The standard build still includes the GIL for backwards compatibility.

⚠ GIL Does Not Eliminate All Race Conditions

Even with the GIL, race conditions can occur during OS-level context switches. The GIL is released periodically — by default every 5 ms (`sys.getswitchinterval()`) — and briefly between the old thread releasing it and the new thread acquiring it. During that window, the OS scheduler may preempt execution mid-operation.

Compound operations that *appear* atomic in Python source code (e.g. `x += 1`, which compiles to LOAD/ADD/STORE bytecode instructions) are **not guaranteed to be atomic** under concurrent thread execution. A lock (`threading.Lock`) must still be used to protect shared state.

For a detailed analysis of the GIL’s interaction with the OS scheduler, see: <https://www.dabeaz.com/python/GIL.pdf>

9.12. Threading vs Multiprocessing in Python

	Threading (<code>threading</code>)	Multiprocessing (<code>multiprocessing</code>)
Memory	Shared across threads	Separate address space per process
GIL	Constrained by GIL (one thread executes at a time)	No GIL constraint (each process has its own interpreter)
Best for	I/O-bound tasks (disk, network)	CPU-bound tasks (numerical computation)
Communication	Shared variables (but need locks)	Queues, pipes, shared memory objects

💡 Tip

Other libraries extend these capabilities: **joblib** provides a unified interface to both threading and multiprocessing backends.

```
# joblib example: run square() in parallel across 4 processes
from joblib import Parallel, delayed

def square(x):
    return x ** 2

results = Parallel(n_jobs=4, backend='multiprocessing')(
    delayed(square)(i) for i in range(10)
)
# results = [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

mpi4py exposes the MPI (Message Passing Interface) standard to Python, enabling distributed-memory parallelism across cluster nodes — the same model used in large-scale HPC applications.

9.13. Hands-on: Concurrency in Python

The following scripts demonstrate the progression from single-threaded execution through multi-threaded and multi-process programming in Python.

Note: These scripts are designed to be run from the terminal (`python3 script_name.py`). They are not executed during book rendering.

9.13.1. 00 — Single-Threaded Baseline

A simple CPU-bound task: sum all integers in a range. This serves as the performance baseline before introducing concurrency. The wall-time here is the cost of pure sequential execution on a single core.

9.13.2. 01 — Two Concurrent Threads

The same summation split across two `threading.Thread` instances. `start()` is non-blocking (launches the thread), while `join()` blocks until that thread finishes. Due to the GIL, wall-time will not improve for CPU-bound work — both threads still share a single interpreter lock.

9.13.3. 02 — Thread-Local Data

Uses `threading.local()` to create storage that is private to each thread. Even though the variable name is the same across threads, each thread sees its own independent copy — no sharing, no synchronisation needed.

9.13.4. 03extra — Minimal Race Condition

Ten threads each increment a shared `counter` 1 000 000 times without any lock. The read–modify–write sequence (`counter += 1`) is not atomic, so threads interleave and overwrite each other's updates. The final value is almost certainly **less than** 10 000 000.

9.13.5. 03 — Multi-Threaded Without Lock (Race Condition)

Two threads concurrently write to `COMMON_COUNTER` and `COMMON_LIST` with no synchronisation. The mismatch between the counter and the list length at the end reveals the race condition in action.

9.13.6. 04extra — Minimal Race Condition Fixed with Lock

The counter example from above, now protected by `threading.Lock()`. Every `counter += 1` is bracketed by `acquire()` / `release()`, so only one thread modifies the counter at a time. The final value is always exactly 10 000 000.

9.13.7. 04 — Multi-Threaded With Lock

The full race-condition scenario corrected: each access to `COMMON_COUNTER` and `COMMON_LIST` is guarded by `the_lock.acquire()` / `the_lock.release()`. The lock is passed explicitly as an argument so each thread uses the same shared lock object.

9.13.8. 05 — Thread Pool with Lock (`concurrent.futures`)

`ThreadPoolExecutor` manages a pool of reusable worker threads. `submit()` dispatches a task and returns a `Future`; `wait()` blocks until all futures complete. The return value from each thread is retrieved with `f.result()` — a cleaner pattern than writing to global state.

9.13.9. 06 — Thread Pool with Queue

Replaces the explicit lock with a `queue.Queue`, which is inherently thread-safe (its internal lock is managed automatically). Threads call `put()` to deposit results; the main thread reads them after all workers finish. Queues are the idiomatic Python pattern for producer–consumer communication.

9.13.10. 07 — Multi-Processing

`multiprocessing.Process` spawns a completely separate OS process with its own Python interpreter and memory space, bypassing the GIL entirely. True CPU parallelism is possible: both processes can execute Python bytecode simultaneously on different cores.

9.13.11. Synchronous vs Asynchronous Execution

Before the next two scripts, it is useful to define these terms precisely:

- **Synchronous (blocking)**: the calling code waits for the operation to complete before continuing. Execution is sequential.
- **Asynchronous (non-blocking)**: the calling code submits the operation and continues immediately. Results are collected later via a handle or callback.

In the context of `multiprocessing.Pool`: - `starmap()` → **synchronous**: blocks until all worker processes finish. - `starmap_async()` → **asynchronous**: returns an `AsyncResult` immediately; call `.get()` when results are needed.

9.13.12. 08 — Multiprocessing Pool (`starmap`, blocking)

`Pool.starmap()` distributes a list of argument tuples across a pool of worker processes. The call **blocks** until all workers finish, then returns the collected results as a list. Each worker independently sums its sub-range and returns the partial sum.

9.13.13. 09 — Multiprocessing Pool (`starmap_async`, non-blocking)

`starmap_async()` returns immediately with an `AsyncResult` object, allowing the main process to continue (or poll) while workers run. `.get()` blocks until all results are ready. This pattern is useful when the main process has other work to do while waiting.

9.14. Benchmarking: I/O-bound vs CPU-bound Tasks

The following pairs of scripts directly compare single-threaded, multi-threaded, and multi-process execution on two contrasting workloads.

9.14.1. I/O-Bound Tasks

I/O-bound tasks spend most of their time waiting for external operations (disk, network). The GIL is released while waiting for I/O, so **threading is effective** here — multiple threads can overlap their wait times even though only one runs Python bytecode at a time.

Single-threaded — downloads five images sequentially; each download blocks until complete:

Multi-threaded — the same downloads dispatched concurrently via `ThreadPoolExecutor`; all five network waits overlap. Significant speedup is expected:

Multi-process — same task using `multiprocessing.Pool`. For pure I/O work the process-spawn overhead typically outweighs any benefit; performance is similar to or worse than threading:

9.14.2. CPU-Bound Tasks

CPU-bound tasks are limited by raw computation speed. Because the GIL serialises Python bytecode execution, **threading provides no speedup** for CPU-bound code — threads take turns rather than running in parallel. Multiprocessing spawns separate interpreter instances that truly run in parallel on different cores.

The example performs a Monte Carlo numerical integration of $f(x) = \sin^2\left(\frac{1}{x(2-x)}\right)$ over $[0, 2]$ across multiple independent chunks.

Single-threaded — 10 chunks processed one after another:

Multi-threaded — 10 chunks dispatched to a `ThreadPoolExecutor`; the GIL prevents true parallelism, so the result is similar to (or slower than) single-threaded due to context-switch overhead:

Multi-process — 10 chunks dispatched to a `ProcessPoolExecutor`; each chunk runs in its own interpreter with no GIL contention. True speedup proportional to the number of cores:

💡 Environment Setup

To run these scripts, clone the repository and ensure you have Python 3. For older Python versions (to test GIL behaviour) you can use:

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.9    # or another old version
```

Use `taskset` to pin threads/processes to specific CPU cores:

```
taskset -c 4 python3 script.py    # CPU 4 only
taskset -c 1,3 python3 script.py  # CPUs 1 and 3
taskset -c 0-4 python3 script.py  # CPUs 0 to 4
```

Monitor CPU usage with `htop` while the scripts run.

10. Distributed Processing and MapReduce

10.1. Scaling Processing

As discussed in the previous chapter, processing performance can scale with the number of processing units, provided that the computation is at least partially parallelisable. Speedup studies typically confirm that wall-time scales favourably with n for heavily data-parallel tasks (SPMD-style).

The natural next step is to tackle very large, possibly distributed, datasets by increasing n beyond what a single machine can offer. Two fundamental strategies exist:

Strategy	Description
Vertical scaling (Scale Up)	Increase the capacity or performance of existing hardware — buy a faster machine.
Horizontal scaling (Scale Out)	Add more machines — buy more commodity hardware.

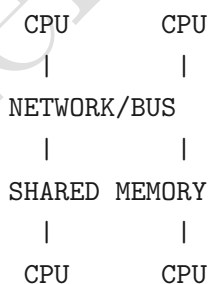
Horizontal scaling is almost always preferred for large-scale data processing: commodity hardware is cost-effective, and the system can grow incrementally.

10.2. Parallel Computing Architectures

Different memory-sharing architectures lead to different trade-offs in performance and scalability.

10.2.1. Shared Memory Multiprocessor (SMP)

In a **Shared Memory Multiprocessor**, a set of CPUs share a common memory space connected by a high-speed bus or fabric.



- All modern workstations (including smartphones) are SMP systems.

- Memory access is uniform and transparent to the programmer.
- Scaling this architecture to a very large number of processors is expensive and technically demanding.

i NUMA: Non-Uniform Memory Access

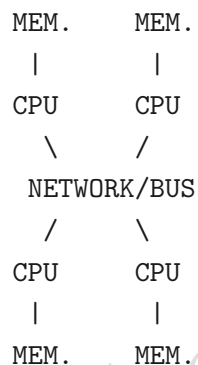
Multi-socket servers partition memory into **NUMA nodes** — one per CPU socket, each with local RAM. The `lscpu` output for a dual-socket server shows **NUMA node(s): 2**. Accessing local memory (same NUMA node as the running core) is significantly faster than accessing remote memory (on the other socket). The OS and job schedulers can pin processes to specific NUMA nodes with `numactl`:

```
numactl --cpunodebind=0 --membind=0 python3 my_script.py
```

NUMA topology is a key consideration when scaling shared-memory parallelism beyond a single socket.

10.2.2. Distributed Memory Multiprocessor (Cluster)

In a **cluster**, each processor has its own local memory — a complete unit called a **node**. All nodes are connected via a high-speed network.



- Good overall performance even from commodity hardware.
- The complexity lies in the communication across nodes.
- Scales horizontally to thousands or millions of nodes.

10.3. Distributed Computing: Clusters

A cluster distributes the processing workload across multiple networked nodes, exposing itself to users as a single virtual machine.

Typical cluster components:

- **Head node** — the access point for users; distributes workload to compute nodes.
- **Computing nodes (CN)** — where the actual processing runs; usually invisible to users.

- **Interactive nodes** — accessible to users as standard computers for testing, storage access, or launching jobs.
- **Specialised nodes** — nodes equipped with specific hardware (GPUs, large RAM, fast NVMe) for particular workloads.

10.3.1. Supercomputers as Clusters

Most of today's supercomputers are clusters:

CINECA LEONARDO (world's #10 in November 2025) - 1 536 nodes with 2× Intel Sapphire Rapids (56-core CPUs) - 3 456 nodes with 1× Intel Xeon 8358 (32-core CPU) and 4× NVIDIA Ampere GPUs

On the opposite extreme, clusters of Raspberry Pi boards or repurposed desktop PCs (**COTS** — **Commodity Off-The-Shelf** hardware) can efficiently tackle a wide range of parallel problems through **opportunistic computing** (using shared resources when they are idle).

10.4. Batch Systems

Users typically submit jobs to a cluster through a **batch system** rather than logging in interactively:

1. A job consists of an executable, optional arguments, and resource requirements (CPUs, memory, walltime, etc.).
2. The batch scheduler accepts the job and places it in a queue.
3. The scheduler selects which job runs next based on priority, previous resource usage, and availability.
4. The job's tasks are dispatched to computing nodes for execution.

Popular batch schedulers include **PBS**, **SLURM**, **LSF**, and **HTCondor**. Example HTCondor submission:

```
$ condor_submit my_job.sub
Submitting job(s).
50 job(s) submitted to cluster 5146936.

$ condor_q
OWNER      BATCH_NAME  SUBMITTED  DONE  RUN  IDLE  TOTAL  JOB_IDS
jpazzini   ID:5146936  10/4 17:29   12   25   13     50   5146936.0-49
```

An HTCondor submit file:

```
executable      = my_exec
arguments       = --input input_file_$(ProcId).txt
transfer_input_files = input_file_$(ProcId).txt
log             = my_job_$(ProcId).log
```

```

error                = my_job_$(ProcId).err
output               = my_job_$(ProcId).out
request_disk         = 8GB
request_memory       = 2GB
queue 50

```

10.5. Programming Frameworks for Parallel-Distributed Systems

Optimising complex tasks for parallel and distributed execution requires knowledge of specific programming standards:

Framework	Purpose
OpenMP	Parallelisation directives for shared-memory multi-core CPUs on a single node
MPI (Message Passing Interface)	Communication and synchronisation across nodes of a cluster
CUDA / HIP	Vector/matrix processing on NVIDIA and AMD GPUs

In this course we focus on higher-level, Python-accessible frameworks that abstract away the inner workings of distributed communication — in particular, the **MapReduce** paradigm and its implementations in Hadoop and Spark.

i GPU Computing: Going Further

This course focuses on CPU-based parallelism and distributed processing. GPU programming (CUDA, OpenCL, HIP, Numba, CuPy) and heterogeneous CPU+GPU architectures are covered in the optional companion course “**Modern Computing for Physics**”, which addresses SIMD/vectorisation, GPU kernel programming, and MPI+GPU hybrid models.

10.6. Big Data and the 3 Vs

From the early 2000s, a wave of data-processing innovations emerged, driven largely by technology companies that needed to handle datasets at an unprecedented scale. These are often summarised as **Big Data**, characterised by the three Vs:

- **Volume** — datasets at the scale of terabytes, petabytes, or beyond.
- **Velocity** — data arriving at high speed (streaming sensors, transactions, logs).
- **Variety** — structured, semi-structured, and unstructured data.

The five key ingredients of a BigData processing stack are:

1. **APIs / programming language** — the interface developers use.
2. **Software framework** for distributed processing.
3. **Cluster orchestrator / scheduler** — resource management.
4. **Distributed file system (DFS)** — storage layer.
5. **Computing nodes** — physical or cloud hardware.

10.7. Hadoop

Hadoop was born in 2006 within Yahoo! and was based on a 2003 Google paper on the Google Distributed File System. It rapidly became the de facto standard for BigData storage and processing.

Large-scale real-world deployments have included:

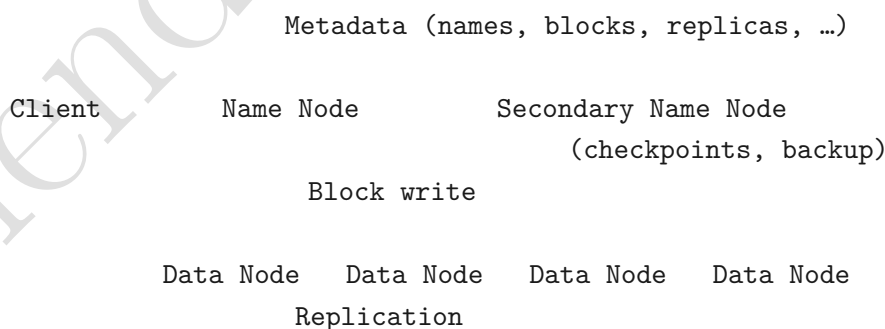
- **Facebook:** 1 100-node cluster (8 800 cores, ~12 PB storage); 300-node cluster (~3 PB)
- **Twitter:** clusters storing over 500 PB, largest with >10 000 nodes
- **Yahoo!:** >100 000 CPUs across >40 000 computers

Hadoop is open source (Apache Foundation), written in Java, and designed to scale horizontally. Its main components are:

Component	Role
Hadoop Common	Java-based shared libraries and utilities
HDFS	Hadoop Distributed File System — storage
YARN	Yet Another Resource Negotiator — cluster manager and scheduler
Hadoop MapReduce	Distributed programming model

10.8. HDFS: A Reminder

HDFS stores data as large blocks distributed across data nodes, with the name node keeping metadata:



- The **Name Node** manages the filesystem namespace and metadata.
- The **Secondary Name Node** periodically checkpoints the metadata.
- **Data Nodes** store the actual data blocks and replicate them for fault tolerance.

10.9. The MapReduce Paradigm

MapReduce is not a programming language, but a **paradigm** for developing distributed algorithms on potentially very large datasets. It decomposes computation into two phases:

10.9.1. 1. Map

Split the input data into **<key, value>** pairs, and apply a user-defined map function to each pair, producing a list of intermediate **<key, value>** pairs:

$$\text{map}(k_{\text{in}}, v_{\text{in}}) \rightarrow \text{list}[(k_1, v_1), (k_2, v_2), \dots]$$

Each key-value pair is processed **independently**, enabling full data parallelism across the cluster.

10.9.2. 2. Reduce

Collect all intermediate pairs with the same key, then apply a user-defined aggregation function to produce a final result:

$$\text{reduce}(\text{list}[(k, v_1), (k, v_2), \dots]) \rightarrow [(k, v_{\text{out}})]$$

10.9.3. Word Count Example

The canonical example: count the occurrences of each word in a text corpus.

Input: "Car Car River Deer Car Bear"

Map phase:

("Car", 1), ("Car", 1), ("River", 1), ("Deer", 1), ("Car", 1), ("Bear", 1)

Shuffle & Sort (grouping by key):

"Bear" → [1]

"Car" → [1, 1, 1]

"Deer" → [1]

"River" → [1]

Reduce phase (sum):

Bear → 1

Car → 3

Deer → 1

River → 1

10.10. MapReduce in Hadoop

10.10.1. Map Phase

Data is read from HDFS, partitioned among computing nodes, and sent to each node as a set of key-value pairs. The Map tasks process input records **independently** and write intermediate results to the **local disk** of the node. This **data locality** principle — bringing computation to the data rather than the data to the computation — is central to Hadoop’s performance.

10.10.2. Reduce Phase

Once all Map tasks complete, a **shuffle and sort** step groups and transfers the intermediate key-value pairs to the Reduce tasks. The Reduce tasks then apply the aggregation function to each group.

10.11. YARN: Yet Another Resource Negotiator

YARN acts as the “operating system” of the Hadoop cluster, managing the scheduling and execution of jobs.

Key components:

Component	Role
Resource Manager (RM)	Manages cluster-wide resource allocation; contains the Scheduler and the Application Manager
Application Master (AM)	Negotiates resources from the RM and coordinates the execution of a single application
Node Manager (NM)	Manages the user processes on each individual machine
Container	A set of allocated resources (memory, CPU) used to run a specific task

10.12. Hadoop Higher-Level Abstractions

Writing raw MapReduce jobs in Java is verbose. Higher-level tools provide more accessible interfaces:

Tool	Description
Pig	High-level scripting language translated into MapReduce programs (Yahoo!, latest release 2017)
Hive	SQL-like query language (HiveQL) for data stored in HDFS (Facebook/Netflix)
Oozie	Workflow scheduler for chaining multiple MapReduce jobs

10.13. The Hadoop Ecosystem

Hadoop grew into a rich ecosystem covering the full BigData stack:

- **APIs / programming language:** Pig, Hive, custom Java/Python
- **Distributed processing framework:** Hadoop MapReduce
- **Cluster orchestrator:** YARN
- **Distributed file system:** HDFS

The broader BigData landscape includes many additional tools (Spark, Kafka, Cassandra, HBase, etc.) that either complement or replace individual components of the Hadoop stack depending on the use case.

The full BigData stack can be visualised as five layers, with Hadoop occupying all four lower layers and higher-level tools sitting on top:

APIs / Languages	Pig · Hive · HiveQL · custom Java/Python
Processing	Hadoop MapReduce · Apache Spark · Storm
Cluster Scheduler	YARN · Mesos · Kubernetes
Distributed FS	HDFS · Amazon S3 · Azure Data Lake
Computing Hardware	Commodity nodes · Cloud VMs · Bare metal

Beyond this core stack, the broader ecosystem includes: **Kafka** (streaming), **HBase** / **Cassandra** (wide-column NoSQL), **ZooKeeper** (distributed coordination), **Sqoop** (RDBMS ↔ HDFS transfer), **Oozie** (workflow scheduling), and **Mahout** (distributed machine learning).

11. Apache Spark

11.1. Motivation

As discussed in the MapReduce chapter, Hadoop provided a solid foundation for distributed processing, but it has notable limitations for certain workloads:

- **Complex/iterative algorithms** (e.g. k-means clustering) require chaining multiple MapReduce jobs, making them awkward to implement and slow due to repeated disk I/O between stages.
- **MapReduce I/O bottleneck**: every intermediate result is written to disk (HDFS), then read back for the next stage.
- **Batch-only**: Hadoop MapReduce processes data “at rest” in large batches; it does not natively support real-time or streaming processing.

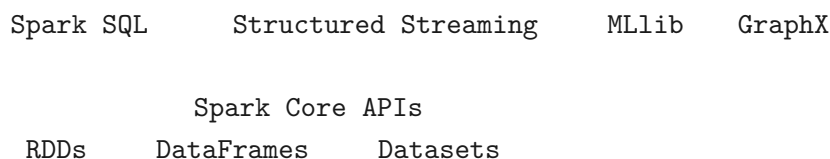
Apache Spark addresses all three limitations:

Hadoop MapReduce	Apache Spark
Strictly Map → Reduce pattern	Arbitrary DAG of transformations — not tied to MapReduce
Disk-based intermediate storage	In-memory processing — massive speedup for iterative algorithms
Batch processing only	Both batch and real-time streaming processing

Spark was originally developed at UC Berkeley AMPLab in 2009 and became an Apache project. It is open source, written in Scala, with official APIs for Python (PySpark), Java, and R.

11.2. Spark Ecosystem

Spark provides a layered set of APIs:



- **RDD (Resilient Distributed Dataset)** — the low-level, unstructured API.

11. Apache Spark

- **DataFrame / Dataset** — the high-level, structured API built on top of RDDs.
- **Spark SQL** — SQL interface for structured data (DataFrames).
- **Structured Streaming** — continuous stream processing via the DataFrame API.
- **MLlib** — distributed machine learning library.
- **GraphX** — graph computation.

i PySpark vs Scala Performance

Spark is written in **Scala** and runs on the JVM. The PySpark API communicates with the JVM via the **Py4J bridge**, which introduces serialisation overhead when data is passed between Python and Java processes. For most DataFrame/SQL operations this overhead is minimal because computation runs on the JVM regardless. However, Python UDFs (User Defined Functions applied row-by-row) bypass the JVM and can be significantly slower than equivalent Scala UDFs.

In this course PySpark is used for accessibility. For production pipelines with strict performance requirements, Scala/Java provides the highest throughput. Vectorised Pandas UDFs (via PyArrow) offer a middle ground.

11.3. Spark Cluster Architecture

Spark uses a **Master/Slave** architecture.

11.3.1. Driver and Executors

When a Spark application is submitted:

1. A **Driver** process is created. The Driver is the application's brain: it converts user code into a plan of smaller execution units (tasks), and distributes and schedules those tasks across the executors.
2. A **Spark Context** is created within the Driver for each application, allowing it to access cluster resources via the Resource Manager.
3. A set of **Executors** are created on the cluster's worker nodes. Each Executor:
 - Executes the tasks assigned by the Driver.
 - Reports the computation state back to the Driver.
 - Stores data in memory or disk across the lifetime of the application.

The Driver and Executors live in Java Virtual Machines (JVMs) within the worker nodes. One Executor cannot span multiple nodes, but one node can host multiple Executors.

11.3.2. Cluster Resource Managers

Spark works with several cluster managers:

- **Spark standalone** cluster manager (built-in)
- **YARN** (Hadoop's resource manager)

- Mesos
- Kubernetes

11.4. RDD: Resilient Distributed Dataset

The **RDD** is Spark's low-level data abstraction: an immutable, partitioned collection of records that can be operated on in parallel.

Key properties:

Property	Description
Distributed	Split into partitions, each hosted on an Executor (data locality)
Resilient	Contains full lineage information to recover from partition loss
Immutable	Transformations produce new RDDs; the original is never modified
In-memory	Spark keeps RDDs in Executor memory across the application lifetime
Unstructured	Data stored as Java/Python objects (no imposed schema)

One Executor processes one partition at a time. More Executors can process more partitions simultaneously — this is the unit of parallelism.

11.5. Transformations and Actions

Operations on RDDs fall into two categories:

11.5.1. Transformations

Operations that take one or more RDDs as input and produce a **new RDD**. Transformations are **lazy** — they are not executed until an Action is triggered.

- **Narrow transformations:** each input partition contributes to exactly one output partition (e.g. `map`, `filter`, `flatMap`). Can be pipelined without data shuffling.
- **Wide transformations:** input partitions contribute to multiple output partitions (e.g. `groupByKey`, `reduceByKey`). Require a **shuffle** — data is repartitioned across the cluster.

11.5.2. Actions

Operations that trigger execution and return a value to the Driver (or write data to storage).

Examples: `count()`, `collect()`, `reduce()`, `take(n)`, `saveAsTextFile()`

11.6. Laziness and the DAG Scheduler

Spark defers execution until an Action is triggered. This **laziness** brings two main benefits:

1. **Minimises driver-executor communication:** transformations are accumulated as a recipe, not executed immediately.
2. **Enables optimisation:** Spark can optimise the entire transformation pipeline before executing it.

Spark represents the full computation plan as a **DAG (Directed Acyclic Graph)**:

- The **DAG Scheduler** groups transformations that do not require data exchange across partitions (narrow transformations) into **stages**.
- Operations within a stage are **pipelined**.
- All tasks are dispatched by the Driver to the Executors as an optimised execution plan.

11.7. Spark DataFrame

The **DataFrame** is the high-level structured API, built on top of RDDs:

- Similar interface to Pandas DataFrames.
- Offers SQL queries via the SparkSQL interface.
- Distributed and fault-tolerant (backed by RDDs).
- Suitable for structured and semi-structured datasets with a defined schema.

A DataFrame always has an underlying RDD. The schema information allows Spark to apply additional query optimisations not available to raw RDDs.

11.8. Real-Time Analytics and Structured Streaming

Processing unbounded data streams in real time is common across many domains:

- Sensor readings from physics detectors
- IoT device readings
- Financial transaction monitoring
- Web application event streams

The challenges are significant: potentially out-of-order data, high throughput, exactly-once processing guarantees, and load balancing.

Two common approaches:

Approach	Characteristics
One-Element-at-a-Time	Fixed pipeline; triggered on each new record. Low latency, limited throughput.

Approach	Characteristics
Micro-batching	Accumulate data over a fixed time window (e.g. 500 ms), then process as a batch. Higher throughput, latency limited by batch interval.

A common real-time streaming pattern is the **ETL pipeline**:

- **Extract** — read raw events from a source (e.g. Kafka topic, sensor feed)
- **Transform** — apply business logic: filtering, aggregation, enrichment, join with static data
- **Load** — write processed results to a sink (e.g. database, dashboard, another Kafka topic)

Spark Structured Streaming expresses the full ETL pipeline as a single streaming query, making the code nearly identical to a batch DataFrame transformation.

11.8.1. Spark Structured Streaming

Spark Structured Streaming is the modern streaming API (based on DataFrames, replacing the deprecated Spark Streaming/DStream API (not fully supported since Spark 3.5)).

The key idea: treat the continuous input stream as an **unbounded table** to which new rows are continually appended. Queries over this table are expressed identically to queries over static DataFrames.

Three output modes:

- **Complete**: the entire output table is written to the sink on each trigger.
- **Append**: only rows added since the last trigger are written.
- **Update**: only rows updated since the last trigger are written.

Input sources include Kafka, Amazon S3, TCP sockets, and others. Output sinks include databases, files, Kafka topics, and the console.

11.8.2. Sources and Sinks

Structured Streaming uses a unified **Source** / **Sink** abstraction:

Role	Description	Examples
Source	The origin of the unbounded data stream	Kafka topic, HDFS/S3 file directory, TCP socket, Rate (synthetic test source)
Sink	The destination where processed results are written	Kafka topic, HDFS/S3 file, JDBC database, Console (debugging), Memory (testing)

A streaming query is expressed as: *Source* → *Transformations* → *Sink*.

11.9. Hands-on: Apache Spark

The following hands-on sessions use a standalone Spark cluster deployed on your machine via Docker Compose. The cluster provides:

- A Spark **master** accessible at `spark://spark-master:7077`
- A Spark **UI** at `localhost:8080` (master) and `localhost:4040` (active application)

Note: These notebooks require the Docker cluster to be running and are not executed during book rendering.

11.9.1. Lecture 1: Introduction to PySpark — RDD

Distributed computing over a cluster is a widely adopted strategy in the case of many applications both in- and out-side academia.

Imagine you are interested in working on a very large dataset, at the scale of hundreds of Terabytes or even Petabytes. As discussed in the Data Management part of the course, a large horizontally scalable storage solution is going to be necessary to host the entire dataset. Even if we assume the storage solution is sorted out, we are left with the issue... how can we deal with all these data in order to produce the “final result” we aim at?

- Data are distributed over many nodes, and collecting all data in one node (e.g. loading the entire dataset in a single computer memory) would be impossible
- Not all data in the entire dataset is necessarily going to be relevant to solve your problem. You may want to first load all data, explore it, filter it and pre-process it
- Then, you may want to move to the actual data analysis phase, and possibly to the application machine learning models on the preprocess data. However, even the preprocessed data may not be enough to fit your local machine memory, and once again a distributed system is going to be necessary.

What you would like to do is to run the pre-processing on the nodes of the cluster where the data is distributed, and return only the high level quantities that will be used in the analysis, or even the end-result of the entire analysis.

This is the core concept of distributed computing frameworks like Hadoop and Spark: the code/execution goes to the data, rather than the data coming to the machine where the code resides (data locality).

11.10. Docker Cluster

For this set of lectures we will work with a standalone Spark cluster deployed on your machine via Docker. This is a mock-up of a larger scale cluster setup, and can also be used to develop and test your application before deploying it on a “production” cluster.

We are using `docker compose` to create the cluster with multiple Docker containers acting as computing nodes.

The cluster dashboard is exposed at `localhost:8080` and the master is already available at `spark://spark-master:7077`

12. Spark Session

We can now create the Spark session.

With the following command we are asking to the master (and the resource manager) to create an **application** with the required resources and configurations.

In this case we are using all the default options (e.g. number of core and number of executors), but we can also specify them by hand with `.config("spark.some.config", "value")`.

The list of available configurations can be found [here](#).

```
# import the python libraries to create/connect to a Spark Session
from pyspark.sql import SparkSession

# build a SparkSession
#   connect to the master node on the port where the master node is
    ↪ listening (7077)
#   declare the app name
#   configure the executor memory to 512 MB
#   either *connect* or *create* a new Spark Context
spark = SparkSession.builder \
    .master("spark://spark-master:7077") \
    .appName("My first spark application") \
    .config("spark.executor.memory", "512m") \
    .getOrCreate()
```

Check the content of the Spark Session `spark` object.

This is the entry point to the main Spark functionalities.

```
spark
```

The **Spark UI** link provided won't work, as it refers to the Docker container running the Spark application.

Port mapping is however provided for the 4040 port and you can open the Spark UI link to the application pointing your browser to `localhost:4040`.

From the Spark Session we can access the Spark Context.

The Spark Context is the driver application program we will use to submit applications to Spark, and it is used to work with RDDs

```
# create a spark context
sc = spark.sparkContext

# print its status
sc
```

12.1. Parallelize!

The main feature of Spark is to distribute data in a number of **partitions**, either using the *memory* or the *disk* available in the executors.

Do also keep in mind the difference between the Worker and the Executor in Spark!

The first command we will use is `parallelize()`.

This is a function of the `SparkContext` (`sc`) object and it is used to create a Resilient Distributed Datasets (**RDD**) from a list collection. In other words, it takes a collection and split it amongst the executors.

We will start with a basic example to get familiar with the main functions of Spark RDDs.

Starting from a python “dataset” we create an RDD, i.e. a distributed dataset residing on the cluster, `dist_data`, and then operate on it in parallel.

```
# a simple python list, as the starting dataset
data = [1,2,3,4,5,6,7,8]
```

```
# parallelize it over the available executors!
dist_data = sc.parallelize(data)
```

Go and check the Spark web UI - <http://localhost:8080> for the Master node web UI - <http://localhost:4040> for the SparkContext web UI (where your application will reside)

In principle we could expect the application to report something related to our request of parallelizing data over the cluster...

But instead... nothing happened in the Web UI, even though we asked Spark to parallelize our data into a number of partitions!

That's because `parallelize` is not an **action**, but a **transformation**, and thus is *lazy* by nature.

We can trigger it by using an action, for instance the `count` action, a function used to count the number of elements in the RDD.

```
# count elements of the rdd
dist_data.count()
```

Let's go back and check the Spark web UIs and interpret its content...

If you are using two executors (i.e. two cores of your CPU in this example), in the WebUI you will see two parallel lines for this task.

That's because the RDD consists of two **partitions**.

Indeed, under the hood, the RDDs are stored and represented as partitions, “small blocks” of the original dataset that will be **unit of parallelism**.

One partition can be processed by one PU (e.g. a core of a CPU) at a time. More PUs can thus operate on multiple partitions at the same time, with the limit of 1 PU per partition.

In other words, if the RDD is composed of one partition, even if we have two total available cores, then only one task at a time will be run.

On the other hand, if the RDD partitions are four and we still have two available cores, only two of the partitions will be processed in parallel.

To split our data in an arbitrary number of partitions we can use `numSlices`

```
# parallelize using a different number of partitions
sc.parallelize(data, numSlices=8).count()
```

We can ask Spark to count the number of partitions of a given RDD with the method `getNumPartitions()`

```
# sc.parallelize() is itself an RDD
# thus we can call getNumPartitions() on the RDD created via the parallelize
  ↪ transformation
sc.parallelize(data, numSlices=8).getNumPartitions()
```

By default, Spark will try to split the dataset into a number of partitions equal to the number of executors.

Not always successfully though...

To parallelize your computation effectively, you should make sure what is the number of slices your dataset is parallelized into, and how it compares with the available computing resources.

```
# get the number of partitions of the RDD created previously
dist_data.getNumPartitions()
```

It's important to visualize that **the RDD resides in the worker nodes**, and not in your client/local machine.

If we print the `dist_data` object from Jupyter we will not see the actual data contained in the RDD, but only that this is a distributed dataset.

```
# print the dist_data object from here
print(dist_data)
```

12. Spark Session

This is actually a very important concept, as in a distributed system the datasets we are processing are typically much larger than the available memory of the driver (it's one of the main reasons we use a distributed system in the first place).

Returning the content of the whole dataset in a single machine is not a clever idea.

However, a distributed dataset can be collected in the driver with `collect()`.

By issuing this action, **all the workers will send back all the blocks of the RDD.**

This is generally used to collect only the end results after a computationally heavy processing is first run on the worker nodes, and a slim and manageable result can be retrieved.

This function should be **used with caution** since **it fetches the entire RDD to a single machine** and can cause the driver to run out of memory.

```
# collect the distributed dataset (so far, a mere list)
dist_data.collect()
```

If we have a massive dataset and we are looking for inspecting only a small subset of elements we can use `take(num)`.

`take` allows to return only the first `num` number of elements from the RDD, thus limiting the amount of data we are collecting from the cluster.

Under the carpet, Spark will look at one partition's data, and will decide if one is enough to satisfy your request, or it will need to retrieve data from more than one.

```
# retrieve a few elements from the RDD
dist_data.take(3)
```

12.2. Map and Reduce

We can now write our first map-reduce application.

The method `map(f)` will apply a function `f` to each *element* of the RDD.

For instance, let's create a simple function to increment by 1 each of the elements of the RDD.

```
[1,2,3,4,5,6,7,8] --> [2,3,4,5,6,7,8,9]
```

The function itself is very simple, we can use a Python `lambda` function to take care of that:

```
lambda(x: x+1)
```

Remember, **this is a transformation**, not an action. Being a `map` phase-equivalent to the Hadoop Map-Reduce, we have the idea that all partitions will be operated in parallel.

To trigger the transformation, we can chain it with an action, such as `collect`.

```
# increment each value of the array with map
# and collect the result
dist_data.map(lambda x: x+1).collect()
```

Differently from plain Hadoop, in Spark **we can chain together multiple transformations, without having to strictly follow the Map-then-Reduce pattern.**

For instance, we can even run an entire chain of map functions one after the other with no reduce function at all.

Let's chain a +1 increment function with a /2 function to divide the result of the previous multiple map transformations.

We can continue using map and lambda functions to do this, and chain a .map() after other .map() functions.

```
# multiple map transformations
# plus collect (action)
dist_data.map(lambda x: x+1)\
    .map(lambda x: x/2)\
    .collect()
```

So far we can see that we have already moved away from the strict pattern of Map-Reduce “a-la-Hadoop”, where every operation is a single Map phase followed by a single Reduce phase.

In this example we have instead 2 Map phases, and a single Reduce phase (the collect).

The optimization provided by the DAG scheduler will hide the map functions under the same processing stage, and will take care of running them pipelined as fast as possible.

Another important operation available is an explicit **reduce** function.

Reduce is an **action** that aggregates all the elements of the RDD using some user-provided function, and returns the result to the driver.

We will see later on another version of reduce, **reduceByKey**, which performs the reduction for elements with the same key, and we'll reproduce the wordcount example.

For the time being, let's explore a simple **reduce** implementation.

Reduce actions can take a **function as argument**, which will be applied to *all the elements of the RDD*, to perform some kind of **aggregation**.

For example, we can think about summing the elements of a RDD in pairs with a lambda function.

```
# increment each element of the dataset (transformation)
# and sum all of them in pairs (action)
dist_data.map(lambda x: x+1)\
    .reduce(lambda x, y: x+y)
```

A schematic view of this simple Map-Reduce application can be seen sketched here:

12. Spark Session

Please note how the reduce function does not necessarily have to complete its task in a single iteration. In this case, the sum in pairs will continue on until we are left with a single scalar.

12.2.1. Exercise 1

Write a simple reduce function to find the minimum of `dist_data`.

A simple approach is to evaluate the min of pairs of values `x` and `y`, by reducing the `min(x,y)` function.

```
# reduce the min function over pairs of values for the dist_data dataset
```

12.2.2. Exercise 2

Create a set of 100 points in a 3D space, rapresented as tuples of randomly-generated floats, and calculate the maximum distance of the points from the origin (0,0,0).

1. Create the collection of tuples on the client (Python)
2. Parallelize the collection in a Spark RDD
3. Use Spark to evaluate the euclidean distance of each point from the origin
4. Use Spark reduce to evaluate the maximum distance

```
# 1. Create the collection of tuples on the client (Python)
```

```
# 2. Parallelize the collection in a Spark RDD
```

```
# 3. Use Spark to evaluate the euclidean distance of each point from the  
↪ origin
```

```
# 4. Use Spark reduce to evaluate the maximum distance
```

We should keep in mind that every time we move data **from the client to the cluster** or **from the cluster to the client** we are potentially paying a large price: - moving data from the client to the workers implies a large amount of data transfers, through the driver, and to the executors. If we can read data directly from the worker nodes (we'll see later a couple of examples) this should always be preferred. - on top of this issue, moving data from the workers to the client implies an additional risk. We might have more data on the cluster than what we can afford to store locally on our client machine. Moving back data (i.e. doing a `collect`) from the cluster might crash the driver, and crash your application. Refrain from asking Spark to `collect` an entire GB-, TB- or even PB-sized dataset to your laptop.

12.2.3. — *Intermezzo* — Broadcasting small shared data

Sometimes many parallel tasks need access to the same small piece of common data.

A typical example is a lookup table, a dictionary, or a small configuration object.

In Spark, we can explicitly broadcast this object to the workers using `sc.broadcast`.

This avoids sending the same object repeatedly with every task, and instead it will preserve it in the memory of all the executors for the duration of the entire task.

```
# some common data
lookup = {
    "a": 1,
    "b": 2,
    "c": 3,
}

# broadcast the lookup dictionary to all executors
lookup_broadcast = sc.broadcast(lookup)

# create an RDD with some keys
rdd = sc.parallelize(["a", "b", "c", "a", "d"])

# use the broadcast variable to look up values for each key in the RDD
result = rdd.map(lambda x: lookup_broadcast.value.get(x, 0))

# collect the results
result.collect()
```

12.2.4. Exercise 3 - compute π

Estimate π by simulating random points in the unit square (side length of 1) and counting how many fall in the unit circle.

The probability of one point falling inside the circle is

$$P = \frac{\text{Area circle}}{\text{Area square}} = \frac{\pi}{4}$$

We can estimate this probability by counting the number of simulated points inside the circle

$$P = \frac{\text{\#Points in circle}}{\text{\#Points}}$$

Thus obtaining

$$\pi \approx 4 \cdot \frac{\text{\#Points in circle}}{\text{\#Points}}$$

This can be done in Spark running the computation in parallel with the following steps: 1. Create a “dummy RDD” containing placeholders for all the points (e.g. use the value 0 as placeholder for each data point) 2. Generate the points as random (x,y) pairs and check if each of them falls inside/outside the circle 3. Count the points inside the circle

12. Spark Session

Think about which transformations are needed to generate the points and to check if they fall inside the circle. Combine all the transformations and the action in a single pipeline.

```
num_points = # a reasonable amount, such as 100_000

# instantiate a "placeholder RDD"
points_rdd =

def in_circle(dummy):
    # generate a point randomly and check if
    # it is inside the circle
    # by returning 0 or 1

# count the number of points inside the circle

# print the final result
```

The same result can be achieved using the `filter` transformation.

`filter(f)` returns a new RDD containing only the element of source RDD on which `f` is `true`.

```
# perform the same exercise using filter
points_inside = points_rdd \
    .map(""" the map function """) \
    .filter(""" the filter function """) \
    .count()

# print result
```

Reduce by key and flat map

Consider the following dataset where each element consists of a tuple (`group`, `value`) (a simple example of a **key-value pair**).

This can be for example: - `group` = product class, and - `value` = revenue

Or readings from a set of sensors: - `group` = sensor identifier - `value` = reading

```
class_list = [('group1', 10),
              ('group2', 4),
              ('group3', 1),
              ('group2', 7),
              ('group1', 8)]
class_rdd = sc.parallelize(class_list)
```

We may be interested in operating only on the *values* of the dataset, discarding the *keys*.

We can write a map function to do this, using the python syntax to access only the `[0]` (the key)

or the [1] element of the tuple.

For instance, we can implement a simple increment of +1 to all values via a `map` function.

```
# operate only on the values
class_rdd.map(lambda el: (el[0], el[1]+1)).collect()
```

The same result can be achieved using `mapValues`.

This allows to write an equivalent `map` function that applies only to the values of the groups disregarding the keys.

```
# do the same using map values
class_rdd.mapValues("increment values by 1").collect()
```

We can perform a reduce function for each class using `reduceByKey`: in this way we are applying the reduce function only to all the elements of the same class.

Despite its name, `reduceByKey` is not an action, but a transformation, since it returns a distributed dataset (the result of an aggregation by key could still give a very large number of items, if the number of keys is large).

Let's compute the minimum value of all elements of each group using `reduceByKey` and collecting the result.

```
# compute the minimum using reduce by key
class_rdd.reduceByKey("compute the minimum per group")\
    .collect()
```

We can further filter our results by using `takeOrdered` to get the first `n` results sorted by a given field.

By default, `takeOrdered(n)` takes the first `n` elements of the RDD in ascending order. To get the descending order (or any other arbitrary order) we can specify a function, such as:

```
lambda x: -x
```

Get the maximum value for each group, and return only the 2 with the largest values

```
class_rdd.reduceByKey("compute the maximum per group") \
    .takeOrdered("select the 2 elements with largest value")
```

If we are not interested in treating the dataset as key-value pairs, or in general we have a non-flat dataset (e.g. each element of the RDD is a list, or another object), we can use `flatMap` to “explode” each element returning a plain sequence of elements.

This can be extremely useful if you have data stored as arbitrary data structures, but you would like to use them as plain lists of items.

12. Spark Session

```
# flatten the class rdd
class_rdd.flatMap(lambda el: el)\
    .collect()
```

```
# create a list of elements, and flatten them into a single list of items
sc.parallelize([[1,2,3], [2,3,4]]) \
    .flatMap(lambda x: x) \
    .collect()
```

Exercise 3: word count

Let's rerun the word-count example seen in class when discussing the Map-Reduce programming paradigm, starting from the following arbitrary text.

```
message = [
    'Three Rings for the Elven-kings under * the sky,',
    'Seven for the Dwarf-lords in $ their halls of stone,',
    'Nine for Mortal # Men doomed to die,',
    'One for [ the Dark Lord on his dark throne',
    'In the Land of _ Mordor where the Shadows lie.\n',
    'One Ring to } rule them all, One @ Ring to find them,',
    'One Ring $ to bring them all and in the darkness bind them',
    'In the Land of Mordor * where the Shadows lie.\n'
]

print(''.join(message))
```

The goal is counting the occurrence of each word using a map-reduce approach in Spark.

But first, you may want to need to clean-up the dataset by removing all “stray” symbols such as @ and \$.

To avoid double counting the same word when in lower- or upper-case, you may also want to change all the words to lower case.

Hint: use `string.punctuation` or a regular expression to remove the unwanted characters

```
import string
string.punctuation
```

```
# parallelize the message
message_rdd = sc.parallelize(message)
message_rdd.collect()
```



```

# step 1: create the words_rdd where each element is a word
# eg: ['one', 'ring', 'to', 'rule', 'them']

def clean_row(row):
    # this function should:
    # 1. clean a row from punctuation
    # 2. put each word in lowercase
    # 3. and return the list of words
    word_list = []

    # your code

    # ---

    return word_list

# apply transformation
words_rdd = """ your transformation """

# print a few items
words_rdd.take(10)

```

```

# step 2: perform word count!
# -> crate pairs (word, 1)
# -> reduce based on the word and count

word_count_rdd = """ your word-count """

# print your result
word_count_rdd.collect()

```

The method `takeOrdered(n, key)` allows to collect only the first n elements based on the ordering provided by key. Ordering is by default ascending.

```

# collect the 10 most occurent words
word_count_rdd.""" your takeOrdered """

```

12.2.5. Exercise 4 - Distributed linear regression

The files in `../datasets/lecture1/` contain a series of measurements of a sensor. Each measure is in the form

Measure N: (t,val)

12. Spark Session

There could be pairs such as `(t, None)`, corresponding to missing values. This measure should be removed.

We are interested in knowing if there is any sort of relation between `t` and `val`.

The first task consists in reading each file and creating an RDD containing as element the pairs `(t, val)`.

We can first inspect the first lines of a file to get a sense of the data:

```
!head ../datasets/lecture1/file_1.txt
```

Imagine that each file contains millions of measurements and there may be thousands of files.

Reading all the files with Python on our local machine (client-side), and then using `parallelize` should be avoided at all costs. This would be a very inefficient way to read the data, as we would have to read all the files in the driver (possibly getting out of memory), then splitting the data in the workers.

Instead, we can parallelize the *list of files* and then read all files in parallel directly from the Spark executors.

```
base_path = '/mapd-workspace/datasets/lecture1/file_{}.txt'
file_list = [base_path.format(i) for i in range(1,5)]

files_rdd = sc.parallelize(file_list)
files_rdd.collect()
```

12.2.5.1. Note

In this case we could have used the built-in Spark function `textFile`, which takes as input the path to the files and reads them directly into an RDD.

To access a file with this function we need to prepend `file://` if the file is in the local file system. If the file is in Hadoop the path will be similar to `hdfs://namenode.com:8020/path_to_the_file`. In the same way, if the file is stored on object stores using the `s3` API (such as the one used in CloudVeneto) the path will be `s3://`.

The Spark function `textFile` could be used in this case as the input files are stored in a plain text `.txt` format. In many other cases, however, a custom data loader is required instead, as the data will be stored in a specific format, impossible for Spark to interpret without some instructions. On the other hand, we should try to use the built-in data readout functions as much as possible since they are much more performant than our custom Python code.

```
text_rdd = sc.textFile("file://" + base_path.format('*'))
text_rdd.take(4)
```

In this example we will write a custom data loader, that could be generalized to all kinds of files and data formats.

Starting from the RDD files, write a map function that convert data into an RDD, `data_rdd`, where each element is a tuple (`t`, `val`).

During this preprocessing phase remember to remove points with `None` as measure.

The elements of `data_rdd` should finally be something like:

```
[(0.0,9.93), (-1.0,9.02), ...]
```

In order to do this, we can use *regular expressions (or regex)*.

A regular expression is a sequence of characters that allows to identify and select a **pattern** in a string, instead of a specific substring. Most editors (including Jupyter) allow to use regex to find and replace text based on patterns, and several interesting sites offer a simple interface to learn and experiment with regexp, e.g. <https://regex101.com/>

All following characters are considered “meta-characters” in regex

```
. ^ $ * + ? { } [ ] \ | ( )
```

In a regular expression, all these symbols will be **not** interpreted as their characters, but are used to build up the patterns and search for matches in the text.

The main notable examples are:

[] square brackets meaning exactly 1 character

regexp	interpretation
[a]	the letter a
[ax]	the letter a or x
[a-x]	any letter from a to x
[a-zA-F]	any letter from a to z and from A to F
[0-9]	any number from 0 to 9

^ represents a **not**

regexp	interpretation
[^b02]	all characters but b , 0 and 2

. represents any character at all

regexp	interpretation
.	all characters

? makes the preceding character in the regular expression optional

12. Spark Session

regexp	interpretation
colou?r	color or colour

* match the preceding character (or combination) zero or more times

regexp	interpretation
go*gle	ggle,gogle,google,gooogle,...

\t matches a tab

\n matches a new-line

^ (outside of []) matches the start of a line

\$ matches the end of a line

Regex can be used in plain Python with the `re` module.

Let's use the `re` module to implement the appropriate regular expression to clean up our data.

We should find all float values possibly starting with a +/- sign and collecting them in pairs.

```
# test the regex pattern with a fake measurement
import re

fake_meas = 'Measure 0: (.05,0.70)'
float_pattern = '-?[0-9]*\.\.[0-9]*'
test_ = re.findall(f'{float_pattern},{float_pattern}', fake_meas)
test_
```

```
def parse_file(file):
    # this function should:
    # 1. open the file and read all lines
    # 2. apply the regex to extract the values
    # 3. sanify the data by removing all measurement pairs with
        ↪ non-numerical values
    # 4. create a `points` list where each entry is a tuple of the two
        ↪ values/coordinates
    points = []

    # your code

    # ---

    return points
```

```
data_rdd = """ parallelize the reading and parsing of the measurements """
data_rdd.take(10)
```

With `sample(with_replacement, fraction)` we can sample some points from the original RDD.

This is useful if we want to collect only a given `fraction` of our data, without dumping the entire dataset with a `collect`.

```
import matplotlib.pyplot as plt
import numpy as np

# collect a sub-sample of your data
data = np.array(data_rdd.sample(False,0.2).collect())

plt.scatter(data[:,0], data[:,1]);
plt.xlabel(r'$p_x$');
plt.ylabel(r'$p_y$');
```

We will now implement a distributed gradient descent and use it to find the best parameters for a linear model.

Given an input X the output of the model is

$$Y = WX = w_0 + w_1x_1 + \dots + w_px_p$$

In our case, we have $Y = y(w, x)$, $W = [w_0, w_1]$ and $X = [1, x]^T$, in other words

$$y(w, x) = w_0 + w_1x$$

We are interested in estimating the optimal parameters w^* of the line fitting our data.

To do this we will use **gradient descent**, an iterative procedure that allows us to find a local minimum of a differentiable function.

In our case, we would like to minimize the square residuals, i.e.

$$J(W, X) = \frac{1}{2n} \sum_{i=1}^n [Y(W, X) - y_i]^2 = \frac{1}{2n} \sum_{i=1}^n [(w_0 + w_1x) - y_i]^2$$

where y_i is the true value.

In each step of the gradient descent we use the following update rule

$$W_{i+1} = W_i - \gamma \nabla J_W(W_i, X)$$

where γ is the learning rate, a variable used to reduce the size of each step.

12. Spark Session

In other word we are moving in the opposite direction of the gradient, i.e. towards the minimum of the function.

Recalling that $W = [w_0, w_1]$ and $X = [1, x]$ we have that each component of the gradient, i.e. $\frac{\partial}{\partial w_p} J(W, X)$

$$\nabla J(W, X) = \left[\frac{1}{n} \sum_{i=1}^n [Y(W, X) - y_i] \cdot 1, \frac{1}{n} \sum_{i=1}^n [Y(W, X) - y_i] \cdot x_i \right]$$

We can now write a map-reduce job used to estimate the parameters using gradient descent on the full dataset.

We start by defining the weights vector and initializing it to some values, e.g. (10, 0.5) is a good guess...

```
# use numpy to create the W weight array
W = """ a simple numpy array """
```

Implement then the functions computing the prediction given as input x and the current weights W

```
# predict y for any given x and weights W
def predict(x, W):
    # return prediction

    # ---

# test the function in local

# ---
```

Implement the function computing the gradient for one example, given as inputs: 1. the point $P = (x, y)$, and 2. the current set of weights W .

Remember that the gradient has two components, one per parameter.

Furthermore, the normalization $\frac{1}{n}$ can be omitted since we can apply it afterwards, having summed the gradients of all examples.

```
# compute the gradient
def gradient(P, W):
    # return the prediction given the x value of P and the weights W
    # and compute gradient

    # ---

    return gradient
```

```
# test the function in local

# ---
```

We are now ready to implement the gradient descent and find the optimal line parameters.

Hint: compute the gradient for each point in parallel, and then sum all of them. Finally, use this sum to update the weights vector.

```
# count points based on the partitioned data
num_points = """ count the points in the partitioned data """

# re-declare the weight vector here:
# this is useful if the cell is run multiple times
W = """ a simple numpy array """

# define the learning rate
lr = 0.01

# define the number of iterations
num_it = 20

for i in range(num_it):
    # run the gradient descent in parallel and then sum the gradients
    grad = """ the parallelized gradient evaluation """
    # ---

    # update the weights according to the learning rate and the gradient
    W = """ the weights update """
    # ---

# print the results
print("Final parameters: x0={:.2f}, x1={:.2f}".format(W[0], W[1]))
```

Check out the results by plotting the data points and the resulting best fit line.

```
data = np.array(data_rdd.collect())

plt.scatter(data[:,0], data[:,1]);
x = np.arange(-10,11)
y = W[0] + W[1]*x
plt.plot(x, y, color='red', lw=2);
plt.xlabel(r'$x_{0}$');
```

```
plt.ylabel(r'$x_{1}$');
```

Finish up by computing the residuals using Spark, and plot their distribution in a histogram. The residual of the point (x_i, y_i) with respect to the model $y(x)$ is simply defined as

$$R_i = y(x_i) - y_i$$

```
# compute the residuals
residuals_rdd = """ residual evaluation """
```

```
# plot its distribution
plt.hist(residuals_rdd.collect(), range=(-1,1), bins=50);
plt.xlabel("residuals");
plt.ylabel("counts");
```

Note

By running the previous cell we asked Spark to return the entire **residual** RDD object. This is not a sizeable dataset in this specific example, but it might be the case in a real-life scenario, where you may have lots of records stored in a distributed cluster. Returning the entire dataset just to produce a histogram is extremely inconvenient, as it implies large data transfers from the executors to the driver, and finally to the client (possibly resulting in getting out of the driver memory).

What you should be doing instead is to evaluate the histogram remotely and in parallel, and return the histogram as a set of bins and counts. This way the data transfer from the executors to the driver is minimized, avoiding possible issues.

You should be able to write a Map-Reduce function to create an histogram (consider this an **Extra-exercise**), but Spark is also providing some built-in functions to compute simple histograms remotely.

```
# use the spark RDD histogram method to return the histogram from the
  ↪ cluster
hist = residuals_rdd.histogram(np.linspace(-1,1,50).tolist())
plt.stairs(hist[1], hist[0]);
plt.xlabel("residuals");
plt.ylabel("counts");
```

12.3. Caching

From the WebUI we can see that in each iteration Spark is computing every operation from the very beginning, i.e. **starting from reading the list of filenames and running the parallelization of the text files!**. This would not really necessary, as this first operation should not need to be re-executed every time we need to access the dataset.

With `persist()` we can instruct spark to put intermediate results into the executors' memory, e.g. after the function parsing the files.

In this way the same dataset will be loaded in the next iterations much faster, at the cost of having the dataset stored in memory.

Note-1: to be precise, there could be different levels of *persistence* of data into the executors' memory.

Note-2: one should be very careful not to *abuse* the caching of datasets. Caching itself takes time and resources, and caching all intermediate stages of your dataset might end up taking a toll on the performance (and saturating the available memory of the executors).

```
# persist the original RDD
data_rdd = data_rdd.persist()

# persist is a transformation, it will be triggered
# only once we apply an action on the RDD!
data_rdd.count()
```

It is worth noting that, in Spark, `persist()` marks the RDD for persistence and returns the same logical RDD handle.

In practice, both of the following are valid:

```
data_rdd.persist()
```

and

```
data_rdd = data_rdd.persist()
```

In this and later notebooks, we will prefer the second form, with explicit assignment. This is not strictly required by Spark, but it makes the intent visible in the code: from this point onward, we want to work with the persisted version of the dataset. It is also a useful habit because other frameworks, such as Dask, do not always follow exactly the same caching semantics, and explicit reassignment makes the pattern more portable.

```
# run the same code as previously
# and have a look at the spark application webUI
# under the Storage tab

print("Final parameters: x0={:.2f}, x1={:.2f}".format(W[0], W[1]))
```

The RDD can (actually, **should**) be removed from memory with `unpersist()`, freeing up the executors' memory when needed.

12. Spark Session

```
# free up the memory  
data_rdd = data_rdd.unpersist()
```

13. Stop worker and master

```
# stop the running Spark context  
sc.stop()
```

```
# stop the running Spark session  
spark.stop()
```

Finally, use `docker compose down` to stop and clear all running containers.

13.0.1. Lecture 2: Spark DataFrames

Spark SQL is the module within Spark dedicated to storing and processing structured datasets.

As we discussed at the beginning of this course, structured datasets are common in both scientific and non-scientific environments.

Structured data is typically represented using abstractions such as Pandas Dataframes or relational database tables.

In Spark, structured datasets are referred to as **Spark Dataframes**. A Spark Dataframe is built on top of the RDD low-level API, offering the same underlying logic concerning storing and partitioning data across multiple nodes, but also providing a richer set of functionalities and optimizations.

13.1. Create and Start a Spark Session

```
# import the python libraries to create/connect to a Spark Session  
from pyspark.sql import SparkSession  
  
# build a SparkSession  
# connect to the master node on the port where the master node is  
  ↳ listening (7077)  
# declare the app name  
# configure the executor memory to 512 MB  
# either *connect* or *create* a new Spark Context  
spark = SparkSession.builder \
```

```
.master("spark://spark-master:7077")\  
.appName("My dataframe spark application")\  
.config("spark.executor.memory", "512m")\  
.config("spark.sql.execution.arrow.pyspark.enabled", "true")\  
.config("spark.sql.execution.arrow.pyspark.fallback.enabled", "false")\  
.getOrCreate()
```

```
spark.sparkContext.setLogLevel("ERROR")
```

```
# create a spark context  
sc = spark.sparkContext  
  
# print its status  
sc
```

13.2. Creating a Spark DataFrame

A Spark DataFrame is a **distributed** collection of data grouped into named columns.

A Spark DataFrame is already very similar to the Pandas DataFrame API, and it can be created in a multitude of possible ways:

- Creating and appending a list of Spark Rows (Spark objects for records) with an *implicit* schema definition.
- Creating and appending a list of Python tuples with an *explicit* schema definition.
- Importing a Pandas DataFrame.
- From the results of a SQL query over a database or another DataFrame.
- From a collection of input files provided with a schema.

13.2.1. Create a Spark DataFrame from a List of Rows

The Spark Row is the native Spark object that hosts the data of a record in a Spark DataFrame. It's equivalent to a Python tuple or a dictionary.

```
# create a spark dataframe from a list of Rows  
  
# import the Row class from the pyspark.sql module  
from pyspark.sql import Row  
from datetime import date, datetime  
  
# create a list of Row objects  
# each Row object contains a list of values
```

```

# the list of values can be of different types
# the list of values must be in the same order as the list of columns
df = spark.createDataFrame([
    Row(a=1, b=2., c='string1', d=date(2000, 1, 1), e=datetime(2000, 1, 1,
↪ 12, 0)),
    Row(a=2, b=3., c='string2', d=date(2000, 2, 1), e=datetime(2000, 1, 2,
↪ 12, 0)),
    Row(a=4, b=5., c='string3', d=date(2000, 3, 1), e=datetime(2000, 1, 3,
↪ 12, 0))
])

# print the spark dataframe
df

```

13.2.2. Create a Spark DataFrame from a List of Tuples

PySpark allows passing records as a list of Python tuples. However, to allow Spark to interpret the tuple fields correctly, an **explicit schema definition** is required.

```

# create a spark dataframe from a list of Python tuples

# create a list of Python tuples
# when creating the spark dataframe we must also
# pass the definition of the data types (i.e. the schema)
df = spark.createDataFrame([
    (1, 2., 'string1', date(2000, 1, 1), datetime(2000, 1, 1, 12, 0)),
    (2, 3., 'string2', date(2000, 2, 1), datetime(2000, 1, 2, 12, 0)),
    (3, 4., 'string3', date(2000, 3, 1), datetime(2000, 1, 3, 12, 0))
], schema='a long, b double, c string, d date, e timestamp')

# print the spark dataframe
df

```

13.2.3. Create a Spark DataFrame from a Pandas DataFrame

PySpark can create a DataFrame starting from a Pandas DataFrame.

It should be noted that a Pandas DataFrame is meant to be stored in the memory of a local machine, while Spark is designed for hosting large datasets on storage/memory across multiple nodes.

Moving data that can be hosted on a local machine using Pandas to a remote cluster using Spark is typically not a very common use case.

13. Stop worker and master

```
# import pandas
import pandas as pd

# create an equivalent pandas dataframe
pandas_df = pd.DataFrame({
    'a': [1, 2, 3],
    'b': [2., 3., 4.],
    'c': ['string1', 'string2', 'string3'],
    'd': [date(2000, 1, 1), date(2000, 2, 1), date(2000, 3, 1)],
    'e': [datetime(2000, 1, 1, 12, 0), datetime(2000, 1, 2, 12, 0),
          ↪ datetime(2000, 1, 3, 12, 0)]
})

# create the spark dataframe from the pandas one
df = spark.createDataFrame(pandas_df)

# print the spark dataframe
df
```

As in the case of the creation of an RDD, the mere creation of a Spark DataFrame does not imply any work is submitted to the executors.

Whenever we ask Spark to retrieve something from the dataset, we instead trigger the execution of the jobs (as usual, subdivided into stages and tasks), starting from loading the dataset into the memory/storage of the remote cluster nodes.

To use a Spark DataFrame we have similar interfaces as in Pandas: - `DataFrame.show()` to visualize its content (i.e. its state/instance) - `DataFrame.printSchema()` to visualize the schema of the structured dataset

```
# print the first 10 rows (if available) of the dataframe
df.show()
```

```
# print the schema of the DataFrame
df.printSchema()
```

13.3. Loading Structured Data

By default, Spark can create a DataFrame from data stored in many formats such as `csv`, `json`, `avro`, `parquet`, and many others listed [here](#).

If your dataset is stored in a format that Spark DataFrame cannot understand and interpret natively, it is always possible to first create an RDD as in the previous lecture and later convert the RDD into a DataFrame with the `toDF()` functionality.

Here we create some mock-up structured data in the form of a simple set of records with 2 features: `feature1` and `feature2`.

We can parallelize the creation of the `Rows` in Spark as if we were loading and “unpacking” or parsing data from a set of files.

```
# import numpy for random
import numpy as np

# mock-up function to load data from a file
def read_custom_data(file_name):
    # this is a placeholder code for some operation
    # you might have to perform when reading the files
    # for example, you might have to read the file line by line
    # and parse the lines into numerical values
    custom_data = []
    # create 5 random events
    for _ in range(5):
        # each event is a pair of random numbers
        event = {
            'feature1': np.random.random(),
            'feature2': np.random.random()
        }
        # append each event to the list as a Spark Row object
        custom_data.append(Row(**event))
    return custom_data

# fake list of files to read
file_list = ['file1', 'file2']

# create an RDD parallelizing the read function on the file list
rdd = sc.parallelize(file_list)\
    .flatMap(read_custom_data)

# the RDD contains a list of Row objects, which can be converted
# to a DataFrame by using the toDF() function
df = rdd.toDF()
```

The `df` object is now a Spark `DataFrame` containing a list of `Rows`.

However, Spark will always remember the `DataFrame` is actually a `RDD`, and we can always operate and retrieve the underlying `RDD` object from the `DataFrame` API, simply by using `collect`.

```
# print the whole set of values in the RDD
rdd.collect()
```

13. Stop worker and master

```
# perform the same operation on the dataframe  
df.collect()
```

Issuing a `show()` or other high-level API functionalities designed for the `DataFrame` will instead “wrap” around the `RDD` to present the output with the proper `DataFrame` schema.

```
# call a DataFrame show() method to display the first 5 rows of the  
↪ DataFrame.  
df.show(5)
```

13.4. The `pyspark.pandas` API

Quite recently (in the last 2/3 years), PySpark introduced a dedicated `Pandas` API on Spark, geared mainly for new users to make it even easier to move to and from the `Pandas DataFrame` and the `PySpark DataFrame` equivalent.

```
# import the pandas api from pyspark  
import pyspark.pandas as ps
```

With the `pyspark.pandas` API, `DataFrame` creation is now streamlined and almost identical to plain `Pandas`.

Let’s create a simple `Pandas DataFrame` and its equivalent `PySpark` one using the `pyspark.pandas` API.

```
# create a dummy pandas dataframe  
pandas_df = pd.DataFrame(  
    {'a': [1, 2, 3, 4, 5, 6],  
     'b': [100, 200, 300, 400, 500, 600],  
     'c': ["one", "two", "three", "four", "five", "six"]},  
    index=[10, 20, 30, 40, 50, 60])
```

```
# print the pandas dataframe  
pandas_df
```

```
# create the equivalent pyspark pandas dataframe  
pyspark_pandas_df = ps.DataFrame(  
    {'a': [1, 2, 3, 4, 5, 6],  
     'b': [100, 200, 300, 400, 500, 600],  
     'c': ["one", "two", "three", "four", "five", "six"]},  
    index=[10, 20, 30, 40, 50, 60])
```



```
# print the pyspark pandas dataframe
pyspark_pandas_df
```

A good fraction of the common Pandas functions will still work on `pyspark.pandas`, so most probably a good fraction of the pure-Pandas code you have might as well work in Spark with little effort, and thus be parallelized and sped up with little to no effort.

However, not all Pandas functionalities are implemented in PySpark. For this reason, we will mostly use the PySpark DataFrame in this notebook to continue with the more “traditional” and robust approach.

Nonetheless, you are suggested to take a look at the documentation on the Pandas API on Spark at the [link](#), and to the [best practices](#) and [supported Pandas API](#) if you want to use the PySpark-Pandas API more in your project.

NOTE

We also have to keep in mind that *Pandas and PySpark are very different “under the hood”*. Where **Pandas** is **single-threaded** and will host the DataFrame data in **local memory**, **PySpark** is designed to host data into **partitions** using the **executors’ memory (or storage)** and process it in a **distributed** way.

This imply that many operations that are ideally very simple might be either impossible or simply very inefficient in PySpark.

For instance... how do you think the ordering of the entries in the table is handled when your dataframe is split into possibly hundreds of partitions? What if you want a dataframe to be sorted?

The data in a Spark dataframe does not preserve the natural order by default... We can force Spark to rearrange data in its “natural” (e.g. from an index) order but this causes a performance overhead with an additional stage of internal data sorting (a wide-dependency transformation).

13.5. DataFrame Analysis Example

Let’s run an end-to-end example of a simple data analysis performed using the PySpark DataFrame API.

For this example, we’ll use a subsample of data produced by a particle physics experiment at CERN and attempt to reconstruct the mass spectrum of a particle from its decay products.

13.5.1. The Dimuon Mass Spectrum

Several particles decay into a pair of oppositely charged leptons (electrons, muons, and taus).

The dimuon spectrum, computed by calculating the invariant mass of muon pairs with opposite charge, features the presence of a number of narrow resonances corresponding to the mass of the parent particle: from the ρ meson at about 548 MeV up to the Z boson at about 91 GeV.

Rare processes are also associated with this very same final state, such as the Bs dimuon decay (first observed in 2012 at CMS and LHCb) and the elusive Higgs dimuon decay (for which there is

13. Stop worker and master

statistical evidence, but not yet an observation); new yet undiscovered particles might also show up as new resonances in the dimuon spectrum as the statistics and accelerator energy are increased.

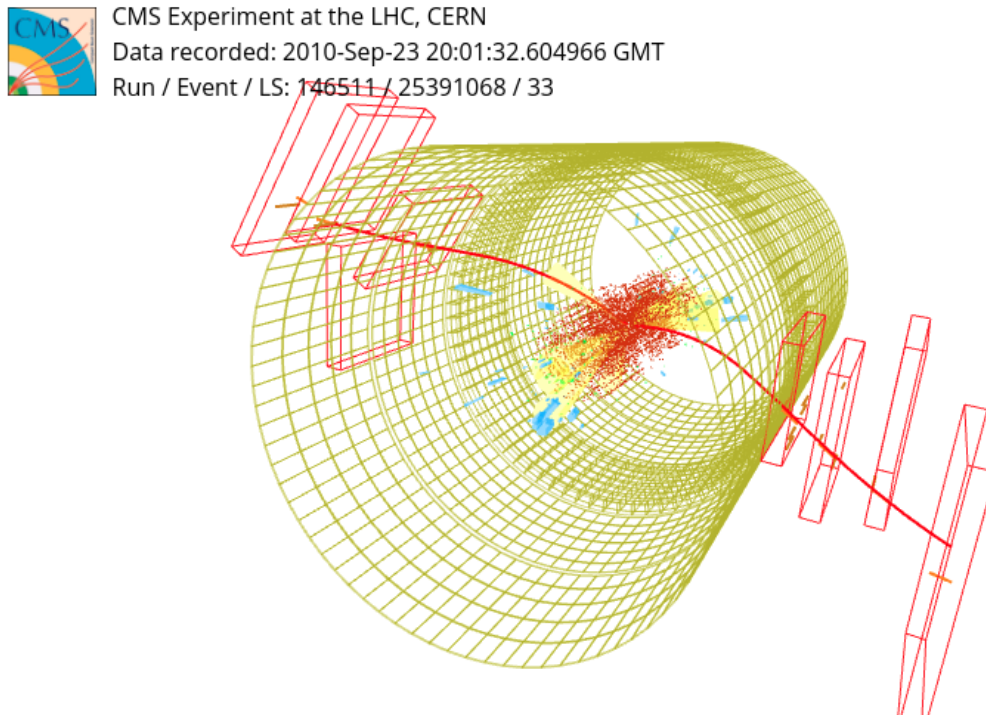


Figure 13.1.: Event Display

The dataset used in this exercise is taken from the CERN Open Data portal (<https://opendata.cern.ch/>) and represents only a tiny portion of the whole data collected by the CMS collaboration at the Large Hadron Collider in 2010.

This dataset comprises only the fraction of events (i.e., particle collisions) retained by online selections (trigger) which identify collisions where muons have been produced, thus storing only about 10 events out of the about 40 million collisions produced every second at the LHC.

The whole dataset collected by the CMS collaboration since the start of LHC operations in 2010 is comprised of roughly 100 PBs of data, and even more if taking into account the simulations.

A small subset of the events has been extracted and preprocessed by Matteo, former T.A. of this course, who has stored the resulting records as `json` files.

```
# list all available files
!ls -l ../datasets/lecture2/dimuon/*.json
```

13.5.1.1. Step 0 - Load and inspect the data

Load the `json` files directly in a Spark DataFrame using the built-in data loading functions

```
# load the dataset found in the datasets/lecture2/dimuon folder
# spark.read --> read from a source of data
```

```
# option    --> specify the options for the data source
# format    --> specify the format of the data source
# load      --> specify the path of the input data
df = spark.read \
    .option("inferTimestamp","false") \
    .option("prefersDecimal","false") \
    .format('json') \
    .load('/mapd-workspace/datasets/lecture2/dimuon/*.json')
```

Once again, the data at this stage is not loaded in a DataFrame just yet. Reading and loading files from source to an RDD (the low-level representation of the DataFrame) is a *lazy* operation.

More specifically, it's not really a transformation (since there's no previous DataFrame or RDD to start from), and it's not an action (since it doesn't trigger execution).

PySpark just builds a logical plan describing how to read the data (inferring its schema, etc.).

Let's inspect the schema of our dataset

```
# print the schema of the dataframe
```

The schema is quite interesting, as the data is not entirely *flat*.

We have two scalar objects, `nMuon` and `sample`, respectively an integer and a string, per record. However, every record also contains a data structure named `Muons`, an array where each element contains the five numerical fields `E`, `charge`, `px`, `py`, `pz`.

We can inspect a few elements by showing their content.

```
# show the first 5 rows of the dataframe
```

In this example, `sample` is a simplified element representing whether the records are from *real data* or from a *simulated sample* generated with a MonteCarlo technique.

We can evaluate how populated are the two samples with an aggregation operation using `groupBy`.

`groupBy` acts in a similar way to its SQL and Pandas counterparts, by creating groups from the DataFrame data using the specified columns, onto which we can apply an aggregation function.

For a complete list of aggregation functions you can refer to the [documentation](#).

```
# count the number of records in each sample group
#
# 1. apply a .groupBy() transformation on the dataframe, grouping on the
    ↪ sample
# 2. use a .count() aggregator
# 3. show the results
```

13. Stop worker and master

All transformations on the DataFrame will start from loading the files from storage. This is particularly inefficient if we will need to run multiple sets of operations (transformations+action) on the same dataset.

We can however place the DataFrame in memory as for the RDD.

```
# persist the dataframe to memory to avoid recomputing  
# everything from scratch at each iteration
```

```
# let's trigger persist with a count operation
```

13.5.1.2. Step 1 - First feature aggregation and plotting

Let's finally start investigating the dataset by extracting some meaningful information.

A first interesting piece of information is to produce the distribution (an histogram) of the number of muons we have per event (thus per record) both in the **data** and in the **mc** samples, to compare them and see whether they do match.

We can start by grouping the records by the number of muons per record and sample and count over them.

```
# group the records by the sample  
# and the number of muons  
#  
# 1. get the number of muons per sample  
# 2. count them  
# 3. sort them based on the number of muons  
# 4. collect the results  
num_muons_dist =
```

```
# print the first 10 entries  
num_muons_dist[:10]
```

We have retrieved a pre-computed quantity from the cluster, thus having Spark do the “heavy lifting” in a distributed way. The result is still a collection of **Row** objects, actually an RDD... not really the plot we would like, nor a simple Python object we can use to plot. We could retrieve directly a plot from Spark if we wanted, having it compute all features required to populate a graph, but that would require a lot of code.

Fortunately though, there is a trick we can use: it is possible to convert a Spark dataframe back into a Pandas Dataframe with `toPandas()`.

Again, as it happens for `collect()`, the entire resulting dataset is fetched into the master and it may not fit in the memory, so do not use `toPandas()` very lightly.

[Side Note on Apache Arrow]

Apache Arrow comes into play in this context: it is an in-memory columnar data format that is used in Spark to efficiently transfer data between the JVM and Python processes. When it is not enabled Spark communicates with Python processes by serializing and deserializing one element at a time. With Arrow this operation is “vectorized”, i.e. one column at a time is transferred. This is possible because both spark and pandas included Arrow representation.

Here you can find a nice blog post explaining how it works.

In this way, operations between PySpark and Pandas are more efficient and we can try to get the best from both worlds.

```
# import pandas
import pandas as pd

# do not collect the results as a plain RDD
# but transform it into a pandas dataframe
# using the .toPandas() function
num_muons_dist =
    """ ... """
    .toPandas()

# print the first few rows of the pandas dataframe
num_muons_dist.head()
```

This way we have “closed the loop”.

- We have loaded data from a large number of files (possibly stored on a remote or distributed File System)
- We have run a computationally expensive operation in parallel using a computing cluster
- We have returned only the meaningful result (something that fits in memory) to our local machine
- We have obtained a Python (pandas) object we can easily work with for the remainder of our analysis (plotting, etc)

```
# verify the type of this python variable
type(num_muons_dist)
```

Produce the simple plot we were attempting to do in the previous cell: - a histogram of the number of muons per record, comparing data and simulation.

```
# use matplotlib to plot
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))
```

13. Stop worker and master

```
# fill the histogram with the simulation (mc)
mc_counts = num_muons_dist.loc[num_muons_dist['sample']=='mc']
plt.bar(
    mc_counts['nMuon'], mc_counts['count'],
    label = 'Montecarlo', width=1, alpha=0.6
)

# use an errorbar plot to overlay data
# the error is the square root of the counts
data_counts = num_muons_dist.loc[num_muons_dist['sample']=='data']
plt.errorbar(
    data_counts['nMuon'], data_counts['count'],
    yerr = np.sqrt(data_counts['count']),
    label = 'Data', color='black', fmt='o'
)
plt.xlim(-0.5, 16.5)
plt.xlabel("Number of muons")
plt.ylabel("Counts")
plt.legend()
plt.semilogy()
plt.show()
```

13.5.1.3. Step 2 - Filter and flatten the dataset

The end goal of this task is to study dimuon events, i.e. events/records having exactly 2 muons.

Selecting such records is a simple filtering task, that can be performed in a variety of ways using the Spark DataFrame API: - **pandas**-like selections, e.g. `df[df.column > xyz]` - **where** or **filter** Spark DataFrame operations, analogous to the **filter** se used with RDDs - fully-fledged SQL statements

pandas-like selections do work similarly to their **pandas** counterparts by operating on columns of the DataFrame:

```
df[df.column > xyz]
```

```
# select only events with two muons
dimuon_df = df[df['nMuon']==2]

# print the first 5 rows
dimuon_df.show(5)
```

Also **where**/**filter** operations work similarly to the equivalent **pandas** logic. The two are aliases of the same function (specifically **where** is an alias of **filter**) therefore there is no difference in

writing one or the other.

We have already used `filter` on plain RDDs writing a Python function to apply to arbitrary data.

Using `where/filter` with DataFrames and having it operate on all items of a given column is, in principle, a slightly different concept.

Spark has to “understand” that we are working with a columnar object, and not with a plain collection of arbitrary data. This is what we refer to when we say that we have low-level APIs and high-level APIs. From the user’s point of view, the method call is going to be the same, but the underlying “wiring” of the function is different (we are offered a similar *interface* to different *inner functions* in the two cases).

We can write the same `where/filter` function in multiple ways: - specifying that we are operating on a column named `column`

```
df.where(col('column')>xyz)
```

- recalling the column from the DataFrame

```
df.where(df['column']>xyz)
```

- using the column name in the filter field

```
df.where("column > xyz")
```

```
# import the column object from pyspark
from pyspark.sql.functions import col

# apply where explicitly on the spark column object
df.where(col('nMuon')==2)\
    .show(5)
```

```
# apply where recalling the column from the dataframe
df.where(df['nMuon']==2)\
    .show(5)
```

```
# apply where (or filter) using the column name in the filter field
df.filter("nMuon == 2")\
    .show(5)
```

Spark DataFrame has an underlying optimization engine for structured filters based on SQL.

In dealing with structured data, a common use case is to use Spark to interface with SQL: - use SQL statements to query large and possibly distributed structured datasets - connect to a remote SQL

13. Stop worker and master

database and process data in parallel - preprocess large amounts of raw data and then translate them into tables of a DB for data management - ...

In PySpark using the structured data API (actually named `pyspark.sql` as we have seen) we can create a table *view* from our dataset, and then use plain SQL code to run the same filters or aggregations.

The underlying scheduling and task optimization of Spark is always going to be there and will help execute queries and data filters quickly and efficiently.

```
# create a temporary table view of the dataframe
# this will allow spark to execute SQL queries on the dataframe
# treating the dataframe as a table in a RDBMS
df.createOrReplaceTempView("dimuon_df_table")

# select all rows from the table
# where the number of muons is 2
spark.sql("""
    SELECT * FROM dimuon_df_table
    WHERE nMuon = 2
""")\
    .show(5)
```

After filtering the dataset and retaining only the data we want, we can start accessing the features and producing higher-level quantities and results.

However, at this stage, the Dataset is not flat, as we have nested content in the Muons column.

```
# print the dataframe schema
dimuon_df.printSchema()
```

We can flatten our dataset for further processing: in this way, we will be able to perform operations between columns as in a “normal” `pandas DataFrame`.

A new column can be created by using `withColumn(name, func)` where : - `name` is the name to be assigned to the new column - `func` is a function taking as input one or more columns

For instance, a function may be to multiply together values in different columns, or simply copy the elements of a nested structure into a plain column.

However, any arbitrary complex function can be used to produce new columns: later we will define arbitrary **User Defined Functions (UDFs)** that will be applied to every row of the `DataFrame`.

To refine our selection by retaining only a subset of columns, we can use the `select` statement, similar to the `SELECT` SQL keyword.

On top of this, we can also drop all unused columns from the dataset with the `drop` function of the `DataFrame`.

To recap: - `select()` projects a set of expressions and returns a new `DataFrame` - `withColumn()`

creates a new attribute from the given column - `drop()` removes columns from the table

```
# starting from the dimuon dataframe (the filtered one)
dimuon_flat = (
    # select only the columns we need for the analysis
    dimuon_df.select([col('sample'), col('nMuon'), col('Muons')[0],
    ↪ col('Muons')[1]])
    # and then flatten the Muons array into one column per feature:
    # - Energy E
    # - Momentum along x-axis px
    # - Momentum along y-axis py
    # - Momentum along z-axis pz
    # - Charge c
    .withColumn('E1', col('Muons[0].E'))
    .withColumn('px1', col('Muons[0].px'))
    .withColumn('py1', col('Muons[0].py'))
    .withColumn('pz1', col('Muons[0].pz'))
    .withColumn('c1', col('Muons[0].charge'))
    .withColumn('E2', col('Muons[1].E'))
    .withColumn('px2', col('Muons[1].px'))
    .withColumn('py2', col('Muons[1].py'))
    .withColumn('pz2', col('Muons[1].pz'))
    .withColumn('c2', col('Muons[1].charge'))
    # drop the two Muons nested structures
    .drop('Muons[0]', 'Muons[1]')
)

# print the schema of the new dataframe
dimuon_flat.printSchema()
```

13.5.1.4. Step 3 - Computing new features

At this stage, we are left with pairs of muons with flat data structures.

We would like to select only those pairs of muons with opposite electric charge and compute the invariant mass from the four-momenta of the two selected particles.

This is a relativistic invariant property that resonates around a given value in the case the two muons are produced by the decay of a “parent particle”.

```
# starting from the flat dimuon dataset
# select only the pairs with opposite charge
dimuon_flat =

# count the number of opposite-charge pairs
```

13. Stop worker and master

```
dimuon_flat.count()
```

To compute the invariant mass of the two candidate muons we need first to compute the four-momentum of the parent particle

\$

$$p = (E_1 + E_2, \quad p_{x,1} + p_{x,2}, \quad p_{y,1} + p_{y,2}, \quad p_{z,1} + p_{z,2}) = \quad (13.1)$$

$$= (E, \quad p_x, \quad p_y, \quad p_z) = \quad (13.2)$$

$$= (E, \quad \mathbf{p}) \quad (13.3)$$

\$

Create four new columns in the DataFrame, one per each of the four coordinates: (E, p_x, p_y, p_z)

```
# create four new columns for the four components of the momentum  
# and add them to the dataframe  
dimuon_flat =
```

```
dimuon_flat.printSchema()
```

From the 4-momentum of the candidate we can compute the invariant mass as

\$

$$M = \sqrt{p \cdot p} = \sqrt{(E^2 - \|\mathbf{p}\|^2)} = \quad (13.4)$$

$$= \sqrt{E^2 - (p_x^2 + p_y^2 + p_z^2)} \quad (13.5)$$

\$

We can compute it by manipulating directly the columns of the DataFrame.

```
# compute the invariant mass  
  
# import the spark sqrt function  
from pyspark.sql.functions import sqrt  
  
# create a new column for the invariant mass  
dimuon_flat =  
  
# persist the dataframe after these new columns have been added  
dimuon_flat =  
  
# show the first 5 values of the M column of the dataframe  
dimuon_flat.select('M').show(5)
```

As we have anticipated, we can define custom functions and use them to evaluate complex features from the columns of our DataFrame.

A User Defined Function (UDF) is any arbitrary Python function that can be evaluated by Spark on any row of your DataFrame. If there is any risk a UDF might fail (e.g. if the value of a column is `None`) you must remember to protect the function (e.g. by adding an `if _ is not None` condition).

- PySpark UDFs are extremely useful when **there is no native Spark function available for the computation you need to perform**.
- They are also a nice way of writing **simple Python code** and letting Spark run it on the data stored in the partitions.
- We can easily **write and reuse complex functions** in multiple occurrences in our distributed computation.

BUT... the PySpark UDFs are custom Python code executed on Java VMs (the executors)... Spark will thus have to start Python processes on the executors, execute the function row by row on that data in the Python process, and then finally return the results of the row operations to the JVM and Spark. **This typically takes a toll on the performances**

```
# create a pyspark udf
from pyspark.sql.functions import udf
import math

# create a function that takes in four columns and returns a float
# this is the function that will be used as the udf
@udf('float')
def invariant_mass(E, px, py, pz):
    # the function evaluates plain python code
    # no vectorization (numpy) is involved
    return math.sqrt(E*E - (px*px + py*py + pz*pz))

# apply the udf to the appropriate columns
# and return the first 5 rows
dimuon_flat.select(
    invariant_mass(col('E'), col('px'), col('py'), col('pz'))
).show(5)
```

To partially overcome the performance issues, we can use **Pandas UDFs in PySpark**, also referred to as **vectorized UDFs**.

This allows to processing PySpark's DataFrames and columns as Pandas DataFrames and Pandas series, respectively.

Differently from the standard UDFs, these Pandas UDFs perform the same operation on the entire pandas series (vectorized operation) and do not run one row at a time.

This provides a boost in the efficiency of the computation and thus should be preferred whenever

13. Stop worker and master

there is the possibility.

They result particularly efficient in the “grouped maps”, i.e. by applying a user-defined function after a `groupby` to perform dedicated aggregation on the data.

More on this can be found [here](#) and [here](#).

```
# create a pyspark pandas udf
from pyspark.sql.functions import pandas_udf

# the pandas udf takes four pandas series
# and returns a pandas series (a single column)
@pandas_udf('float')
def invariant_mass_udf(
    # the four input pandas series (with explicit type definition)
    E: pd.Series,
    px: pd.Series,
    py: pd.Series,
    pz: pd.Series
) -> pd.Series:
    # the resulting pandas series (a numpy array)
    # once again with the explicit type definition
    return np.sqrt(E*E - (px*px + py*py + pz*pz))

# apply the pandas udf to the appropriate columns
# and return the first 5 rows
#
# we can also use an alias to rename the column
dimuon_flat.select(
    invariant_mass_udf(col('E'), col('px'), col('py'), col('pz')).alias('M
    ↪ pandas udf')
).show(5)
```

Compute the average energy E of the muons found in each of the two `sample` groups.

To do this, compare the two approaches: 1. use only PySpark native functions to run `groupBy` and aggregations (look up [here](#) about all the available built-in aggregations) 2. use a vectorized UDF to evaluate the average for each group

Use only PySpark native functions to run `groupBy` and aggregations

```
# compute the mean energy for muons in each
# sample using pyspark native functions
```

Use a vectorized-UDF to run the aggregation

```
# compute the mean energy for muons in each
# sample using a pyspark vectorized udf function
```

13.5.1.5. Step 4 - (Re-)Discovering particles

Now that we have loaded our dataset in the cluster, inspected it, pre-processed it by selecting and flattening variables, and we have computed all the high-level quantities, we can finally use the dataset to perform the final analysis.

For example, let's plot the invariant mass M of the dimuon system in its whole range.

```
# compute the histogram of M
#
# 1. select the M column
# 2. get the underlying RDD
# 3. retrieve the M values into a histogram
# (remember the .histogram() RDD function from the previous notebook)
#
# - use 0-150 GeV as the histogram range
# - use 2 GeV as the bin width

range_min = 0
range_max = 150
bin_width = 4

bins, counts = dimuon_flat.

# print the bins and counts to check the logic
print(f'{bins=}\n')
print(f'{counts=}\n')
```

```
# create two histograms, one per sample (data/mc)

# it could be convenient to use a dictionary
# to host the two samples' bins and counts
histogram_results = {}

# perform the same pyspark function on both samples
for sample in ['mc', 'data']:
    histogram_results[sample] = {}

    bins, counts = dimuon_flat.
```

13. Stop worker and master

```
    histogram_results[sample]['bins'] = bins
    histogram_results[sample]['counts'] = counts

# create a plot containing
# - a filled histogram for the Simulation (mc)
# - an errorbar plot for the Data (data), with sqrt(count) as the y-error
# - show the resulting plot in semi-log scale
plt.figure(figsize=(8,6))

mc_res = histogram_results['mc']
bin_centers = mc_res['bins'][:-1] + np.diff(mc_res['bins'])/2
plt.hist(bin_centers,
         weights=mc_res['counts'],
         bins=mc_res['bins'],
         label='Simulation',
         alpha=0.6
)

data_res = histogram_results['data']
bin_centers = data_res['bins'][:-1] + np.diff(data_res['bins'])/2
plt.errorbar(
    bin_centers,
    data_res['counts'],
    yerr=np.sqrt(data_res['counts']),
    fmt='o',
    ms=4,
    lw=1,
    color='black',
    label='Data'
)

plt.xlabel("$m_{\mu \mu}$ [GeV]")
plt.ylabel(f"Events/{bin_width} GeV")
plt.legend()
plt.semilogy()
plt.tight_layout()
plt.show()
```

The plot shows a number of features we are not going to talk about in this class.

However, to complete the exercise, we'll focus on the range 75-115 GeV, where it appears to be a large resonance.

We can create a new plot of data and simulations in that specific range with higher binning “resolution” (0.5 GeV), to study how well the simulation agrees with the experimental data.

```

# create a plot containing
# - a filled histogram for the Simulation (mc)
# - an errorbar plot for the Data (data), with sqrt(count) as the y-error
# - show the resulting plot in semi-log scale
histogram_results = {}
for sample in ['mc', 'data']:
    histogram_results[sample] = {}

    bins, counts = dimuon_flat.where(col('sample')==sample)\
                                .select('M')\
                                .rdd.map(lambda r: r.M)\
                                .histogram(list(np.arange(70,115,0.5)))

    histogram_results[sample]['bins'] = bins
    histogram_results[sample]['counts'] = counts

plt.figure(figsize=(8,6))

mc_res = histogram_results['mc']
bin_centers = mc_res['bins'][:-1] + np.diff(mc_res['bins'])/2
plt.hist(bin_centers,
         weights=mc_res['counts'],
         bins=mc_res['bins'],
         label='Simulation',
         alpha=0.6
)

data_res = histogram_results['data']
bin_centers = data_res['bins'][:-1] + np.diff(data_res['bins'])/2
plt.errorbar(
    bin_centers,
    data_res['counts'],
    yerr=np.sqrt(data_res['counts']),
    fmt='o',
    ms=4,
    lw=1,
    color='black',
    label='Data'
)

plt.xlabel("$m_{\mu \mu}$ [GeV]")
plt.ylabel(f"Events/0.5GeV")
plt.legend()

```

13. Stop worker and master

```
plt.tight_layout()
plt.show()
```

We can try to perform some selections, to improve the quality of the signal and remove the background, and hopefully this selection might help restore a decent Data-Simulation agreement.

To do this we can try to look at dimuon candidates with a large transverse momentum

\$

$$p_T = \sqrt{p_x^2 + p_y^2} = \quad (13.6)$$

$$= \sqrt{(p_{x,1} + p_{x,2})^2 + (p_{y,1} + p_{y,2})^2} \quad (13.7)$$

\$

- Create a column with name `highpt` and for each row set the value to `True` if $p_T \geq 30$ GeV and `False` otherwise
 - Start from the two muons and use a UDF for this
 - Use the PySpark `when` and `otherwise` function (have a look at the documentation [here](#))
- Plot the Data-Simulation comparison for dimuon candidates passing the `highpt` selection

```
# create the udf to evaluate the dimuon pT
```

```
# apply the udf to the dimuon_flat dataframe
dimuon_flat = dimuon_flat.withColumn("pT",
                                     """...""")
```

```
# import the when function from pyspark.sql
from pyspark.sql.functions import when

# create the `highpt` feature with where/otherwise
dimuon_flat = dimuon_flat.
```

```
# inspect a few rows of the dataframe
# to verify the feature is set correctly
dimuon_flat['sample', 'M', 'pT', 'highpt'].show(5)
```

```
# create the two data-simulation comparison plots
# for low- and high-pt dimuon candidate
```


13.6. Stop worker and master

```
sc.stop()
spark.stop()
```

Finally, use `docker compose down` to stop and clear all running containers.

13.6.1. Lecture 3: Stream Processing with Spark

Spark Structured Streaming is an extension of the Spark API that enables scalable stream processing.

It's a well-documented [API link](#), which is however different from Spark Streaming, which has recently become deprecated since Spark version 3.4.0 (~a couple of years ago)

The main difference between Spark **Streaming** and **Structured Streaming** is that the former is based on the low-level RDD API, while the latter is based on DataFrames.

Both APIs have however been designed to help with the continuous processing of streaming applications.

The continuous stream of input data can be ingested from many data sources such as **Kafka**, **Amazon S3**, or **TCP sockets**.

Processed data can be exported to an external database and used to make live dashboards or offline analyses, stored in files, or used in a further stage of a Kafka pipeline.

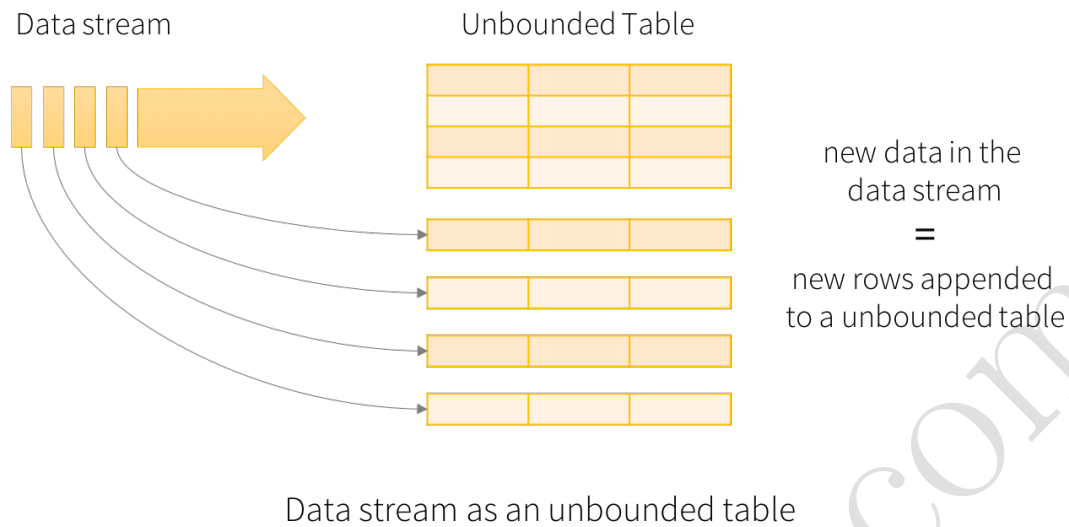
Overall, the practice of reading data from a set of sources, pre-processing it, and then storing it in a different format for later analysis is extremely common and has its own name: **real-time ETL pipelines**. - Extract - Transform - Load

13.7. Structured Streaming

The key idea of this stream processing model is to treat the continuous stream as a table that is continuously appended to.

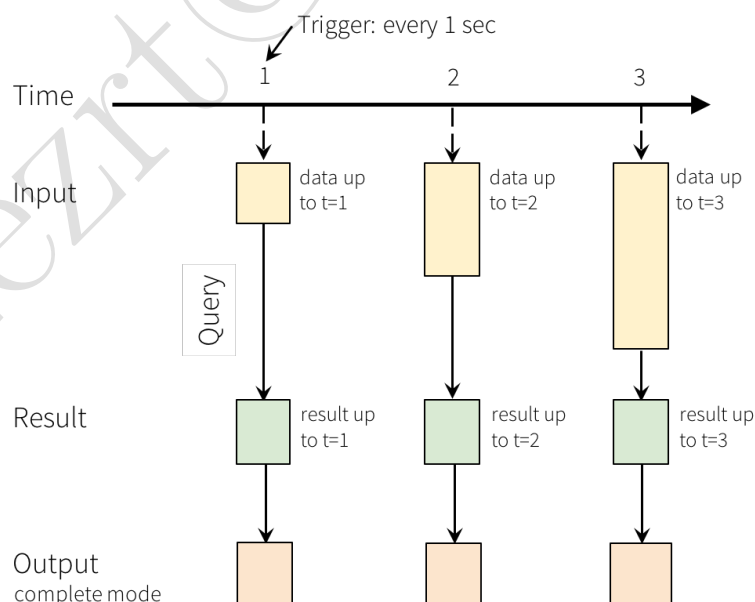
This allows users to view the continuously incoming data as a DataFrame with new records being new rows to be included. It further expresses stream processing as a standard batch-like query on a static table, similar to what we have done in the Spark DataFrame lecture.

13. Stop worker and master



Internally, the stream is divided into micro-batches produced by a “trigger,” which could represent any given condition (e.g., every 1 second):

- The input stream is a table (DataFrame) or the new rows to be appended to the previous DataFrame.
- Every operation/query will produce a result table (DataFrame).
- When the output table is updated, it can be written somewhere thanks to an output module/type.



Programming Model for Structured Streaming

NB: Although the input table can be viewed as an always-growing DataFrame, **Spark does not**

actually materialize the entire table.

- Only the latest data are processed (latest batch) and then discarded.
- Conversely, the result table can be updated to keep track of the results from previous batches.

There are three output modes:

- **Complete Mode** where the entire output table is written to the Sink.
- **Append Mode** where only the new rows appended in the Result Table since the last trigger will be processed by the Sink.
- **Update Mode** where only the rows that were updated in the result table since the last trigger will be written to the Sink.

In this notebook, the *Update Mode* and *Console* output sink will be used.

- Results (i.e., only the updated records) will be displayed on the screen (i.e., on the “terminal”).
- However, the Sink could be a database, a file, Kafka, or more, depending on the needs and applications.

13.8. Create and Start a Spark Session

```
# import the python libraries to create/connect to a Spark Session
from pyspark.sql import SparkSession

# build a SparkSession
# connect to the master node on the port where the master node is
#   ↳ listening (7077)
# declare the app name
# configure the executor memory to 512 MB
# either *connect* or *create* a new Spark Context
spark = SparkSession.builder \
    .master("spark://spark-master:7077") \
    .appName("My streaming spark application") \
    .config("spark.executor.memory", "512m") \
    .config("spark.sql.execution.arrow.pyspark.enabled", "true") \
    .config("spark.sql.execution.arrow.pyspark.fallback.enabled", "false") \
    .config("spark.sql.streaming.forceDeleteTempCheckpointLocation",
#   ↳ "true") \
    .config("spark.sql.adaptive.enabled", "false") \
    .getOrCreate()
```

```
spark.sparkContext.setLogLevel("ERROR")
```

13.8.1. TCP Socket Source

For this example, Spark will read data from a TCP socket using Spark Structured Streaming.

A TCP socket is a communication endpoint used to establish a connection between two devices over a network. You can think of it as a telephone connection: two endpoints have to establish a connection; once the connection is established, communication can occur, with data transfer; as soon as one of the two ends interrupts the connection, the whole communication is lost.

We will generate a dummy data stream representing fake credit card transactions.

A simple Python program will be used to create this data stream. You will be able to find it in `utils/producer.py`. When executed, the producer will try to establish a TCP connection and send data on port 5555 of a given `host` (`spark-master` in our case).

Before executing the producer program, take a moment to review the `producer.py` code to understand how it works. It's important to understand the logic of the program before using it to generate the streaming data.

```
! cat utils/producer.py
```

The producer will generate new records in the form of a random combination of: - `name` - `surname` - `amount`: amount of the credit card transaction - `delta_t`: time between transactions - `flag`: random flag to indicate if potentially fraudulent or not

This information will be formatted into a `.json` data format

13.8.2. Creating the streaming DataFrame from a TCP socket source

To inform Spark that the data source will be a TCP socket located at a specific `hostname` and `port`, we can use the options `host` and `port` methods. * When declaring this source a message appears, warning that the TCP source is only meant for testing purposes.

Refer to the [documentation](#) for additional available options.

Notice that the syntax is equivalent to the one used for example to read a set of files from a disk.

```
# the hostname and port number
hostname = "spark-master"
portnumber = 5555

rawMessagesDf = (
    spark
    .readStream
    .format("socket")
    .option("host", hostname)
    .option("port", portnumber)
    .load()
)
```

13.8.3. Start the python producer.py script

From a terminal/WSL, connect to the `spark-master` Docker container using the command

```
docker exec -it spark-master bash
```

From inside the docker container, move to the `/mapd-workspace` folder and execute the python script with the option `--hostname spark-master`:

```
python /mapd-workspace/notebooks/utils/producer.py --hostname spark-master
```

The producer application will be automatically closed when the streaming application terminates.

13.8.4. Running the first streaming application

The first streaming query will be a simple `show` of the `DataFrame` * No processing between `rawMessagesDf` and output sink (`writeStream`)

Output mode is set to `update`, hence every new message received by the TCP source will be processed by the output sink. * Output format `console` indicates that the output will be displayed on the screen before being discarded

The streaming query is triggered every 2 seconds * refer to the [documentation](#) for additional trigger types.

```
query = (
    rawMessagesDf
    .writeStream
    .outputMode("update")
    .format("console")
    .trigger(processingTime='2 seconds')
    .option("truncate", False)
    .start()
)
```

Run this cell to stop the streaming query execution

```
query.stop()
```

13.9. Example of streaming query: data parsing

Data received from the TCP source is seen as a `string` by the spark application

We first must develop an application parsing the string and creating a column for each `json` field in order to start processing the dataset using the `DataFrame` API functionalities

13. Stop worker and master

It can be useful to start with a set of test data, to develop this query.

The same Spark code (transformations/actions) used for processing a “static” DataFrame can be used for the streaming context!

```
# dummy data for testing purposes
testData = [
    '{"name": "Jill", "surname": "Millers", "amount": 736.56, "delta_t":'
    ↪ 7.78, "flag": 0}',
    '{"name": "John", "surname": "Johnson", "amount": 986.47, "delta_t":'
    ↪ 3.9, "flag": 0}',
    '{"name": "John", "surname": "Jones", "amount": 249.9, "delta_t": 0.62,'
    ↪ "flag": 1}',
    '{"name": "Andy", "surname": "Jones", "amount": 950.95, "delta_t": 6.02,'
    ↪ "flag": 0}',
    '{"name": "Jill", "surname": "Millers", "amount": 724.32, "delta_t":'
    ↪ 9.19, "flag": 0}',
    '{"name": "John", "surname": "Johnson", "amount": 850.07, "delta_t":'
    ↪ 7.33, "flag": 1}',
    '{"name": "Andy", "surname": "Smith", "amount": 557.48, "delta_t": 9.64,'
    ↪ "flag": 0}',
    '{"name": "Alice", "surname": "Darby", "amount": 424.75, "delta_t":'
    ↪ 7.76, "flag": 0}'
]
```

Create a Spark DataFrame by importing the testData.

It can be useful to investigate the `pyspark.sql.types` to check whether there is any helping function that could be used for this purpose.

```
from pyspark.sql.types import StringType

# create a spark dataframe from testdata
testDf = spark.createDataFrame(testData, StringType())

# show the dataframe content
testDf.show(n=5, truncate=False)
```

Data are now in the same format as the one received by the socket data source.

The function `from_json` can be used to parse a json string with a given schema. As always, have a look at the [documentation](#) before using it.

Start by defining a schema for our data.

```
import pyspark.sql.functions as f
from pyspark.sql.types import StructField, StructType, StringType,
    DoubleType, IntegerType

# create the schema
schema = StructType(
    [
        StructField("name", StringType()),
        """
        ...
        ...
        ...
        """
    ]
)
```

And create a “parsed” DataFrame to verify if our schema is properly addressing the data types.

```
# parse the dataframe and show its structure
parsedTestDf = testDf.select(f.from_json('value', schema=schema))

parsedTestDf.printSchema()
```

The structure obtained using `from_json` is actually inherently nested... The `json` messages were correctly parsed, but a nested DataFrame is returned, which should be flattened before being able to use it as a “plain DataFrame”.

As discussed in the DataFrame notebook, this can be easily solved in a variety of ways, for instance by selecting the columns we are interested in using from now on...

As an example, the `name` and `surname` columns could be selected.

```
# show values from the parsedTestDf
parsedTestDf.show(n=5, truncate=False)
```

```
# re-create the parsedTestDf by
# - aliasing the values produced by from_json as `data`
# - selecting the columns `data.name` and `data.surname`
parsedTestDf = (
    testDf
    .select("""___""")
    .select("""___""")
)
```

13. Stop worker and master

```
# NB: data.* will select all the columns, as in plain SQL
```

```
# print the new schema  
parsedTestDf.printSchema()
```

```
# and show the new content  
parsedTestDf.show(5)
```

13.9.1. From static to streaming queries

Now that we have developed a way to extract and interpret the data using a static example, we can reuse the very same code for the streaming query, by simply “chaining” the application to the source and directing its results to the sink.

NB: Remember to restart the producer application before starting the queries.

Re-create the “raw” DataFrame connecting the Spark Structured Streaming Context to the input TCP socket.

Do not start the stream just yet!

```
# the hostname and port number  
hostname = "spark-master"  
portnumber = 5555  
  
# recreate the streaming raw dataframe  
rawMessagesDf = (  
    spark  
    .readStream  
    .format("socket")  
    .option("host", hostname)  
    .option("port", portnumber)  
    .load()  
)
```

Starting from the streaming DataFrame, issue the appropriate transformations to interpret its json format, and prepare a new parsed DataFrame including all (*) the columns.

```
# parse the json lines using the previous schema  
# select all columns  
parsedDf = (  
    rawMessagesDf."..."  
)
```



```
parsedDf
```

Start the application as done previously, with: - `outputMode=update` - `format=console`

Choose the trigger as you prefer. A suggestion is to keep it simple, and use a time-based trigger of 2 seconds.

```
query = (
  parsedDf
    .writeStream
      """..."""
      """..."""
      """..."""
    .option("truncate", False)
    .start()
)
```

```
query.stop()
```

13.9.2. Process each batch to identify possibly fraudulent transactions

1. compute the *number of flagged transactions per batch per user* (create a unique `userID` field as the combination of *FirstLastname* to identify individual users)
2. identify all the “suspicious” transactions per user: all users with more than one flagged transaction per batch will be assigned a `isFraud` boolean variable
3. format the resulting `userID` and `isFraud` information in a `DataFrame` to mimick a “live-report” of the suspicious transactions

```
# the hostname and port number
hostname = "spark-master"
portnumber = 5555

rawMessagesDf =
```

```
# parse json lines, use a schema
parsedDf = (
  rawMessagesDf.

)
```

13. Stop worker and master

```
# find number of transactions for each user when flag = 1
#   declare a new column to create a unique user identifier
#   this can be easily done by concatenating first- and last-name fields
#   check the concat function from pyspark.sql.functions
numTransactions = (
    parsedDf

)
```

```
# find suspicious transactions
#   filter only users with more than one transaction per batch
#   create a "fraud" column with a value of 1 for the selected users (check
    ↪ the lit function)
#   from the dataframe, project the unique id, fraud flag and number of
    ↪ transaction columns
susTransactions = (
    numTransactions

)
```

```
# this line is a trick to force Spark to use a small number of partitions (4
    ↪ in this example)
spark.conf.set("spark.sql.shuffle.partitions", 4)
```

```
query = (
    susTransactions
    """
    ...
    ...
    ...
    """
    .start()
)
```

```
query.stop()
```

Since this query contains aggregations, output mode can be changed to **complete**. In this case, the full set of results is displayed for every batch.

```
query = (  
    susTransactions  
    ""  
    .  
    .  
    .  
    ""  
    .start()  
)
```

```
query.stop()
```

13.10. Stop spark worker

```
spark.stop()
```


14. Dask

14.1. Introduction

Dask is a flexible parallel computing library for Python, designed to scale Python's scientific ecosystem (NumPy, Pandas, Scikit-learn) to larger-than-memory datasets and multi-machine clusters.

Compared to Spark:

	Apache Spark	Dask
Language	Scala (Python via PySpark API)	Pure Python
Data structures	RDD, DataFrame, Dataset	Bag, Array, DataFrame
Scheduling	JVM-based DAG scheduler	Python-based task scheduler
Ecosystem	Standalone (integrates with Hadoop/YARN)	Native Python — integrates with NumPy/Pandas/Scikit-learn
Best for	Large JVM-native clusters, Java/Scala users	Python-centric workflows, data scientists

Dask's scheduler is written in Python. While this provides tight integration with the Python ecosystem, it also means that Python memory management limitations apply over the lifecycle of long-running applications.

Dask provides three high-level data structures:

- **dask.bag** — a distributed list of arbitrary Python objects, analogous to a Spark RDD.
- **dask.array** — distributed NumPy arrays.
- **dask.dataframe** — distributed Pandas DataFrames.

All three APIs support **lazy evaluation** and leverage Dask's DAG-based scheduler for optimisation and parallel execution.

14.2. Dask Fundamentals

14.2.1. Lazy Evaluation and Task Graphs

Like Spark, Dask uses **lazy evaluation**: operations are not executed immediately. Instead, each operation builds a **task graph** — a DAG (Directed Acyclic Graph) representing the full compu-

tation. Execution is triggered by calling `.compute()`.

```
import dask

@dask.delayed
def add(x, y):
    return x + y

@dask.delayed
def square(x):
    return x ** 2

# Build the task graph (nothing executes yet)
a = add(1, 2)
b = square(a)

# Visualise the graph
b.visualize() # renders the DAG as an image

# Execute
result = b.compute() # triggers actual computation + 9
```

`dask.delayed` is the fundamental building block: wrap any Python function with `@dask.delayed` and it becomes a lazy node in the task graph.

14.2.2. Partitioning

All Dask collections (Bag, Array, DataFrame) split data into **partitions** that are processed in parallel. Choosing the right partition count is important:

Too few partitions	Too many partitions
Insufficient parallelism — workers sit idle	Scheduler overhead dominates — dispatching thousands of tiny tasks is expensive
Large memory footprint per worker	Excessive inter-partition communication

Rule of thumb: aim for 10–100 MB per partition, with the number of partitions roughly $2\text{--}5 \times$ the number of workers. Use `df.repartition(n)` to adjust.

⚠ Python Memory Management in Dask

Because Dask is written in pure Python, it is subject to Python’s memory management. Over the lifecycle of a long-running application, **unmanaged memory** (memory held by Python objects not yet garbage-collected) can accumulate on workers, eventually causing out-of-memory errors even when the logical data fits within the cluster’s total RAM. This is

a key difference from Spark, which runs on the JVM with a more predictable memory model. For production workloads, monitor worker memory via the Dask dashboard and consider restarting workers periodically.

14.3. Cluster Setup

A Dask cluster can be used locally (leveraging multi-core CPUs) or distributed across multiple machines. The `Client` object connects to a running Dask scheduler:

```
from dask.distributed import Client
client = Client('dask-scheduler:8786')
```

A web dashboard at `localhost:8787` provides live visualisation of workers and task graphs.

14.4. Hands-on: Dask

The following hands-on sessions use a Dask cluster deployed via Docker Compose. The scheduler is accessible at `dask-scheduler:8786` and the dashboard at `localhost:8787`.

Note: These notebooks require the Docker cluster to be running and are not executed during book rendering.

14.4.1. Lecture 1: Task Parallelization with Dask

14.5. Recap on Task Parallelization (this time using Dask)

By now, we should be quite familiar with the programming pattern discussed in previous lectures, where we explored Hadoop MapReduce and the Spark framework.

Apart from the specifics of the API, which vary depending on the framework implementation, and the internal workings of job scheduling and resource management, most distributed computing frameworks and tools offer ways to parallelize our tasks through graph schedulers and task optimizers.

The workflow remains largely consistent, regardless of the tool used: - **Subdivide large datasets that won't fit into memory into smaller subsets (partitions)** - **Design the entire data processing pipeline before execution and optimize it by breaking it into smaller tasks** - **Schedule and distribute processing close to the data, minimizing data movement (data locality)**

In Dask, we have access to a variety of dataset representations:

14. Dask

- Dask bag (roughly equivalent to the Spark RDD)
- Dask array
- Dask dataframe (similar to the Spark DataFrame)

However, it's important to note that Dask primarily serves as a **scheduler**, written in Python as opposed to Scala ^[*], which allows for the lazy execution of Python-like code, distributing it across multiple workers.

^[*] Please note that the fact that Dask's scheduler is written in Python does not necessarily make it better than Spark for most purposes. Both Python and Scala/Java have their own strengths and weaknesses. For instance, due to Python's memory management approach, Dask still faces challenges with unmanaged memory over the lifecycle of an application. For more information, you can refer to this [link](#).

14.6. Starting a cluster

We can use Dask locally to leverage the multitasking/processing capabilities of our local machine, or we can set up a cluster and deploy a scheduler and multiple worker nodes.

Once the cluster is set up, we can initialize a `Client` by providing it with the address of a scheduler.

```
# import the dask distributed client
from dask.distributed import Client

# instantiate the client by providing
# the address:port of the scheduler
client = Client('dask-scheduler:8786')

# inspect the client
client
```

We can monitor the status of the cluster with a dedicated webui by accessing the location (IP address) of the scheduler/master node at port 8787. Using Docker with the appropriate port-mapping (provided in the Docker compose file) we can access the dashboard on `localhost:8787`.

The dashboard provides an overview of both the status of the workers, and the execution of the DAG task graphs (when we begin using them).

14.7. Simple parallelization of python functions

In the following, we will follow a similar approach to the one provided by the excellent Dask documentation.

The simplest example of parallelizing any arbitrary Python code in Dask can be demonstrated with a couple of simple operations, represented by two functions and an arbitrary sleep time of 1 second.

The sleep time is meant to represent arbitrarily complex code (**your task**) and the time required for its execution.

```
from time import sleep

# dummy function incrementing the input value by 1
def increment(x):
    # sleep for 1s
    sleep(1)
    # given the input x, return x+1
    return x + 1

# dummy function decrementing the input value by 1
def decrement(x):
    # sleep for 1s
    sleep(1)
    # given the input x, return x-1
    return x - 1

# dummy function summing two input values
def add(x, y):
    # sleep for 1s
    sleep(1)
    # given the inputs x and y, return x+y
    return x + y
```

These are purely Python functions...

We can test the functions locally by running them on the client (not on the cluster).

```
%%time

# we should expect a wall-time of roughly 3 seconds (3x 1sec sleep)
x = increment(1)
y = decrement(2)
z = add(x, y)
```

In order for Dask to leverage the processing units assigned to the cluster, we need to construct the Directed Acyclic Graph (DAG) corresponding to the execution of the code we want to deploy on the cluster. Subsequently, we allow the scheduler to dispatch the tasks to the workers.

This is accomplished in Dask by marking a function as **delayed**.

The **delayed** method in Dask accepts two main arguments: - The first argument is the function to be executed in parallel. - The subsequent arguments are the arguments upon which the original function will operate.

14. Dask

Now, we aim to transform the `increment`, `decrement`, and `add` functions, thereby making them **lazy**.

```
# import the dask delayed module
from dask import delayed

%%time

# make the function behave as lazy with delayed
#
# result = delayed(your_function)(<your_function_arguments>)
x = delayed(increment)(1)
y = delayed(decrement)(2)
z = delayed(add)(x, y)
```

At this stage, as is typical with lazy operations, the results are not yet stored in `z`.

At this stage, `z` simply represents the “plan” of the code execution, created by the Directed Acyclic Graph (DAG) task scheduler.

To visualize the plan of execution, we can utilize the `visualize()` method (note that the `graphviz` Python package is required for this).

```
# dask offers a nice and simple
# visualization of the DAG
z.visualize()
```

This visualization represents the task graph **before** any optimization is applied.

We can also request Dask to provide the DAG **after** optimization by using the `optimize_graph=True` option.

For this simple task, we shouldn’t anticipate significant optimization. However, with more complex tasks, we can expect to see a noticeable difference.

```
# optimized DAG
z.visualize(optimize_graph=True)
```

Alternatively, Dask also offers a high-level task visualization tool that can be accessed by clicking on it from within a Jupyter notebook.

```
# high-level task graph
z.dask
```

To actually execute the job, we need to instruct Dask to trigger the execution by requesting the results of the lazy operation with the `compute()` method.

Under the hood, Dask will send the computational graph to the scheduler and dispatch the tasks to the workers, similar to what was discussed for Spark.

It's important to note that `compute()` is a blocking operation, meaning that the program will wait for the results to be computed before proceeding.

```
%time

# the execution time should now be
# less than the 3 seconds measured above
#
# ideally, we should expect 2 seconds:
# - 1 second (sleep time) to run the increment and decrement functions --in
  ↪ parallel--
# - 1 second (sleep time) to run the add function based on the results of
  ↪ the previous stages
z.compute()
```

14.8. Futures and Eager computation with `submit`

The eager operation alternative to `delayed` in Dask is `submit`.

Using `submit` instructs Dask to immediately submit our task to the cluster and begin executing it on the available computing resources.

This approach is quite similar, and in fact, almost equivalent to the concept of a batch system.

```
# submit the function increment with argument 1
# return a "promise" of getting the actual result
future = client.submit(increment, 1)
```

The function `increment` with its argument 1 is submitted to the cluster, and the `future` object is returned immediately. However, this `future` object **does not contain the result of the computation**; rather, it represents an execution **promise** of the instruction that was submitted. It's important to note that the execution may not have been completed yet, as the cluster might take some time to process it.

The `future` variable **does not contain the result** itself, but rather a promise of it for when the computation is completed.

The result of the computation will remain on the worker nodes of our cluster and will not be sent back to our client until we explicitly request it.

This concept is particularly useful when dealing with large computation results or when subsequent tasks need to be executed on the data resulting from previous tasks.

To retrieve the result from the cluster to the client, we can invoke the `gather` function on the `future` object.

```
# the promise of the result
future
```

```
# the result, gathered from the cluster
client.gather(future)
```

This approach can be extremely useful in situations where we need to submit a task multiple times, potentially with different input parameters.

This scenario is common in machine learning, especially when optimizing an algorithm across a hyperparameter space for a specific dataset.

The concept behind this approach allows us to `map` the instruction we want to execute to each element of a dataset, effectively submitting the same operation across the entire dataset.

In practical terms, we are parallelizing our function in a manner similar to how it's done with the Python `multiprocessing` module, but leveraging a large and horizontally scalable set of computing resources.

```
# restart from a list of integers
data = [0, 1, 2, 3, 4, 5, 6, 7, 8]

# submit the increment function on ALL elements of the list
# using a `map` approach (each element is independent from the others)
future_results = client.map(increment, data)

# this execution is Eager and Asynchronous
future_results
```

```
# check the status of the jobs
future_results
```

```
# retrieve the data from the cluster
new_data = client.gather(future_results)

print(new_data)
```

`client.map` operates **asynchronously**, meaning that the cluster computes the results without blocking our local Python code for task completion.

Think of it in terms of `process.start()` and `process.join()` in `multiprocessing`.

However, **if** we need to wait for the result of a `submit` computation to be ready — for instance, if we require it as an input for other computations — we can use the `wait` method. This method blocks the execution of new code and waits for the computation of the future.

```
# import the dask wait
from dask.distributed import wait

# start the computation
new_future = client.map(increment, new_data)

# block this python process
# and wait for the remote task to be completed
wait(new_future)
```

It is important to notice that `wait` and `gather` are not the same operation.

The `wait` function only blocks the local Python process until the submitted computation is completed on the cluster.

However, it does **not** move the result back to the client.

The result is still stored on the worker nodes, and it will be transferred to the client only when we explicitly call `gather`.

This is useful when we want to make sure that some computation is completed before continuing, but we do not yet need the actual result locally.

```
# the result, gathered from the cluster
client.gather(new_future)
```

In complete analogy to what we have discussed with the `delayed` lazy execution, we can combine multiple instructions that need to be submitted to the cluster to create a more complex job to run on our cluster.

It's important to remember that the results of the `submit` execution reside on the cluster until a `gather` is used.

This means that **we can submit a task that takes as an argument an execution promise** of an instruction that has been previously submitted*.

The `gather` function should be invoked at the end of the program, only when the results need to be effectively retrieved from the cluster.

```
# submit the increment function
x = client.submit(increment, 1)
# submit the decrement function
y = client.submit(decrement, 2)

# submit the add function
#
# this will run on the promises of the results
# of both the x and y (possibly not yet available)
```

```
total = client.submit(add, x, y)
```

```
# this is still the execution promise
print(total)
```

```
# this is the final result
client.gather(total)
```

14.9. Exercise 1: Parallelization of a for loop

At this point, with the previous knowledge of what discussed with Spark, and the `delayed` and `compute` Dask operations, we should already be able to run simple “dummy” tasks.

Starting from a list $\vec{x}$ of values: 1. write a function to increment each element x_i by a random value δx_i (in the 0-1 range) 2. write a function to multiply the resulting value by 10 3. loop over $\vec{x}$ and apply both functions on each x_i element using the `delayed` 4. retrieve the sum of all the new updated elements 5. visualize the task graph

```
import random
import time

# input data
data = [1, 2, 3, 4, 5, 6, 7, 8]

# 1. increment function
def add_rand(x):
    return x+random.random()

# 2. multiplication function
def mult_ten(x):
    return x*10

# placeholder for the updated array
results = []

# 3. for each element
# - add a random value
# - multiply by ten
# - append the new data in a list
for x in data:
```

```
# 4. sum all elements of the list
total =
```

```
# 5. visualize the task graph
total.
```

```
# visualize the Optimized task graph
#
# once again, we should not expect any real optimization
# to be viable for this map-like operations
total.
```

```
%%time

# compute the result and time it
result = total.
```

```
# check the result
print("result: ",result)
```

14.9.0.1. Run the same task with the eager submit execution approach

```
# input data
data = [1, 2, 3, 4, 5, 6, 7, 8]

# placeholder for the updated array
results = []

# for each element
# - add a random value
# - multiply by ten
# - append the new data in a list
for x in data:

# sum of all elements of the list
total = client.
```

```
# print the future object
total
```

```
# print the result
print("result: ",client.    )
```

There is clearly another alternative to the previous approach using the **map** Dask functionality:

```
# input data
data = [1, 2, 3, 4, 5, 6, 7, 8]

# map both functions on all data elements

# submit the sum function on the z future
total = client.
```

```
# print the future object
total
```

```
# print the result
print("result: ",client.    )
```

14.10. Exercise 2: Map Reduce with Dask

So far, we have primarily used *map*-like operations and then collected all outputs in a single *sum* operation.

Now, we need to explore how we could write in Dask an equivalent *reduce* function to evaluate the sum of a list of elements pair-wise.

We can utilize the *add* function previously defined, which includes a 1-second sleep time, to visualize the time taken to run the task.

Here is a pair-reduction algorithm implemented with a (“*nasty*”) nested *for* loop and some simple Python logic.

```
%%time

L = [1, 2, 3, 4, 5, 6, 7, 8, 9]

while len(L) > 1:
```



```

# Temporary list to store the sum of pairs
_ = []

# Iterate over the indices of L with a step of 2 to process pairs of
  ↪ elements
for i in range(0, len(L), 2):
    if i+1 < len(L):
        # Sum pairs of elements in the list
        pair_sum = add(L[i], L[i + 1])
    else:
        # If the length of L is odd, add the last element with 0
        pair_sum = add(L[i], 0)
    _.append(pair_sum) # Append the sum to the temporary list

# Replace L with the temporary list containing pair sums
L = _

# Print the final result
print("result:",L[0])

```

To parallelize this reduce task, we can define the pair-wise sum of neighbor elements as `delayed`.

```

%%time

L = [1, 2, 3, 4, 5, 6, 7, 8, 9]

# rewrite the same algorithm using delayed
while len(L) > 1:

```

```

# visualize the task graph for L[0]
L[0].visualize()

```

```

%%time

# compute the result
result = L[0].compute()

```

```

print("result",result)

```

14.11. Exercise 3: Count how many words are present in a series of documents

Given a dataset composed of pieces of text taken from `sklearn`, the task is to count how many words are present in each document and calculate the total number of words in the dataset.

The dataset consists of approximately 8000 documents.

While one way to proceed would be to loop over all documents and count the words in each, a more efficient approach is needed, and we should now know how to do it.

```
from sklearn.datasets import fetch_20newsgroups
from dask.distributed import Client
import time

categories = [
    'comp.graphics',
    'comp.os.ms-windows.misc',
    'comp.sys.ibm.pc.hardware',
    'comp.sys.mac.hardware',
    'comp.windows.x',
    'misc.forsale',
    'rec.autos',
    'rec.motorcycles',
    'rec.sport.baseball',
    'rec.sport.hockey',
    'sci.crypt',
    'sci.electronics',
    'sci.med',
    'sci.space'
]

dataset = fetch_20newsgroups(subset='train', categories=categories ).data

print("Documents in the dataset:",len(dataset))
```

```
# a simple function to split a body of text
# into words and count them
def count_words(text):
    words = text.split()
    return len(words)
```

A simple example of single-threaded execution in plain Python can be the following

```
%%time

# initialize a word-per-document list
total_words_per_document = []

# count the number of words in each document
for document in dataset:
    total_words_per_document.append(count_words(document))

# calculate the total number of words in the dataset
total_words_dataset = sum(total_words_per_document)

print(f"Total number of words in the dataset: {total_words_dataset}")

import matplotlib.pyplot as plt

plt.hist(total_words_per_document, bins=range(0, 1000, 10));
plt.xlabel('words per document');
plt.ylabel('counts');
```

14.11.0.1. Implement the distributed version using the delayed lazy execution

```
%%time

# initialize a word-per-document list

# count the number of words in each document
# using dask delayed

# calculate the total number of words in the dataset
# using dask delayed
total_words_lazy =
```

```
%%time

# execute the tasks and retrieve the result
lazy_result =
```

```
print(f"Total number of words in the dataset: {lazy_result}")
```

14.11.0.2. Implement the distributed version using the map and submit eager execution

```
%%time

# count the number of words in each document
# using dask map

# calculate the total number of words in the dataset
# using dask submit
total_words_eager =

# check the future object
total_words_eager

eager_result =
print("Total number of words in the dataset: {}".format(eager_result))
```

14.12. Exercise 4: Calculate the first n Fibonacci numbers

Let's define the plain Python algorithm to evaluate the sequence of Fibonacci up to the n -th element:

Once again, let's start from some simple single-threaded Python code for this:

```
def fibonacci_sequential(num):
    iteration = 1
    fibonacci = []
    if num <= 0:
        pass
    elif num == 1:
        fibonacci.append(1)
    elif num == 2:
        fibonacci.append(1)
        fibonacci.append(1)
    elif num > 2:
```

```

    fibonacci.append(1)
    fibonacci.append(1)
    while iteration < (num - 1):
        fibonacci.append(fibonacci[iteration] + fibonacci[iteration-1])
        iteration+=1
    return fibonacci

```

```

# test it
n = 17

print(f"The first {n} fibonacci numbers are: {fibonacci_sequential(n)}")

```

14.12.0.1. Implement the same algorithm using the Dask lazy evaluation, with delayed, and inspect the task graph

```

# in order to generalize the append we need
# to create a function that does not get
# an error when called with an empty array
def append(arr = [], val = 0):
    if val != None:
        arr.append(val)
    return arr

# define the delayed version of the fibonacci code
def fibonacci_delayed(num):

```

```

# test it
result = fibonacci_delayed(8)

```

```

# visualize the graph
result.visualize(rankdir="LR")

```

```

# compute
result.compute()

```

14.13. Exercise 5: Monte Carlo Integration

Let's assume we want to integrate of a function via MonteCarlo technique, as you have discussed in LCP Module A.

14. Dask

Let's use the function

$$f(x) = \sin^2\left(\frac{1}{x(2-x)}\right)$$

and let's integrate in the range (0, 2)

```
import numpy as np

def f(x):
    return (np.sin(1/(x*(2-x))))**2

x=np.linspace(-0.2,2.2,4000)

plt.figure(figsize=(16,6));
plt.plot(x,f(x),'grey','.');
plt.fill_between(x[np.where((x>0) & (x<2))],[1]*len(np.where((x>0) &
    ↪ (x<2))), alpha=0.2);
plt.fill_between(x[np.where((x>0) & (x<2))],f(x[np.where((x>0) & (x<2))]),
    ↪ alpha=0.2);
plt.vlines([0, 2], 0, 1, colors = ["k", "k"], linestyle = ["dashed",
    ↪ "dashed"],linewidths=[3,3],zorder=20);
plt.xlabel('x');
plt.ylabel('$f(x)$');
```

Create the single-threaded Python code to execute this task, and evaluate the integral over N=100 000 points

```
%time
# Monte Carlo integration using Python's built-in random module

# number of points to use for integration
N = 100000

# list of counts for points that pass the function test
count = []

# function to check if a random point is passing
def pass_function():
    # generate a random x value between 0 and 2
    # generate a random y value between 0 and 1
    x = 2 * random.random()
    y = random.random()
    # check if the point (x, y) is under the curve defined by f(x)
    return 1 if y < f(x) else 0
```

```

# iterate over all points and count how many pass the function test
for i in range(N):
    count.append(pass_function())

# compute the integral by dividing the sum of the counts by the total number
    ↪ of points,
# and multiply by the width of the integration range (2)
I = 2 * sum(count) / N

# print the result
print(f"Integral = {I}")

```

14.13.0.1. Apply the minimal changes to the code to deploy the same integral computation on the Dask cluster using delayed

NOTE: Do NOT use 100 000 points in this case, but *limit the computation to $N=1\ 000$ points*

```

%%time
# Monte Carlo integration using Python's built-in random module

# number of points to use for integration
N = 1_000

```

We can even use Python **decorators** to declare that an entire function is going to be interpreted as delayed

```

@dask.delayed
def my_delayed_function(arg):
    ...
    return val

```

```

%%time
# Monte Carlo integration using Python's built-in random module

# import dask to use its delayed decorator
import dask

```

Why are we experiencing degraded performance compared to single-threaded execution?

The Dask code may exhibit slower performance than the equivalent plain Python code due to the **overhead** introduced by Dask itself.

When utilizing Dask, we construct a task graph that Dask must traverse to execute the compu-

tation. This process involves additional computation and communication overhead between the scheduler and the workers, contributing to increased execution time compared to the plain Python version.

Each delayed task inherently incurs an overhead of hundreds of microseconds. With a large number of computations issued, this overhead can become significant.

Furthermore, our usage of `delayed` to parallelize the computation may not fully exploit Dask's parallelism capabilities. By creating a list of `delayed` objects and then computing them all at once with `delayed(sum)(count)`, we essentially execute 1000 `count` functions as individual processes on the worker nodes.

This approach can be inefficient since it still processes the loop sequentially before aggregating the results in parallel. A more effective strategy could involve leveraging Dask's `dask.bag` to parallelize the computation more efficiently.

Overall, it's crucial to ensure that we maximize Dask's capabilities to parallelize the computation effectively. In some scenarios, utilizing Dask may not yield any performance benefits and could potentially be slower than plain Python if the task is not optimized for parallel execution.

14.14. Exercise 6: π via MonteCarlo

We can re-run the evaluation of π using the Monte Carlo technique, similar to what was done during the Spark hands-on session.

However, it's important to note that, at present, we haven't partitioned the data in Dask (yet).

Instead, we're instructing Dask to execute a simple task for each entry in our list, which incurs significant overhead.

Therefore, it's advisable to start with a very limited number of points, approximately 10, and gradually increase to a maximum of around 100. This cautious approach allows us to monitor the job's status from the dashboard and assess its performance effectively.

```
%time

import random

# set the number of points to use
num_points = 100

# list to hold the points inside the circle
points_in_circle = []

# function to check if a point is inside the circle
def in_circle():
    # generate a random x and y value between 0 and 1
```



```

# iterate over all the points and count how many are inside the circle

# count the number of points inside the circle
num_points_inside =

# compute pi
pi = 4*num_points_inside/num_points

# print result
print (f"{pi}")

```

14.14.0.1. Use delayed to parallelize the computation

```

%%time

# print result
print (f"{pi}")

```

14.14.0.2. Use map or submit to parallelize the computation using the Dask eager execution

With submit we have to be very careful...

Naively calling `client.map` with a function that returns a random variable may result in the same exact random value being returned every time. Essentially, we have *one function* executed 10000 times.

However, we can inform Dask that this function is **impure**, meaning it can produce different output values for the same input:

```
client.submit(your_function, pure=False)
```

This approach ensures that we obtain 10000 different random variables. However, it's important to note that while this guarantees diversity in the random variables, it can pose challenges for Dask's parallelization. We may encounter unnecessary overhead and performance penalties due to the increased complexity introduced by impure functions.

```
%%time
```

```
# print result
print (f"{pi}")
```

14.14.1. Take-away Message

Dask offers flexibility in scheduling our Pythonic code and tasks, allowing for both lazy and eager execution strategies.

However, as discussed, parallelization tends to be most beneficial for tasks involving substantial data or computationally intensive operations that can be divided into smaller chunks and executed concurrently.

The simple functions used in our examples do not represent computationally “heavy” tasks well, and thus, the benefits of parallelization (if any at all) may not be fully realized.

Moreover, Dask Distributed can yield significant performance gains when computations are distributed across multiple nodes or machines, rather than just across a few CPU cores on a single machine (as in our case using Docker compose).

Given these factors, it’s always essential to thoroughly evaluate the performance of parallelized code and compare it against single-threaded implementations to ensure that the desired efficiency gains are achieved.

14.15. Stop client

```
client.close()
```

Finally, use `docker compose down` to stop and clear all running containers.

14.15.1. Lecture 2: Dask Data Structures

Dask provides several Pythonic data structures designed to handle and manipulate data that exceeds our local memory capacity:

- `dask.bag`: A distributed generic Python list of objects, analogous to a PySpark RDD.
- `dask.array`: Distributed NumPy arrays.
- `dask.dataframe`: Distributed pandas dataframes, offering functionality similar to pandas but capable of handling larger-than-memory datasets.

All these high-level data structure APIs are optimized to leverage the Directed Acyclic Graph (DAG) optimization features of the Dask scheduler. Consequently, they rely on lazy computation, allowing for efficient execution of operations on large datasets.

14.16. Start the Dask cluster

```
from dask.distributed import Client

# use the provided master
client = Client('dask-scheduler:8786')

# print the status of the client
client
```

14.17. Dask Bag

Bags are highly versatile and flexible data structures in Dask.

Dask Bag offers a level of flexibility similar to PySpark's RDDs. They serve as parallelized collections of objects, similar to Python's built-in `list`, capable of holding any Python objects, whether they're custom classes or built-in types.

This flexibility allows for the storage of complex data structures like raw text or nested JSON data, which can be easily navigated.

Due to this versatility, Dask bags are commonly employed to parallelize simple computations on unstructured or semi-structured data, such as text data, log files, JSON records, or user-defined Python objects. They facilitate MapReduce-like approaches for loading, inspecting, filtering, and processing arbitrary datasets, whether structured or unstructured.

Dask Bag provides operations such as `map`, `filter`, `groupby`, and aggregations on collections of Python objects. It accomplishes these tasks in parallel using Python iterators, resembling a parallel version of `itertools`.

After completing an initial stage of data preparation with Dask Bag, it's often customary to reduce and convert the data into more suitable data structures, such as Dask Arrays or Dask DataFrames, which will be covered in subsequent sections.

14.17.1. Create and Take from a Bag

We can create a Bag from various data sources, including Python sequences, files, and cloud storage services like Amazon AWS S3, among others. For a comprehensive overview on accessing remote data from Distributed File Systems, S3, and other sources, please refer to the official documentation [here](#).

Additionally, we can create a Bag from a function declared as `delayed`. This approach allows us to generate data within a distributed application and subsequently access it using the Bag API before computing a result.

The data within the Bag is partitioned into blocks, typically with multiple items per block. The number of partitions (`npartitions`) depends on factors such as the dataset size, cluster resources, and our specified partitioning strategy.

14. Dask

Let's start by creating a simple data **bag** from a Python list.

In Python, we can easily create data from a list. Since Python is a dynamically typed language, the list can contain a variety of data types such as integers, strings, or even objects.

For example, we can create a simple array of integers:

```
import dask.bag as db

# create a Dask Bag from a Python sequence
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# create a Dask Bag from the sequence with 4 partitions
b = db.from_sequence(data, npartitions=4)
```

As previously mentioned, Dask data structures embody the lazy programming paradigm. The data is thus not yet stored on the cluster, as we have not performed an operation such as **compute**.

In general, we don't want to retrieve the entire data stored on the cluster, but we might want to inspect a few elements. We can do that with the **take(n_elements)** method. The returned data will be a tuple containing the first **n_elements** of the Bag.

```
# retrieve the first 3 elements of the Bag
b.take(3)
```

Data from text files can be extracted in Dask by providing either a list of all files or using the ***** wildcard.

By default, the resulting Dask Bag will have one item per line and one file per partition. It's important to be mindful of partitioning when working with large datasets to avoid inefficient processing.

Dask provides automatic handling of standard compression libraries (e.g., **gzip**, **bz2**, **xz**) when reading text files. The compression type can be inferred from the file name extension or explicitly specified using the **compression='gzip'** keyword argument.

For example, to load a number of compressed files from a local folder into a Dask Bag:

```
# list the files provided
! ls datasets/accounts_json/.
```

```
import os

# read text files from the specified directory and use gzip compression
b = db.read_text(os.path.join('datasets',
                              'accounts_json',
                              'accounts.*.json.gz'),
```

```

files_per_partition=4)

# take the first element from the bag
example = b.take(1)

# print the type of the example variable
print(type(example))

# print the example variable
print(example)

```

Bag objects in Dask support standard functional APIs, such as `map`, `filter`, `groupby`, and more. These operations create new bags, allowing for chaining multiple operations together to manipulate the data until the desired result is obtained.

To execute the computations on a Bag object and obtain the final result, we can use the `.compute()` method, similar to how we handle `delayed` objects.

Since a Bag is inherently a delayed object, there's no need to explicitly specify that the functions applied to the dataset are further delayed. Dask handles the delayed execution transparently, allowing for efficient processing of large datasets.

```

# a silly function to check if a number is even
def is_even(n):
    return n % 2 == 0

# create a bag from a sequence of numbers
b = db.from_sequence([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# 1. filter the bag elements to retain only the even ones
# 2. then, map a lambda function to square each even number
c = b.filter(is_even).map(lambda x: x ** 2)

# print the resulting bag
c

```

```

# visualize the computational graph
c.visualize()

```

```

# compute the results
c.compute()

```

14.17.2. MapReduce in Dask: using fold

In PySpark, we have seen how to express computations using the MapReduce paradigm: - a **map** step transforms the data - a **reduce** step aggregates it into a final result

Dask follows the same model.

We have already used `map` on Dask Bags.

For the reduction step, Dask provides the `fold` operation.

While this plays the same role as `reduce` in Spark, it makes the parallel nature of the computation explicit by separating: 1. the reduction within each partition 2. the combination of partial results across partitions

```
# Map step: square each element
mapped = b.filter(is_even).map(lambda x: x ** 2)

# Reduce step: sum the results
result = mapped.fold(
    binop=lambda x, y: x + y,    # combination within each partition
    combine=lambda x, y: x + y  # combination of partial results across
    ↪ partition
)
```

```
result.visualize()
```

```
result.compute()
```

One of the crucial parameters when optimizing a computing task on a cluster is ensuring the effective utilization of available computing resources. This involves rewriting the computational task to be distributed efficiently and optimizing the number of partitions used to store data and perform map-like data transformations.

14.17.3. Considerations for Partitioning

Consider the following (“extreme”) scenarios: - Having only one partition with 100 available CPUs
- Having 10,000 tiny partitions with 3 available CPUs

14.17.3.1. Inefficient Single Partition

Having only one partition with a large number of available CPUs would be inefficient as there would be no parallelization at all.

14.17.3.2. Excessive Tiny Partitions

Conversely, having an excessive number of tiny partitions with few available CPUs each would likely be inefficient due to the overhead of starting and stopping computation on each individual tiny partition.

14.17.3.3. Optimal Partitioning Strategy

The optimal number of partitions depends on factors such as the amount of shuffling required, especially for operations like `groupby`. For instance, having a moderate number of partitions with a balanced distribution of available CPUs might be more efficient than having a smaller or larger number of partitions. However, the optimal choice will ultimately depend on your specific workload and cluster characteristics.

14.17.3.4. Experimentation and Fine-Tuning

Determining the optimal partitioning strategy is a task that requires experimentation and educated guesses based on the available processing units in your cluster. There is no one-size-fits-all answer to this question. You'll need to experiment and fine-tune your partitioning strategy based on your specific workload, computational requirements, and cluster characteristics.

```
# create a new bag with the same data as in the previous example
# but a different number of partitions
b = db.from_sequence([1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
                    npartitions=3)

# 1. filter the bag elements to retain only the even ones
# 2. then, map a lambda function to square each even number
c = b.filter(is_even).map(lambda x: x ** 2)

# visualize the computation graph
c.visualize()
```

```
# compute the results
c.compute()
```

14.17.4. Broadcasting data to all workers

As discussed in Spark, in some situations, we want to send the **same piece of data to all workers**, so that it can be reused by many tasks without being transferred multiple times: **broadcasting**.

In Dask we can achieve this using `client.scatter` with the option `broadcast=True`. This will tell the scheduler to distribute data across all worker nodes, exploiting data locality for this common resource.

14. Dask

This is particularly useful when: - we have a read-only object (e.g. configuration, lookup table, model parameters) - the same object is needed by many parallel tasks

By broadcasting it once, each worker keeps a local copy, avoiding repeated data transfers.

NB: If the option `broadcast` is set to `False` then the same data will be scattered across the cluster, but not fully copied in all nodes. Typically, data will be distributed in a round-robin fashion grouping by number of cores of the worker nodes.

```
lookup = {
    0: "a",
    1: "b",
    2: "c",
}

lookup_future = client.scatter(lookup, broadcast=True)

def func(x, lut):
    return f"{x} {lut.get(x % 3)}"

futures = client.map(func, range(20), lut=lookup_future)

client.gather(futures)
```

14.18. Exercise 1 - Open and Preprocess JSON Data

We'll start with a dummy dataset of gzipped JSON data located in your data directory. This dataset simulates data that you might collect from a document store database (e.g., MongoDB) or by scraping a website using a dedicated API.

Each line of each document is a JSON-encoded dictionary with the following keys:

- **id:** Unique identifier of the customer.
- **name:** Name of the customer.
- **transactions:** A list of key-value pairs in the form of `transaction-id` and `amount` pairs, representing each transaction made by the customer in that file.

1. Create a Bag reading out the dataset from the text files
2. Map the `json.loads` function on each message to convert the records in the form of Python dictionaries

```
# 1. create a Dask Bag from the files

# load up the files from dask directly in a bag

# take the first 3 elements from the Bag
```



```

# print the examples

# print the type of examples and the type of its first element

import json

# 2. read the data from the JSON format

# map each line to a JSON object using json.loads

# take the first 3 elements from the Bag

# pretty-print the examples

# print the type of examples and the type of its first element

```

Once the JSON data is mapped into the appropriate Python objects (dictionaries, lists, etc.) within a Dask Bag, we can perform specific operations by creating small Python functions to run on our data.

The most fundamental operations we can perform on a Dask Bag include:

- **map:** Apply a function to each element.
- **filter:** Retain only the elements that satisfy a given function.
- **pluck:** Select a specific nested field, such as `element[field]` from a Python dictionary.
- **flatten:** Unfold the dictionary into a list-like object.

These operations serve as building blocks for manipulating and transforming the data within a Dask Bag, providing flexibility in data processing pipelines.

14.18.0.1. 1. Compute the average number of transactions for each entry of a user named “Alice”

To achieve this task using Dask Bag operations, we’ll follow these steps:

1. Filter the dataset to retain only entries for users named “Alice”.
2. Map each entry to extract the number of transactions.
3. Compute the average number of transactions for each entry.

```

# retain only the records from users named "Alice"
db_js.

```

```
# retain only the records from users named "Alice"
# AND count the total number of transactions for each entry in the dataset

# function reformatting each record with the new information
def count_transactions(d):
    return

db_js.
```

```
# retain only the records from users named "Alice"
# AND count the total number of transactions (as 'count') for each entry in
    ↪ the dataset
# AND return only the 'count' values
db_js.
```

```
# retain only the records from users named "Alice"
# AND count the total number of transactions (as 'count') for each entry in
    ↪ the dataset
# AND return only the 'count' values
# AND compute the average of the counts
db_js.
```

```
# visualize the graph of the tasks composing the job
db_js.
```

14.18.0.2. 2. Compute the average amount of transactions users named “Alice”

To compute the average amount for all transactions made by users named “Alice” using Dask Bag operations, we’ll follow these steps:

1. Filter and pluck the dataset to retain only entries for users named “Alice”.
2. Flatten the list of transaction amounts.
3. Compute the average of all transaction amounts.

```
# retain only the records from users named "Alice"
```

```
# retain only the records from users named "Alice"
# AND flatten and pluck to return only the "amount" in a bag
```

```
# retain only the relevant transactions
# AND flatten and pluck to return only the "amount" in a bag
# AND compute the average of all transactions amounts
```

```
# visualize the graph of the tasks composing the job
```

14.18.1. Additional operations on Dask Bags: Groupby and Foldby

Additional standard operations on Dask Bags can be performed using the `groupby` and `foldby` methods, together with data aggregation functions.

14.18.1.1. Groupby

The `groupby` method shuffles the data so that all items with the same key are grouped together in the same key-value pair. This operation is useful for tasks that involve grouping data by a specific key.

14.18.1.2. Foldby

The `foldby` method walks through the data and accumulates a result per key. It combines the functionality of `groupby` and a reduction operation, making it suitable for efficient parallel split-apply-combine tasks. While more complex to use, `foldby` significantly reduces computational time compared to `groupby`, especially for tasks involving heavy data shuffling.

14.18.1.3. Considerations

It's important to note that operations involving heavy data shuffling, such as `groupby`, can be computationally expensive as they require moving data across workers. In such cases, using the `foldby` method provides a more efficient alternative.

Consider using the `foldby` method whenever possible to optimize performance and minimize data shuffling overhead, especially in large-scale distributed computing tasks.

```
names_data = ['Alice', 'Bob', 'Charlie', 'Dan', 'Edith', 'Frank']

# create a bag from the list of names
b = db.from_sequence(names_data)

# group names by length
res = b.groupby(len)

# visualize this "simple" graph before computing the results
res.visualize()
```

```
# compute the results
res.compute()
```

When the result of a `groupby` operation in Dask is a tuple and we need to apply functions to the elements of these tuples, we can use the `starmap` function.

The `starmap` function in Dask allows us to apply a function using argument tuples, similar to what the standard `itertools.starmap` does in Python.

For example:

```
# create a simple bag from a list of integers
b = db.from_sequence(list(range(10)))

# group numbers into even/odd groups
b.groupby(lambda x: x % 2).compute()
```

```
# return the max value for all elements in each group
b.groupby(lambda x: x % 2)\
  .starmap(lambda k, v: (k, max(v)))\
  .compute()
```

```
# return the sum of the elements in each group
b.groupby(lambda x: x % 2)\
  .starmap(lambda k, v: (k, sum(v)))\
  .compute()
```

```
# visualize the graph of this latest "extremely simple" computation
res = b.groupby(lambda x: x % 2)\
      .starmap(lambda k, v: (k, sum(v)))

res.visualize()
```

14.18.2. Understanding foldby in Dask

`foldby` in Dask can initially seem peculiar, but it shares similarities with functions in other libraries:

- `toolz.reduceby`
- `pyspark.RDD.combineByKey`

When using `foldby`, you need to provide:

1. A **key function to group elements** (similar to `groupby`).

2. Either:

- A **binary operator** (function that takes 2 elements and returns 1 of the same type) that performs **reduction within each group**.
- Or a **combine binary operator** that can **combine the results of two reduce calls on different parts of your dataset**.

In Dask, a foldby call like this:

```
dask_bag.foldby(key, binop, init)
```

is equivalent to a combination of two operations: `groupby` and `reduce`:

```
def reduction(group):
    return reduce(binop, group, init)

dask_bag.groupby(key).map(lambda (k, v): (k, reduction(v)))
```

The reduction operation must be associative and is executed in parallel within each partition of the dataset. The intermediate results are then combined using the `combine` binary operator.

Let's re-write the equivalent group-by + starmap operation with a foldby call

```
b.groupby(lambda x: x % 2)\
  .starmap(lambda k, v: (k, sum(v)))\
  .compute()
```

```
# create a simple bag from a list of integers
b = db.from_sequence(list(range(10)))
```

```
# groupby even/odd numbers with a foldby and find the total sum per group
#
# write down a **binary filter function** to select only even or odd
↪ numbers
# write down a **reduce-like operation** to sum all elements
is_even = lambda x: x % 2 == 0
add      = lambda x, y: x + y
b.foldby(key=is_even,
        binop=add,
        initial=0).compute()
```

```
# have a look at the graph and compare it
# with the groupby implementation
```

```
#
# the `split_every` option instructs foldby to group
# partitions into groups of this size while performing the reduction.
# (`split_every` defaults to 8)
b.foldby(is_even,
        add,
        initial=0,
        split_every=8).visualize()
```

14.19. Exercise 2 - Account Data

In this exercise, we'll work with account data to get the total number of users with the same name from the dataset. We'll compare the computational time of achieving this task using both the `groupby` and `foldby` functions in Dask.

- Take a moment to look at the `foldby` API documentation [here](#).

1. Using Groupby Function:

- Utilize the `groupby` function to group users by their name and count the number of users with the same name.
- Measure the computational time required for this operation.

2. Using Foldby Function:

- Explore the `foldby` function in the Dask Bag API to achieve the same task.
- Measure the computational time required for this operation.

```
%%time
# groupby on the 'name' key
# count the number of items in each group
result_groupby = db_js.
```

Let's inspect what `groupby` is doing in Dask...

```
sorted(result_groupby)
```

What about `foldby`?

```
%%time
# foldby on 'name' key
# increment by one each time we see an element (binop function)
# use a final combination function with add (combine function)
result_foldby = db_js.
```

```
sorted(result_foldby)
```

14.20. Exercise 3 - Computing Total Transfers Amount per Name

In this exercise, we'll work with account data to compute the total transfers amount per each name using a foldby operation in Dask.

1. Create a Function for Summing Amounts:

- Create a function that takes an input dictionary containing the name and transactions, and produces the sum of the amounts.
- For example:

```
{'name': 'Alice', 'transactions': [{'amount': 1, 'id': 123},
  ⇨ {'amount': 2, 'id': 456}]}
```

The sum of the amounts for this input would be 3.

2. Modify Binary Operator for Foldby:

- Modify the binary operator of the foldby examples to accumulate the sum of the transferred amounts instead of counting the number of entries.

```
# add the sum_transactions values together
def add_values(tot, _):
    return

# compute the sum of transaction amounts per name
def sum_amount(d):

    return

# apply sum_amount function to each item in db_js
# perform foldby operation on 'name' key to accumulate the sum of
  ⇨ transactions for each name
result_foldby = db_js.
```

```
result_foldby.visualize()
```

```
%%time
result_foldby = result_foldby.compute()
```

```
sorted(result_foldby)
```

14.21. From Bag to pre-processed output datasets

Dask Bags are often used as an entry point to ingest, decode, and preprocess data before further processing or analysis. Once the data has been processed and transformed, it's desirable to convert the Dask Bag into a structured data format, such as a Dask DataFrame, for easier manipulation and analysis.

Dask provides several methods to convert Bags into output data objects, including:

- `to_textfiles`: Writes the Bag data to multiple text files.
- `to_avro`: Writes the Bag data to Avro files.
- `to_delayed`: Converts the Bag into a list of delayed objects.

By far, the most widely used approach in data preprocessing using Dask Bags follows the Extract-Transform-Load (ETL) process:

1. **Extract**: Raw data is extracted from the original input source.
2. **Transform**: The extracted data is transformed by applying functions to filter, reduce, or create features from the original (often messy) dataset.
3. **Load**: The transformed and cleansed dataset is loaded into a database or further data processing pipeline based on structured data.

To convert a Dask Bag into a Dask DataFrame, the dataset needs to be flattened and normalized before invoking the `to_dataframe` function on the Bag. This flattening and normalization process ensures that the data is in a structured format that can be easily loaded into a DataFrame.

This conversion process is similar to converting from RDD to a Spark DataFrame and is a common operation in data processing workflows.

14.22. Flattening Deeply Nested Account Data

As a purely illustrative example, our account data is deeply nested and not suitable for being transformed into a table-like DataFrame structure.

We may want to retain only the first transaction per customer or retain other features such as the max-amount transfer or an aggregated quantity per user.

To achieve this, we can flatten the dataset by mapping a dedicated function. This function will extract the desired features from the nested structure and create a flattened representation suitable for further processing or conversion into a Dask DataFrame.

```
# print one element of the json bag
pprint( db_js.take(1) )
```



```
# function to flatten the nested structure of the record
# and extract specific fields
def dummy_flatten(record):
    return {
        'id': record['id'],
        'name': record['name'],
        'first_transaction_id': record['transactions'][0]['transaction-id'],
        'first_transaction_amount': record['transactions'][0]['amount']
    }

# apply the dummy_flatten function to each record in db_js
# and take the first element from the resulting Bag
pprint(db_js.map(dummy_flatten).take(1))
```

To create a Dask distributed DataFrame, we can use the `to_dataframe` method on the flattened Dask Bag.

The Dask DataFrame is still a Dask distributed object, i.e.:

- **Partitioned Across Workers:** Like other Dask data structures, Dask DataFrames are partitioned across the workers in the cluster. This allows for parallel processing of data.
- **Lazy Operations:** Operations performed on Dask DataFrames are lazy, meaning that they are not executed immediately. Instead, a task graph is created, and computations are deferred until the result is explicitly needed.

```
# create a Dask DataFrame from the flattened Bag
dd = db_js.map(dummy_flatten).to_dataframe()
```

Unlike a Pandas DataFrame, printing a Dask DataFrame won't show its records but rather its structure and the number of partitions it has been divided into.

- **Structure Display:** When you print a Dask DataFrame, you'll see its structure, including column names, data types, and the number of partitions.
- **Absence of Records:** The actual data records are not shown in their entirety. Displaying all records would be equivalent to collecting all data in the scheduler and passing it to the client, which is similar to the `collect` operation. This could lead to performance issues with large datasets.

```
# show the DataFrame
dd
```

In Dask DataFrames, issuing the `head` or `tail` methods triggers computation and retrieves results from the cluster.

This behavior is different from the printing of the DataFrame structure, which only displays meta-data without fetching the actual data.

14. Dask

- **Computation Trigger:** When you call the `head` or `tail` methods on a Dask DataFrame, Dask will compute a small portion of the DataFrame to display. This computation is performed on the cluster, and the results are then returned to the client for display.
- **Performance Consideration:** It's important to consider the performance implications of calling `head` or `tail`, especially for large datasets. Triggering computation for a large portion of the DataFrame can lead to significant overhead, as it involves transferring data between workers and the client.

```
# show the first 10 elements of the dask dataframe  
dd.head(10)
```

14.23. Stop client

```
client.close()
```

Finally, use `docker compose down` to stop and clear all running containers.

14.23.1. Lecture 3: Dask DataFrames and Arrays

Dask provides several Pythonic data structures designed to handle and manipulate data that exceeds our local memory capacity:

- `dask.bag`: A distributed generic Python list of objects, analogous to a PySpark RDD.
- `dask.array`: Distributed NumPy arrays.
- `dask.dataframe`: Distributed pandas dataframes, offering functionality similar to pandas but capable of handling larger-than-memory datasets.

All these high-level data structure APIs are optimized to leverage the Directed Acyclic Graph (DAG) optimization features of the Dask scheduler. Consequently, they rely on lazy computation, allowing for efficient execution of operations on large datasets.

14.24. Start the Dask cluster

```
from dask.distributed import Client  
  
# use the provided master  
client = Client('dask-scheduler:8786')  
  
# print the status of the client  
client
```

The `dask.dataframe` API provides a parallel DataFrame object that closely resembles the Pandas DataFrame interface.

A Dask DataFrame consists of multiple Pandas DataFrames or Series stored in memory across all machines in the cluster. The table of a Dask DataFrame is partitioned along the index axis ^{*}.

When performing operations on a Dask DataFrame, it triggers corresponding operations on the constituent Pandas DataFrames, taking into account potential parallelism and memory limitations.

** Quick question: In this context, what type of data partitioning does “partitioned along the index” refer to: Vertical or Horizontal?*

14.24.1. (On Data Partitioning)

Partitioning is a crucial aspect of data management and processing in distributed systems, such as frameworks like Spark and Dask.

Distributing our dataset into numerous small partitions allows for parallel processing on each node. However, it’s essential to acknowledge that every call the central scheduler makes to access data on a remote partition (on a worker) incurs some time, typically in the order of a few hundred milliseconds.

As users of distributed processing systems, it is our responsibility (not Dask’s or Spark’s) to determine and optimize the number of partitions. While the distributed framework may make an initial choice or guess, optimizing performance relies on us making the appropriate choice based on the task at hand.

For instance, when initially loading data from a set of files, the number of partitions might align with the number of CSV files we are importing. Nonetheless, as we alter the size of our DataFrames through filtering or joining, it may be prudent to reassess the number of partitions to optimize the Dask scheduler’s overhead.

There is always a cost associated with having too many or too few partitions, and unfortunately, there is no single rule to determine the “right” number of partitions.

As a *rule of thumb*, partitions should comfortably fit in memory. However, they should not be excessively numerous, as the overhead of the central scheduler can impact computation performance.

This suggests that the number of partitions should fall within a “reasonable range,” and their number and size should be selected by the user to optimize execution.

14.25. Basic operations on DataFrames

Let’s start by reading a set of structured files (comma-separated) into a DataFrame.

In pure Pandas, we would have to loop over all the files, open each one of them as a DataFrame, and concatenate all the DataFrames into a large DataFrame.

The resulting DataFrame is a single entity stored in memory, allowing for single-threaded data access.

```
! ls datasets/accounts_csv
```

```
! head -10 datasets/accounts_csv/accounts.0.csv
```

```
# import the necessary libraries
from glob import iglob
import pandas as pd
import os

# define the path to the CSV files
path = os.path.join('datasets', 'accounts_csv', 'accounts.*.csv')

# use glob to find all matching file paths
all_files = iglob(path, recursive=True)

# create a generator to read each CSV file as a Pandas DataFrame
dataframes = (pd.read_csv(f) for f in all_files)

# concatenate all DataFrames into a single large DataFrame
large_dataframe = pd.concat(dataframes, ignore_index=True)

# print the pandas DataFrame
large_dataframe
```

In Dask, DataFrames can be created from a `glob` pattern using the wildcard `*`, which reads all files in the specified path matching that pattern into the same Dask DataFrame.

It's important to remember that Dask DataFrames are a collection of Pandas DataFrames scattered across the workers.

When reading data into a Dask DataFrame, Dask automatically creates parallel jobs to read the data in parallel. By default, it creates one (parallel) job per chunk, which is typically the number of input files. However, it is recommended to consider how we want to partition our dataset to make the best use of our workers.

The Dask DataFrame API don't optimize the query execution for you like Spark or a SQL DBMS would.

This means that users waste a lot of computation not profiting from the number of optimization techniques normally available in structured data processing tools.

Starting only from very recent versions (2024.4), Dask is now transitioning to set a `query-planning` optimizer by default.

We should make however sure that the `query-planning` is activated, by issuing:

```
import dask
dask.config.set({'dataframe.query-planning':True})
```

```
# import the necessary libraries
import dask.dataframe as dd

# define the path to the CSV files
path = os.path.join('datasets', 'accounts_csv', 'accounts.*.csv')

# read the CSV files into a Dask DataFrame
df = dd.read_csv(path)
```

As always, reading data is a lazy operation, meaning it is postponed until we have something to compute.

Let's compute the length of the entire DataFrame.

```
# compute the length of the Dask DataFrame
len(df)
```

At this stage, each file was loaded into a separate Pandas DataFrame and scattered across the nodes.

Computing `len()` means that `len` was applied to each individual Pandas DataFrame, and then the sub-results were aggregated and combined to provide the overall total length of the DataFrame.

```
# "print" the Dask DataFrame
df
```

We can reassign the number of partitions to the Dask DataFrame object using the `repartition` method.

```
# change the number of partitions to 8
df = df.repartition(npartitions=8)
```

```
# "print" the Dask DataFrame
df
```

```
# compute the length of the Dask DataFrame
len(df)
```

```
# compute the number of partitions of the Dask DataFrame
df.npartitions
```

It is worth mentioning that unlike Pandas, Dask only reads a sample from the beginning of the file (or the list of files) to start inferring the data types.

These inferred data types are then enforced when reading all partitions, which can potentially lead to incorrect data type assignments.

For example, consider the case where we start reading a CSV file in which the first n entries of a column are all integers, but the column is actually supposed to be of type *string*.

```
df = dd.read_csv(os.path.join('datasets', 'nycflights', '*.csv'))
```

```
df.head()
```

```
# read CSV files - explicitly parse dates
df = dd.read_csv(os.path.join('datasets', 'nycflights', '*.csv'))

# re-format the date
df['Date'] = dd.to_datetime(
    df.rename(
        columns = {'Year':'year', 'Month':'month', 'DayofMonth':'day'}
    )[['year', 'month', 'day']],
    meta=pd.Series(dtype="datetime64[ns]")
)
```

```
# the head of the *distributed* Dask DataFrame
df.head()
```

```
# the tail of the *distributed* Dask DataFrame
df.tail()
```

In this case, the datatypes inferred for `CRSElapsedTime` and `TailNum` are in fact incorrect.

However, we can force Dask to interpret the data types as we want by using the `dtype` assignment. This is usually the most viable and robust solution, but not the only one.

Alternatively, we could: - Increase the size of the sample used to infer the data types. - Use `assume_missing` to make Dask assume that columns inferred to be `int` (which don't allow missing values) are actually `floats` (which do allow missing values).

However, in general, the best solution is still to hard-code the data types.

```

# read CSV files - explicitly parse dates
df = dd.read_csv(os.path.join('datasets', 'nycflights', '*.csv'),
                 dtype={'TailNum': str,
                       'CRSElapsedTime': float})

# re-format the date
df['Date'] = dd.to_datetime(
    df.rename(
        columns = {'Year': 'year', 'Month': 'month', 'DayofMonth': 'day'}
    )[['year', 'month', 'day']],
    meta=pd.Series(dtype="datetime64[ns]")
)

# repartition the dataframe
df = df.repartition(npartitions=8)

# print the tail of the dataframe
df.tail()

```

While on the topic of reading data from files, it is worth mentioning that Dask offers a wide range of APIs for importing structured data from various file formats into a DataFrame object, including:

- CSV - JSON - Avro - Parquet

For a more extensive list, please refer to the official documentation at the following [link](#).

14.26. Computations with Dask DataFrames

Dask DataFrames are designed to closely mimic the Pandas DataFrame API, with nearly a 1-to-1 correspondence. Dask provides a wrapper around the Pandas API to manage the same calls on the distributed collection of Pandas DataFrames, which in our case are the partitions of our dataset.

Using Dask DataFrames is beneficial when Pandas reaches its limits, such as when dealing with datasets larger than memory or tasks that can be efficiently parallelized.

However, when working with small datasets or tasks that are not easily parallelizable, Pandas will always be the better choice.

For this reason, the typical computing pattern when working with Dask DataFrames is as follows:

1. Use **Dask Bag** to ingest data from semi/unstructured sources and preprocess it into Dask DataFrames.
2. Use **Dask DataFrame** for parallel computations and data reduction to produce a slimmed-down DataFrame that can fit in memory. Then, convert it into a single Pandas DataFrame.
3. Use **Pandas** for simple (yet fast) DataFrame operations for non-parallelizable tasks.

We can always retrieve data from all partitions and create a Pandas DataFrame from a Dask DataFrame using the `compute` method on the DataFrame itself.

14. Dask

However, it's important to note that this operation should be done carefully and only at the appropriate time, especially when dealing with very large (possibly terabytes-sized) Dask DataFrames. It should only be performed when we have reduced the DataFrame to a manageable size that can fit into our computer's memory.

```
# convert Dask DataFrame to Pandas DataFrame
pandas_df = df.compute()

# display the Pandas DataFrame
pandas_df
```

Let's work with basic DataFrame operations and compare their impact in Dask vs Pandas.

In Dask, operations on columns and individual items are easily parallelized, making them very fast.

```
# retrieve the max of the DepDelay column
df.DepDelay.max().compute()
```

```
# visualize the execution plan for this operation
df.DepDelay.max().visualize()
```

Unfortunately, despite the expression graph is a much closer representation of the underlying Dask optimization of the DAG, recent Dask versions won't allow to access anymore to the task graph:

```
# find the average arrival delay (ArrDelay)
# of all flights with carrier UnitedAirlines (UA)
```

```
# visualize the graph for this last operation
```

Dask DataFrames also provide efficient implementations of operations that involve minimal shuffling, such as group-based aggregations and merge operations.

This means that operations like `groupby`, `resample`, `rolling`, and similar operations are still *reasonably* fast, thanks to extensive under-the-hood optimizations.

```
# find the max airtime by combinations of origin and destination
```

```
# visualize the graph for this last operation
```

As we can see from the graph, the `groupby` method returns a single object from the computation, stored in a single partition. This is generally a reasonable choice, as the result of a `groupby` aggregation is typically small enough to fit into a single worker's memory, eliminating the need to split the result into multiple partitions.

However, there may be cases where the returned object is very large, depending on the input datasets. In such situations, we can optimize the number of output partitions by using the `split_out` argument.

```
# find the max airtime by combinations of origin and destination
# split the result of the groupby operation in 4 partitions
df.groupby(['Origin', 'Dest']).AirTime.max(split_out=4).visualize(True)
```

Note: Dask also supports the same aggregate syntax as Pandas, allowing us to run several aggregations simultaneously on the same group.

```
# group the DataFrame by 'Origin' AND 'Dest', and calculate the mean AND
  ↪ standard deviation of 'AirTime'
df.groupby(['Origin', 'Dest'])['AirTime'].aggregate(['mean',
  ↪ 'std']).compute()
```

There are **certain operations** for which the **mapping with the standard Pandas APIs does not hold** in Dask.

For example, slicing and feature-based indexing using `loc` work as expected in Dask, but the position-based indexing operator `iloc` does not behave the same way as it does in Pandas.

```
# select rows where 'Dest' column equals 'DEN' and retrieve the
  ↪ 'UniqueCarrier' column
df.loc[df['Dest'] == 'DEN', ['UniqueCarrier']].compute()
```

```
# visualize the graph for this last operation
df.loc[df['Dest']=='DEN', ['UniqueCarrier']].visualize(True)
```

In Dask, using `iloc` for position-based indexing will raise an exception instead of behaving as it does in Pandas.

```
# use iloc to locate the first i-th rows of the *distributed* DataFrame
df.iloc[0:10]           # Won't work --> a subset of rows (of which Pandas
  ↪ DF?) for all columns
#df.iloc[0:10, [6, 8]] # Won't work --> a subset of rows (of which Pandas
  ↪ DF?) of a given set of columns
```

This behavior occurs because Dask DataFrame does not consider the length of partitions or the row ordering within each partition. When records are sharded across multiple nodes, determining the true ordering of rows becomes ambiguous.

While you can still use `iloc` to select the index of a specific column for all rows in all partitions, such as `iloc[:, 0:10]`, it is important to note that `iloc` can be extremely inefficient in Dask due to the underlying data distribution and lack of explicit row ordering.

```
# use iloc to locate *all* rows of the *distributed* DataFrame
df.iloc[:, [6, 8]]      # Will work --> all rows of a given set of columns
```

Other operations that are **inefficient** in Dask include actions that require *re-indexing* of the entire DataFrame, *sorting*, or *merging based on a column that is not the DataFrame's index*.

Dask optimizes the data distribution by utilizing the range of the index column to subdivide the data into partitions. However, setting an index on a DataFrame requires sorting the entire dataset by the specified column, which can be an expensive process. While sorting can be slow, it can be beneficial to perform it (although infrequently) to accelerate subsequent computations.

Dask also leverages the index in merge/join operations. Performing a join on a column that is not the DataFrame's index is also an expensive operation.

After performing an unavoidable and expensive operation such as reshuffling the data, you can persist the new DataFrame to speed up subsequent computations.

```
# select rows where 'UniqueCarrier' is 'UA' and 'Dest' is 'DEN', and set the
↪ index to 'Date'
df_reindexed = df[(df.UniqueCarrier=='UA') &
↪ (df['Dest']=='DEN')].set_index('Date')
```

```
# compute a 5-days rolling average of the related ArrivalDelay
df_reindexed.ArrDelay.rolling('5D').mean().compute()
```

```
# visualize the graph for this last operation
df_reindexed.ArrDelay.rolling('5D').mean().visualize(True)
```

14.27. Shared, repeated and intermediate computations

When performing the computations mentioned above, there may be cases where the same operation needs to be repeated multiple times.

In Dask DataFrame, for most operations, the arguments are hashed, allowing for duplicate computations to be shared and computed only once.

For example, let's consider computing the mean and standard deviation of departure delay for all non-canceled flights. Since Dask operations are lazy, the computed values are not the final results yet. They represent the recipe required to obtain the final result.

If we change the approach and compute these values with two individual calls to `compute`, there will be no sharing of intermediate computations, resulting in an overall speedup of the computation.

```
# cache the distributed DataFrame in the memory of the workers
df = df.persist()
```

`persist()` is not an in-place operation.

It returns a new `DataFrame` backed by data already computed and stored in memory on the workers.

While calling `df.persist()` without assignment may still speed up later computations (due to graph reuse), the recommended and explicit pattern is:

```
df = df.persist()
```

```
# show the head of the DataFrame
df.head()
```

```
import dask

# create non-cancelled dataframe (lazy)
non_cancelled = df[df.Cancelled==0]

# create mean DepDelay series (lazy)
mean_delay    = non_cancelled.DepDelay.mean()

# create std DepDelay series (lazy)
std_delay     = non_cancelled.DepDelay.std()
```

```
%%time

# compute the time to run both functions
mean_delay_res = mean_delay.compute()
std_delay_res  = std_delay.compute()
```

```
%%time

# compute the time to combining both functions into a single compute call
mean_delay_res, std_delay_res = dask.compute(mean_delay, std_delay)
```

Using `dask.compute` takes approximately half the time compared to the other approach. This is because when calling `dask.compute`, the task graphs for both results are merged, allowing shared operations to be performed only once instead of twice. Specifically, `dask.compute` performs the following operations once:

- Calls to the `read_csv` function
- Filter on cancelled flights
- Some necessary reductions (such as sum and count)

Let's take a look at the computation graph:

What we can observe from the computation graph is that many operations have been merged and used only once, resulting in an optimization of the overall computation time.

14.28. Re-run the same exercise from the Spark notebook using Dask

To apply all we discussed about Dask high- and low-level APIs, we can re-run the same task we used as an example analysis pipeline in PySpark for the DiMuon invariant mass of the LHC collision data.

Starting from the JSON files already used in the previous PySpark example, we will need to:

1. Load all files using the Dask Bag API.
2. Parse the JSON event collections into Python dictionaries.
3. Use a `foldby` operation to count the number of events per sample (mc and data).
4. Plot the distribution of the number of muons in the MC and data samples from the Dask Bag object.
5. Select only events with exactly 2 muons with opposite charge and fill a Dask DataFrame with the normalized results.
6. Create the transverse momenta $p_{T,1}$ and $p_{T,2}$ features and create a 2D plot of these features.

$$p_T = \sqrt{p_x^2 + p_y^2}$$

7. Retain only the dimuon pairs where both muons' p_T is greater than 15 (GeV). 8. Create the components of the dimuon 4-momentum

$$(E, P_x, P_y, P_z) = (E_1 + E_2, p_{x,1} + p_{x,2}, p_{y,1} + p_{y,2}, p_{z,1} + p_{z,2})$$

9. create and plot the invariant mass spectrum

$$M = \sqrt{E^2 - (P_x^2 + P_y^2 + P_z^2)}$$

Before starting, from a terminal (external to the docker container you are working in), copy the `MAPD-B/spark/datasets/lecture2/dimuon` folder into the `MAPD-B/dask/notebooks/datasets/` path, or create a symbolic link to that path.

```
# restart the client to free up the workers' memory
client.restart()
```

14.28.0.1. 1. Load all files using the Dask Bag API.

```
# load the dimuon json files in a bag, each containing the text lines
```

14.28. Re-run the same exercise from the Spark notebook using Dask

14.28.0.2. 2. Parse the JSON event collections into Python dictionaries.

```
# extract the json structure from the lines
```

14.28.0.3. 3. Use a foldby operation to count the number of events per sample (mc and data).

```
# count the number of events per each sample
```

14.28.0.4. 4. Plot the distribution of the number of muons in the MC and data samples from the Dask Bag object.

```
# plot the distribution of the number of muons in the mc and data sample
```

14.28.0.5. 5. Select only events with exactly 2 muons with opposite charge and fill a Dask DataFrame with the normalized results.

```
# select only events with exactly 2 muons with opposite charge and fill a  
↪ dask dataframe with the results
```

14.28.0.6. 6. Create the transverse momenta $p_{T,1}$ and $p_{T,2}$ features and create a 2D plot of these features.

```
# create the new pT1 and pT2 features  
# pT = sqrt(px^2 + py^2)
```

```
# plot the 2 dimensional distribution of the dimuon pair pTs in the mc  
↪ sample
```

14.28.0.7. 7. Retain only the dimuon pairs where both muons' p_T is greater than 15 (GeV).

```
# retain only those dimuon pairs where both muons' pT is greater than 15
```

14. Dask

14.28.0.8. 8. Create the components of the dimuon 4-momentum

```
# create the 4-momentum features
```

14.28.0.9. 9. create and plot the invariant mass spectrum

```
# compute the invariant mass of the dimuon pair  
#  $M = \sqrt{E^2 - (P_x^2 + P_y^2 + P_z^2)}$ 
```

```
# extract the slimmed down dataframe containing only the sample, Mass, and  
↪ pT of the objects  
# and store it in a Pandas DataFrame
```

```
# plot the distribution of the dimuon invariant mass in the mc and data  
↪ sample
```

14.29. Stop client

```
client.close()
```

Finally, use `docker compose down` to stop and clear all running containers.

15. Apache Kafka

15.1. Introduction to Message Streaming

Modern applications increasingly rely on the ability to ingest, route, and process continuous data streams in real time — sensor networks, financial transactions, application event logs, IoT devices, and more. A dedicated **message broker** is often placed at the centre of such architectures.

Apache Kafka is a distributed event streaming platform originally developed at LinkedIn (2011) and later open-sourced through the Apache Software Foundation. It provides:

- **High-throughput, low-latency** publish/subscribe messaging.
- **Durable, fault-tolerant** storage of streams as ordered logs.
- **Scalable** partitioned architecture that distributes load across many brokers.

15.1.1. Core Concepts

Concept	Description
Topic	A named channel to which producers publish and from which consumers read. Topics are partitioned across broker nodes.
Partition	An ordered, immutable log of records within a topic. New messages are always appended.
Offset	The position of a message within a partition. Consumers track their reading offset.
Producer	An application that publishes messages to Kafka topics.
Consumer	An application that subscribes to topics and reads messages.
Consumer Group	A set of consumers that cooperate to read from a topic; each partition is assigned to exactly one consumer in the group.
Broker	A Kafka server that stores and serves partitions. A cluster has multiple brokers for fault tolerance.

15.2. Kafka Architecture

15.2.1. Brokers and Clusters

A Kafka cluster consists of multiple **brokers** — server processes that store and serve partitions. Topics are spread across brokers; each partition lives on exactly one broker at a time (its **leader**), with additional **follower** copies on other brokers for fault tolerance.

Topic "events" - 3 partitions, replication factor 2:

Broker 1	Broker 2	Broker 3
Partition 0 (L)	Partition 0 (F)	
	Partition 1 (L)	Partition 1 (F)
Partition 2 (F)		Partition 2 (L)
L = Leader (handles all reads and writes for that partition)		
F = Follower (replicates the leader; takes over if leader fails)		

The **replication factor** (number of copies of each partition) is set per topic. A replication factor of 3 tolerates 2 broker failures without data loss.

15.2.2. Consumer Offset Management

Each consumer tracks its position in a partition using an **offset** — the sequence number of the last message processed. Offsets are committed back to Kafka so that a consumer can resume after a restart.

The `auto_offset_reset` parameter controls where a consumer starts when no committed offset exists:

Value	Behaviour
'earliest'	Start from the very beginning of the partition log
'latest'	Start from the next new message (default)
'none'	Raise an error if no committed offset is found

15.3. Delivery Semantics

Kafka provides three delivery guarantees, depending on how producers and consumers are configured:

Semantic	Description	Configuration
At-most-once	Messages may be lost; never duplicated	Producer: <code>acks=0</code> ; Consumer: auto-commit before processing
At-least-once	Messages are never lost; may be delivered more than once	Producer: <code>acks=all</code> + retries; Consumer: commit after processing
Exactly-once	Messages processed exactly once — no loss, no duplication	Transactional producer + idempotent consumer; requires Kafka 0.11

Most applications use **at-least-once** semantics and make their processing logic **idempotent** (safe to re-execute with the same message). True exactly-once requires transactional APIs and is more complex to implement.

i Note

Delivery semantics are a key concern when Kafka feeds a Spark Structured Streaming pipeline (as shown in the `KafkaSpark_StructuredStreaming` notebook). Spark's checkpointing mechanism can achieve end-to-end exactly-once semantics when combined with an idempotent sink and Kafka's transactional API.

15.3.1. Pub/Sub Decoupling

A key architectural feature of Kafka is the **complete decoupling** of producers and consumers. A producer does not need to know which consumers exist; a consumer does not need to know which producers are publishing. Messages are written to the topic log and remain available for consumption according to retention policy (time- or size-based), not according to whether a consumer has read them.

This enables flexible, scalable architectures: new consumers can read from the beginning of a topic log without affecting producers or other consumers.

15.3.2. Partitions and Keys

When a producer sends a message without a key, Kafka distributes messages across partitions using a round-robin (or load-based) strategy. When a message includes a **key**, Kafka uses a hash of the key to consistently route all messages with the same key to the same partition — guaranteeing ordering within that group.

Partitioning provides both **scalability** (different partitions can be processed by different consumers in parallel) and **ordering guarantees** (within a partition).

15.4. Hands-on: Apache Kafka

The following notebooks use a Kafka cluster deployed via Docker Compose. The Kafka broker is accessible at `kafka-broker:9092`.

Note: These notebooks require the Docker cluster to be running and are not executed during book rendering.

15.4.1. Producer

15.5. Interacting with Kafka Producer from the Shell

The Producer is the Kafka abstraction for publishing data *to* a Kafka topic.

It is completely detached from any possible Consumer process; therefore, there is no need to directly connect a Producer (Sender) and a Consumer (Receiver) in Kafka due to its Pub/Sub messaging model.

However, for a Producer to publish a message, we need to specify at least two things: 1. The **topic** to which the messages will be published. 2. The **entry point of the cluster** over the network.

The goal is to work with Kafka programmatically (from Python/C/etc.) to interface it with other applications. However, initially, we will interact with the Kafka cluster in *interactive mode* from the shell.

Apache Kafka provides a set of bash scripts to interact with and operate the cluster for basic operations and testing, such as: - Topic creation, configuration, and inspection. - Shell-based message producer. - Shell-based message consumer. - Shell-based performance testing. - ...

Let's first connect to the Kafka cluster from the shell by logging into the Kafka-Broker container:

```
$ docker exec -it <your_kafka-broker_container_name> bash
```

e.g.:

```
$ docker exec -it kafka-kafka-broker-1 bash
```

The Kafka installation is located in the folder whose path is stored as an environmental variable `$KAFKA_HOME`. Inside that folder, you can find the binaries (the “executables”) to use Kafka directly.

To navigate to the `bin` folder, use the following command:

```
cd $KAFKA_HOME/bin
```

From there, you can use the basic Kafka binaries from the shell to issue commands such as:

- List all available topics:

```
./kafka-topics.sh --list --bootstrap-server kafka-broker:9092
```

- Create a new topic:

```
./kafka-topics.sh --create --topic my_awesome_topic --bootstrap-server  
↪ kafka-broker:9092
```

- Describe a specific topic:

```
./kafka-topics.sh --describe --topic my_awesome_topic --bootstrap-server  
↪ kafka-broker:9092
```

We can also use the shell to send messages to the newly created topic:

To start the Kafka console producer, use the following command:

```
./kafka-console-producer.sh --topic my_awesome_topic --bootstrap-server  
↪ kafka-broker:9092
```

At this point, you should be able to send messages to the topic you just created via the `kafka-console-producer`.

So far, no consumer is available to process or display those messages. However, the messages are successfully sent to the topic, increasing the log(s) in the partition(s).

Let's create a console consumer and subscribe to the topic.

To do this, connect to the same Docker container running the Kafka broker and navigate to the folder containing the Kafka binaries.

From there, you can:

- Consume the messages from our previously created topic:

```
./kafka-console-consumer.sh --topic my_awesome_topic --bootstrap-server  
↪ kafka-broker:9092 [--from-beginning]
```

Note: We can instruct the Consumer to go back and read data from the beginning, but the Producer can only append new data to the log.

15.6. Programmatically produce messages to Kafka with Python

There are various Python modules available to interact with Kafka programmatically, including: - `kafka-python` - `confluent-kafka` - `pyKafka`

The differences between these modules are relatively minor. As always, take your time to review the documentation of all alternatives before starting a project.

15. Apache Kafka

For now, we'll use `kafka-python` to handle topics and producers.

```
! pip install kafka-python # confluent-kafka
```

To instantiate a Kafka producer using `kafka-python`, you can use the following code:

```
from kafka import KafkaProducer

# Create a Kafka producer instance
producer = KafkaProducer(
    bootstrap_servers=['62.30.10.23:9092'], # List of brokers
    security_protocol="SSL", # Security protocol (if any)
    ssl_cafile="./ca.pem", # Certificate details (if any)
    ssl_certfile="./service.cert", # ...
    ssl_keyfile="./service.key", # ...
    value_serializer=msgpack.dumps # Message value serialization
    ↪ function
                                     # (e.g., interpreting the
                                     ↪ message as a specific
                                     ↪ format)
)
```

We'll work with the vanilla version of the producer, which does not require any certificates or specific serialization in this example.

Here's an example of a simple producer instantiated by pointing it to the Kafka brokers:

```
# define the list of brokers in the cluster
KAFKA_BOOTSTRAP_SERVERS = ['kafka-broker:9092']
```

```
from kafka import KafkaProducer

# Create a Kafka producer instance
producer = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS)
```

Let's publish a message to the topic we previously created *without specifying any key*.

```
# publish a message to a topic without a key
producer.send(topic='my_awesome_topic',
              value=b'message 1')
```

The output message `<kafka.producer.future...>` explicitly tells us that the record has been created and will be sent at some point.

However, when we pressed return on the previous cell, the message had not been sent yet.

The `KafkaProducer.send()` method is *asynchronous*, which means it enqueues the message on an internal queue. The actual sending of the message to the broker happens later, based on tunable parameters like the maximum buffering time or the number of messages.

This behavior is beneficial because packing multiple messages in small batches improves data transfer efficiency. While sending a single message may not show noticeable performance differences, imagine sending billions of messages per day, which translates to millions of messages per second. Optimizing data transfer and minimizing communication overhead between producers and topics becomes crucial.

It's important to be aware that messages won't necessarily be sent immediately. If a high message rate is sent and the `exit()` command is issued right after a `producer.send()` call, it's possible that no messages are actually sent because the maximum buffering time or the number of messages has not been reached before the program exits.

To send messages synchronously, you can use the `flush()` method of the producer. It ensures that all outstanding messages are sent before proceeding.

For more information on the tunable parameters and additional functionalities, you can refer to the KafkaProducer API documentation: [link to KafkaProducer API](#).

```
# publish a message and flush the queue right away
producer.send('my_awesome_topic', b'a new message')
producer.flush()
```

It's important to realize that producers and consumers are completely decoupled in Kafka. This means that the functioning of a consumer is not affected even if a producer “dies”.

The decoupled nature of Kafka allows consumers to access the topic and consume messages regardless of the state of the producer.

Even if a producer becomes unavailable or experiences an issue, the messages previously sent and stored on the brokers are still accessible to consumers. This ensures data durability and enables fault tolerance in the Kafka ecosystem. Consumers can continue to consume messages from the topic without any interruption, as long as the messages are available on the brokers.

This decoupling of producers and consumers in Kafka is a key architectural feature that enables scalability, fault tolerance, and flexibility in building distributed systems.

It's worth noting that while consumers are not directly impacted by the state of the producer, they may experience delays in receiving messages if there are any disruptions, such as network issues or producer errors, affecting the producer's ability to send messages. However, once the producer resumes normal operation, the consumers can continue consuming messages from where they left off.

This decoupled nature of Kafka makes it well-suited for building robust and scalable distributed systems where producers and consumers can operate independently and asynchronously.

```
# stop the producer
producer.close()
```

```
# create a new Kafka producer and send a new message to the same topic
producer = KafkaProducer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS)
producer.send('my_awesome_topic', b'a message from a newly revived
    ↪ producer')
```

15.7. Messages with Key

In Kafka, messages are structured as `<key, value>` pairs.

So far, we have produced messages only with a given `value`, but it's also possible to include a `key` for each message. The `key` serves as an optional identifier for the message and can be used for various purposes, such as routing messages to specific partitions within a topic.

When a producer sends a message with a key, Kafka uses the key to determine the partition to which the message should be assigned. By default, Kafka uses a *hashing algorithm* to evenly distribute messages across partitions based on their keys. This ensures that messages with the same key are always assigned to the same partition, providing a form of message ordering within a partition.

Adding a key to messages can be useful in scenarios where you want to ensure ordering or group related messages together based on a common key. It enables consumers to process messages in a consistent and predictable manner.

To include a key when producing messages, you can modify the previous code example as follows:

```
# send messages to Kafka using a specific key
producer.send(topic='my_awesome_topic',
              key=b'some_key',
              value=b'a message with key')
producer.flush()
```

15.8. Create a topic from kafka-python

Using the `kafka-python` library, you can administer a Kafka cluster and create new topics with specific configuration parameters, such as the replication factor.

```
# import the necessary packages to administer the Kafka cluster
from kafka.admin import KafkaAdminClient, NewTopic

# connect to the cluster to run admin functions
kafka_admin = KafkaAdminClient(
    bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
)
```

It is possible to retrieve the list of topics, equivalent of issuing the following using the Kafka binaries from shell:

```
./kafka-topics.sh --list --bootstrap-server kafka-broker:9092
```

```
# list the available topics
kafka_admin.list_topics()
```

Topics in Kafka are partitioned entities. Each topic is divided into multiple partitions, and within each partition, events are added to the end of a log, resulting in an ordered list of records.

When you publish a new message to a partitioned topic, the message is appended to the end of the log retained by the owner of the specific partition to which the message is assigned.

Each partition has one leader and zero or more follower replicas. If replication is enabled for the topic, the message will be replicated and distributed to the follower replicas of the partition.

The leader partition handles all read and write operations for the partition, while the follower replicas replicate the log from the leader to ensure fault tolerance and high availability. If the leader partition fails, one of the follower replicas will be elected as the new leader to continue processing messages for that partition.

The use of partitions provides several benefits in Kafka including scalability and parallelism.

It's important to note that the distribution and replication of messages across partitions are managed internally by Kafka and are transparent to the producer and consumer applications. The producer only needs to specify the topic and optionally the key for a message, and Kafka takes care of routing the message to the appropriate partition.

```
# create a new topic with the following parameters:
#   number of partitions = 2
#   replication factor   = 1 (i.e. no replication)
a_new_topic = NewTopic(name='a_partitioned_topic',
                        num_partitions=2,
                        replication_factor=1)

kafka_admin.create_topics(new_topics=[a_new_topic])
```

```
# list the available topics
kafka_admin.list_topics()
```

15.9. Publish messages for the Spark Structured Streaming example

Kafka can be used as a source for incoming messages in Spark Streaming and Structured Streaming.

We'll use the pySpark Structured Streaming API to implement the example previously seen in the Spark hands-on sessions.

```
import json
import time
import random

first_names = ('John', 'Andy', 'Joe', 'Alice')
last_names = ('Johnson', 'Smith', 'Jones', 'Millers')

# while 1: # Uncomment this to send a continuous stream of messages
for i in range(20):
    msg = {
        'name': random.choice(first_names),          #
        ↪ Select a random first name
        'surname': random.choice(last_names),        #
        ↪ Select a random last name
        'amount': float('{:.2f}'.format(random.random() * 1000)), #
        ↪ Generate a random amount
        'delta_t': float('{:.2f}'.format(random.random() * 10)),  #
        ↪ Generate a random delta_t
        'flag': random.choices([0, 1], weights=[0.8, 0.2])[0]    #
        ↪ Randomly choose a flag value
    }
    producer.send('a_partitioned_topic',
                  json.dumps(msg).encode('utf-8') # Convert the message to
    ↪ JSON and encode as UTF-8
                  )
    producer.flush() # Flush the producer buffer
    time.sleep(0.25) # Sleep for a short duration before sending the next
    ↪ message
```

Let's create a new topic to store the results of the Spark processing of the Kafka stream.

```
# create a new topic
a_new_topic = NewTopic(name='results',
                        num_partitions=2,
                        replication_factor=1)
```



```
# create the new topic
kafka_admin.create_topics(new_topics=[a_new_topic])

# check the list of available topics
kafka_admin.list_topics()
```

15.9.1. Consumer

Using the `kafka-python` module, Kafka consumers can be instantiated via the `KafkaConsumer` class:

```
from kafka import KafkaConsumer

# Create a Kafka consumer instance
consumer = KafkaConsumer(
    bootstrap_servers=['62.30.10.23:9092'], # list of brokers
    security_protocol="SSL", # security protocol (if any)
    ssl_cafile="./ca.pem", # certificate details (if any)
    ssl_certfile="./service.cert", # ...
    ssl_keyfile="./service.key", # ...
    value_deserializer=msgpack.unpackb, # message value deserialization
    ↪ function (e.g. unpack the message from a specific format)
    auto_offset_reset='earliest', # automatically bring the
    ↪ reading offset to the earliest message
    group_id="group_A", # identify this consumer as
    ↪ part of group_A
)
```

Once more, we'll use a simple implementation of the consumer, with no specific configurations used in this example.

```
# define the list of brokers in the cluster
KAFKA_BOOTSTRAP_SERVERS = ['kafka-broker:9092']
```

```
from kafka import KafkaConsumer

# create a Kafka consumer instance
consumer = KafkaConsumer(
    bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS, # list of Kafka brokers
```

15. Apache Kafka

```
    consumer_timeout_ms=10000                # maximum time to wait for a
    ↪ new message

                                            # before stopping the
                                            ↪ consumer
)
```

Inspect the available topics on the brokers:

```
# list all available topics on the kafka brokers
consumer.topics()
```

In the Publish/Subscribe (Pub/Sub) model, before consuming messages from a specific topic, you need to subscribe to that topic. By subscribing, the consumer expresses its interest in receiving messages from the specified topic.

It's important to note that subscribing to a topic doesn't immediately consume any messages.

Instead, it establishes a connection between the consumer and the partitions of the subscribed topic hosted on the Kafka brokers. This connection allows the consumer to start polling for messages from those partitions.

Once you have subscribed to the topic, the consumer can start polling for messages, fetching messages from the subscribed partitions and return them to the consumer for further processing.

```
# subscribe to a topic
consumer.subscribe(['my_awesome_topic'])

# check the active subscriptions
consumer.subscription()
```

The `KafkaConsumer` class also offers the possibility to inspect topics (for instance, in terms of the number of partitions), but **not** to modify them.

We can inspect how many partitions the specific topic is made of:

```
# print the list of partition IDs
# e.g. a topic with three partitions will have partition IDs {0, 1, 2}
consumer.partitions_for_topic('my_awesome_topic')
```

We can instruct the consumer to `poll` (i.e., ask for all new messages stored in the topic) with a given cadence or logic.

For instance, we can set the consumer to read only 10 messages at a time, with a timeout between subsequent readouts of a given Δt .

```
# set up the polling strategy for the consumer
```

```

consumer.poll(timeout_ms=0,          # do not enable dead-times before one
    ↪ poll to the next
            max_records=None,        # do not limit the number of records to
    ↪ consume at once
            update_offsets=True      # update the reading offsets on this
    ↪ topic
        )

```

Now the consumer is ready to poll messages (until it is stopped or it reaches a timeout).

Let's look for all messages in the consumer:

```

# this consumer will keep polling for messages
# until stopped by the user
# (or reaches the consumer_timeout_ms, if specified)
for message in consumer:
    print (message)

```

The reading offset can also be reset to the beginning of the topic, allowing for re-reading the entire topic:

```

# go back to the beginning of the topic
consumer.seek_to_beginning()

# read the entire topic and print it out
for message in consumer:
    print (message)

```

The message content (`ConsumerRecord`) can be quite messy, but it can be easily inspected by parsing only the relevant information.

```

# import packages to interpret dates
from datetime import datetime

# go back to the beginning of the topic
consumer.seek_to_beginning()

# break down the message into its main components
for message in consumer:
    print ("%d[%d] @%s k=%s v=%s" % (message.partition,
                                     message.offset,
                                     ↪ datetime.fromtimestamp(message.timestamp/1000).time(),
                                     message.key,

```

```
message.value))
```

Let's change the topic to which the consumer is subscribed and make it a partitioned one:

```
# subscribe to a partitioned topic
consumer.subscribe(['a_partitioned_topic'])
consumer.subscription()
```

By inspecting the number of partitions for this topic, we can now see that there are 2 partitions: partition #0 and partition #1.

```
# check the partitions in the partitioned topic
consumer.partitions_for_topic('a_partitioned_topic')
```

When reading from a partitioned topic, it's easy to observe that the messages are sent to the two partitions in a seemingly arbitrary way.

```
# import json to unpack json strings
import json

# go back to the beginning of the topic
consumer.seek_to_beginning()

# read messages from the beginning of the topic
# decode the json into the python dictionary format
for message in consumer:
    print(f"{message.partition}[{message.offset}]\t"
          ↪ {json.loads(message.value)}")
```

15.10. Creating a Consumer Accessing Only One Partition

Publishing records to a partitioned topic is typically transparent to the user: the producer publishes to the topic, and the Kafka cluster will redirect the message to the partition leader, later replicating it to the followers.

The same applies to a generic consumer. As we have just seen, data is consumed from all partitions within the topic.

In some cases, however, it may be more suitable to instantiate multiple consumers, with each one reading from a specific partition of a topic.

Let's create a consumer specifically designed to access the data from partition #0 of the previous partitioned topic.

```

# import TopicPartition to be able to assign a partition to a consumer
from kafka import TopicPartition

# create a standard consumer with a timeout of 10 seconds
consumer_part_0 = KafkaConsumer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
                                client_id='consumer_n_0',
                                consumer_timeout_ms=10000)

# assign the consumer to a specific topic-partition combination
consumer_part_0.assign([TopicPartition(topic      = 'a_partitioned_topic', #
    ↪ topic
                                partition = 0
                                #
    ↪ partition id
                                )])
)

```

```

# go back to the beginning of the topic (for this specific partition)
consumer_part_0.seek_to_beginning()

# read messages from the beginning of the topic (for this specific
    ↪ partition)
# decode the json into the python dictionary format
for message in consumer_part_0:
    print(f"{message.partition}[{message.offset}]\t"
    ↪ {json.loads(message.value)}")

```

15.11. Creating a Consumer Group

Multiple consumers can read from the same topic.

In Kafka, every consumer is part of a consumer group (even a single consumer is part of its own consumer group).

A consumer group consists of one or more cooperating consumers that gather data from the same topic. The consumer group dynamically balances the load across its members and redistributes the consume calls.

If a consumer within a consumer group fails, the remaining consumers in the same group will continue to read the entire data from the subscribed topic.

```

# create `consumer_one` to read from a specific partition
# assign this consumer to a group
consumer_one = KafkaConsumer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
                              client_id='consumer_one',

```

```

        group_id='my_group',
        ↪ the same group will be used by all consumers
        auto_offset_reset='earliest',
        consumer_timeout_ms=10000)

```

```

# subscribe `consumer_one` to the partitioned topic
consumer_one.subscribe(['a_partitioned_topic'])

```

15.11.1. Consuming messages as a group

Use the ConsumerGroup notebook to: 1. create a second consumer `consumer_two` 2. assign it to the same consumer group `my_group` 3. subscribe to the same topic `a_partitioned_topic`

Each consumer within a group is going to be an independent process (should be run in parallel from the others) and will provide access to a fraction of the incoming data

```

# check the partitions assigned to `consumer_one`
consumer_one.assignment()

```

Start the two consumers and process data in parallel from the two partitions.

Typically, one would achieve this by running the two consumers in separate threads, processes, or executors, depending on the desired computing architecture.

```

# use multiple consumers in parallel (`consumer_one` in this notebook)
for message in consumer_one:
    print(f"{message.partition}[{message.offset}]\t"
        ↪ {json.loads(message.value)})

```

```

# check the partitions assigned to `consumer_one`
consumer_one.assignment()

```

15.12. Reading from the Kafka+Spark results topic

Let's subscribe to the `results` topic and monitor the frauds

```

consumer.subscribe(['results'])

```

```

for message in consumer:
    print ("%d[%d] k=%s v=%s" % (message.partition,
                                message.offset,
                                message.key,
                                message.value))

    print ('        --> sending alert message to user
    ↪ {} \n'.format(message.key.decode('ascii')))

```

15.12.1. Consumer Groups

```

# define the list of brokers in the cluster
KAFKA_BOOTSTRAP_SERVERS = ['kafka-broker:9092']

```

```

from kafka import KafkaConsumer

# create `consumer_two` to read from a specific partition
# assign this consumer to a group
consumer_two = KafkaConsumer(bootstrap_servers=KAFKA_BOOTSTRAP_SERVERS,
                             client_id='consumer_two',
                             group_id='my_group',
                             ↪ the same group will be used by all consumers
                             auto_offset_reset='earliest',
                             consumer_timeout_ms=10000)

```

```

import json

# subscribe `consumer_two` to the partitioned topic
consumer_two.subscribe(['a_partitioned_topic'])

```

```

# check the partitions assigned to `consumer_two`
consumer_two.assignment()

```

```

# use multiple consumers in parallel (`consumer_two` in this notebook)
for message in consumer_two:
    print(f"{message.partition}[{message.offset}]\t
    ↪ {json.loads(message.value)}")

```

```
# check the partitions assigned to `consumer_two`
consumer_two.assignment()
```

15.12.2. Kafka and Spark Structured Streaming

The final hands-on section (`KafkaSpark_StructuredStreaming`) demonstrates Kafka acting as both a **Source** and a **Sink** for a Spark Structured Streaming pipeline. This connects the Source/Sink abstraction introduced in Apache Spark with Kafka's pub/sub model:

- **Source:** Spark reads a stream of JSON messages from a Kafka topic using `.readStream.format("kafka")`
- **Transform:** Spark parses, filters, and aggregates the stream using `DataFrame` operations
- **Sink:** results are written back to a Kafka topic or to the console using `.writeStream`

Achieving end-to-end **exactly-once** semantics requires: (1) Kafka's transactional producer, (2) Spark checkpointing, and (3) an idempotent sink.

Kafka can be integrated relatively easily with the streaming APIs of Spark to act as either a source or a sink for the streaming data.

We'll reproduce the PySpark example we discussed when testing the Structured Streaming, now interfacing both the source and the sink of Spark with Kafka topics.

```
# define the list of brokers in the cluster
KAFKA_BOOTSTRAP_SERVERS = 'kafka-broker:9092'
```

```
import os
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .master("spark://spark-master:7077") \
    .appName("Spark structured streaming application") \
    .config("spark.executor.memory", "512m") \
    .config("spark.sql.execution.arrow.pyspark.enabled", "true") \
    .config("spark.sql.adaptive.enabled", "false") \
    .config("spark.sql.execution.arrow.pyspark.fallback.enabled", "false") \
    .config("spark.sql.shuffle.partitions", 4) \
    .config("spark.jars.packages", \
        "org.apache.spark:spark-sql-kafka-0-10_2.13:4.1.1") \
    .getOrCreate()
```

```
sc = spark.sparkContext
sc
```


15.13. Read data from Kafka

First, we create a `DataFrame` representing the stream of input lines from Kafka by connecting to the appropriate servers and topic.

`readStream` and `format("kafka")` are the key components for connecting to the data source.

```
rawMessagesDf = spark\
    .readStream\
    .format("kafka")\
    .option("kafka.bootstrap.servers", KAFKA_BOOTSTRAP_SERVERS)\
    .option("kafka.consumer.pollTimeoutMs", 4000)\
    .option("startingOffsets", "latest") \
    .option("subscribe", "a_partitioned_topic")\
    .option("failOnDataLoss", "false")\
    .load()
```

At this stage, nothing happens because the streaming has not started yet. We have only defined the source of the streaming application.

The `rawMessagesDf` can be treated as if it were a static `DataFrame` in Spark, **but** it is a streaming `DataFrame` that continuously grows.

```
rawMessagesDf.isStreaming
```

Currently, `rawMessagesDf` contains the messages from Kafka, which are further composed of a `<key, value>` pair, a timestamp, and other information.

```
rawMessagesDf.printSchema()
```

We are clearly interested in the `value` of the message, which is in JSON format.

We can utilize the `pyspark.sql` functions to define a schema for our structured data and interpret the data accordingly.

```
from pyspark.sql.functions import from_json, col
from pyspark.sql.types import StructField, StructType, StringType,
    ↳ DoubleType, IntegerType

# the schema of the json data format used to create the messages
schema = StructType(
    [
        StructField("name", StringType()),
        StructField("surname", StringType()),
        StructField("amount", DoubleType()),
        StructField("delta_t", DoubleType()),
```

```

        StructField("flag", IntegerType())
    ]
)

# a new DF can be created from the previous by using the pyspark.sql
↳ functions
parsedDf = (
    rawMessagesDf
    .select(
        from_json(col("value").alias("value").cast("string"), schema)
        .alias("data")
    )
    .select(col("data.*"))
)

```

```
parsedDf.printSchema()
```

It's possible to inspect the data actually reaching the `parsedDf` by starting a stream with a `console` sink. The output of the streaming processing in this case will be displayed in the shell running the Jupyter Notebook.

We should expect a DataFrame with a single column, containing a JSON-like object per message.

```

query = (
    parsedDf.writeStream\
        .outputMode("append")\
        .format("console")\
        .start()
)

```

```
query.stop()
```

15.14. Simple Fraud Detection + Kafka Sink

Let's integrate the same simplistic "fraud detection" logic we previously used in our Spark Streaming example.

We might be interested in having Spark perform the heavy lifting and then allowing other consumers to access the results of these operations. For example, we can display the results on a dashboard or trigger some action.

We can use Kafka as a sink for the resulting data.

```

from pyspark.sql.functions import concat, col, lit, countDistinct

# find number of transactions for each user when flag = 1
num_transactions = parsedDf \
    .where(col('flag')==1) \
    .withColumn('id', concat(col('name'), col('surname'))) \
    .groupBy('id') \
    .count()

# find suspicious transactions
sus_transactions = num_transactions \
    .where(col('count')>1) \
    .withColumn('fraud', lit(1)) \
    .select(col("id").alias("key"),
            col("fraud").cast(StringType()).alias("value"))

```

At this point, it may seem that these newly derived DataFrames are not connected in any way or form to the input streaming Kafka source.

However, we can verify this by using the `isStreaming` method on a DataFrame.

```
sus_transactions.isStreaming
```

```
sus_transactions.printSchema()
```

The `sus_transactions` DataFrame is still a streaming DataFrame. It can be seen as the result of the transformations applied to the original `inputDF` derived from the streaming source.

```

query = (
    sus_transactions.writeStream\
        .outputMode("update")\
        .format("console")\
        .start()
)

```

```
query.stop()
```

Finally, let's wrap up the message and write it to a `results` topic back to Kafka, so that other consumers can make use of its data.

```

query = (
    sus_transactions.writeStream\

```

```
        .outputMode("update") \
        .format("kafka") \
        .option("kafka.bootstrap.servers",
↪ KAFKA_BOOTSTRAP_SERVERS) \
        .option("topic", "results") \
        .option("checkpointLocation", "checkpoint") \
        .option("failOnDataLoss", "false") \
        .start()
    )
```

```
query.stop()
```

15.15. Stop worker and master

```
# stop the running Spark context and Spark session
sc.stop()
spark.stop()
```

Finally, use `docker compose down` to stop and clear all running containers.